



Elmar Wings

AI with an Arduino Nano 33 BLE Sense

Realtime Object Detection with the ArduCAM

Version: 1765
April 21, 2025

Contents

Contents	3
List of Figures	15
List of Listings	19
1. Introduction	23
1.1. Introduction	23
1.2. Challenges and Solutions	23
1.3. Structure	23
I. Arduino Nano 33 BLE Sense - Onboard Sensoren	25
2. Arduino Nano 33 BLE Sense	27
2.1. Arduino Nano 33 BLE Sense	27
2.2. Unterschied zwischen dem Arduino Nano 33 BLE Sense Rev1, dem Arduino Nano 33 BLE Sense Rev2 und dem Arduino Nano 33 BLE Sense Lite	28
2.2.1. What is the difference between Rev1 and Rev2?	28
2.2.2. Changes in the Sketch	29
2.2.3. Arduino Nano 33 BLE Sense Lite	30
2.3. On-Board Sensor Description	30
2.3.1. Gesture, Proximity, and Color Detection Sensor ADPS-9960 . .	32
2.3.2. Accelerometer, Gyroscope, and Magnetometre Sensor LSM9DS1	33
2.3.3. Pressure Sensor LPS22HB	34
2.3.4. Relative Humidity and Temperature Sensor HTS221	35
2.3.5. Digital Microphone MP34DT05-A	35
2.3.6. Bluetooth Module nRF52840	37
2.4. Bluetooth Module INA B306	38
2.5. Arduino Nano 33 BLE Pin Configuration	39
2.6. Was fehlt	39
3. Reset Button on the Arduino Nano 33 BLE Sense	41
3.1. Purpose and Functionality	41
3.1.1. Functions of the Reset Button	41
3.2. Timing and Sequence	42
3.3. Use Cases	42
3.4. Conclusion	42
4. Built-inLED	43
4.1. General Information	43
4.2. Built-in LED	43
4.3. Specification	43
4.3.1. Pin Assignment	43
4.3.2. Power Consumption	44

4.4.	Simple Code	44
4.4.1.	Simple Code for the Built-in LED	44
4.4.2.	Simple Code for Testing the Built-in LED	45
4.5.	Tests	45
4.5.1.	Simple Function Test	45
4.5.2.	Test all Functions	46
4.6.	Simple Application	46
4.7.	Further Readings	46
5.	Power LED	47
5.1.	General Information	47
5.2.	Power LED	47
5.3.	Specification	48
5.3.1.	Pin Assignment	48
5.3.2.	Power Consumption	48
5.4.	Simple Code	48
5.4.1.	Simple Code for LEDs	48
5.4.2.	Simple Code for the Power LED	50
5.4.3.	Simple Code for Testing the Power LED	50
5.4.4.	Test all Functions	51
5.5.	Simple Application	51
5.6.	Further Readings	54
6.	Built-in RGB-LED	55
6.1.	General Information	55
6.2.	Specific Sensor	55
6.3.	Specification	56
6.3.1.	Pin Assignment	56
6.3.2.	Power Consumption	56
6.4.	Simple Code	56
6.5.	Tests	57
6.5.1.	Simple Function Test	57
6.5.2.	Test all Functions	58
6.6.	Simple Application	62
6.7.	Further Readings	63
7.	TinyML Shield: Built-in Push Button	65
7.1.	General	65
7.2.	Built-in Push Button	65
7.3.	Specification	66
7.4.	Simple Code	66
7.5.	Tests	66
7.6.	Interrupt Function on the Arduino Nano 33 BLE Sense	68
7.7.	Simple Application	68
7.8.	Further Readings	70
8.	Pressure and Temperature Sensor LPS22HB	71
8.1.	General	72
8.2.	Specific Sensor	72
8.3.	Specification	73
8.4.	Library	74
8.4.1.	Description	74
8.4.2.	Installation	74
8.4.3.	Functions	75
8.4.4.	Example - Manual	76

8.4.5. Example	76
8.4.6. Example - Code	76
8.4.7. Example - Files	76
8.5. Calibration	76
8.6. Simple Code	78
8.7. Sleep Mode	78
8.8. Simple Application	80
8.9. Tests	80
8.9.1. Simple Function Test	80
8.9.2. Test all Functions	80
8.10. Simple Application	80
8.11. Further Readings	80
9. Sensormodul APDS-9960 for Gesture, Proximity, and Color Detection	81
9.1. Functionality of the APDS-9960	81
9.2. Key Technical Specifications	81
9.3. Library Arduino_APDS9960	83
9.3.1. Installation of APDS-9960 Library	83
9.3.2. Functions	83
9.4. Simple Function Tests	85
9.4.1. Example Color Detection	85
9.4.2. Example Color Detection - Manual	86
9.4.3. Setting Up the Color Sensor Measurement Program	86
9.4.4. Example	86
9.4.5. Example Color Detection - File	86
9.4.6. Proximity	87
9.4.7. Proximity - Manual	87
9.4.8. Example Proximity - Code	87
9.4.9. Example Proximity - File	87
9.4.10. Gesture Detection	88
9.4.11. Example Gesture Detection	88
9.4.12. Example Gesture Detection - Manual	88
9.4.13. Example Gesture Detection - Code	88
9.4.14. Example Gesture Detection - File	89
9.4.15. Troubleshoot	90
9.5. Calibration Color Detection	91
9.6. Calibration Color Detection	91
9.6.1. Calibration Setup	91
9.6.2. Step 1: Measuring Reference Values	91
9.6.3. Step 2: Normalizing the Sensor Values	91
9.6.4. Step 3: Implementing the Calibration in Code	92
9.6.5. Calibration File	92
9.6.6. Step 4: Validating the Calibration	96
9.7. Tests	96
9.7.1. Simple Function Test	96
9.7.2. Test all Functions	96
9.8. Simple Application	96
9.9. Further Readings	100
10. Sensor APDS 9960 Gesture	101
10.0.1. Sensormodul APDS 9960 for Gesture	101
10.1. General	101
10.2. Specific Sensor	101
10.3. Specification	101

10.4. Bibliothek	102
10.4.1. Description	102
10.4.2. Installation	102
10.4.3. Functions	102
10.4.4. Example - Manual	102
10.4.5. Example	102
10.4.6. Example - Code	102
10.4.7. Example - Files	102
10.5. Calibration	102
10.6. Simple Code	102
10.7. Simple Application	102
10.8. Tests	102
10.8.1. Simple Function Test	102
10.8.2. Test all Functions	102
10.9. Simple Application	102
10.10 Further Readings	102
11. Sensor APDS 9960 Color	103
11.1. General	103
11.2. Specific Sensor	105
11.3. Specification	105
11.4. Bibliothek	105
11.4.1. Description	105
11.4.2. Installation	105
11.4.3. Functions	105
11.4.4. Example - Manual	105
11.4.5. Example	105
11.4.6. Example - Code	105
11.4.7. Example - Files	105
11.5. Calibration	105
11.6. Simple Code	106
11.7. Simple Application	106
11.8. Tests	106
11.8.1. Simple Function Test	106
11.8.2. Test all Functions	106
11.9. Simple Application	106
11.10 Further Readings	106
12. Sensor APDS 9960 Proximity	107
12.1. General	107
12.2. Specific Sensor	107
12.3. Specification	107
12.4. Bibliothek	107
12.4.1. Description	107
12.4.2. Installation	107
12.4.3. Functions	107
12.4.4. Example - Manual	107
12.4.5. Example	107
12.4.6. Example - Code	107
12.4.7. Example - Files	107
12.5. Calibration	107
12.6. Simple Code	108
12.7. Simple Application	108
12.8. Tests	108
12.8.1. Simple Function Test	108

12.8.2. Test all Functions	108
12.9. Simple Application	108
12.10. Further Readings	108
13. Sensor Ambient Light	109
13.1. General	109
13.2. Specific Sensor	109
13.3. Specification	109
13.4. Bibliothek	109
13.4.1. Description	109
13.4.2. Installation	109
13.4.3. Functions	109
13.4.4. Example - Manual	109
13.4.5. Example	109
13.4.6. Example - Code	109
13.4.7. Example - Files	109
13.5. Calibration	109
13.6. Simple Code	110
13.7. Simple Application	110
13.8. Tests	110
13.8.1. Simple Function Test	110
13.8.2. Test all Functions	110
13.9. Simple Application	110
13.10. Further Readings	110
14. Application of the Color Sensor with OLED Display	111
14.1. Manual	111
14.2. Code Explanation	112
14.3. Application Test	115
14.4. Improvement Suggestions	115
15. Mikrophone MP34DT05-A	117
15.0.1. Ton und Frequenzen	117
15.0.2. Unterschied Ton, Klang und Geräusch	118
15.0.3. Mikrofone	118
15.0.4. Wavedatei	120
15.0.5. Package PyAudio Version 0.2.14	123
15.0.6. Package Wave Version 3.12	123
15.0.7. Package NumPy Version 1.26	124
15.0.8. Package Guizero Version 1.5.0	125
15.0.9. Package Threading (Version 3.7)	126
15.0.10. Package Librosa Version 0.10.2	127
15.1. Funktionen	127
15.1.1. Laden	127
15.1.2. Abspielen	128
15.1.3. Speichern	128
15.1.4. Anzeigen von zwei Audio-Dateien	130
15.1.5. Glättung von Audio-Dateien	131
15.1.6. Ausschnitt	132
15.1.7. Zoomen in einer Matplotlib	133
15.1.8. Änderung der Tonhöhe und Geschwindigkeit	135
15.1.9. Frequenzanalyse	138
15.1.10. Overlay	140
15.2. General	141
15.2.1. Decibels	141

15.2.2. Puls-Dichtemodulation	142
15.3. Specific Sensor	142
15.4. Specification	142
15.5. Bibliothek pdm.h	143
15.5.1. Description	143
15.5.2. Installation	143
15.5.3. Functions	143
15.5.4. Example - Manual	144
15.5.5. Example	144
15.5.6. Example - Code	144
15.5.7. Example - Files	145
15.6. Calibration	145
15.7. Simple Code	145
15.8. Simple Application	145
15.8.1. Pin-Konfiguration	145
15.8.2. Initialisierung und Setup	145
15.8.3. Messungssteuerung	145
15.8.4. Mikrofondatenverarbeitung	147
15.8.5. Anzeige der Ergebnisse	147
15.8.6. Umrechnung auf den Wert dBSPL	147
15.8.7. Benutzeroberfläche	148
15.9. Tests	148
15.9.1. Simple Function Test	149
15.9.2. Test all Functions	149
15.10 Specific Sensor	149
15.11 Specification	149
15.12 PDM Library	149
15.13 Calibration	150
15.14 Simple Code	150
15.15 Simple Application	150
15.16 Tests	151
15.17 Further Readings	151
16. Inertial Measurement Unit	153
16.1. Introduction	153
16.2. 6-Axis IMU LSM6DSOX	154
16.3. IMU LSM6DSOX Features	154
16.4. IMU LSM6DSOX Data	156
16.4.1. Accelerometer:	156
16.4.2. Gyroscope	157
16.5. Library setup in Arduino IDE	157
16.6. Applications	158
16.7. LSM9DS1(IMU)	158
16.8. Features	158
16.9. Pin description LSM9DS1	159
16.10 Pin connections LSM9DS1	159
16.10.1 Module specifications	161
16.10.2 Block Diagram	162
16.10.3 System Requirements	164
16.10.4 Precautions to be taken	164
16.11 Bibliotheken	165
16.12 Beispiele auf dem Mikrocontroller	165
16.12.1 Testen des Sensors LSM9DS1	165
16.13 Programmierung	165
16.13.1 Programmablaufplan	165

16.13.2 Programmcode und Dokumentation	165
16.13.3 Definition Echtzeit	171
16.14 Kalibrierung	171
16.15 Probleme	172
17. Calibration	173
17.1. Introduction	173
17.2. Standard Operating Procedure	173
17.2.1. Low and high limit method	176
17.2.2. FreeIMU Calibration Application Magnetometer	176
17.2.3. Example	176
18. Errors	179
18.1. Introduction	179
18.2. Affected Parameters	179
18.2.1. Accelerometer:	180
18.2.2. Gyroscope:	180
19. IMU LSM6DSOX - Libraries and Functions	181
19.1. Libraries	181
19.1.1. Wire.h	181
19.1.2. Kalman.h	181
19.1.3. Arduino_LSM6DSOX.h	181
19.1.4. LSM6DSOXSensor.h	181
19.2. Functions	182
19.2.1. setup()	182
19.2.2. loop()	182
19.2.3. IMU.begin()	182
19.2.4. IMU.setAccelerometerRange()	182
19.2.5. IMU.setGyroscopeRange()	182
19.2.6. IMU.setAccelerometerDataRate()	183
19.2.7. IMU.setGyroscopeDataRate()	183
19.2.8. IMU.accelerationSampleRate()	183
19.2.9. IMU.gyroscopeSampleRate()	183
19.2.10 IMU.temperatureAvailable()	183
19.2.11 IMU.readTemperature()	183
19.2.12 IMU.accelerationAvailable()	183
19.2.13 IMU.readAcceleration()	183
19.2.14 IMU.gyroscopeAvailable()	184
19.2.15 IMU.readGyroscope()	184
19.2.16 randomGaussian()	184
19.2.17 kalmanX.setQ(), kalmanY.setQ(), kalmanZ.setQ()	184
19.2.18 kalmanX.setR(), kalmanY.setR(), kalmanZ.setR()	184
19.2.19 kalmanX.update(), kalmanY.update(), kalmanZ.update()	184
19.2.20 kalmanX.getSigma(), kalmanY.getSigma(), kalmanZ.getSigma()	185
19.2.21 delay()	185
19.2.22 lsm6dsoxSensor.Set_X_FS()	185
19.2.23 lsm6dsoxSensor.Set_G_FS()	185
19.2.24 Initialize_IMU()	186
19.2.25 Factors_Calculation()	186
19.2.26 Factors_Calculation()	186
20. Sensor Inertial Mesure Unit	187
20.1. General Description	187
20.1.1. Always On Mode	187

20.1.2. Tilt detection	187
20.1.3. Significant Motion Detection	188
20.2. Specific Sensor	188
20.2.1. LSM9DS1	188
20.2.2. Accelerometer	188
20.2.3. Gyroscope	189
20.3. Specification	190
20.3.1. Accelerometer Sensor	190
20.3.2. PIN Connction	190
20.4. Library	191
20.4.1. Library Description	191
20.4.2. Installation	192
20.5. Function	192
20.5.1. Library <code>Wire.h</code>	193
20.5.2. Function <code>IMU.begin()</code>	194
20.5.3. Function <code>IMU.end()</code>	194
20.5.4. Function <code>IMU.readAcceleration(x,y,z)</code>	194
20.5.5. Function <code>IMU.readGyroscope()</code>	195
20.5.6. Function <code>IMU.accelerationAvailable()</code>	195
20.5.7. Function <code>IMU.gyroscopeAvailable()</code>	195
20.5.8. Function <code>IMU.accelerationSampleRate()</code>	196
20.5.9. Function <code>IMU.gyroscopeSampleRate()</code>	196
21.Distance, Speed and Acceleration Detection Algorithms	197
21.1. Review of Distance Measurement Methods	197
21.2. Review of Speed and Acceleration Detection Algorithms	198
II. Arduino Nano 33 BLE Sense - External Sensors and Actors	201
22.Tiny Machine Learning Kit	203
22.1. Das Arduino Tiny Machine Learning Shield	203
22.2. Die OV7675 Kamera	204
22.3. USB-Micro-B- auf USB-A-Kabel	205
23.Powering the TinyML Shield	207
23.0.1. USB Power Delivery	207
23.1. Battery	207
23.2. Checking the Battery Voltage	208
23.3. Batterieclip	209
23.4. Spannungssensor	209
23.4.1. Durchführung	210
23.4.2. Ergebnisse	212
24.Connectors of Type JST	213
24.1. Connector Series	213
24.2. JST VH	213
24.2.1. Product Profile	213
24.2.2. Specification	213
24.3. JST PH	214
24.3.1. Product Profile	214
24.3.2. Specification	214
24.3.3. JST PHR-2	215

25. Grove	217
25.1. Grove-Universal-Kabel	218
25.2. Verbindung Grove-Schnittstelle zu 4-Pin-Lösungen	218
25.3. Pinbelegung	218
26. BlueTooth	221
26.1. Quick Start	221
26.2. Supplies	221
26.3. Step 1: Introduction	221
26.4. How pfodApp is optimised for short BLE style messages	223
26.5. Step 2: Creating the Custom Android Menus and Generating the Code	223
26.6. BLE Mobile App “Nordic Semiconductors nRF Connect”	224
26.7. Arduino BLE Example – Explained Step by Step	226
26.7.1. Arduino BLE Example Code Explained	226
26.7.2. Arduino BLE – Bluetooth Low Energy Introduction	226
26.7.3. Arduino BLE Example 1 – Battery Level Indicator	226
26.7.4. Arduino BLE Tutorial Battery Level Indicator Code	228
26.7.5. Arduino Bluetooth Battery Level Indicator Code Explained . .	228
26.7.6. Installing the App for Android	229
26.7.7. Example 2 – Arduino BLE Accelerometer Tutorial	229
26.8. Arduino Library BLEAK	235
26.9. Test	235
26.10. Test with Bluetooth Module Connection	235
26.11. BLE Stack	236
26.12. Control Arduino Nano BLE with Bluetooth & Python	237
26.13. Peripheral Device aka Arduino Nano	237
26.13.1. Taking the Project Further	240
27. Ultraschallsensor HC-SR04	241
27.1. Einleitung	241
27.2. Domänenwissen	241
27.3. Grundlagen zur Verwendung eines Ultraschallsensors	242
27.3.1. Grundlagen Schall	242
27.3.2. Schallgeschwindigkeit und Wellenlänge	242
27.3.3. Absorption und Reflexion	243
27.3.4. Weg- und Abstandsmessung mit Ultraschall	244
27.3.5. Anwendungen	245
27.3.6. Herausforderungen	245
27.3.7. Lösungsansätze	246
27.4. Ultraschall Abstandsensor	246
27.5. Specification	246
27.6. Bibliothek	247
27.6.1. Description	247
27.6.2. Installation	247
27.6.3. Functions	247
27.6.4. Example - Manual	247
27.6.5. Example	247
27.6.6. Example - Code	247
27.6.7. Example - Files	247
27.7. Calibration	247
27.8. Kalibrierung	247
27.9. Simple Code	247
27.10. Einfacher Beispiel zur Verwendung des Ultraschallabstandsensors . . .	247
27.10.1. Durchführung	247
27.10.2. Ergebnisse	249

27.11 Simple Application	249
27.12 Tests	249
27.12.1 Simple Function Test	249
27.12.2 Test all Functions	249
27.13 Simple Application	249
27.14 Further Readings	249
28. Entwicklerboard - Motorsteuerung DEBO MotoDriver2 L298N	251
29. Terminal Adapter Board mit Schraubklemmen kompatibel mit Nano V3 und Arduino	253
30. Drehwinkel-Encoder	255
30.1. Encoder	255
30.2. Drehwinkel-Encoder	255
31. Getting Started	257
31.1. Download and Install the NRF Connect App on Your Mobile	257
31.1.1. For Android	257
31.1.2. For iOS	257
31.2. Power ON the Arduino Nano 33 BLE Sense Lite	258
31.2.1. Connect to Arduino Nano 33 BLE Sense Lite through NRFconnect	258
31.2.2. For Android	258
31.2.3. For iOS	258
III. Applikation <Title>	261
32. Application	263
33. Conclusion XY	265
34. To DoX Y	267
IV. Templates	269
35. Sensor	271
35.1. General	271
35.2. Specific Sensor	271
35.3. Specification	271
35.4. Library	271
35.4.1. Description	271
35.4.2. Installation	271
35.4.3. Functions	271
35.4.4. Example's Manual	271
35.4.5. Inside the Example	271
35.4.6. Example's Code	271
35.4.7. Example's Files	271
35.5. Calibration	271
35.6. Simple Code	272
35.7. Simple Application	272
35.8. Tests	272
35.8.1. Simple Function Test	272
35.8.2. Test all Functions	272
35.9. Simple Application	272

Contents	13
35.10 Further Readings	272
36. Package <code>Example</code>	273
36.1. Introduction	273
36.2. Description	273
36.3. Installation	273
36.4. Example - Manual	273
36.5. Example	273
36.6. Example - Code	273
36.7. Example - Files	273
36.8. Further Reading	273
 V. Anhang	 275
37. Materialliste	277
 VI. Hints - Examples for LaTeX - Read and Use	 279
38. Hints	281
38.1. Allgemeine mathematische Beschreibung Bézier-Kurve	282
39. Verrundung mit einem Kreisbogen	285
39.1. Gleichungen	285
39.2. Bewertung	287
39.3. Examples	288
40. Maple files	291
40.1. Geometries with Maple	291
40.2. Geometry elements used	291
40.3. Building the data structure	292
40.4. Structure of a module	292
40.4.1. General structure of a module	294
40.4.2. Saving a module	295
40.4.3. Verwendung eines Moduls	295
40.4.4. Creating a Maple module	295
40.5. Functions of the modules	296
40.5.1. Module <code>MPoint</code>	296
40.5.2. Module <code>MLine</code>	297
40.5.3. Module <code>MArc</code>	297
40.5.4. module <code>MBezier</code>	298
40.5.5. Module <code>MPolygon</code>	298
40.5.6. module <code>MGeoList</code>	298
40.5.7. Module <code>MHermiteProblem</code>	299
40.5.8. Module <code>MHermiteProblemSym</code>	299
40.5.9. Module <code>MConstant</code>	300
40.5.10. Module <code>MGeneralMath</code>	300
40.5.11. Module <code>Biarc</code>	301
40.5.12. Module <code>MBiarc</code>	301
40.6. Programme Flowchart	302
40.6.1. Overall Flow	302
40.6.2. Verrundung der Kurve	302
 41. Representation of Python Programs	 305

42. Representation of Programs written for Arduino Boards	307
43. First Chapter	309
44. CAGD	311
A. drawings with tikz	313
B. Criteria for a good $\text{\LaTeX}$ project	317
Literaturverzeichnis	319
Stichwortverzeichnis	326
Index	327

List of Figures

2.1. Arduino Nano 33 BLE Sense Rev. 2, see Arduino Store	28
2.2. Arduino Nano 33 BLE Sense Rev. 1	29
2.3. Arduino Nano 33 BLE Sense Lite	30
2.4.	31
2.5. Pin assignment of the Arduino Nano 33 BLE Sense	32
2.6. Circuit diagram microphone	37
2.7. Connection with BLE and smartphone	38
2.8. Simple sketch to seen Arduino battery	38
2.9. Arduino Nano 33 BLE Pin Configuration	40
3.1. Arduino Nano 33 BLE Sense's Reset Button	41
4.1. Arduino Nano 33 BLE Sense's built-in LED with Pin 13	43
5.1. Arduino Nano 33 BLE Sense's Power LED with Pin 25	47
6.1. Arduino Nano 33 BLE Sense's RGB LED with Pin 22, Pin 23, and Pin 24	55
7.1. The Tiny Machine Learning Shield	65
8.1. Arduino Nano 33 BLE Sense's pressure and temperature sensor LPS22HB	71
9.1. Arduino Nano 33 BLE Sense's APDS-9960	81
9.2. APDS-9960 Library Instalation	83
9.3. Gesture, Proximity, Color Sensor Output Window	96
11.1. Schematic of the main function of a color sensor	103
11.2. Functional Block Diagram	104
14.1. Circuit diagram for connecting the Arduino Nano 33 BLE Sense with a 1.30" IIC OLED display.	111
15.1. Arduino Nano 33 BLE Sense's RGB LED with Pin 22, Pin 23, and Pin 24	117
15.2. Liste verschiedener Mikrofonarten mit Funktionsprinzip	119
15.3. Datenblatt des betrachteten Mikrofons	119
15.4. Aufbau eines Kondensator Mikrofons	120
15.5. Funktionsprinzip eines Analog zu Digital Wandlers	120
15.6. Symbole und numerische Werte des ASCII Formats	121
15.7. Aufbau des Wave Dateiformats mit Erklärung [Wil03]	122
15.8. Beispiel Anzeige von zwei Audio-Dateien	131
15.9. Window-Funktion	136
15.10Ausgabe Sample Frequenzanalyse	140
15.11MP34DT05, Digital Microphone	148
15.12MP34DT05, Serial Plotter	149
16.1. Examples in the library Arduino_LSM9DS1	153
16.2. <i>Axis Acceleration of Accelerometer Sensor</i>	156
16.3. <i>Angular Speed of Gyroscope Sensor</i>	157

16.4. Library setup in Arduino IDE	157
16.5. Pin connections LSM9DS1	159
16.6. Pin description LSM9DS1	160
16.7. Sensor Characteristics	161
16.8. Temperature Sensor Characteristics	161
16.9. Absolute Maximum Temperature	162
16.10 Accelerometer and gyroscope block diagram	162
16.11 Magnetometer block diagram	163
16.12 LSM9DS1 electrical connections	164
16.13 Output des Testprogramms	165
16.14 Der Programmablaufplan	166
16.15 Anzeige des Arduino OLED Displays sobald der Arduino eingeschaltet ist	171
16.16 Ausgabe von Messdaten	171
16.17 Der Reset Button des Arduino	172
20.1. LSM9DS1 axis on this card	189
20.2. LSM9DS1 axis on this card	189
20.3. Explication of pin mode	191
20.4. Wire include	192
20.5. Board card installation	192
20.6. Library choice	192
21.1. Design of OAK-D Pro camera	198
22.1. Das Tiny Machine Learning Kit	203
22.2. Das Tiny Machine Learning Shield	204
22.3. Pin-Belegung des Tiny Machine Learning Shields	204
22.4. Das OV7675 Kameramodul	205
22.5. Ein USB-A auf Micro-USB Kabel	205
23.1. Montage des Clips	208
23.2. Komplettes System mit Batterie	208
23.3. Batterieclip für 9-Volt-Block[Rei24b]	210
23.4. Voltage Sensor 170640 von dem Hersteller <i>Shenzhen Global Technology Co., Ltd</i> [She]	210
23.5. Testoutput des Spannungssensors	212
24.1. JST connector type VH https://www.jst-mfg.com/product/index.php?series=262	214
24.2. JST connector type VH [Jstc]	215
24.3. JST connector type PHR-2 [Jstc]	215
25.1. Mikrocontroller gesteckt auf dem Shield	217
25.2. Grove-Universal-Kabel[Rei24a]	218
25.3. Grove Jumper zu Grove 4 Pin-Kabel[Rei24c]	218
26.1. BLE Devices – Peripheral and Central Devices	222
26.2. Step 2: Creating the Custom Android Menus and Generating the Code	223
26.3. App nRF is searching the Arduino Nano BLE Sense	224
26.4. App nRF is searching the Arduino Nano BLE Sense	225
26.5. App nRF is getting data the Arduino Nano BLE Sense	225
26.6. BLE Devices – Peripheral and Central Devices	228
26.7. Battery app “nRF Connect”	230
26.8. nRF Connect Screen	233
26.9. Accelerometer Values Read from Arduino Using BLE	234
26.10 Bluetooth Connection	235

27.1. Ausbreitungsgeschwindigkeit von Schall in Luft	243
27.2. Abnahme der Schallintensität in Abhängigkeit der Frequenz bei einem Meter, nach [HS23]	244
27.3. Spannungsverlauf Schallwandler bei einer Echo-Laufzeit-Messung, nach [HS23]	245
27.4. Ultraschallabstandssensor der Marke JOY-IT[Simb]	246
27.5. Testoutput des Ultraschallsensors	249
28.1. Motorsteuerung DEBO MotoDriver2 L298N	252
28.2. Treiber mit Beschriftung	252
29.1. Terminal Adapter Board mit Schraubklemmen kompatibel mit Nano V3 und Arduino	254
31.1. NRF Connect Application	257
31.2. Landing page	258
31.3. List of devices	259
38.1. Bernstein polynomial of degree 3	283
38.2. Bézier curve for example 38.1	284
39.1. Smoothing a corner with the help of an arc - triangle	285
39.2. Angular change with blending arc	287
39.3. Curvature progression for a blending arc	287
39.4. Maximum range for a blending arc of a circle - $L(\varepsilon = \text{const}, \alpha)$	288
39.5. Deviation when specifying the distance L when rounding with a circular arc	288
40.1. Programme flow chart „Corner rounding“	303
40.2. Programme flowchart „Selection of the rounding strategies“	304

List of Listings

2.1. Simple example using of the builtin microphone of the Arduino Nano 33 BLE Sense	36
4.1. Header file without comments for using the Built-in LED	44
4.2. Code file without comments for using the Built-in LED	44
4.3. Simple sketch without comments to control the built-in LED	45
4.4. Simple sketch to test the built-in LED	45
4.5. A simple watch dog: A simple watchdog: the built-in LED is switched on for 1 second every 30 seconds.	46
5.1. Header file without comments for using a LED	49
5.2. Code file without comments for using a LED	49
5.3. Header file with doxygen comments for using the Power LED	50
5.4. Code file with doxygen comments for using the Power LED	50
5.5. Simple sketch to control the power LED	50
5.6. Simple sketch to check the battery state using the power LED	51
5.7. Simple code for signs of life	52
5.8. Simple code for signs of life	52
5.9. Simple sketch to check the battery state using the power LED	53
6.1. Header file without comments for using the RGB-LED	56
6.2. Code file without comments for using the RGB-LED	57
6.3. Simple sketch to test the RGB LED	57
6.4. value between 0 and 255 to write to the RGB LED	59
6.5. Different brightness levels for the RGB-LED colors	61
6.6. A simple watch dog: A simple watchdog: the built-in RGB-LED is switched on for 1 second every 30 seconds.	62
7.1. Defining the built-in button's pin as an input.	66
7.2. Read the built-in button's state	66
7.3. Simple sketch to test the push button and the built-in LED	66
7.4. Simple sketch connects the push button with an interrupt.	68
8.1. Example code for the sensor LPS22HB on the Arduino Nano 33 BLE Sense	74
8.2. Simple sketch calibrating the sensor LPS22HB	76
8.3. Sketch for the Arduino Nano 33 BLE Sense to switch the sensor LPS22HB into sleep mode	78
9.1. Simple sketch using the sensor APDS9960 for colors	86
9.2. Simple sketch using the sensor APDS9960 for measuring the proximity	88
9.3. Simple sketch using the sensor APDS9960 for gesture detection	89
9.4. APDS-9960: Example Calibration	92
9.5. Simple sketch using the sensor APDS9960	96
14.1. Application of the sensor APDS9960: Setup	112
14.2. Application of the sensor APDS9960: Loop	113
14.3. Application of the sensor APDS9960: Main function	114

15.1. Simple sketch to control the built-in LED	144
15.2. Einlesen der Daten	145
15.3. Messungssteuerung	146
15.4. Überwachung	146
15.5. Initialisierung der Datenverarbeitung	147
15.6. Umrechnung des Mikrofonsignals	147
15.7. Aktualisierung des Bildschirms	148
15.8. PDM Microphone Example Code	150
17.1. Example Microphone	177
17.2. Example IMU (Accelerometer and Gyroscope)	177
17.3. Example ADPS-9960	178
20.1. Simple sketch using the sensor LSM9DS1 detection	193
23.1. Beispiel-Sketch zum Testen der Batterie	211
26.1. Arduino BLE Tutorial Battery Level Indicator Code	227
30.1. Testprogramm für den Encoder	256
41.1. „Hello World“ in Python – Variant 1	306
42.1. „Hello World“ in Python – Variant 1	308

Acronyms

AOP	Acoustic Overload Point
CNC	Computerized Numerical Control
GND	Ground
GPIO	General Purpose Input Output
I²C	Inter-Integrated Circuit
IDE	Integrated Development Environment
IMU	Inertial Measurement Unit
IoT	Internet of Things
JST	Japan Solderless Terminal
LED	Light Emitting Diode
mcd	Millicandela
PCB	Printed Circuit Board
PDM	Pulse Density Modulation
PWM	Pulse with Modulation
SCL	Serial Clock
SDA	Serial Data
SPI	Serial Peripheral Interface
SPS	Speicherprogrammierbare Steuerung
UART	Universal Asynchronous Receiver Transmitter
VCC	Voltage at the Common Collector

1. Introduction

1.1. Introduction

Bézier curves are parameter curves that can be used to represent free-form curves. They are named after the French engineer Pierre Bézier, who developed them in the 1960s at Renault to design car body shapes. During the same period, French physicist and mathematician Paul de Casteljau developed these curves independently of Pierre Bézier at Citroën. Paul de Casteljau's results were available earlier, but they were not published. This is the reason why Pierre Bézier is the namesake of these parameter curves.[Far02]

The Bézier curves are the basis for computer-aided design of models. This is referred to either as Computer Aided Design (CAD) or, when the emphasis is more on a mathematical view, Computer Aided Geometric Design (CAGD). [BN11] Computer requires a mathematical description of shapes to represent them. The most suitable description method for this is the use of parametric curves and surfaces. Here Bézier curves play the central role, because they are the most numerically stable polynomial bases used in CAD/CAGD software.[Far02]

Typography on the computer also uses Bézier curves. There are two ways of representing type with the computer. The simplest way is to save each individual letter with a fixed resolution and size as a bitmap and copy it to the memory area of the screen as needed. These so-called bitmap fonts can be displayed quickly but require a lot of memory if the characters are to be available in different sizes. In this case, an extra bitmap must be created for each size. The quality also decreases when these bitmap fonts are scaled in size and resolution. Vector fonts are an alternative. Their name is derived from the fact that they use curves defined in a two-dimensional vector space to represent the characters. The advantage here is that the characters displayed in this way can be scaled without loss of quality. Two standards have developed for vector fonts. One is the TrueType font and the other is the PostScript font. The TrueType fonts use square Bézier curves while the PostScript fonts use cubic Bézier curves. The cubic Bézier curves have more control points which leads to a better quality.

Another field of application is the control of machine tools. When moving to a corner, the axis must be braked to a standstill at the corner point and then accelerated again after direction correction. This procedure ensures that it is not possible to move at a constant speed, which has negative consequences for the cycle time and quality. A possible solution to this problem is to place a curve between the two sections instead of a sharp corner, which can be run at a constant speed. Bézier curves are suitable for this purpose because only two points and two tangents are needed to form them. In the case of a corner, the points as well as the tangents would lie on the sections that form the corner. The resulting error would be analytically controllable and would allow an adjustment of the curve tolerance.[SS14]

1.2. Challenges and Solutions

1.3. Structure

Part I.

Arduino Nano 33 BLE Sense - Onboard Sensoren

2. Arduino Nano 33 BLE Sense

Arduino is an open-source electronics platform based on flexible, easy-to-use hardware and software. It provides us on-board microcontroller and microprocessor kits for making digital and analogue devices. The microcontroller, often called a tiny computer embedded on the Arduino board, normally in order to run these microcontrollers we need some type of electronics e.g.; diodes, resistors, capacitors, and transistors for making the voltage and current balancing. But the Arduino team makes a user-friendly environment for getting rid of these electronics complications to run the hardware on software, just power the board as per the required voltage, write the desired program and upload it in a few seconds, it will bring everything on the board and make us independent from any worry about the electronics complications. By the technological advancement in semiconductor and electronics industry, the control problems are now being solved by using these small size microcontrollers instead of mechanical and electrical switches. All Arduino boards have one thing in common which is a microcontroller, it is basically a really small computer, which helps us to make an edge computing application.[Ard21]

WS:Advertisement!
Rewrite more as an
engineer!

2.1. Arduino Nano 33 BLE Sense

The Arduino Nano Family is a set of boards with a tiny footprint, packed with features. It ranges from the inexpensive, entry model Nano, to the more feature-packed Nano BLE Sense / Nano RP2040 Connect which comprises of Bluetooth / Wi-Fi radio modules. These boards also have a set of embedded sensors, such as temperature, humidity, pressure, gesture, microphone and more. They can also be programmed with MicroPython and supports Machine Learning. [Raj19].

Arduino Nano 33 BLE Sense is one type of Arduino family board, which is come up with Bluetooth Low Energy (BLE) capability for communication and set of sensors. This compact and reliable Nano board is built around the NINA B306 module for BLE and Bluetooth 5.0 communication; Bluetooth 5.0 is the latest version of the Bluetooth wireless communication standard. Use of microcontrollers has become inevitable in almost every field of engineering. The Arduino Nano 33 BLE Sense module is based on Nordic nRF52480 processor that contains a powerful Cortex M4F and the board has a rich set of sensors that allow the creation of innovative and highly interactive designs. Its reduced power consumption, compared to other same size boards, together with the Nano form factor opens up a wide range of applications.

The model name Arduino Nano 33 BLE Sense, itself puts out some important information. It is called “Nano” because of its compact nano size. “33” is included in the model name to indicate that the board operates on 3.3V. Then the name “BLE” indicates that the module supports Bluetooth Low Energy and the name “Sense” indicates that it has on-board sensors like accelerometer, gyroscope, magnetometer, temperature and humidity sensor, Pressure sensor, Proximity sensor, Colour sensor, Gesture sensor, and even a built-in microphone. [Raj19].

Also, for making the RGB color and person detection, we need to make the interface of BLE 33 Sense with camera shield Arducam OV2640. The Arducam OV2640 camera use to detect the RGB, object detection and also the gesture too. Arduino Nano 33 BLE Sense and camera shield Arducam is a perfect match for making ML and AI application, by having the set of sensors on board we just need to install the respective

WS:Advertisement!
Rewrite more as an
engineer!

library on Arduino board and it supports the functionality of sensors and Machine learning application.

The following figure 2.1 shows a Arduino Nano 33 BLE Sense Revision 2, shows how compact it is and small in size.

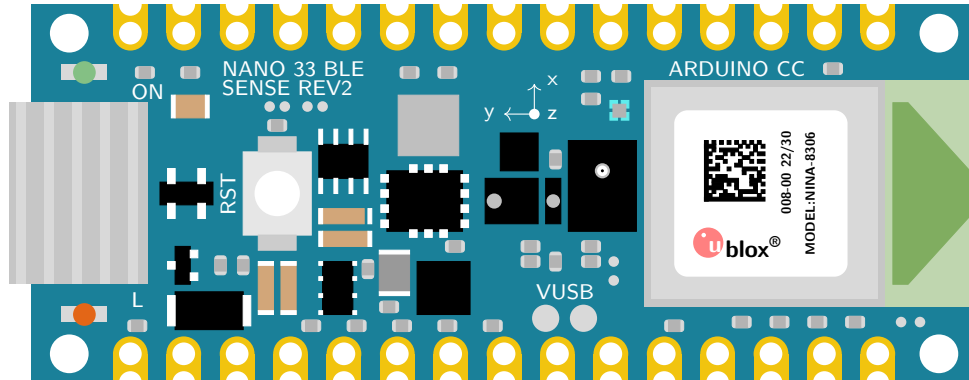


Figure 2.1.: Arduino Nano 33 BLE Sense Rev. 2, see Arduino Store

The BLE (Bluetooth Low Energy) compact and reliable Nano board are built on NINA B306 module for BLE and Bluetooth 5 communication; the NINA B306 module based on Nordic nRF52480 processor that contains a powerful Cortex M4F CPU, Its architecture fully compatible with Arduino IDE Online and Offline. The Arduino Nano BLE 33 Sense have a following set of sensors on board, ADPS-9960, LPS22HB, HTS221, LSM9DS1, and MP34DT05-A. It is small in size, having all the required sensor on board. [Ard21]

The Arduino Nano 33 BLE Sense have the following set of Sensors, BLE module and its functionality below.

- The Bluetooth is managed by a NINA B306 module.
- The ADPS-9960 is a digital proximity, ambient light, RGB and gesture sensor.
- The LSM9DS1 is a system-in-package featuring a 3D digital linear acceleration sensor, a 3D digital angular rate sensor, and a 3D digital magnetic sensor.
- The LPS22HB reads barometric pressure and environmental temperature.
- The HTS221 senses relative humidity.
- The MP34DT05 is support the sound detection.

2.2. Unterschied zwischen dem Arduino Nano 33 BLE Sense Rev1, dem Arduino Nano 33 BLE Sense Rev2 und dem Arduino Nano 33 BLE Sense Lite

2.2.1. What is the difference between Rev1 and Rev2?

The differences between the Arduino Nano 33 BLE Sense and the Arduino Nano 33 BLE Sense Rev2 concern the IMU, the temperature and humidity sensor, the microphone, the crypto chip and the voltage converter. In the second revision, the LSM9DS1 IMU has been replaced by two modules: the BMI270 acceleration and angular rate sensor and the BMI150 magnetometer. The HTS221 humidity sensor has been replaced by the HS3003, with the latter promising greater accuracy. The values

of the microphones have remained the same despite the change from the MP34DT05 to the MP34DT06JTR. Similarly, only the component designations of the voltage converters differ. The first generation uses the MPM3610, the Rev2 uses the MP2322. However, the crypto chip is no longer present in the Rev2.

The changes are summarized below:

- Replacement of the IMU of LSM9DS1 (9 axes) with a combination of two IMUs (BMI270 - 6 axes IMU and BMM150 - 3 axes IMU).
- Replacement of the temperature and humidity sensor from HTS221 to HS3003.
- Replacing the microphone from MP34DT05 to MP34DT06JTR.

In addition, some components and changes have been made to increase user-friendliness:

- Replacing the power supply from MPM3610 to MP2322.
- Add a VUSB solder pad to the top of the board.
- New test points for USB, SWDIO and SWCLK.

The figure 2.2 presents the layout of the first version of the Arduino Nano 33 BLE,

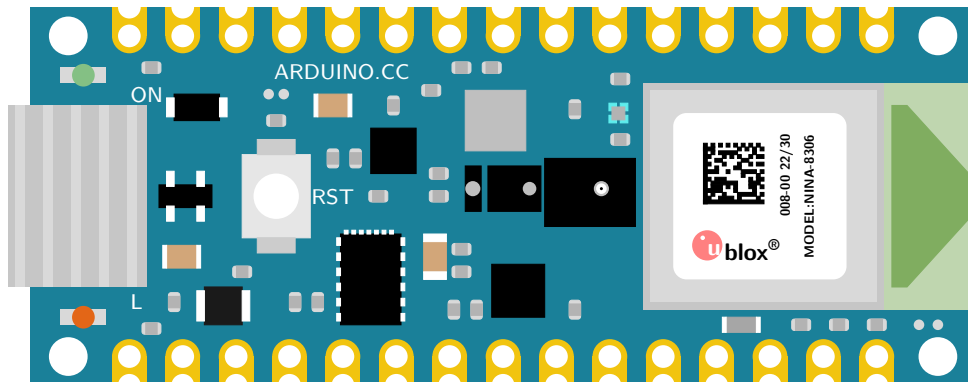


Figure 2.2.: Arduino Nano 33 BLE Sense Rev. 1

2.2.2. Changes in the Sketch

For sketches that were created with libraries such as LSM9DS1 for the IMU or HTS221 for the temperature and humidity sensor, these libraries must be changed to the following for the new revision:

- `Arduino_BMI270_BMM150` for the new combined IMU, and
- `Arduino_HS300x` for the new temperature and humidity sensor.

Rev 1 [TensorFlow Lite lib](#)

2.2.3. Arduino Nano 33 BLE Sense Lite

WS:citation The Tiny Machine Learning Kit contains only the Arduino Nano 33 BLE Sense Lite.

The Arduino Nano 33 BLE Sense Lite differs only slightly from the Arduino Nano 33 BLE Sense Revision 1. The only difference between the two models is that the Lite version does not have the HTS221, the sensor for relative humidity and temperature. The LPS22HB sensor, which is on the board, is a pressure sensor, but it can also be used to measure the temperature. It is therefore not possible to measure humidity with the Arduino Nano 33 BLE Lite. To measure relative humidity, a separate sensor must be connected. The reason for this decision is that the Arduino company has difficulties maintaining the stock. [FD22]

The figure 2.3 presents the layout of the Arduino Nano 33 BLE Lite,

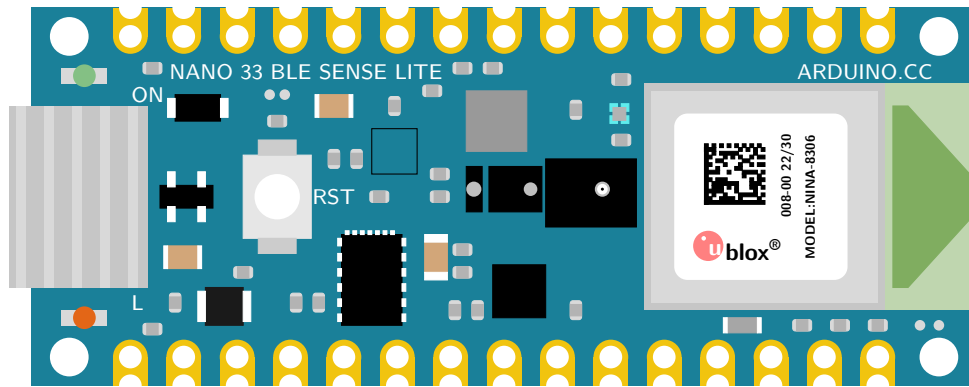


Figure 2.3.: Arduino Nano 33 BLE Sense Lite; the empty black square marks the position of the missing sensor HTS221

2.3. On-Board Sensor Description

Arduino Nano 33 BLE sense come up with the set of embed sensor on the board. The available embed sensors are commonly use for measuring both the analog and digital values around the surrounding. Arduino Nano 33 BLE sense is very small 45mm × 18mm in size, which makes it very useful for Internet of things (IOT) and Artificial intelligence (AI) application as a embed device where space is the main constrained issue. It is low power consumption board and operate normally on 3.3 V, we can say that this small size low power consumption board can operate on small batteries even for many months. Due to on-board available sensor, the low power consumption and mini architecture we can use this nano board anywhere. The Arduino Nano 33 BLE Sense is a completely new board on a well-known form factor. For getting detail information about each component of Arduino Nano 33 BLE Sense and data sheets of each sensor the following links give us a detail information [Ard21]. The short description of each sensor are as follow.

- The ADPS-9960 is a digital proximity, ambient light, RGB and gesture sensor. it can measure the proximity distance, light, color and gestures when moving close with the board.
- The sensor LSM9DS1 is a 9 axis Inertial Measurement Unit (IMU) use as a accelerometer, gyroscope, and magnetometer, this 9 axis sensor is ideal for wearable devices.

- The sensor LPS22HB is a barometric pressure sensor, it measures the environmental pressure which is useful for simple weather station monitoring.
- The sensor HTS221 senses the relative humidity, and temperature, to get highly accurate measurements of the environmental conditions.
- The sensor MP34DT05 is the digital microphone. it is useful for capturing, analyzing and detecting the sound in real time.
- The USB port allows you to connect Arduino Nano 33 BLE sense to your machine.
- There are 3 different LEDs that can be accessed on the Nano BLE Sense: RGB Programmable LED, the built-in orange Programmable LED and the Power LED.

The figure 2.4 show the embed sensors on the board, with powerful processor as compared to other arduino boards the nRF52840 from Nordic Semiconductors, a 32-bit ARM[®] Cortex[™]-M4 CPU processor running at 64 MHz are as follow.

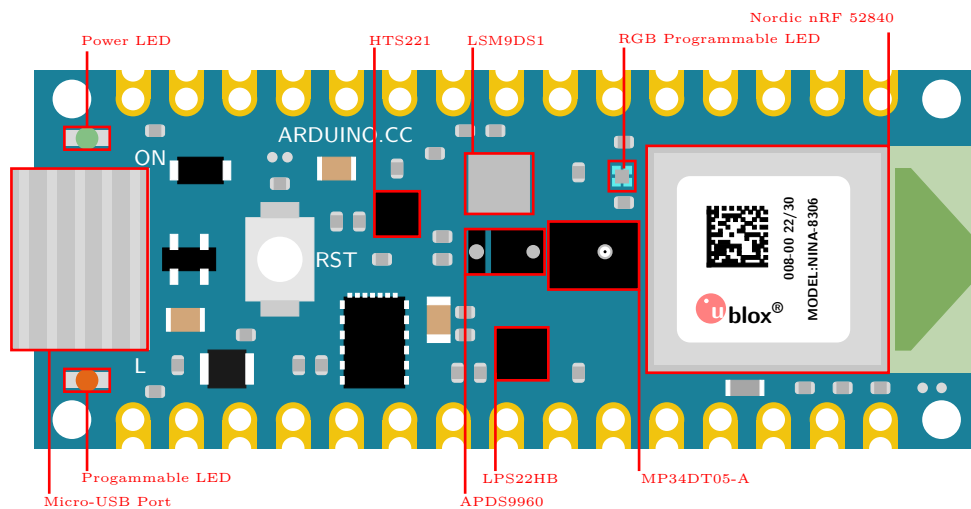


Figure 2.4.

Another point to bear in mind is the overall 'trueness' of sensor readings based on where do you place the actual sensors in the environment, as this could be quite critical. The operating temperature should not exceed 85⁰C and not lower than -40⁰C. Other factors like humidity level and air pressure values should also be kept in check.

WS:Auch mit dem Rev

MICROCONTROLLER	nRF52840
OPERATING VOLTAGE	3.3V
INPUT VOLTAGE (LIMIT)	21V
DC CURRENT PER I/O PIN	15 mA
CLOCK SPEED	64MHz
CPU FLASH MEMORY	1MB
SRAM	256KB
LED_BUILTIN	13
IMU (Accelerometer, Gyroscope, Magnetometer)	LSM9DS1
MICROPHONE	MP34DT05
GESTURE, LIGHT, PROXIMITY, COLOUR	APDS9960
BAROMETRIC PRESSURE	LPS22HB
TEMPERATURE, HUMIDITY	HTS221

Table 2.1.: Technical Specifications of Arduino Nano 33 BLE Sense [Ard21]

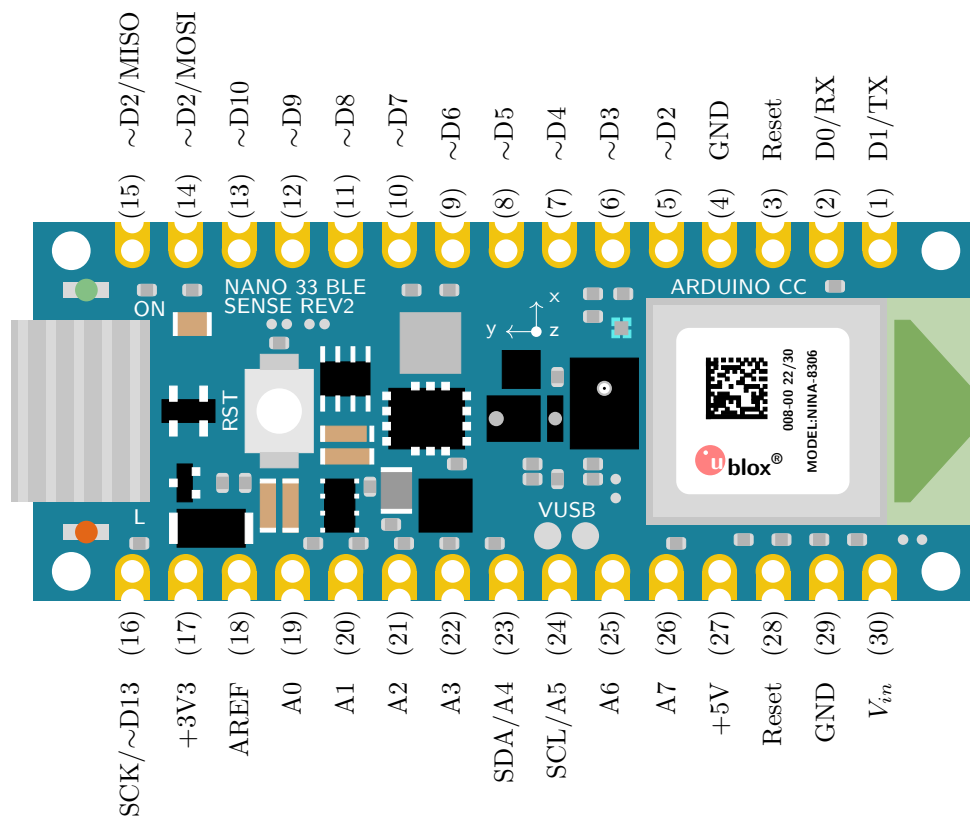


Figure 2.5.: Pin assignment of the Arduino Nano 33 BLE Sense and for the Arduino Nano 33 BLE Sense Rev 2; note that the orange built-in LED is connected to pin D13 and the power LED to pin D1. The built-in RGB LED occupies pins D18, D19 and D20.

WS:Beide Diagramme

2.3.1. Gesture, Proximity, and Color Detection Sensor ADPS-9960

The APDS-9960 device features advanced Gesture detection, Proximity detection, Digital Ambient Light Sense (ALS) and Color Sense (RGB). [Ard21] Gesture detection

utilizes four directional photodiodes to sense reflected IR energy (sourced by the integrated LED) to convert physical motion information (i.e. velocity, direction and distance) to a digital information.

For the following Applications the sensor is in use:

- Gesture Detection
- Color Sense
- Ambient Light Sensing
- Proximity Sensing

2.3.2. Accelerometer, Gyroscope, and Magnetometre Sensor LSM9DS1

The LSM9DS1 is a system-in-package featuring a 3D digital linear acceleration sensor, a 3D digital angular rate sensor, and a 3D digital magnetic sensor. IMU's work by detecting rotational movements across the 3 axis known as Pitch, Roll and Yaw. To achieve the same, it depends on Accelerometer, Gyroscope and Magnetometer. The accelerometer gives the velocity at which the IMU module moves. The gyroscope measure the rotational movement rate on the IMU. Magnetometer measures the force of gravity acting on the IMU.

For the following Applications the IMU is in use:

- Indoor navigation
- Advanced gesture recognition
- Gaming and virtual reality input devices
- Display/map orientation and browsing
- Consumer electronics; Smartphones, tablets, fitness trackers for motion sensing and orientation.
- Compact transportation solutions like Segway.
- Sports Technology - helping athletes to know how they can improve their movements.

There are also certain disadvantages of using the IMU and points that need to be kept in mind while using an IMU sensor. Accumulated error or '*Drift*' is the main disadvantage of IMUs, present due to its constant measuring of changes and rounding off its calculated values off. When such a process happens for a prolonged period of time, it can lead to significant errors. The best way to avoid the *drift* factor is to use a good quality IMU Sensor and make sure the IMU sensor is calibrated. [Stmb]

WS:The explanation of drift is too small; citations!

Calibration of an IMU

On research it was found that there are various methods to calibrate the sensors involved, the time period between each calibration is also not defined specifically, however it is advised that regular calibration is done especially when there are strange outputs noticed. Few methods of calibration are briefed below:

WS:What is calibration citations!

Low and High Limit Method

In this method the sensor is rotated in circles along each axis a few times. The midpoint is then found between the two extremes. If there is no offset, the midpoint is close to zero, but if there is a slight deviation from zero, this figure is the hard iron offset, which is the result of the distortion caused by the Earth's magnetic field. This method is mainly used to calibrate the Magnetometer [Mal15]

Magneto V1.2

In this method, the raw magnetometer data is pre-processed with axis specific gain correction to convert the raw output into nanoTesla:

```
Xm_nanoTesla = rawCompass.m.x*(100000.0/1100.0);
Gain X [LSB/Gauss] for selected input field range
Ym_nanoTesla = rawCompass.m.y*(100000.0/1100.0);
Zm_nanoTesla = rawCompass.m.z*(100000.0/980.0);
```

This converted data is saved into the file `Mag_raw.txt` that you open with the Magneto program. To start using this method, we first need to replace the (100000.0/1100.0) scaling factors with values that convert your specific sensors output into nanoTesla. Rather than simply finding an offset and scale factor for each axis, Magneto creates twelve different calibration values that correct for a whole set of errors: bias, hard iron, scale factor, soft iron and misalignment.

A side benefit of this is that it can be used to calibrate accelerometers as well. You might again need to pre-process your specific raw accelerometer output, taking into account the bit depth and G sensitivity, to convert the data into milliGalileo. Then enter a value of 1000 milliGalileo as the “norm” for the gravitational field. [Mal15]

VS:Here, we need more information because we want to use it:

- General information about imu
- angle to rotation matrix with example!
- calibration
- drift
- ...
- specials for this imu
- description of the parameters for coding
- example code, see Code/- Nano33BLESense/IMU/Arduino

2.3.3. Pressure Sensor LPS22HB

The LPS22HB is an ultra-compact piezoresistive absolute pressure sensor which functions as a digital output barometer.

For the following Applications the sensor is in use:

- Altimeters and barometers for portable devices
- Weather station equipment
- Sports watches

A sensor element is installed as well as an IC interface that communicates via an I²C or SPI bus. The function is given in a temperature range from -40°C to $+85^{\circ}\text{C}$. [STM17]

- Absolutdruckbereich: 260 bis 1260 hPa
- Versorgungsspannung: 1,7 bis 3,6 Volt
- 24-bit Druckdatenausgabe
- 16-bit Temperaturdatenausgabe

2.3.4. Relative Humidity and Temperature Sensor HTS221

The HTS221 is an ultra-compact sensor for relative humidity and temperature. It includes a sensing element and a mixed signal to provide the measurement information through digital serial interfaces.

For the following Applications the sensor is in use:

- Air conditioning, heating and ventilation
- Air humidifiers
- Refrigerators
- Smart home automation
- Industrial automation

Dieser Sensor kommuniziert über den I²C- und SPI-Bus. Dieser Sensor ist einsetzbar in einem Temperaturbereich von -40°C bis $+120^{\circ}\text{C}$. Zur Spannungsversorgung werden 1,7 bis 3,3 Volt benötigt. Eine Temperaturmessung erfolgt mit einer Genauigkeit von $\pm 5^{\circ}\text{C}$. [STM23]

- GND - Ground
- DRDY - Data ready output signal
- SCL/SPC - I2C serial clock (SCL) & SPI serial port clock (SPC)
- VDD - Stromversorgung
- SDA/SDI/SDO - I²C serial Data (SDA) & 3 wire-SPI serial data input/output (SDI/SDO)
- SPI enable - I²C/SPI mode selection

2.3.5. Digital Microphone MP34DT05-A

The MP34DT05-A is an ultra-compact, low-power, omni directional, digital microphone built with a capacitive sensing element and an IC interface. The sensing element, capable of detecting acoustic waves, is manufactured using a specialized silicon micromachining process dedicated to producing audio sensors. The MP34DT05 is a low-distortion microphone with a signal-to-noise ratio of 64 dB. The sensitivity is $-26 \text{ dBFS} \pm 3 \text{ dB}$. The Acoustic Overload Point (AOP) is 122.5 dB SPL.

The output signal of the microphone is a PDM signal. This signal is a binary signal modulated by Pulse Density Modulation (PDM) from the analogue signal. [Stmc]

For the following Applications the sensor is in use:

- Speech recognition
- Portable media player
- Mobile Terminal

The Arduino nano 33 BLE Sense has a built-in microphone that uses PDM (Pulse-Density Modulation) to convert sound into digital data. You can use the PDM library to access the microphone data and perform various tasks with it. Here is a simple example of using the builtin microphone for an Arduino nano 33 BLE Sense:

The code 2.1 will print the sound level of the microphone to the serial monitor every 100 milliseconds.

```

1000     // Include the PDM library
1001     #include <PDM.h>
1002
1003     // Buffer to store the microphone data
1004     short sampleBuffer[256];
1005
1006     // Variable to store the sound level
1007     int soundLevel = 0;
1008
1009     // Callback function for PDM data
1010     void onPDMdata() {
1011         // Read the PDM data
1012         int bytesAvailable = PDM.available();
1013         PDM.read(sampleBuffer, bytesAvailable);
1014
1015         // Calculate the sound level
1016         soundLevel = 0;
1017         for (int i = 0; i < bytesAvailable / 2; i++) {
1018             soundLevel += abs(sampleBuffer[i]);
1019         }
1020         soundLevel /= bytesAvailable / 2;
1021     }
1022
1023     void setup() {
1024         // Initialize serial communication
1025         Serial.begin(9600);
1026         while (!Serial);
1027
1028         // Initialize PDM with a sample rate of 16 kHz and 16-bit
1029         resolution
1030         PDM.begin(1, 16000);
1031         PDM.onReceive(onPDMdata);
1032     }
1033
1034     void loop() {
1035         // Print the sound level to the serial monitor
1036         Serial.println(soundLevel);
1037         delay(100);
1038     }

```

Listing 2.1.: Simple example using of the builtin microphone of the Arduino Nano 33 BLE Sense

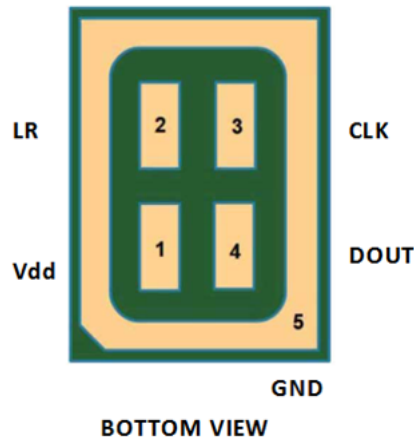


Table 1. Pin description

Pin #	Pin name	Function
1	Vdd	Power supply
2	LR	Left/Right channel selection
3	CLK	Synchronization input clock
4	DOUT	Left/Right PDM data output
5 (ground ring)	GND	Ground

Figure 2.6.: Circuit diagram microphone [Stmc]

You can modify this code to perform other tasks with the microphone data, such as controlling LEDs, playing sounds, or sending data to other devices. For more information and examples, you can check out the PDM library documentation or the Arduino nano 33 BLE Sense page.

WS:How to install the library PDM
description of the lib
functions

2.3.6. Bluetooth Module nRF52840

The nRF52840 is an advanced, highly flexible single chip solution for today's increasingly demanding Ultra Low Power (ULP) wireless applications for connected devices on our person, connected living environments and the Internet of Things (IoT) at large. It is designed ready for the major feature advancements of Bluetooth 5 and takes advantage of Bluetooth 5's increased performance capabilities. [Ard21]

Applications

- Smart Home products
- Industrial mesh networks
- Smart city infrastructure
- Connected watches
- Advanced personal fitness devices
- Wearables with wireless payment
- Connected Health

2.4. Bluetooth Module INA B306

The module is a Bluetooth Low Energy (BLE). The connection with our smartphone is possible, to do this we have to use application "Nordic Semiconductors nRF Connect" They used 5.0 generation bluetooth. 2.4



Figure 2.7.: Connection with BLE and smartphone

We can use for example to share Arduino batterie on smartphone, 2.8 here you see an code example for how we can do this.

Figure 2.8.: Simple sketch to seen Arduino battery

```

1000 #include <ArduinoBLE.h>
1002 BLEService batteryService("1101");
1003 BLEUnsignedCharCharacteristic batteryLevelChar("2101"
1004 BLERead | BLENotify);
1006 void setup() {
1007     Serial.begin(9600);
1008     while (!Serial);
1010     pinMode(LED_BUILTIN, OUTPUT);
1011     if (!BLE.begin()) { //Search connection but they wasn't function
1012         Serial.println("Starting BLE failed!");
1013         while (1);
1014     }
1016     BLE.setLocalName("BatteryMonitor");
1017     BLE.setAdvertisedService(batteryService);
1018     batteryService.addCharacteristic(batteryLevelChar);
1019     BLE.addService(batteryService);
1020     BLE.advertise();
1021     Serial.println("Bluetooth device active, waiting for connections...");
1022 }
1024 void loop() {
1025     BLEDevice central = BLE.central();
1026     if (central) {
1027         Serial.print("Connected to central: ");
1028         Serial.println(central.address());
1029         digitalWrite(LED_BUILTIN, HIGH);
1030         while (central.connected()) {
1031             int battery = analogRead(A0);
1032             int batteryLevel = map(battery, 0, 1023, 0, 100);
1033             Serial.print("Battery Level is now: ");
1034             Serial.println(batteryLevel);
1035             batteryLevelChar.writeValue(batteryLevel);
1036             delay(200);
1037         }
1038         digitalWrite(LED_BUILTIN, LOW);
1039         Serial.print("Disconnected from central: ");
1040         Serial.println(central.address());
1041     }
1042 }

```

../Code/Nano33BLESense/Bluetooth/bluetooth.ino

2.5. Arduino Nano 33 BLE Pin Configuration

Arduino Nano 33 BLE is an advanced version of Arduino Nano board that is based on a powerful processor the nRF52840. The figure 2.9 shows that the board has the following pin configuration. Pin Configuration

Digital pin The number of digital I/O pins are 14 which receive only two values HIGH or LOW. These pins can either be used as an input or output based on the requirement. When these pins receive 5V, they are in a HIGH state and when they receive 0V they are in a LOW state.

Analog pin Total 8 analog pins available on the board A0 – A7. These pins get any value as opposed to digital pins that only receive two values HIGH or LOW. These pins are used to measure the analog voltage ranging between 0 to 5V.

PWM pin All digital pins can be used as PWM pins. These pins generate analog results with digital means.

SPI pin The board supports serial peripheral interface (SPI) communication protocol. This protocol is employed to develop communication between a controller and other peripheral devices like shift registers and sensors. Two pins are used for SPI communication i.e. Master Input Slave Output (MISO) and Master Output Slave Input (MOSI) are used for SPI communication. These pins are used to send or receive data by the controller.

I2C pin The board carries the I2C communication protocol which is a two-wire protocol. It comes with two pins SDA and SCL.

UART pin The board features a UART communication protocol that is used for serial communication and carries two pins Rx and Tx. The Rx is a receiving pin used to receive the serial data while Tx is a transmission pin used to transmit the serial data.

External Interrupts pin All digital pins can be used as external interrupts. This feature is used in case of emergency to interrupt the main running program with the inclusion of important instructions at that point.

LED at Pin 13 and AREF pin There is an LED connected to pin 13 of the board. And AREF is a pin used as a reference voltage for the input voltage.

2.6. Was fehlt

- Beschreibung der Sensoren
 - Hintergrundwissen
 - Beschreibung
 - Eigenschaften
 - Kalibrierung
- Beschreibung der Bibliothek
 - Beschreibung
 - Installation
 - Funktionen

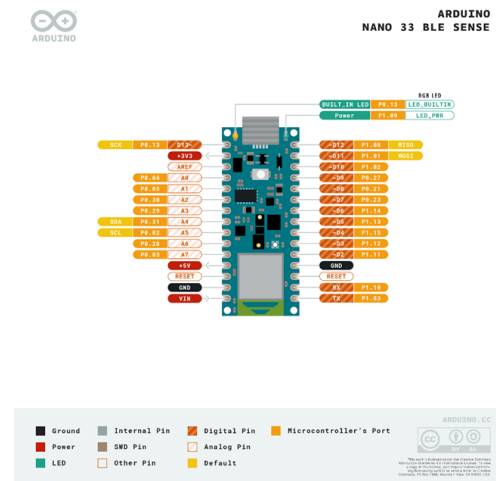


Figure 2.9.: Arduino Nano 33 BLE Pin Configuration

- Laufzeit, Speicherplatz, ...
- Software-Dokumentation
- Verwendung von Werkzeugen, z.B. Doxygen
 - Beschreibung, inklusive im Quellcode; z.B. Header
 - Handbuch
 - Ablaufdiagramm
 - ...
- Interpretation der Ergebnisse
- Literatur, Bilder
- ...

3. Reset Button on the Arduino Nano 33 BLE Sense

The reset button on the Arduino Nano 33 BLE Sense plays a crucial role in managing the board's operation. It allows for restarting the board, entering Bootloader Mode, and addressing issues related to the program or functionality. The following section outlines its functions, appropriate use cases, and technical specifications.

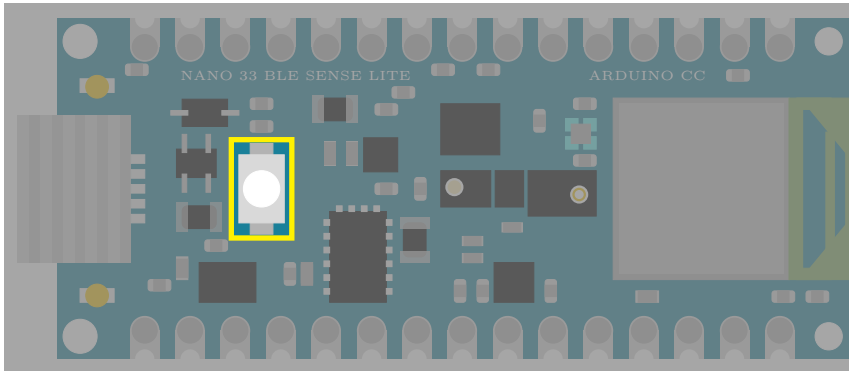


Figure 3.1.: Arduino Nano 33 BLE Sense's Reset Button

3.1. Purpose and Functionality

The reset button on the Arduino Nano 33 BLE Sense is positioned on the top side of the board, close to the USB connector. This small tactile push button communicates with the nRF52840 microcontroller, performing specific actions depending on the timing and sequence of the presses.

3.1.1. Functions of the Reset Button

- **System Reset:** A single press of the reset button triggers a system reset. This action momentarily interrupts the power to the microcontroller, restarting the board and reinitializing the uploaded program.
- **Bootloader Mode:** The reset button also allows entry into Bootloader Mode, which is essential for uploading new sketches or recovering the board when it becomes unresponsive due to a faulty program.
 - **Activation:** To activate Bootloader Mode, press the reset button twice in quick succession (within approximately 1 second). The double-press sequence triggers the bootloader, which prepares the board to receive new firmware or sketches via the USB interface.
 - **Indication:** When Bootloader Mode is active, the built-in LED blinks quickly, indicating that the board is ready for a firmware upload.
 - **Purpose:** Bootloader Mode is particularly helpful for uploading sketches when the Arduino IDE cannot recognize the board and for recovering from unresponsive or faulty programs that disrupt normal functionality.

-

3.2. Timing and Sequence

The reset button's behavior depends on the timing and sequence of presses:

- Single Press: Executes a system reset, restarting the microcontroller and the loaded sketch.
- Double Press: Activates Bootloader Mode, readying the board for new firmware uploads.
 - The timing of the double-press must be precise (quickly within 1 second); otherwise, the board will perform only a standard reset.

3.3. Use Cases

The reset button provides critical functionality in several scenarios:

1. Restarting the Board: During development, a single press allows developers to restart the board and observe the program from its initial state.
2. Uploading Sketches: When the Arduino IDE fails to detect the board or cannot upload a sketch, entering Bootloader Mode manually resolves the issue.
3. Recovering from Errors: If a faulty sketch causes the board to freeze or become unresponsive, Bootloader Mode enables users to bypass the issue and upload a new program.

3.4. Conclusion

The reset button on the Arduino Nano 33 BLE Sense is a vital feature for developers, providing tools to restart the board, enter Bootloader Mode, and recover from programming errors. Its dual functionality—system reset and Bootloader Mode activation—ensures flexibility and reliability during development and debugging. The requirement for a precise double-press to activate Bootloader Mode enhances user control, making it a robust tool for managing the board.

4. Built-inLED

Some Arduino boards have a LED on board which can use for the application. [Ard24a; Ard23a; Ard23b]

4.1. General Information

LEDs are used in the applications. Depending on the action performed by a user or the sketch, corresponding feedback can be provided. This can take the form of the LED switching on, switching off or flashing. [Kur21]

4.2. Built-in LED

The built-in LED of the Arduino Nano 33 BLE Sense is a single orange LED that is connected to pin 13 on the board. The built-in LED is active-high, which means that setting the pin to **HIGH** will turn the LED on, and setting the pin to **LOW** will turn the LED off. The built-in LED can be used for various purposes, such as indicating the status of the board, displaying sensor data, creating visual effects, or debugging. [Ard23a; Ard23b; Arda]

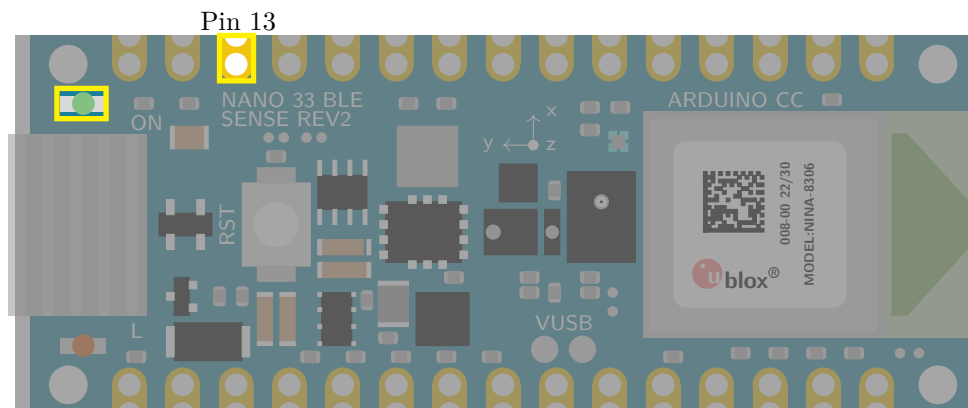


Figure 4.1.: Arduino Nano 33 BLE Sense's built-in LED with Pin 13

4.3. Specification

4.3.1. Pin Assignment

The built-in LED is a orange LED and connected to pin 13. The brightness can not be controlled. [Ard23a; Ard23b]

Built-in LED: `LED_BUILTIN = 13u`

Built-in LED is active-high and connected to pin 13.

The built-in LED can be controlled programmatically by setting the pin to **HIGH** or **LOW**.

The pin 13 must be defined as an output in the function `setup` by setting `pinMode(13, OUTPUT)`, otherwise the LED cannot be switched on.

The pin 13 can also be used otherwise. Then the LED is not in use.

4.3.2. Power Consumption

The power consumption has to be considered. It is the same value as for the power LED, see section 5.3.2.

4.4. Simple Code

4.4.1. Simple Code for the Built-in LED

For using the built-in LED, a simple library is introduced here.

In the header file `BuiltinLED.h`, see code ??, are the declaration of the variables and the functions. A variable is connected to pin 13. The pin 13 is defined as an output in the function `BuiltinLEDinit`. There are 2 functions:

1. `BuiltinLEDinit`: This function initializes the digital pin 13 of the built-in LED as an output.
2. `BuiltinLED`: This function switches the built-in LED on or off.

Listing 4.1.: Header file without comments for using the Built-in LED

```
1000 #ifndef LEDBuiltin_h
1001 #define LEDBuiltin_h
1002
1003
1004 /**
1005
1006 /**
1007
1008 #endif
```

../Code/Nano33BLESense/LEDs/LEDBuiltin.h

In the file `BuiltinLED.cpp`, see code 4.2, the functions are implemented.

Listing 4.2.: Code file without comments for using the Built-in LED

```
1000 #include <LEDGeneral.h>
1001 #include <LEDBuiltin.h>
1002 * @return 0 - success
1003 * @return 1 - error
1004 */
1005 int LEDBuiltinsetup()
1006 {
1007     // initialize digital pin LED_BUILTIN as an output.
1008     LEDSetup(LED_BUILTIN);
1009 *
1010 * @return 0 - success
1011 * @return 1 - error
1012 */
1013 int LEDBuiltin(bool On)
1014 {
```

../Code/Nano33BLESense/LEDs/LEDBuiltin.cpp

4.4.2. Simple Code for Testing the Built-in LED

In the sketch 15.1, a variable is connected to pin 13. The pin is defined as an output in the function `setup`. In the function `loop`, the LED is switched on for 1 second and switched off for 1 second so that the LED flashes accordingly.

Listing 4.3.: Simple sketch without comments to control the built-in LED

```

1000 * Turns the built-in LED on for one second, then off for one second,
      * repeatedly.
      *
1002 * The LED is switched on for 1 second and switched off
      *
1004 * Initialization of the pin LED_BUILTIN as output
      *
1006 * @param —
      *
1008 * swichting the led on / off for 1sec.
      *
1010 * @param —
      *
1012 * @return void
      */
1014 void loop() {
      // Turn the LED on
1016     LEDBuiltin(false);
      // Wait for one second

```

[../Code/Nano33BLESense/LEDs/examples/TestLEDBuiltin/TestLEDBuiltin.ino](#)

This is just a simple example. The variable `LED_BUILTIN` is already defined, so the assignment is not necessary. The command `delay` should be avoided in an Arduino sketch. Instead, variables of the type `elapsedMillis` should be used.

4.5. Tests

4.5.1. Simple Function Test

The simplest test is the flashing of the LED at 2 Hz, see sketch 4.4.

Listing 4.4.: Simple sketch to test the built-in LED

```

1000 * Turns the built-in LED on for one second, then off for one second,
      repeatedly.
      *
1002 * The LED is switched on for 1 second and switched off
      *
1004 * Initialization of the pin LED_BUILTIN as output
      *
1006 * @param —
      *
1008 * swichting the led on / off for 1sec.
      *
1010 * @param —
      *
1012 * @return void
      */
1014 void loop() {
1016     // Turn the LED on
      LEDBuiltin(false);
      // Wait for one second

```

[../Code/Nano33BLESense/LEDs/examples/TestLEDBuiltin/TestLEDBuiltin.ino](#)

4.5.2. Test all Functions

As the built-in does not allow any special manipulations, all functions are covered with the sketch 4.4.

4.6. Simple Application

In many applications, it is necessary to save power so that the LED is not switched on continuously. Nevertheless, feedback is required as to whether the system, e.g. a fire detector, is switched on and functional. In this case, the LED is switched on for 1 second at a defined time interval, in this case 30 seconds. This is implemented in the example 4.5.

Listing 4.5.: A simple watch dog: A simple watchdog: the built-in LED is switched on for 1 second every 30 seconds.

```

1000 #include <../LEDs/BuiltinLED.h>
1001 #include <../LEDs/LED.h>
1002 #include <../LEDs/SignsOfLife.h>
1003
1004 #define CycleTimeOn 1000 /*< Duty cycle [ms] */
1005
1006 #define CycleTimeOff 29000 /*< Switch-off time [ms] */
1007
1008 void setup() {
1009     // Initialize the function SignsOfLife
1010     SignsOfLifeInit(LED_BUILTIN, CycleTimeOn, CycleTimeOff)
1011
1012     // Application
1013     // ...
1014 }
1015
1016 void loop() {
1017     // Switch builtin LED on/off
1018     SignsOfLife();
1019
1020     // Application
1021     // ...
1022 }

```

../Code/Nano33BLESense/Test/TestLEDBuiltinApplication.ino

4.7. Further Readings

- Babu, Neelakandan Ramesh and Chenchireddy, Kalagotla and Reddy, V. Harsha Vardhan and Samhitha, Dr. and Apparao, P. and Kalyan, Ch. Pavan: *Case Study on Ni-MH Battery*. 2023 2nd International Conference on Applied Artificial Intelligence and Computing (ICAAIC), 2023. [Bab+23]
- Scherer, Thomas: *Elektor special: Stromversorgung und Batterien*. Elektor Special. 2022. [Sch22]

5. Power LED

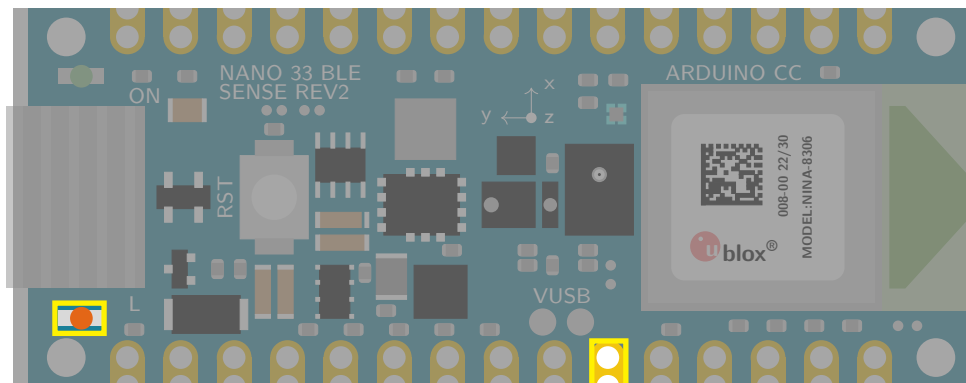
Arduino boards have a power LED on board. Normally, the power LED indicates the status of the board.

5.1. General Information

The power LED on an Arduino board is a small, usually red or green, Light Emitting Diode (LED) that indicates whether the board is receiving power. It is typically located near the USB connector on the board. When the board is powered on, the LED will illuminate. The specific color and behavior of the power LED may vary depending on the Arduino board model. For example, on the Arduino Uno, the power LED is red and illuminates steadily when the board is powered on. On the Arduino Nano, the power LED is green and blinks slowly when the board is powered on. The power LED is a helpful visual indicator that can be used to troubleshoot power supply issues. If the power LED is not illuminated, it means that the board is not receiving power. This could be due to a number of reasons, such as a loose USB connection, a damaged power supply, or a problem with the board itself. By checking the power LED, you can quickly identify and resolve power supply problems with your Arduino board. [Ard24a; Ard23a; Ard23b; Arda; Kur21]

5.2. Power LED

While labeled as a power LED, the single green LED on the Nano 33 BLE Sense isn't solely for power indication. It has multi-functionality depending on board state and user code interaction. During typical operation, it lights steadily green when powered, similar to other Arduino models. However, it flashes rapidly during serial communication and bootloader mode. Notably, when user code enters deep sleep mode, the LED turns off entirely for power saving. This includes power status, communication activity, and sleep mode activation. Understanding these LED behaviors can aid in troubleshooting and code debugging. The green power LED's primary function remains indicating power and basic board states. [Ard24a; Ard23a; Ard23b; Arda]



Pin 25

Figure 5.1.: Arduino Nano 33 BLE Sense's Power LED with Pin 25

5.3. Specification

5.3.1. Pin Assignment

The power LED is a green LED and connected to pin 25. The brightness can be controlled via Pulse with Modulation (PWM). [Ard23a; Ard23b]

Power LED: `LED_PWR = 25u`

Power LED is active-high and connected to pin 25.

The power LED can also be controlled programmatically by setting the pin to `HIGH` or `LOW`.

The pin must be defined as an output in the function `setup` by setting `pinMode(LED_PWR, OUTPUT)`, otherwise the LED cannot be switched on.

5.3.2. Power Consumption

The power consumption has to be considered. There, the power and current in a simple circuit using a resistor and an LED has to be calculated.

The led onboard works in a circuit with an 200 Ohm resistor, a 5V power source from an Arduino, and an LED with 2V across it. To find out how much power is being used by the LED and the resistor, the current must be calculated in the first step. Using Ohm's Law $V = I \cdot R$ we can find the current flowing through the resistor. Since 3V is across the resistor, we can plug in the values:

$$\frac{3V}{200\Omega} = 0.015A = 15mA$$

This means that 15mA of current is flowing through the resistor and the LED. In the next step, we calculate the power. Now that we know the current, we can calculate the power dissipated in the LED and the resistor.

- For the LED: $2V \times 15mA = 30mW$
- For the resistor: $3V \times 15mA = 45mW$
- Total Power:

To find the total power coming out of the Arduino, we multiply the voltage by the current:

$$5V \times 15mA = 75mW$$

Notice that the total power (75mW) is equal to the sum of the power dissipated in the LED (30mW) and the resistor (45mW). This makes sense, since all the power coming out of the Arduino is being used by either the LED or the resistor.

The pin 25 can also be used otherwise. Then the LED is just switched on if the board is connected to a power source.

5.4. Simple Code

5.4.1. Simple Code for LEDs

Using of LEDs is a standard problem for Arduino's programmer. Therefore, a simple library is introduced here.

In the header file `LED.h`, see code 5.1, are the declaration of the functions. There are 3 functions:

1. `LEDInit`: This function initializes a digital pin of the LED as an output.
2. `LED`: This function switches the LED on or off.
3. `LEDBrightness`: This function sets the LED's brightness.

Listing 5.1.: Header file without comments for using a LED

```

1000 #define LED_h
1002
1004 /**
1005  * @brief initialize digital pin of the LED as an output.
1006  * @brief function for switching on or off the LED
1007  *
1008  */
1009
1010 #endif

```

../Code/Nano33BLESense/LEDs/LEDGeneral.h

In the file `LED.cpp`, see code 5.2, the functions are implemented.

Listing 5.2.: Code file without comments for using a LED

```

1000 #include "LEDGeneral.h"
1001 #include <Arduino.h>
1002 *
1003 * @return 0 - success
1004 * @return 1 - error
1005 */
1006 int LEDSetup(int PinNo) {
1007     // initialize digital pin LED_BUILTIN as an output.
1008     pinMode(PinNo, OUTPUT);
1009 *
1010 * @return 0 - success
1011 * @return 1 - error
1012 */
1013 int LED(int PinNo, bool On)
1014 {
1015     if (On)
1016     {
1017         digitalWrite(PinNo, HIGH); // turn the LED on (HIGH is the voltage
1018                                     level)
1019     } else
1020     {
1021         digitalWrite(PinNo, LOW); // turn the LED off by making the
1022                                     voltage LOW
1023     }
1024 *
1025 * @return 0 - success
1026 * @return 1 - error
1027 */
1028 int LEDBrightness(int PinNo, int setBrightness)
1029 {
1030     int b;
1031
1032     if (setBrightness <= 0) {
1033         b = 0;
1034     } else
1035     if (setBrightness >= 255) {
1036         b = 255;
1037     } else
1038     {
1039         b = setBrightness;
1040     }
1041 }

```

```

1040 analogWrite(PinNo, b);
      ../../Code/Nano33BLESense/LEDs/LEDGeneral.cpp

```

5.4.2. Simple Code for the Power LED

For using the power LED, a simple library is introduced here.

In the header file `PowerLED.h`, see code 5.3, are the declaration of the variables and the functions. A variable is connected to pin 25. The pin 25 is defined as an output in the function `PowerLEDinit`. There are 2 functions:

1. `PowerLEDinit`: This function initializes a the digital pin 25 of the power LED as an output.
2. `PowerLED`: This function switches the power LED on or off.

Listing 5.3.: Header file with doxygen comments for using the Power LED

```

1000 #ifndef LEDPower_h
1001 #define LEDPower_h
1002
1003 /**
1004  *
1005  */
1006
1007 /**
1008  *
1009  */
1010 #endif
      ../../Code/Nano33BLESense/Nano33BLESenseLED/LEDPower.h

```

In the file `PowerLED.cpp`, see code 5.4, the functions are implemented.

Listing 5.4.: Code file with doxygen comments for using the Power LED

```

1000 #include <LEDGeneral.h>
1001 #include <LEDPower.h>
1002
1003 #define LED_PWR 25 /*< Define the pin for the power LED */
1004 * @return 1 - error
1005 */
1006 int LEDPowersetup()
1007 {
1008     // initialize digital pin LED_BUILTIN as an output.
1009     LEDSetup(LED_PWR);
1010 * @return 0 - success
1011 * @return 1 - error
1012 */
1013 int LEDPower(bool On)
1014 {
1015     LED(LED_PWR, On); // turn the LED on = true or off = false
      ../../Code/Nano33BLESense/Nano33BLESenseLED/LEDPower.cpp

```

5.4.3. Simple Code for Testing the Power LED

In the sketch 5.5, the power LED is tested. First, the function `PowerLEDinit` is called in the function `setup`. In the function `loop`, the power LED is switched on for 1 second and switched off for 1 second so that the power LED flashes accordingly.

Listing 5.5.: Simple sketch to control the power LED

```

1000 *
1001 */
1002 * @brief Setup function runs once when the sketch starts.
1003 *
1004 * Initializes the power LED for use in the main loop. The pin for the
1005 * LED is configured here.
1006 */
1007 void setup() {
1008     LEDPower.turnOff(); ///< Turns off the power LED
1009     delay(500);          ///< Wait for 500 milliseconds (LED is OFF)
1010 }

```

../Code/Nano33BLESense/LEDs/examples/TestLEDPower/TestLEDPower.ino

This is just a simple example. The variable `LED_PWR` is already defined, so the assignment is not necessary. The command `delay` should be avoided in an Arduino sketch. Instead, variables of the type `elapsedMillis` should be used. [Ard24b]

5.4.4. Test all Functions

The brightness of the power LED can be controlled. This is demonstrated in the example sketch 5.6.

Using the pulse width modulation, the brightness is gradually increased to the maximum value and then gradually reduced to 0 again.

Listing 5.6.: Simple sketch to check the battery state using the power LED

```

1000 #include <../LEDs/PowerLED.h>
1001 #include <../LEDs/LED.h>
1002
1003 int Brightness = 0; ///< Define the initial brightness values (0-255) */
1004
1005 int redStep = 5; ///< Define the increment/decrement value */
1006 void setup() {
1007     // Initialize the pin as an output
1008     PowerLEDinit();
1009 }
1010 void loop() {
1011     // Write the PWM values to the LED pin
1012     LEDBrightness(LED_PWR, redBrightness);
1013
1014     // Update the brightness values
1015     Brightness += Step;
1016
1017     // Check if the brightness values are out of range and reverse the
1018     // direction
1019     if (Brightness <= 0 || Brightness >= 255) {
1020         Step = -Step;
1021     }
1022
1023     // Wait for 10 milliseconds
1024     delay(10);
1025 }

```

../Code/Nano33BLESense/Test/TestLEDPowerBrightness.ino

5.5. Simple Application

There are different situations where it might be useful to program the power LED of the Arduino Nano. For example, you could use it to:

- Indicate the status of the board, such as whether it is connected to a power source, a computer, or a sensor.
- Display the battery level of the board, by changing the brightness or color of the power LED.
- Create a visual alarm or notification, by making the power LED blink or flash in a certain pattern.

The next sketch is a frame for applications with a sign of life. The power LED is switched on for 1 sec and off for 29 sec. The file [SignsOfLife.h](#), see 5.8, contains the declarations and the file [SignsOfLife.cpp](#) contains the implementations.

Listing 5.7.: Simple code for signs of life

```

1000 #ifndef LEDSignsOfLife_h
1001 #define LEDSignsOfLife_h
1002
1003
1004 /**
1005  * @brief initialize digital pin of the LED as an output.
1006  *
1007  * call the function once, in the function setup, when you press reset
1008  * or power the board
1009  *
1010  * Initialization of the pin as an output
1011  *
1012  * The program doesn't check if the parameter PinNo is reasonable.
1013  *
1014  * @param PinNo - number of pin for the LED
1015  * @return false - LED's actual state is off
1016  */
1017 extern bool SignsOfLife();

```

../Code/Nano33BLESense/LEDs/LEDSignsOfLife.h

The library contains one function for initialization and one function [SignsOfLife](#), which controls the LED.

Listing 5.8.: Simple code for signs of life

```

1000 #include "LEDGeneral.h"
1001 #include "LEDSignsOfLife.h"
1002 #include <Arduino.h>
1003
1004 /**
1005  * @brief initialize digital pin of the LED as an output.
1006  *
1007  * call the function once, in the function setup, when you press reset
1008  * or power the board
1009  *
1010  * Initialization of the pin as an output
1011  *
1012  * The program doesn't check if the parameter PinNo is reasonable.
1013  *
1014  * @param PinNo - number of pin for the LED
1015  * @param CycleTimeOn - duration of the LED is on
1016  * @param CycleTimeOff - duration of the LED is off
1017  *
1018  * @return true - LED's actual state is on
1019  * @return false - LED's actual state is off
1020  */
1021 bool SignsOfLifeSetup(int PinNo, int CycleTimeOn, int CycleTimeOff)
1022 {
1023     LEDSetup(PinNo);

```



```
../..Code/Nano33BLESense/LEDs/LEDSignsOfLife.cpp
```

Listing 5.9.: Simple sketch to check the battery state using the power LED

```

1000 #include <Arduino.h>
1001 #include <../LEDs/PowerLED.h>
1002 #include <../LEDs/LED.h>
1003
1004 const int BATTERY_PIN = A0; /*< Battery measurement pin */
1005
1006 const float REFERENCE_VOLTAGE = 3.3; /*< Reference voltage for 3.3V (
1007     adjust if using external power) */
1008 void setup() {
1009     SignsOfLifeInit(LED_PWR, 500, 500);
1010     PowerLED(true);
1011 }
1012 int BatteryState(bool SendMessages)
1013 {
1014     int ret;
1015     // Read battery voltage from analog pin
1016     float rawVoltage = analogRead(BATTERY_PIN) * (REFERENCE_VOLTAGE /
1017         1023.0);
1018
1019     // Calculate percentage based on reference voltage (adjust based on
1020     battery specs)
1021     float batteryPercentage = (rawVoltage / 4.2) * 100.0;
1022
1023     if (SendMessages)
1024     {
1025         // Print battery voltage and percentage (for debugging)
1026         Serial.print("Battery voltage: ");
1027         Serial.print(rawVoltage);
1028         Serial.println(" V");
1029         Serial.print("Battery percentage: ");
1030         Serial.print(batteryPercentage);
1031         Serial.println("%");
1032     }
1033
1034     // Optional: blink LED based on battery level (adjust thresholds)

```

```
1034   if (LED_PWR >= 0) {  
1036       if (batteryPercentage < 20) {  
           digitalWrite(LED_PWR, HIGH);  
           delay(500);  
           digitalWrite(LED_PWR, LOW);  
           delay(500);  
           ret = 1;  
1040       } else {  
           digitalWrite(LED_PWR, HIGH);  
1042       ret = 0;  
           }  
1044   }  
   else  
1046   {  
       ret = 0;  
1048   }  
   return ret;  
1050 }  
  
1052 void loop() {  
1054     BatteryState(false);  
1056     // Delay between measurements  
     delay(5000);  
1058 }
```

../Code/Nano33BLESense/Test/TestLEDPowerBattery.ino

5.6. Further Readings

- Kurniawan, Agus: *IoT Projects with Arduino Nano BLE Sense: Step-By-Step Projects for Beginners*. Apress, 2021. [Kur21]
- Kühnel, Claus: *Arduino - Das umfassende Handbuch*. Rheinwerk Verlag GmbH, 2024. [Küh24]
- Smythe, Richard J.: *Advanced Arduino Techniques in Science - Refine Your Skills and Projects with PCs or Python-Tkinter*. Apress, 2021. [Smy21a]
- Smythe, Richard J.: *Arduino in Science - Collecting, Displaying, and Manipulation Sensor Data*. Apress, 2021. [Smy21b]

6. Built-in RGB-LED

Some Arduino boards have a RGB-LED on board which can use for the application. ARG-LED is at least thre LEDs in on.

6.1. General Information

An RGB LED is a type of LED that can emit different colors of light. RGB stands for red, green, and blue, which are the three primary colors of light. An RGB LED consists of three individual LEDs inside a single package, each with its own color and pin. By varying the brightness of each LED using PWM signals, an RGB LED can produce a wide range of colors by mixing the primary colors. An RGB LED is usually active-low, which means that setting the pin to LOW will turn the LED on, and setting the pin to HIGH will turn the LED off. [Ber18; HEG21; KKV14]

6.2. Specific Sensor

The onboard LED of the Arduino Nano 33 BLE Sense is a RGB-LED that consists of three individual LEDs: red, green, and blue. Each LED is connected to a different pin on the board:

- Pin 22 for red,
- Pin 23 for green, and
- Pin 24 for blue.

The RGB-LED can produce different colors by varying the brightness of each LED using PWM signals. The RGB-LED is active-low, which means that setting the pin to **LOW** will turn the LED on, and setting the pin to **HIGH** will turn the LED off. This is different from the power LED and the built-in LED, which are both active-high. [Ard24a; Ard23a; Ard23b]

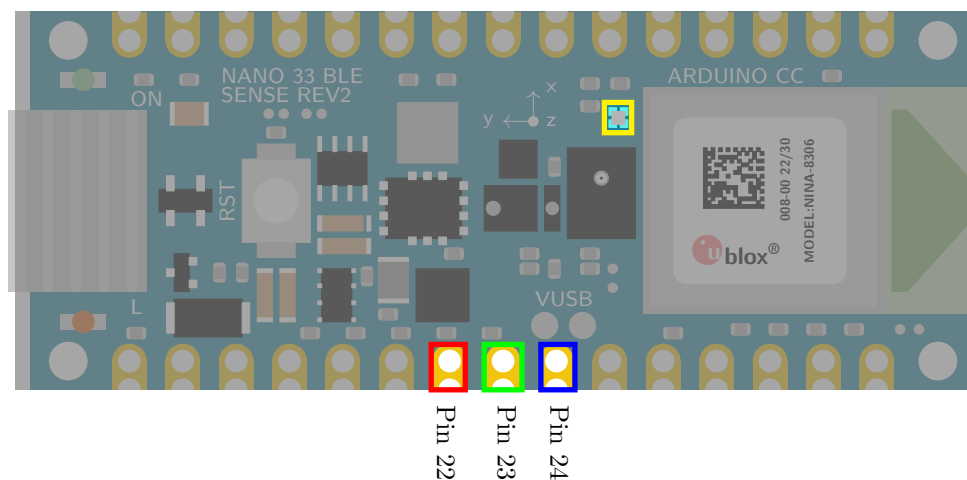


Figure 6.1.: Arduino Nano 33 BLE Sense's RGB LED with Pin 22, Pin 23, and Pin 24

The brightness of the LED varies between 5000-6000 Millicandela (mcd) at a current of 20 mA. The color temperature is controllable and can be set in a range of 5000-6000 Kelvin. The LED should be stored and used between -40°C and +85°C.

6.3. Specification

6.3.1. Pin Assignment

The RGB LED occupies pins on the board internally. These pins are defined via variable names `LEDR`, `LEDG`, and `LEDB`. [Ard23a; Ard23b]

This must be observed when using the pins.

Red LED: `LEDR` = Pin 22

Green LED: `LEDG` = Pin 23

Blue LED: `LEDB` = Pin 24

RGB LED is active-low and connected to pin 22, 23, and 24.

The RGB LED can be controlled programmatically by setting the pin to `LOW` or `HIGH`. The pins 22, 23, and 24 must be defined as an output in the function `setup` by setting `pinMode (22, OUTPUT)`, `pinMode (23, OUTPUT)` and `pinMode (24, OUTPUT)`, otherwise the LED cannot be switched on.

The pins 22, 23, 24 can also be used otherwise. Then the LED is not in use.

6.3.2. Power Consumption

The power consumption has to be considered. It is the same value as for the power LED, see section 5.3.2. Here, a RGB-LED has 3 LEDs inside, therefore the power consumption must be considered for each color.

6.4. Simple Code

In the header file `RGBLED.h`, see code 6.1, are the declaration of the variables and the functions. Variables are connected to pin 22, pin 23, and pin 24. The pins are defined as output in the function `RGBLEDinit`. There are the following functions:

1. `RGBLEDinit`: This function initializes the digital pin 22, pin 23, and pin 24 of the RGB-LED as output.
2. `RGBLED_Red`: This function switches the red part of the RGB-LED on or off.
3. `RGBLED_Green`: This function switches the green part of the RGB-LED on or off.
4. `RGBLED_Blue`: This function switches the blue part of the RGB-LED on or off.
5. `RGBLED_Color`: This function combines the colors.

Listing 6.1.: Header file without comments for using the RGB-LED

```
1000 #ifndef LEDRGB_h
1001 #define LEDRGB_h
1002
1003
1004 /**
1005
1006
```

```
1008 /**
1010 /**
```

../Code/Nano33BLESense/LEDs/LEDRGB.h

In the file `RGBLED.cpp`, see code 6.2, the functions are implemented.

Listing 6.2.: Code file without comments for using the RGB-LED

```
1000 #include <LEDGeneral.h>
1001 #include <LEDRGB.h>
1002 #include <Arduino.h>
1003 * @param ——
1004 *
1005 * @return 0 — success
1006 * @return 1 — error
1007 */
1008 int LEDRGBsetup()
1009 {
1010 *
1011 * swichting the red led on / off
1012 *
1013 * @param On — true: switch on, false: switch off
1014 *
1015 * @return 0 — success
```

../Code/Nano33BLESense/LEDs/LEDRGB.cpp

6.5. Tests

6.5.1. Simple Function Test

The simplest test is the flashing of the LEDs at 2 Hz, see sketch ??.

Listing 6.3.: Simple sketch to test the RGB LED

```
1000 /**
1001 * @file TestLEDRGB.ino
1002 *
1003 * @brief Simple program for testing the RGB-LED
1004 *
1005 * Turns the RGB-LED on for one second, then off for one second,
1006 * repeatedly.
1007 *
1008 * The LED is switched on for 1 second and switched off
1009 * for 1 second so that the LED flashes accordingly.
1010 *
1011 * On the Arduino Nano 33 BLE Sense, it is attached to digital pin 22,
1012 * 23, 24
1013 */
1014
1015 #include <../LEDs/RGBLED.h>
1016 #include <../LEDs/LED.h>
1017
1018 /**
1019 * @brief the setup function runs once when you press reset or power the
1020 * board
1021 *
1022 * Standard function of Arduino sketches
1023 *
1024 * Initialization of the pin 22, pin 23, and 24 as output
```

```

1026 * @param —
1027 *
1028 * @return void
1029 */
1030 void setup() {
1031     // Initialize the pin as an output
1032     RGBLEDinit();
1033 }
1034
1035
1036 /**
1037 * @brief the setup function runs once when you press reset or power the
1038 *       board
1039 *
1040 * Standard function of Arduino sketches
1041 *
1042 * In each loop, the red part is switched on for 1 sec,
1043 * then the green part is switched on for 1 sec, and
1044 * at last the blue part is switched on.
1045 *
1046 * @param —
1047 *
1048 * @return void
1049 */
1050 void loop() {
1051     // Turn the red LED on
1052     RGBLED_Red(SET_ON);
1053     // Wait for one second
1054     delay(1000);
1055     // Turn the LED off
1056     RGBLED_Red(SET_OFF);
1057     // Turn the green LED on
1058     RGBLED_Green(SET_ON);
1059     // Wait for one second
1060     delay(1000);
1061     // Turn the LED off
1062     RGBLED_Green(SET_OFF);
1063     // Turn the blue LED on
1064     RGBLED_Blue(SET_ON);
1065     // Wait for one second
1066     delay(1000);
1067     // Turn the LED off
1068     RGBLED_Blue(SET_OFF);
1069     // Wait for one second
1070     delay(1000);
1071 }

```

../Code/Nano33BLESense/Test/TestLEDRGB.ino

6.5.2. Test all Functions

Different Colors

Colors

A RGB LED is a device that can emit light of different colors by mixing the primary colors of red, green, and blue. The color of the light depends on the relative brightness of each LED, which can be controlled by PWM signals. By varying the brightness of each LED, the RGB LED can produce a wide range of colors, such as yellow, cyan, magenta, white, and more. Some examples of the colors and their corresponding brightness values are:

Red: red = 255, green = 0, blue = 0

Green: red = 0, green = 255, blue = 0

Blue: red = 0, green = 0, blue = 255

Yellow: red = 255, green = 255, blue = 0

Cyan: red = 0, green = 255, blue = 255

Magenta: red = 255, green = 0, blue = 255

White: red = 255, green = 255, blue = 255

Black: red = 0, green = 0, blue = 0

The RGB LED can also create intermediate colors by using different brightness values for each LED. For example, to create

- **orange**, one can use red = 255, green = 127, blue = 0.
- To create **pink**, one can use red = 255, green = 192, blue = 203.
- To create **purple**, one can use red = 128, green = 0, blue = 128.

The sketch 6.4 tests different colors of the LED. The color of the LED changes every 1 second.

Listing 6.4.: value between 0 and 255 to write to the RGB LED

```

1000 /**
1001  * @file TestLEDRGBColors.ino
1002  *
1003  * @brief Simple program for testing the RGB-LED
1004  *
1005  *
1006  * Turns the RGB-LED on for one second, then off for one second,
1007  * repeatedly.
1008  *
1009  * The LED is switched on for 1 second and switched off
1010  * for 1 second so that the LED flashes accordingly.
1011  *
1012  * On the Arduino Nano 33 BLE Sense, it is attached to digital pin 22,
1013  * 23, 24
1014  */
1015
1016 #include <../LEDs/RGBLED.h>
1017 #include <../LEDs/LED.h>
1018
1019 /**
1020  * @brief the setup function runs once when you press reset or power the
1021  * board
1022  *
1023  * Standard function of Arduino sketches
1024  *
1025  * Initialization of the pin 22, pin 23, and 24 as output
1026  *
1027  * @param —
1028  *
1029  * @return void
1030  */
1031 void setup() {
1032     // Set the LED pins as outputs
1033     RGBLEDinit();
1034 }

```

```

1034 |
1036 |
1038 | /**
1039 |  * @brief the setup function runs once when you press reset or power the
1040 |  *       board
1041 |  *
1042 |  * Standard function of Arduino sketches
1043 |  *
1044 |  * In each loop, different colors are tested
1045 |  *
1046 |  * @param —
1047 |  *
1048 |  * @return void
1049 |  */
1050 | void loop() {
1051 |     // red
1052 |     RGBLED_Color(255,0,0);
1053 |     // Wait for one second
1054 |     delay(1000);
1055 |     // green
1056 |     RGBLED_Color(0,255,0);
1057 |     // Wait for one second
1058 |     delay(1000);
1059 |     // blue
1060 |     RGBLED_Color(0,0,255);
1061 |     // Wait for one second
1062 |     delay(1000);
1063 |     // yellow
1064 |     RGBLED_Color(255,255,0);
1065 |     // Wait for one second
1066 |     delay(1000);
1067 |     // cyan
1068 |     RGBLED_Color(0,255,255);
1069 |     // Wait for one second
1070 |     delay(1000);
1071 |     // magenta
1072 |     RGBLED_Color(255,0,255);
1073 |     // Wait for one second
1074 |     delay(1000);
1075 |     // white
1076 |     RGBLED_Color(255,255,255);
1077 |     // Wait for one second
1078 |     delay(1000);
1079 |     // black
1080 |     RGBLED_Color(0,0,0);
1081 |     // Wait for one second
1082 |     delay(1000);
1083 |     // orange
1084 |     RGBLED_Color(255,127,0);
1085 |     // Wait for one second
1086 |     delay(1000);
1087 |     // pink
1088 |     RGBLED_Color(255,192,203);
1089 |     // Wait for one second
1090 |     delay(1000);
1091 |     // purple
1092 |     RGBLED_Color(120,0,128);
1093 |     // Wait for one second
1094 |     delay(1000);
1095 | }

```

../Code/Nano33BLESense/Test/TestLEDRGBColors.ino

Brightness of the RGB-LED

The RGB-LED can also create gradients of colors by changing the brightness values gradually over time. This can create a smooth transition from one color to another, such as from red to green to blue and back to red.

This sketch 6.5 will make the RGB-LED change colors smoothly by varying the brightness of each LED with different speeds. You can adjust the initial brightness values and the increment/decrement values to get different effects.

Listing 6.5.: Different brightness levels for the RGB-LED colors

```

1000 /**
1001  * @file TestLEDRGBBrightness.ino
1002  *
1003  * @brief Simple program for testing the RGB-LED
1004  *
1005  * Turns the RGB-LED on for one second, then off for one second,
1006  * repeatedly.
1007  *
1008  * The LED is switched on for 1 second and switched off
1009  * for 1 second so that the LED flashes accordingly.
1010  *
1011  * On the Arduino Nano 33 BLE Sense, it is attached to digital pin 22,
1012  * 23, 24
1013  */
1014
1015 #include <../LEDs/RGBLED.h>
1016 #include <../LEDs/LED.h>
1017
1018 int redBrightness = 0; /*< Define the initial brightness values for
1019    red (0-255) */
1020 int greenBrightness = 0; /*< Define the initial brightness values for
1021    green (0-255) */
1022 int blueBrightness = 0; /*< Define the initial brightness values for
1023    blue (0-255) */
1024
1025 int redStep = 5; /*< Define the increment/decrement value for red */
1026 int greenStep = 3; /*< Define the increment/decrement value for green */
1027 int blueStep = 7; /*< Define the increment/decrement value for blue */
1028
1029 /**
1030  * @brief the setup function runs once when you press reset or power the
1031  * board
1032  *
1033  * Standard function of Arduino sketches
1034  *
1035  * Initialization of the pin 22, pin 23, and 24 as output
1036  *
1037  * @param —
1038  *
1039  * @return void
1040  */
1041 void setup() {
1042     // Set the LED pins as outputs
1043     RGBLEDinit();
1044 }
1045
1046 /**
1047  * @brief the setup function runs once when you press reset or power the
1048  * board
1049  */

```

```

* Standard function of Arduino sketches
1050 *
* In each loop, the color is changing
1052 *
* @param ——
1054 *
* @return void
1056 */
void loop() {
1058 // Write the PWM values to the LED pins
    RGBLED_Colors(redBrightness, greenBrightness, blueBrightness);
1060
    // Update the brightness values for each color
1062 redBrightness += redStep;
    greenBrightness += greenStep;
1064 blueBrightness += blueStep;

    // Check if the brightness values are out of range and reverse the
    direction
1066 if (redBrightness <= 0 || redBrightness >= 255) {
1068     redStep = -redStep;
    }
1070 if (greenBrightness <= 0 || greenBrightness >= 255) {
    greenStep = -greenStep;
1072 }
1074 if (blueBrightness <= 0 || blueBrightness >= 255) {
    blueStep = -blueStep;
1076 }

    // Wait for 10 milliseconds
1078 delay(10);
}

```

../Code/Nano33BLESense/Test/TestLEDRGBBrightness.ino

6.6. Simple Application

In many applications, it is necessary to save power so that the LED is not switched on continuously. Nevertheless, feedback is required as to whether the system, e.g. a fire detector, is switched on and functional. In this case, the LED is switched on for 1 second at a defined time interval, in this case 30 seconds. This is implemented in the example ??.

Listing 6.6.: A simple watch dog: A simple watchdog: the built-in RGB-LED is switched on for 1 second every 30 seconds.

```

1000 /**
    *
    * @file TestLEDRGBApplication.ino
    *
    * @brief Simple application using the built-in RGB-LED of the Arduino
    * Nano 33 BLE Sense
    *
    * @details The red part of the RGB-LED is switched on for 1 second and
    * switched off for 29 second so that red part of the LED flashes
    * accordingly.
    *
    */
1010 */
1012 #include <../LEDs/RGBLED.h>
    #include <../LEDs/LED.h>
1014 #include <../LEDs/SignsOfLife.h>

```

```

1016 #define CycleTimeOn 1000 /*< Duty cycle [ms] */
1018 #define CycleTimeOff 29000 /*< Switch-off time [ms] */
1020
1022 /**
1024 * @brief the setup function runs once when you press reset or power the
      * board
      *
1026 * standard function of Arduino sketches
      *
1028 * Initialization of the pin LED_RED, the red part of the builtin RGB-
      * LED for the signs of life
      *
1030 * @param ——
      *
1032 * @return void
      */
1034 void setup() {
      // Initialize the function SignsOfLife
1036 SignsOfLifeInit(LED_RED, CycleTimeOn, CycleTimeOff)
      // Application
      // ...
1040 }
1042
1044 /**
1046 * @brief the loop function runs over and over again forever
      *
1048 * standard function of Arduino sketches
      *
1050 * swichting the led on / off for the signs of life
      *
1052 * @param ——
      *
1054 * @return void
      */
1056 void loop() {
      // Switch red part of the RGB LED on/off
      SignsOfLife();
      // Application
      // ...
1060 }

```

../Code/Nano33BLESense/Test/TestLEDRGBApplication.ino

6.7. Further Readings

- Schanda, Janos: *Colorimetry: Understanding the CIE System*. Wiley, 2007. [Sch07]
- Hiller, Gabrielle: *Color measurement - the CIE color space*. Datacolor, 2019. [Hil22]
- Lukac, Rastislav and Plataniotis, Konstantinos N.: *Color Image Processing: Methods and Applications*. CTC Press, 2018. [LP18]

7. TinyML Shield: Built-in Push Button

7.1. General

A push button is a simple switch mechanism used to control various devices and processes. It is typically made of hard materials like plastic or metal. The surface of a push button is designed to be easily depressed or pushed by the human finger or hand. When you press a push button, it either closes or opens an electrical circuit.

In industrial and commercial applications, push buttons can be linked together so that pressing one button releases another. Emergency stop buttons, often with large mushroom-shaped heads, enhance safety in machines and equipment. Pilot lights are sometimes added to push buttons to draw attention and provide feedback when the button is pressed. Color-coding is common to associate push buttons with their specific functions (e.g., red for stopping, green for starting). [Din]

7.2. Built-in Push Button

The Arduino Nano 33 BLE Sense features an onboard push button. This button is a simple electrical switch that can be activated by pressing it. When you press the button, it completes an electrical circuit. The push button is designed for user interaction and can be used for various purposes.

The built-in button `BUTTON_B` is connected with pin 11. Using the function `pinMode(BUTTON_PIN, INPUT_PULLUP)` the pin is declared as an input. As can be seen in the sketch, pressing the button can be used to trigger actions; typical actions include switching on an LED, changing modes, or initiating sensor readings. Overall, the push button provides a convenient way to interact with the Arduino Nano 33 BLE Sense and create responsive projects. [Ard23a; Ard23b; Arda]

WS:Push Button hier
nochmal mit tikz-code
hervorheben

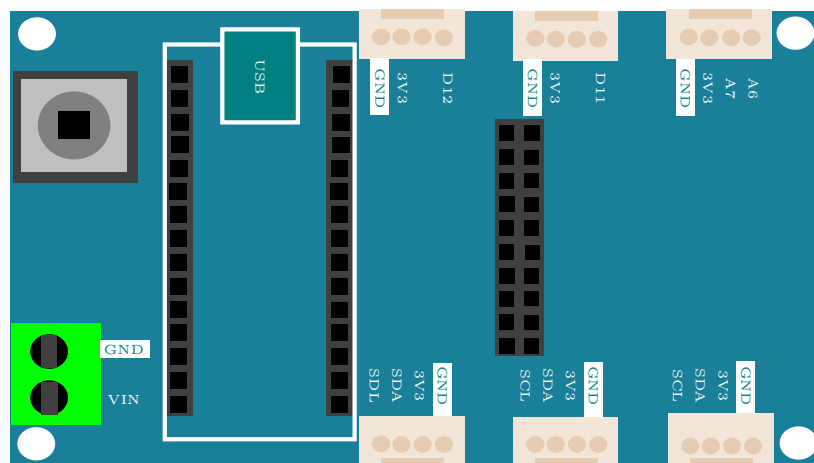


Figure 7.1.: Tiny Machine Learning Shield [Gua21]

7.3. Specification

The built-in button is a small white button and connected to pin 11.

Built-in Button: `BUTTON_B = 11u`

If the pin is declared as input in the function `setup`, then it can be used.

The pin 11 must be defined as an input in the function `setup` by setting `pinMode(11, INPUT_PULLUP)`, otherwise the button cannot be read.

The pin 11 can also be used otherwise. Then the button is not in use. [Ard23a; Ard23b; Arda]

WS:

- cite data sheet
- Circuit Diagram

7.4. Simple Code

As soon as the button is connected, it can be used. It is not necessary to install a special library. Programming takes place in two steps:

1. In the first step, the pin is configured in the function `setup`:

Listing 7.1.: Defining the built-in button's pin as an input.

```
1000 pinMode(BUTTON_B, INPUT_PULLUP)
```

2. In the second step, the button can be used in the function `loop`. To read in the value, use the function `digitalRead`:

Listing 7.2.: Read the built-in button's state

```
1000 buttonState = digitalRead(BUTTON_B);
```

7.5. Tests

The simplest test is the flashing of an LED for 2 seconds, if the button is pressed, see sketch 7.3.

Listing 7.3.: Simple sketch to test the push button and the built-in LED

```
1000 #include <LEDGeneral.h>
1001 #include <LEDPower.h>
1002 #include <LEDRGB.h>
1003 #include <LEDSignsOfLife.h>
1004 #include <LEDbuiltin.h>
1005 #include <Arduino.h>
1006
1007 /**
1008  *
1009  * @file TestPushButton.ino
1010  *
1011  * @brief Simple application reading in the built-in push button states.
1012  *
1013  *
1014  * @details If the built-in push button is pressed, the the internal
1015  *          built-in LED is turn on for 2 seconds
```

```

1016 *
1017 */
1018 #define BUTTON_B 11 /*< Button_B is here defined as pin 11 */
1020 #define BUTTON_PIN BUTTON_B /*< Use the onboard push button (BUTTON_B)
1021 */
1022 #define SET_ON true /*< Define flag for switching on */
1023 #define SET_OFF false /*< Define flag for switching off */
1024
1025 int buttonState = 0; /*< state of the built-in push button */
1026
1027 /**
1028 * @brief the setup function runs once when you press reset or power the
1029 board
1030 *
1031 * standard function of Arduino sketches
1032 *
1033 * Initialization of the built-in LED and the push button
1034 *
1035 * @param —
1036 *
1037 * @return void
1038 */
1039 void setup() {
1040 // Initialize the pin as an output
1041 LEDBuiltinSetup();
1042 // Initialize the pin as an input
1043 pinMode(BUTTON_PIN, INPUT_PULLUP);
1044 }
1045
1046 /**
1047 * @brief the loop function runs over and over again forever
1048 *
1049 * standard function of Arduino sketches
1050 *
1051 * swichting the built-in led on for 2 sec, if the push button is
1052 pressed
1053 *
1054 * @param —
1055 *
1056 * @return void
1057 */
1058 void loop()
1059 {
1060 buttonState = digitalRead(BUTTON_PIN);
1061 if (buttonState == HIGH)
1062 {
1063 // Turn the LED on
1064 LEDBuiltin(SET_ON);
1065 // Wait for two second
1066 delay(2000);
1067 // Turn the LED off
1068 LEDBuiltin(SET_OFF);
1069 // Wait for one second
1070 delay(1000);
1071 buttonState = LOW;
1072 }
1073 }

```

../Code/Nano33BLESense/PushButton/TestPushButton/TestPushButton.ino

7.6. Interrupt Function on the Arduino Nano 33 BLE Sense

An interrupt is a function that allows the microcontroller on the Arduino Nano 33 BLE Sense to immediately respond to an external event, such as pressing a button, without the need for the program to continuously check for that event. Normally, in a program, the microcontroller would repeatedly check if a button is pressed by running through a loop, which consumes processing power and can slow down other tasks. This is not efficient, especially when the program needs to perform other actions at the same time.

With an interrupt, the program can continue running other tasks, and only when the specified event (like a button press) occurs, the program is temporarily paused to execute a special function known as the **Interrupt Service Routine (ISR)**. The ISR is a small, efficient function designed to handle the event, such as changing a variable or triggering another action. After the ISR is executed, the program returns to where it left off, continuing its normal operation.

Using interrupts in this way helps make the program more efficient and responsive, as it doesn't waste time constantly checking for events and can focus on other tasks until something important happens. This is especially useful for real-time applications where quick responses to external events are necessary, such as in embedded systems, robotics, or sensor monitoring.

7.7. Simple Application

This code sketch 7.4 shows how to use an interrupt to control the built-in LED on the Arduino Nano 33 BLE Sense when the built-in push button is pressed.

The program sets up the built-in LED and push button. When the button is pressed, an interrupt runs a special function called an Interrupt Service Routine. The ISR is very short and only sets a flag variable `pushPressed` to `true`, indicating that the button has been pressed.

The main program `loop` checks this flag. If it is set to `true`, the program turns on the LED for 2 seconds, then turns it off and resets the flag to `false`. This ensures the system is ready to detect the next button press. Using the `true` and `false` values makes it easy to manage the button's state.

This method avoids constantly checking the button's state, making the program more efficient. Additionally, the Arduino Nano 33 BLE Sense allows all its pins to be used for interrupts. This makes it easy to attach interrupt functions to any pin, allowing quick and efficient responses to inputs like buttons or sensors. [Arde]

WS:andere Überschrift
für simple Application

Listing 7.4.: Simple sketch connects the push button with an interrupt. Here, pushing the built-in button is handled by an interrupt. Then the built-in LED switch on for 2 sec.

```

1000 #include <LEDGeneral.h>
      #include <LEDPower.h>
1002 #include <LEDRGB.h>
      #include <LEDSignsOfLife.h>
1004 #include <LEDbuiltin.h>

1006 #include <LEDGeneral.h>
      #include <LEDPower.h>
1008 #include <LEDRGB.h>
      #include <LEDSignsOfLife.h>
1010 #include <LEDbuiltin.h>

1012 /**

```



```

1014 *   @file TestPushButtonInterrupt.ino
1016 *   @brief Simple application reading in the built-in push button states
        using an interrupt
1018 *
1018 *   @details If the builtin push button is pressed, the built-in LED is
        switched on for 2 second and switched off again.
1020 *   But in this example, an interrupt is used.
1022 */

1024 #include <LEDGeneral.h>
1024 #include <LEDBuiltin.h>
1026
1028
1028 #define BUTTON_B 11 /*< Button_B is here defined as pin 11 */
1030 #define BUTTON_PIN BUTTON_B

1032
1032 #define SET_ON true /*< Define flag for switching on */
1034 #define SET_OFF false /*< Define flag for switching off */
1036
1038
1038 // Initialize variables
1040 //
1040 volatile bool pushPressed = false; /*< // Flag, whether the button is
        pressed. Declare as volatile for interrupt. safety */
1042
1042 int ledState = 0; /*< LED-Status zur Verarbeitung */
1044
1044 /**
1046 *   @brief the setup function runs once when you press reset or power the
        board
1048 *
1048 *   standard function of Arduino sketches
1050 *
1050 *   Initialization of the built-inb LED and the push button
1052 *
1052 *   @param —
1054 *
1054 *   @return void
1056 */
1056 void setup() {
1056     // Initialize the pin as an output
1058     LEDBuiltinsetup();
1058     // Initialize the pin as an input
1060     pinMode(BUTTON_PIN, INPUT_PULLUP);
1060     // Initialize the interrupt function
1062     attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), buttonPressed,
        CHANGE);
1064 }

1064
1066 /**
1066 *   @brief Interrupt service function
1068 *
1068 *   Attention: as short as possible
1070 *
1070 *   The function changes just one flag.
1072 */
1072 void buttonPressed()
1074 {
1074     if (pushPressed == false)
1076     {

```

```

    pushPressed = true;
1078 }
1080 }
1082 /**
1084 * @brief the loop function runs over and over again forever
1086 * standard function of Arduino sketches
1088 * swichting the built-in led on for 2 sec, if the push button is
1090 * pressed
1092 * @param —
1094 * @return void
1096 */
1098 void loop()
1100 {
1102     if (pushPressed)
1104     {
1106         // Turn the LED on
1108         LEDBuiltin(SET_ON);
1110         // Wait for one second
1112         delay(2000);
1114         // Turn the LED off
1116         LEDBuiltin(SET_OFF);
1118         pushPressed = false;
1120     }
1122     // ...
1124 }

```

../Code/Nano33BLESense/PushButton/TestPushButtonInterrupt/TestPushButtonInterrupt.ino

This is just a simple example. The variable `BUTTON_B` is already defined, so the assignment is not necessary. The command `delay` should be avoided in an Arduino sketch. Instead, variables of the type `elapsedMillis` should be used.

7.8. Further Readings

- Boxall, John: *Arduino Workshop - A Hands-On Introduction with 65 Projects*. No Starch Press, 2021. [Box21]
- Voš, Andreas: *Volumio mit Drehgebern erweitern*. Make Magazin, 2024. [Voš24]
- Kühnel, Claus: *Arduino - Das umfassende Handbuch*. Rheinwerk Verlag GmbH, 2024. [Küh24]

8. Pressure and Temperature Sensor LPS22HB

The sensor LPS22HB is a piezoresistive absolute pressure sensor that is integrated into the Arduino Nano 33 BLE Sense board. It's a digital sensor that measures atmospheric pressure and temperature. The sensor uses a piezoresistive technology to detect changes in pressure. The LPS22HB has an accuracy of ± 1.5 hPa and a resolution of 0.01 hPa. It can measure pressures from 260 to 1260 hPa. The sensor also measures temperature with an accuracy of $\pm 0.12^\circ\text{C}$ and a resolution of 0.01°C . The temperature range is from -40°C to 85°C .

The LPS22HB communicates with the Arduino board using the protocol I²C. It has a low power consumption of $1.5\ \mu\text{A}$ in sleep mode and 1.5 mA in active mode. The sensor is also resistant to shock and vibration. It's a sensor for projects that require accurate pressure and temperature measurements, such as weather stations, altimeters. [Ard23a; Ard23b]

Arduino Nano 33 BLE Sense Lite hat keine HTS221-Temperatur- und Feuchtigkeitssensoren, sondern nur den LPS22HB-Drucksensor, der allerdings auch die Temperatur messen kann

Arduino Nano 33 BLE Sense Rev2 hat einen HTS221-Temperatur- und Feuchtigkeitssensor und den LPS22HB-Drucksensor, der allerdings auch die Temperatur messen kann. Unterschiede:

Inertiale Messeinheit (IMU): Rev 1: Verfügt über eine einzelne 9-Achsen-IMU. Rev 2: Hat eine Kombination aus zwei IMUs: eine 6-Achsen-IMU (BMI270) und eine 3-Achsen-IMU (BMM150), was die Möglichkeiten zur Bewegungserkennung erweitert¹. Design und Zugänglichkeit: Rev 2: Integriert neue Pads und Testpunkte für USB, SWDIO und SWCLK, was den Zugang zu diesen wichtigen Punkten auf der Platine erleichtert¹. Rev 1: Hat diese zusätzlichen Pads und Testpunkte nicht. Stromversorgung:

Rev 2: Verwendet den MP2322 als Stromversorgungs Komponente, was die Leistung verbessert¹.

Rev 1: Nutzt eine andere Stromversorgungs Komponente. Der Arduino Nano 33 BLE Sense Rev 1 verwendet den MPS MP2144 als Stromversorgungs Komponente

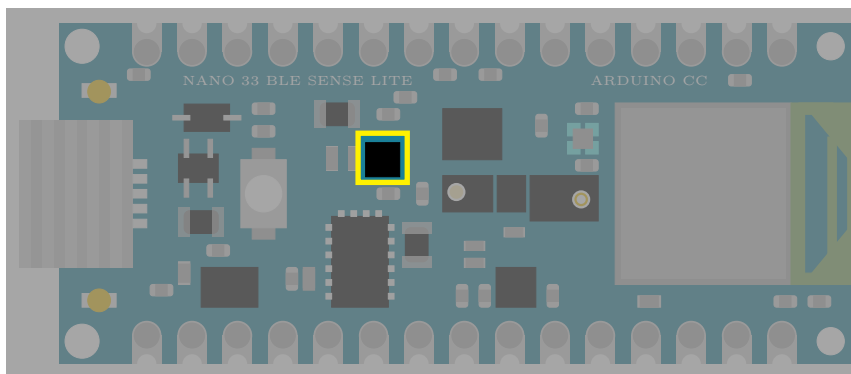


Figure 8.1.: Arduino Nano 33 BLE Sense's pressure and temperature sensor LPS22HB

8.1. General

A pressure sensor is a device that measures the pressure of its surroundings. It works by converting the mechanical energy of the environment into an electrical signal. The sensor typically consists of a diaphragm or membrane, which is a thin, flexible material that changes its shape in response to changes in pressure. The diaphragm or membrane is usually connected to a microcontroller or other electronic device, which reads the electrical signal and converts it into a pressure reading. The pressure sensor can be calibrated to provide accurate readings over a specific pressure range. The sensor can be used in a variety of applications, including industrial control systems, medical devices, and consumer electronics.

Pressure sensors can be classified into two main types: absolute and differential. Absolute pressure sensors measure the absolute pressure of the environment, while differential pressure sensors measure the difference in pressure between two points. Pressure sensors can be used to measure a wide range of pressures, from very low pressures to very high pressures. The accuracy of a pressure sensor depends on its calibration and the quality of its components. Pressure sensors can be affected by various factors, including temperature, humidity, and air flow. Pressure sensors can be used to monitor and control pressure in a variety of applications, including heating and cooling systems, refrigeration systems, and food processing systems. Pressure sensors can be used to detect pressure anomalies and alert users to potential problems. [Gan24]

A temperature sensor is a device that measures the temperature of its surroundings. It works by converting the thermal energy of the environment into an electrical signal. The sensor typically consists of a thermistor or thermocouple, which is a device that changes its electrical resistance or voltage in response to changes in temperature. The thermistor or thermocouple is usually connected to a microcontroller or other electronic device, which reads the electrical signal and converts it into a temperature reading. The temperature sensor can be calibrated to provide accurate readings over a specific temperature range. The sensor can be used in a variety of applications, including industrial control systems, medical devices, and consumer electronics.

Temperature sensors can be classified into two main types: contact and non-contact. Contact temperature sensors, such as thermocouples, come into direct contact with the object being measured. Non-contact temperature sensors, such as infrared sensors, do not come into contact with the object being measured.

Temperature sensors can be used to measure a wide range of temperatures, from very low temperatures to very high temperatures. The accuracy of a temperature sensor depends on its calibration and the quality of its components. Temperature sensors can be affected by various factors, including humidity, air flow, and radiation. Temperature sensors can be used to monitor and control temperature in a variety of applications, including heating and cooling systems, refrigeration systems, and food processing systems. Temperature sensors can be used to detect temperature anomalies and alert users to potential problems.

WS:cite books,
applications, board

8.2. Specific Sensor

The pressure sensor of the LPS22HB is a piezoresistive sensor that changes its electrical resistance in response to changes in pressure. The pressure sensor of the LPS22HB is connected to the Arduino Nano 33 BLE Sense, which reads the electrical signal and converts it into a pressure reading. The pressure sensor of the LPS22HB is calibrated to provide accurate readings over a pressure range of 260 to 1260 mbar. The pressure sensor of the LPS22HB is a non-contact sensor, meaning that it does not come into

direct contact with the object being measured. The pressure sensor of the LPS22HB is affected by various factors, including temperature and humidity. The pressure sensor of the LPS22HB can be used to monitor and control pressure in a variety of applications, including industrial control systems and consumer electronics. The pressure sensor of the LPS22HB can be used to detect pressure anomalies and alert users to potential problems. The pressure sensor of the LPS22HB has an accuracy of ± 0.12 mbar. The pressure sensor of the LPS22HB is a low-power sensor, making it suitable for use in battery-powered devices.

The LPS22HB is a digital pressure sensor that also includes a temperature sensor. The temperature sensor of the LPS22HB is a thermistor that changes its electrical resistance in response to changes in temperature. The thermistor is connected to the microcontroller Arduino Nano 33 BLE Sense, which reads the electrical signal and converts it into a temperature reading. The temperature sensor of the LPS22HB is calibrated to provide accurate readings over a temperature range of -40°C to 85°C . The temperature sensor of the LPS22HB is a non-contact sensor, meaning that it does not come into direct contact with the object being measured.

The temperature sensor of the LPS22HB is affected by various factors, including humidity and air flow. The temperature sensor of the LPS22HB can be used to monitor and control temperature in a variety of applications, including industrial control systems and consumer electronics. The temperature sensor of the LPS22HB can be used to detect temperature anomalies and alert users to potential problems. The temperature sensor of the LPS22HB is a high-accuracy sensor, with an accuracy of $\pm 0.12^{\circ}\text{C}$ and a resolution of 0.01°C .

The temperature sensor of the LPS22HB is a low-power sensor, making it suitable for use in battery-powered devices.

WS:cite board

8.3. Specification

The LPS22HB is a digital pressure sensor that measures pressure in the range of 260 to 1260 mbar. It has a high accuracy of ± 0.12 mbar and a resolution of 0.01 mbar. The sensor is calibrated to provide accurate readings over a temperature range of -40°C to 85°C . It has a non-contact measurement principle, which means that it does not come into direct contact with the object being measured. The LPS22HB is a compact sensor, measuring $3.5 \times 3.5 \times 1.1$ mm in size.

It has a low power consumption of $1.5\mu\text{A}$ in active mode and $0.1\mu\text{A}$ in sleep mode. The sensor has a sampling rate of up to 100 Hz and a data transfer rate of up to 400 kbps. Note that the sampling rate of the sensor LPS22HB can be adjusted using the function `setSamplingRate()` in the Arduino library. The default sampling rate is 100 Hz, but it can be set to 50 Hz, 25 Hz, or 10 Hz using the following code:

```
lps22hb.setSamplingRate(50); // 50 Hz
lps22hb.setSamplingRate(25); // 25 Hz
lps22hb.setSamplingRate(10); // 10 Hz
```

It's worth noting that the sampling rate of the sensor LPS22HB can affect the accuracy and reliability of the measurements. A higher sampling rate can provide more accurate measurements, but it can also increase the power consumption of the sensor.

The sensor is also resistant to shock, vibration, and temperature changes. It has a high sensitivity and a low noise level, making it suitable for use in applications where high accuracy is required.

- cite data sheet
- Circuit Diagram

8.4. Library

To program the sensor LPS22HB on the Arduino Nano 33 BLE Sense, you will need to use the library LPS22HB. [Ardc]

8.4.1. Description

The library LPS22HB is a library for the Arduino platform that provides a simple interface for interacting with the sensor LPS22HB. It allows you to read temperature and pressure values from the sensor, as well as set the sensor's mode and power mode. [Ardc]

8.4.2. Installation

To use the library LPS22HB, you will need to install it on your Arduino IDE. You can do this by following these steps:

- Open the Arduino IDE and navigate to the menu `Sketch`.
- Select `Sketch -> Include Library -> Manage Libraries`.
- Search for “LPS22HB” in the library search bar.
- Click on the library “LPS22HB” and then click on the button “Install”.
- Wait for the library to install and then restart the Arduino IDE.

Once you have installed the library LPS22HB, you can use it to program the sensor LPS22HB on the Arduino Nano 33 BLE Sense. Here is an example ?? of how you can use the library to read temperature and pressure values from the sensor:

Listing 8.1.: Example code for the sensor LPS22HB on the Arduino Nano 33 BLE Sense

```

1000 /**
1001  * @file LPS22HBSimpleExample.ino
1002  *
1003  * @brief Example code for the sensor LPS22HB on the Arduino Nano 33 BLE
1004  *       Sense
1005  *
1006  * @author Elmar Wings
1007  *
1008  * @version 1.0
1009  */
1010 #include <LPS22HB.h>
1011
1012 /**
1013  * @brief LPS22HB sensor object
1014  *
1015  * @details This object represents the LPS22HB sensor and provides
1016  *          methods for reading temperature and pressure values.
1017  */
1018 LPS22HB lps22hb;
1019
1020 /**
1021  * @brief Setup function
1022  *
1023  * @details This function is called once at the beginning of the program
1024  *          and is used to initialize the sensor and serial communication.
1025  */
1026 void setup() {
1027     // Initialize serial communication

```

```

1026 Serial.begin(9600);
1028 //Initialize the LPS22HB sensor
    lps22hb.begin();
1030 }
1032 /**
1033  * @brief Loop function
1034  *
1035  * @details This function is called repeatedly after the setup function
1036  *         and is used to read temperature and pressure values from the sensor.
1037  */
1038 void loop() {
1039     // Read temperature value from the sensor
1040     int16_t temp = lps22hb.readTemperature();
1041
1042     // Read pressure value from the sensor
1043     int32_t pressure = lps22hb.readPressure();
1044
1045     // Print temperature and pressure values to the serial console
1046     Serial.print("Temperature: ");
1047     Serial.print(temp);
1048     Serial.println(" C");
1049     Serial.print("Pressure: ");
1050     Serial.print(pressure);
1051     Serial.println(" mbar");
1052
1053     // Wait for 1 second before taking the next reading
1054     delay(1000);
1055 }

```

../Code/Nano33BLESense/SensorLPS22HB/LPS22HBSimpleExample.ino

This sketch 8.1 uses the library LPS22HB to read temperature and pressure values from the sensor LPS22HB and print them to the serial console.

Note that you will need to have the Arduino Nano 33 BLE Sense board connected to your computer and the Arduino IDE installed on your computer in order to use the LPS22HB library.

8.4.3. Functions

Here are the functions of the Arduino library LPS22HB:

- Initialization Functions
 - `begin()`: Initializes the sensor LPS22HB and sets it up for use.
 - `reset()`: Resets the sensor LPS22HB to its default state.
- Reading Functions
 - `readTemperature()`: Reads the temperature value from the sensor LPS22HB.
 - `readPressure()`: Reads the pressure value from the sensor LPS22HB.
 - `readAltitude()`: Reads the altitude value from the sensor LPS22HB.
 - `readTemperatureAndPressure()`: Reads both the temperature and pressure values from the sensor LPS22HB.
- Setting Functions
 - `setMode()`: Sets the mode of the sensor LPS22HB.
 - `setPowerMode()`: Sets the power mode of the sensor LPS22HB.
 - `setSamplingRate()`: Sets the sampling rate of the sensor LPS22HB.

-
- `setFilter()`: Sets the filter of the sensor LPS22HB.
 - Getting Functions
 - `getMode()`: Gets the current mode of the sensor LPS22HB.
 - `getPowerMode()`: Gets the current power mode of the sensor LPS22HB.
 - `getSamplingRate()`: Gets the current sampling rate of the sensor LPS22HB.
 - `getFilter()`: Gets the current filter of the sensor LPS22HB.
 - Sleep Functions
 - `sleep()`: Puts the sensor LPS22HB into sleep mode.
 - `wakeUp()`: Wakes up the sensor LPS22HB from sleep mode.
 - Other Functions
 - `checkStatus()`: Checks the status of the sensor LPS22HB.
 - `getError()`: Gets the error code of the sensor LPS22HB.
 - `resetError()`: Resets the error code of the sensor LPS22HB.

Note that this list may not be exhaustive, and the library may have additional functions not listed here.

8.4.4. Example - Manual

8.4.5. Example

8.4.6. Example - Code

8.4.7. Example - Files

8.5. Calibration

To calibrate the sensor LPS22HB on the Arduino Nano 33 BLE Sense, you will need to follow these steps. First, upload the calibration code to the Arduino Nano 33 BLE Sense. The calibration code will prompt you to enter the calibration parameters, such as the temperature and pressure values. Enter the calibration parameters, and the code will store them in the sensor's memory. The calibration process will take a few minutes to complete. During the calibration process, the sensor will take multiple readings of the temperature and pressure values. The sensor will then calculate the average of the readings and store it as the calibration value. Once the calibration process is complete, the sensor will be ready to use. To verify the calibration, you can use the calibration code to read the temperature and pressure values from the sensor. Compare the readings with the expected values to ensure that the sensor is calibrated correctly. If the readings are not accurate, you may need to repeat the calibration process. The calibration process can be repeated as many times as necessary to achieve accurate readings. The calibration values can be stored in the sensor's memory and retrieved later. By following these steps, you can calibrate the sensor LPS22HB on the Arduino Nano 33 BLE Sense and ensure accurate readings.

The code 8.2 reads the temperature and pressure values from the sensor LPS22HB, stores them in the sensor's memory, and prints them to the serial console. The `storeCalibration` function is used to store the calibration values in the sensor's memory.

Listing 8.2.: Simple sketch calibrating the sensor LPS22HB

WS:cite method, more
mathematics!


```

1000 /**
1001  * @file LPS22HBCalibration.ino
1002  *
1003  * @brief Calibration code for the sensor LPS22HB on the Arduino Nano 33
1004  *        BLE Sense
1005  *
1006  * @details This code reads the temperature and pressure values from the
1007  *        LPS22HB sensor, stores them in the sensor's memory, and prints them
1008  *        to the serial console. The storeCalibration function is used to
1009  *        store the calibration values in the sensor's memory.
1010  *
1011  * Note that this is just an example code and you may need to modify it
1012  * to suit your specific needs. Additionally, you will need to make
1013  * sure that the LPS22HB sensor is properly connected to the Arduino
1014  * Nano 33 BLE Sense and that the I2C interface is enabled.
1015  *
1016  *
1017  * @author Elmar Wings
1018  *
1019  * @version 1.0
1020  */
1021
1022 #include <Wire.h>
1023 #include <LPS22HB.h>
1024
1025 /**
1026  * @brief LPS22HB sensor object
1027  */
1028 LPS22HB lps22hb;
1029
1030 /**
1031  * @brief Setup function
1032  */
1033 void setup() {
1034     Serial.begin(9600);
1035     Wire.begin();
1036     lps22hb.begin();
1037 }
1038
1039 /**
1040  * @brief Loop function
1041  */
1042 void loop() {
1043     // Read the temperature and pressure values from the sensor
1044     int16_t temp = lps22hb.readTemperature();
1045     int32_t pressure = lps22hb.readPressure();
1046
1047     // Print the temperature and pressure values to the serial console
1048     Serial.print("Temperature: ");
1049     Serial.print(temp);
1050     Serial.println(" C");
1051     Serial.print("Pressure: ");
1052     Serial.print(pressure);
1053     Serial.println(" mbar");
1054
1055     // Store the calibration values in the sensor's memory
1056     lps22hb.storeCalibration(temp, pressure);
1057
1058     // Wait for 1 second before taking the next reading
1059     delay(1000);
1060 }
1061
1062 /**
1063  * @brief Store calibration values in the sensor's memory
1064  * @param temp Temperature value
1065  * @param pressure Pressure value
1066  */
1067 void storeCalibration(int16_t temp, int32_t pressure) {

```

```

1062 // Store the calibration values in the sensor's memory
Wire.beginTransmission(0x5C); // LPS22HB I2C address
Wire.write(0x00); // Calibration register
1064 Wire.write(temp >> 8); // High byte of temperature value
Wire.write(temp & 0xFF); // Low byte of temperature value
1066 Wire.write(pressure >> 24); // High byte of pressure value
Wire.write(pressure >> 16); // Middle byte of pressure value
1068 Wire.write(pressure >> 8); // Low byte of pressure value
Wire.endTransmission();
1070 }

```

../Code/Nano33BLESense/SensorLPS22HB/LPS22HBCalibration.ino

Note that this is just an example code and you may need to modify it to suit your specific needs. Additionally, you will need to make sure that the sensor LPS22HB is properly connected to the Arduino Nano 33 BLE Sense and that the I2C interface is enabled.

8.6. Simple Code

8.7. Sleep Mode

Listing 8.3.: Sketch for the Arduino Nano 33 BLE Sense to switch the sensor LPS22HB into sleep mode

```

1000 /**
    * @file LPS22HBSleep.ino
1002 *
    * @brief Sketch to switch the sensor LPS22HB into sleep mode
1004 *
    * @details Sketch for the Arduino Nano 33 BLE Sense to switch the
           sensor LPS22HB into sleep mode
1006 *
    * @author Elmar Wings
1008 *
    * @version 1.0
1010 */
1012 #include <Wire.h>
#include <LPS22HB.h>
1014
1016 /**
    * @brief LPS22HB sensor object
    * @details This object represents the LPS22HB sensor and provides
           methods for reading temperature and pressure values.
1018 */
LPS22HB lps22hb;
1020
1022 /**
    * @brief Setup function
    *
1024 * @details This function is called once at the beginning of the program
           and is used to
    * - Initialize I2C communication
1026 * - initialize the sensor, and
    * - initialize serial communication.
1028 */
void setup() {
1030 // initialize serial communication
    Serial.begin(9600);
1032
    //Initialize I2C communication
1034 Wire.begin();

```

```

1036 // This line initializes the LPS22HB sensor and sets it up for use.
1037 lps22hb.begin();
1038 }
1039
1040 /**
1041  * @brief Loop function
1042  *
1043  * @details This function is called repeatedly after the setup function
1044  *          and is used to switch the sensor into sleep mode and wake it up.
1045  */
1046 void loop() {
1047     // Switch the sensor into sleep mode
1048     lps22hb.sleep();
1049
1050     // Wait for 1 second
1051     delay(1000);
1052
1053     // Switch the sensor out of sleep mode
1054     lps22hb.wakeUp();
1055
1056     // Wait for 1 second
1057     delay(1000);
1058 }

```

../Code/Nano33BLESense/SensorLPS22HB/LPS22HBSleep.ino

This sketch 8.3 uses the function `sleep()` to switch the sensor into sleep mode, and the function `wakeUp()` to switch the sensor out of sleep mode. The function `sleep()` puts the sensor into a low-power state, and the function `wakeUp()` restores the sensor to its normal operating state.

Note that the function `sleep()` must be called before the sensor can be put into sleep mode, and the function `wakeUp()` must be called before the sensor can be restored to its normal operating state.

Also, note that the function `sleep()` can only be called when the sensor is in a valid state, and the function `wakeUp()` can only be called when the sensor is in a valid state.

You can also use the function `setMode()` to set the sensor to sleep mode, and the function `getMode()` to get the current mode of the sensor.

```

lps22hb.setMode(LPS22HB_MODE_SLEEP);
lps22hb.getMode();

```

This will set the sensor to sleep mode and get the current mode of the sensor.

You can also use the function `setPowerMode()` to set the power mode of the sensor, and the function `getPowerMode()` to get the current power mode of the sensor.

```

lps22hb.setPowerMode(LPS22HB_POWER_MODE_SLEEP);
lps22hb.getPowerMode();

```

This will set the power mode of the sensor to sleep mode and get the current power mode of the sensor.

8.8. Simple Application

8.9. Tests

8.9.1. Simple Function Test

8.9.2. Test all Functions

8.10. Simple Application

8.11. Further Readings

9. Sensormodul APDS-9960 for Gesture, Proximity, and Color Detection

The APDS-9960 sensor, integrated on the Arduino Nano 33 BLE Sense, is a multifunctional optical sensor used for gesture recognition, ambient light detection, color sensing, and proximity measurement. The sensor uses an I²C interface for communication and comes equipped with an additional infrared LED. [Ava15]

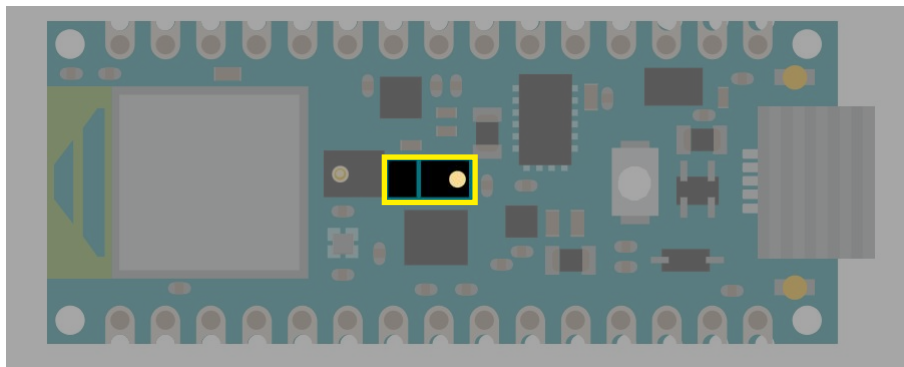


Figure 9.1.: Arduino Nano 33 BLE Sense's APDS-9960

The sensor is located centrally on the Arduino Nano 33 BLE Sense, as indicated in the Figure 9.1.

WS:Figure 6.1 must be converted into a TikZ image to maintain consistency

9.1. Functionality of the APDS-9960

The APDS-9960 is a versatile optical sensor that offers multiple functionalities, including color detection, proximity sensing, ambient light measurement, and gesture recognition. Each of these features plays a crucial role in various applications, ranging from smart lighting adjustments to touchless control systems.

In the following sections, each function of the APDS-9960 will be explored in detail, explaining its working principle, the type of data it provides, and its potential use cases.

9.2. Key Technical Specifications

Power Supply

- Supply Voltage: 2.4 V to 3.6 V
- Maximum Voltage: 3.8 V

Power Consumption

- Active ALS Mode (Ambient Light Sensor): 200-250 μ A
- Proximity and Gesture Modes: 790 μ A (without LED)
- Sleep Mode: 1-10 μ A

Temperature

- Operating Temperature Range: -30°C to +85°C
- Storage Temperature Range: -40°C to +85°C

Optical Properties

- LED Wavelength (max.): 950 nm
- LED Drive Current:
 - 100 mA (Standard), 50 mA, 25 mA, 12.5 mA
 - LED Boost Option: 100%, 150%, 200%, 300% (adjustable current boost)
- Max Detection Distance for Color, Proximity and Gesture Sensor 0mm to 100mm

Proximity Sensor

- Pulse Width for Proximity Measurement: 4 μ s to 32 μ s

Gesture Sensor

- LED Pulse Count: 1 to 64 pulses

Connections

- I²C Communication:
 - Data Rates: Up to 400 kHz
 - Pins:
 - * SDA (I²C Data)
 - * SCL (I²C Clock)
 - * INT (Interrupt, Open Drain, Active Low)
 - * LDR (LED Driver Input)

Dimensions

- Length: 3.94mm
- Width: 2.36mm
- Height: 1.35mm

9.3. Library `Arduino_APDS9960`

In the case of the APDS-9960 sensor, Arduino provides a library `Arduino_APDS9960` for calling functions related to color detection, distance measurement, or gesture recognition.

The library for the sensor is `Arduino_APDS9960`. This allows measuring gestures, colors, light intensity, and distances with the sensor. Communication between the Arduino Nano 33 BLE Sense's chip and the APDS-9960 module occurs via an internal I²C interface [Ava15].

The library is included by using the command `#include <Arduino_APDS9960.h>`.

9.3.1. Installation of APDS-9960 Library

The library is installed as follows:

1. Open the Arduino IDE and navigate to `Tools > Manage Libraries...`.
2. In the Library Manager, use the search bar to look for "Arduino_APDS9960".
3. Several libraries will be displayed. Select the library titled "Arduino_APDS9960" by Arduino and click *Install*.
4. Once installed, the IDE will show a message in the console confirming the installation. The "Install" button will also change to "Remove", as seen in Figure 9.2, indicating the library is ready for use.

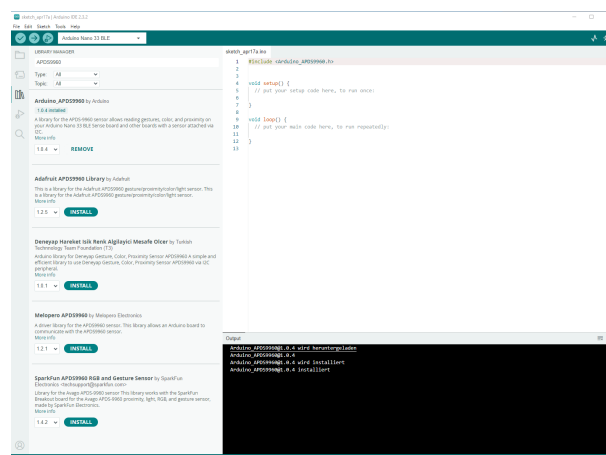


Figure 9.2.: APDS-9960 Library Installation

9.3.2. Functions

After the theoretical overview of the APDS-9960 sensor's functions, the following section will explain the practical implementations. It will demonstrate how the various functions of the sensor, such as color detection, gesture recognition, and proximity sensing, can be applied in practice to activate and evaluate the sensor's capabilities.

Here are the codes to utilize the various functions of the APDS-9960 sensor, such as color detection, gesture recognition, and proximity sensing, see [Ard24]:

- `begin()`: The `begin()` method activates and initializes the APDS9960 sensor. It is typically called during the `setup()` phase.
 - Return value `TRUE`: Initialization was successful.

- Return value `FALSE`: Initialization failed.
- `end()`: The `end()` method deactivates the APDS9960 sensor.
- `colorAvailable()`: This method checks if color data is available to be retrieved.
 - Return value `TRUE`: Color data is available.
 - Return value `FALSE`: No color data is available.
- `readColor(...)`: This method retrieves the color values from the sensor. It can either read just the color values or the color values along with the ambient light intensity.

To read only the color values:

```
int r, g, b;  
APDS.readColor(r, g, b);
```

The variables `r`, `g`, and `b` will contain the updated color values. The value range is 0, 1, 2, ..., 255.

To read both the color values and ambient light intensity:

```
int r, g, b, a;  
APDS.readColor(r, g, b, a);
```

The variables `r`, `g`, and `b` will contain the updated color values, while `a` will contain the ambient light intensity. The value range is 0, 1, 2, ..., 255.

- `proximityAvailable()`: This method checks if proximity data is available.
 - Return value `TRUE`: Proximity data is available.
 - Return value `FALSE`: No proximity data is available.
- `readProximity()`: This method reads the proximity value.
 - Return value `int proximity = APDS.readProximity()`: Returns the proximity value.
 - Return value `-1`: Proximity could not be determined.
- `gestureAvailable()`: This method checks if a gesture has been detected.
 - Return value `TRUE`: A gesture has been detected.
 - Return value `FALSE`: No gesture detected.
- `setGestureSensitivity()`: Gesture detection is influenced by lighting conditions, speed, and distance of movement. This function allows you to adjust the sensitivity of gesture recognition. A higher value detects more gestures, but increases the chance of false positives. A lower value reduces the likelihood of detecting gestures.

The valid range is 1 to 100.

The default value is 80.
- `readGesture()`: If a gesture has been detected, it can be retrieved using the `readGesture()` method. The possible return values are:

- `GESTURE_UP`: An upward movement has been detected.
 - `GESTURE_DOWN`: A downward movement has been detected.
 - `GESTURE_LEFT`: A leftward movement has been detected.
 - `GESTURE_RIGHT`: A rightward movement has been detected.
 - `GESTURE_NONE`: No gesture has been detected.
- `setInterruptPin()`: This method sets the pin used to trigger a measurement. The pin is usually detected automatically, but can be set manually using this function.
If `-1` is passed, no pin is connected.
If `0` or a higher value is passed, that pin will be used.
The default value depends on the board.
 - `setLEDBoost(...)`: The sensor includes an infrared LED that can be temporarily boosted to provide higher brightness. It can be set to provide up to 300% of its normal power. This can be configured using this method.

Usage:

- `0`: Boost to 100% (default power).
- `1`: Boost to 150%.
- `2`: Boost to 200%.
- `3`: Boost to 300%.

Return values:

- `0`: Failure.
- `1`: Success.

9.4. Simple Function Tests

The following code examples can be used to test and try out all functions of the APDS-9960.

9.4.1. Example Color Detection

The library `Arduino_APDS9960` includes an example code demonstrating how to perform color detection with the sensor APDS-9960 on an Arduino Nano 33 BLE Sense. This example also tests the ambient light sensor, as both functionalities rely on the same underlying technology. The measured color channel intensities (Red, Green, and Blue) are displayed in the Serial Monitor for real-time observation.

The sensor APDS-9960 enables accurate RGB color detection by measuring the intensity of red, green, and blue light reflected from an object. These values can be visualized in the Serial Plotter, allowing users to track changes in color intensity over time through a graphical representation. This feature is particularly useful for applications that require real-time color monitoring or dynamic light adjustments.

9.4.2. Example Color Detection - Manual

WS:to do 9.4.3. Setting Up the Color Sensor Measurement Program

To begin the process of creating the color sensor measurement program, open the Arduino Cloud Editor and navigate to the "Libraries" tab. Search for the "Arduino-APDS9960" library and download it. Afterward, click on "More info" to access the GitHub repository, where several examples are provided.

Next, connect the Arduino Nano 33 BLE Sense to the computer, ensuring that the Cloud Editor detects both the board and the corresponding port. If the board is not automatically recognized, follow the instructions to install the necessary plugin to enable the editor to detect the board. Confirm that the correct port is selected. Finally, upload the program to the Arduino, and by opening the serial monitor, the measured values will be recorded in the Arduino IDE.

9.4.4. Example

Color Detection - Code

First, serial communication is started with `Serial.begin(9600)` to send data to the computer. The command `APDS.begin()` initializes the sensor and in the event of a potential error, an error message is output via the serial interface.

The code operates within the function `loop()` of a sketch to continuously read and display color values from the APDS-9960 sensor. Initially, it checks if a color reading is available by using the method `APDS.colorAvailable()`. If no reading is available, the program briefly pauses for 5 milliseconds to avoid excessive CPU usage. Once a reading is ready, the method `APDS.readColor(r, g, b)` retrieves the color data and stores the red, green, and blue values in the respective variables. These values are then printed to the Serial Monitor, labeled clearly as "r", "g", and "b". A one-second delay is added before the program repeats the loop, allowing for clear intervals between consecutive readings.

9.4.5. Example Color Detection - File

The program can be found at

Listing 9.1.: Simple sketch using the sensor APDS9960 for colors

```

1000  /*
1001     APDS-9960 - Color Sensor
1002
1003     This example reads color data from the on-board APDS-9960 sensor of
1004     the
1005     Nano 33 BLE Sense and prints the color RGB (red, green, blue) values
1006     to the Serial Monitor once a second.
1007
1008     The circuit:
1009     - Arduino Nano 33 BLE Sense
1010
1011     This example code is in the public domain.
1012  */
1013  #include <Arduino_APDS9960.h>
1014
1015  void setup() {
1016     Serial.begin(9600);
1017     while (!Serial);
1018
1019     if (!APDS.begin()) {

```

```

1020     Serial.println("Error initializing APDS-9960 sensor.");
1021   }
1022 }

1024 void loop() {
1025   // check if a color reading is available
1026   while (! APDS.colorAvailable()) {
1027     delay(5);
1028   }
1029   int r, g, b;
1030
1031   // read the color
1032   APDS.readColor(r, g, b);
1033
1034   // print the values
1035   Serial.print("r = ");
1036   Serial.println(r);
1037   Serial.print("g = ");
1038   Serial.println(g);
1039   Serial.print("b = ");
1040   Serial.println(b);
1041   Serial.println();
1042
1043   // wait a bit before reading again
1044   delay(1000);
1045 }

```

../Code/Nano33BLESense/APDS9960/TestAPDS9960Color/TestAPDS9960Color.ino

9.4.6. Proximity

A simple code for testing the proximity sensor is also provided by the library. The proximity sensor is designed for measuring distances of up to 100mm.

9.4.7. Proximity - Manual

Open the Arduino Cloud Editor and navigate to the Libraries tab. In the search bar, search for the library [Arduino_APDS9960](#). Once located, open the Examples section within the library and select the sketch “ProximitySensor”. Connect the Arduino Nano 33 BLE Sense to the computer. Check that the Cloud Editor recognizes the board by verifying if both the board and port are listed in the dropdown menu.

9.4.8. Example Proximity - Code

First, serial communication is started with `Serial.begin(9600)` to send data to the computer. The method `APDS.begin()` initializes the sensor and in the event of a potential error, an error message is output via the serial interface.

The code operates within the function `loop()` of a sketch to continuously read and display proximity values from the sensor APDS-9960. It begins by checking if a proximity reading is available using the method `APDS.proximityAvailable()`. If a reading is ready, the method `APDS.readProximity()` retrieves the proximity value, where `0` indicates a close object, `255` indicates a distant object, and `-1` signals an error in reading. This proximity value is then printed to the Serial Monitor. A delay of 100 milliseconds is added before the program repeats the loop, ensuring a brief interval between consecutive readings.

9.4.9. Example Proximity - File

The sketch can be found at

Listing 9.2.: Simple sketch using the sensor APDS9960 for measuring the proximity

```

1000  /*
1001     APDS-9960 - Proximity Sensor
1002
1003     This example reads proximity data from the on-board APDS-9960 sensor
1004     of the
1005     Nano 33 BLE Sense and prints the proximity value to the Serial Monitor
1006     every 100 ms.
1007
1008     The circuit:
1009     - Arduino Nano 33 BLE Sense
1010
1011     This example code is in the public domain.
1012  */
1013  #include <Arduino_APDS9960.h>
1014
1015  void setup() {
1016     Serial.begin(9600);
1017     while (!Serial);
1018
1019     if (!APDS.begin()) {
1020         Serial.println("Error initializing APDS-9960 sensor!");
1021     }
1022  }
1023
1024  void loop() {
1025     // check if a proximity reading is available
1026     if (APDS.proximityAvailable()) {
1027         // read the proximity
1028         // - 0  => close
1029         // - 255 => far
1030         // - -1 => error
1031         int proximity = APDS.readProximity();
1032
1033         // print value to the Serial Monitor
1034         Serial.println(proximity);
1035     }
1036
1037     // wait a bit before reading again
1038     delay(100);
1039  }

```

../Code/Nano33BLESense/APDS9960/TestAPDS9960Proximity/TestAPDS9960Proximity.ino

9.4.10. Gesture Detection

9.4.11. Example Gesture Detection

The library [Arduino_APDS9960](#) also provides an example sketch demonstrating gesture detection with the sensor APDS-9960 on an Arduino Nano 33 BLE Sense. Some LED feedback signals have been programmed to provide quick visual feedback.

9.4.12. Example Gesture Detection - Manual

WS:to do 9.4.13. Example Gesture Detection - Code

In this example sketch, the library [Arduino_APDS9960](#) is integrated first and then the setup is created. In addition, the LED pins are configured so that they behave as outputs.

After that, serial communication is initialized, and the program verifies whether the sensor is correctly initialized using `APDS.begin()`. If initialization fails, an error message is printed. The gesture sensitivity can be adjusted via `APDS.setGestureSensitivity(value)`, where values range from 1 to 100, affecting detection accuracy and sensitivity. The sketch sets a default sensitivity and signals the start of gesture detection by turning off the RGB LEDs with `digitalWrite(LED_R, HIGH)`, `digitalWrite(LED_G, HIGH)`, and `digitalWrite(LED_B, HIGH)`.

In the function `loop()`, the code continuously checks for available gestures using `APDS.gestureAvailable()`. When a gesture is detected, it is read using `APDS.readGesture()` and interpreted within a `switch` statement. Each gesture corresponds to specific LED behavior:

- `GESTURE_UP`: The red LED is briefly turned on, then off after a 1-second delay.
- `GESTURE_DOWN`: The green LED is briefly turned on, then off after a 1-second delay.
- `GESTURE_LEFT`: The blue LED is briefly turned on, then off after a 1-second delay.
- `GESTURE_RIGHT`: All LEDs are turned on simultaneously, then off after a 1-second delay.

The `default` case ensures no action if an undefined gesture is detected.

9.4.14. Example Gesture Detection - File

The sketch can be found at

Listing 9.3.: Simple sketch using the sensor APDS9960 for gesture detection

```

1000  /*
1001     APDS9960 – Gesture Sensor
1002     This example reads gesture data from the on-board APDS9960 sensor of
1003     the
1004     Nano 33 BLE Sense and prints any detected gestures to the Serial
1005     Monitor.
1006     Gesture directions are as follows:
1007     – UP:    from USB connector towards antenna
1008     – DOWN:  from antenna towards USB connector
1009     – LEFT:  from analog pins side towards digital pins side
1010     – RIGHT: from digital pins side towards analog pins side
1011     The circuit:
1012     – Arduino Nano 33 BLE Sense
1013     This example code is in the public domain.
1014  */
1015
1016  #include <Arduino_APDS9960.h>
1017
1018  void setup() {
1019     Serial.begin(9600);
1020     //Red
1021     pinMode(LED_R, OUTPUT);
1022     //Green
1023     pinMode(LED_G, OUTPUT);
1024     //Blue
1025     pinMode(LED_B, OUTPUT);
1026
1027     while (!Serial);
1028     if (!APDS.begin()) {
1029         Serial.println("Error initializing APDS9960 sensor!");
1030     }
1031 }

```

```

// for setGestureSensitivity(..) a value between 1 and 100 is required
// Higher values makes the gesture recognition more sensible but less
// accurate
// (a wrong gesture may be detected). Lower values makes the gesture
// recognition
// more accurate but less sensible (some gestures may be missed).
// Default is 80
//APDS.setGestureSensitivity(80);
Serial.println("Detecting gestures ...");
// Turning OFF the RGB LEDs
digitalWrite(LED_R, HIGH);
digitalWrite(LED_G, HIGH);
digitalWrite(LED_B, HIGH);
}
void loop() {
  if (APDS.gestureAvailable()) {
    // a gesture was detected, read and print to serial monitor
    int gesture = APDS.readGesture();
    switch (gesture) {
      case GESTURE_UP:
        Serial.println("Detected UP gesture");
        digitalWrite(LED_R, LOW);
        delay(1000);
        digitalWrite(LED_R, HIGH);
        break;
      case GESTURE_DOWN:
        Serial.println("Detected DOWN gesture");
        digitalWrite(LED_G, LOW);
        delay(1000);
        digitalWrite(LED_G, HIGH);
        break;
      case GESTURE_LEFT:
        Serial.println("Detected LEFT gesture");
        digitalWrite(LED_B, LOW);
        delay(1000);
        digitalWrite(LED_B, HIGH);
        break;
      case GESTURE_RIGHT:
        Serial.println("Detected RIGHT gesture");
        digitalWrite(LED_B, LOW);
        digitalWrite(LED_R, LOW);
        digitalWrite(LED_G, LOW);
        delay(1000);
        digitalWrite(LED_B, HIGH);
        digitalWrite(LED_G, HIGH);
        digitalWrite(LED_R, HIGH);
        break;
      default:
        break;
    }
  }
}

```

../Code/Nano33BLESense/APDS9960/TestAPDS9960Gesture/TestAPDS9960Gesture.ino

9.4.15. Troubleshoot

Errors in color detection can be caused by insufficient lighting in the room. Please ensure that the environment is bright enough for the color detection function.

9.5. Calibration Color Detection

9.6. Calibration Color Detection

Due to variations in environmental light conditions and the sensitivity of the sensor, calibration is required to ensure accurate color detection. This section outlines the calibration process using a standard color chart under constant lighting conditions. To carry out the calibration, the Arduino, a connection cable (USB A to USB Micro), and a laptop or computer with the appropriate Arduino development environment are required.

The proximity and gesture feature is pre-configured and factory-calibrated to detect proximity and gesture at a distance of 100mm, eliminating the need for customer calibration. [Bro24]

9.6.1. Calibration Setup

A standardized color chart is used, which provides reference values for perfect red, green, and blue under constant lighting conditions. The RGB values from the sensor are read as raw, uncalibrated data, which must be normalized to a standard range (0–255) for accurate color detection.

WS:citation, picture

9.6.2. Step 1: Measuring Reference Values

The first step in the calibration process is to measure the raw sensor values for each primary color (red, green, and blue) using the standardized color chart. By placing the color chart in front of the sensor and reading the raw RGB values, we can establish the maximum possible sensor readings for each color channel.

$$\text{RawRed} = \max(R_{\text{measured}})$$

$$\text{RawGreen} = \max(G_{\text{measured}})$$

$$\text{RawBlue} = \max(B_{\text{measured}})$$

These maximum values are obtained by positioning the chart at a fixed distance of one centimeter and ensuring constant light intensity.

9.6.3. Step 2: Normalizing the Sensor Values

Once the maximum sensor readings for red, green, and blue are obtained, the raw sensor data is normalized to a 0-255 scale. This ensures that the sensor's readings correspond to standard RGB values. The following formula is used for normalization:

$$R_{\text{calibrated}} = \frac{R_{\text{raw}}}{\text{RawRed}} \times 255$$

$$G_{\text{calibrated}} = \frac{G_{\text{raw}}}{\text{RawGreen}} \times 255$$

$$B_{\text{calibrated}} = \frac{B_{\text{raw}}}{\text{RawBlue}} \times 255$$

Where:

- $R_{\text{raw}}, G_{\text{raw}}, B_{\text{raw}}$ are the raw sensor values for each channel.
- RawRed, RawGreen, RawBlue are the maximum measured values from the standard color chart.

9.6.4. Step 3: Implementing the Calibration in Code

The normalization process is implemented directly in the Arduino code to adjust the sensor's readings. To begin calibration, open the Serial Monitor, enter "OK" in the command line, and press Enter to confirm. The following steps will then be explained through text output.

After the user initiates the calibration by typing "OK" into the Serial Monitor, the sketch starts. The sketch guides the user through the calibration of red, green, and blue colors, with each phase confirmed by entering "OK". The pins for the RGB LEDs are defined, and variables store the maximum calibration values for each color. Status variables control the sequence, ensuring each phase is completed before moving to the next. These maximum values are later used for accurate color detection in the Application code.

The following code section begins by initializing serial communication at a baud rate of 9600, enabling interaction through the serial monitor. Next, it sets the RGB LED pins as outputs and turns on all LEDs to create white light, which helps capture accurate color measurements. The code then initializes the sensor APDS-9960, checking for successful setup. If the sensor fails to initialize, an error message is printed, and the program halts. If successful, a message confirms initialization and prompts the user to type "OK" in the serial monitor to proceed with the calibration process explanation.

The next code segment in the loop first checks if the user has entered "OK" to start the calibration process. After confirmation, it displays an explanation and instructions. After showing the explanation, it waits for the user to confirm by typing "OK" again, which then initiates the red color calibration by prompting the user to place the red chart in front of the sensor. The loop pauses at each step until the user confirms.

WS:Screen shot?

Each color calibration starts after the user types "OK" in the Serial Monitor. The sketch measures the highest detected value for each color over 10 seconds and sets it as the calibration maximum. After all colors are calibrated, it displays the final maximum values for red, green, and blue in the Serial Monitor, completing the process. The `maxRed`, `maxGreen`, and `maxBlue` values can later be entered into the application sketch to apply the calibration to the sketch.

9.6.5. Calibration File

The sketch can be found at

Listing 9.4.: APDS-9960: Example Calibration

```

1000 // Code to determine the calibration factors for the application '
      ApplicationAPDS9960.ino '.
      // To start, open the serial monitor and type "OK" into the command line
      , then press Enter.
1002 // After reading the information, always confirm with "OK".

1004 // file: APDS9960Calibration.ino

1006 #include <Arduino_APDS9960.h>

1008 // RGB LED Pins for the Arduino Nano 33 BLE Sense
const int redPin = LEDR;
1010 const int greenPin = LEDG;
const int bluePin = LEDB;
1012 // Maximum RGB values based on calibration measurements
1014 int maxRed = 0;

```



```

1016 int maxGreen = 0;
1017 int maxBlue = 0;

1018 // Calibration state variables
bool initialOKConfirmed = false; // Confirmation to start
    explanation
1020 bool explanationShown = false; // Flag to show calibration
    explanation
bool explanationConfirmed = false; // Confirmation that explanation
    was read
1022 bool redCalibrated = false; // Flag for completed red
    calibration
bool greenCalibrated = false; // Flag for completed green
    calibration
1024 bool blueCalibrated = false; // Flag for completed blue
    calibration
bool readyForRed = false; // Flag to begin red calibration
1026 bool readyForGreen = false; // Flag to begin green calibration
bool readyForBlue = false; // Flag to begin blue calibration

1028 void setup() {
1030     Serial.begin(9600);

1032     // Set up RGB LED pins as outputs
pinMode(redPin, OUTPUT);
1034 pinMode(greenPin, OUTPUT);
pinMode(bluePin, OUTPUT);
1036
1038     // Turn on all LEDs (for white light) to capture accurate color
digitalWrite(redPin, LOW);
digitalWrite(greenPin, LOW);
1040 digitalWrite(bluePin, LOW);

1042     // Initialize the APDS-9960 sensor and check for success
if (!APDS.begin()) {
1044     Serial.println("Error initializing the APDS-9960 sensor!");
while (1); // Stop program if sensor fails to initialize
1046 }

1048     Serial.println("Sensor successfully initialized.");
Serial.println("Type 'OK' to proceed to the explanation of the
    calibration process.");
1050 }

1052 void loop() {
    int r = 0, g = 0, b = 0, a = 0; // Variables to hold RGB and ambient
    values
1054
1056     // Initial confirmation to proceed to calibration explanation
if (!initialOKConfirmed) {
1058     if (Serial.available() > 0) {
String input = Serial.readStringUntil('\n');
input.trim(); // Remove whitespace
1060         if (input.equalsIgnoreCase("OK")) {
initialOKConfirmed = true;
1062             explanationShown = true;
Serial.println("Explanation:");
1064             Serial.println("Welcome to the calibration process.");
Serial.println("You will be prompted to calibrate with the red,
green, and blue color charts.");
1066             Serial.println("Follow the instructions and confirm each step by
typing 'OK'.");
Serial.println("Type 'OK' to begin the calibration process.");
1068         }
    }
1070     return; // Exit loop until 'OK' is entered initially
}

1072     // Wait for user confirmation after showing explanation

```

```

1074 if (explanationShown && !explanationConfirmed) {
1075     if (Serial.available() > 0) {
1076         String input = Serial.readStringUntil('\n');
1077         input.trim(); // Remove whitespace
1078         if (input.equalsIgnoreCase("OK")) {
1079             explanationConfirmed = true;
1080             readyForRed = true;
1081             Serial.println("Explanation confirmed.");
1082             Serial.println("Place the red color chart in front of the sensor
1083                 and type 'OK' when ready.");
1084         }
1085     }
1086     return; // Exit loop until explanation is confirmed
1087 }
1088
1089 // Red calibration step
1090 if (readyForRed && !redCalibrated) {
1091     if (Serial.available() > 0) {
1092         String input = Serial.readStringUntil('\n');
1093         input.trim(); // Remove whitespace
1094         if (input.equalsIgnoreCase("OK")) {
1095             readyForRed = false;
1096             Serial.println("Measuring red value for 10 seconds...");
1097
1098             unsigned long startTime = millis(); // Start time for 10-second
1099             calibration
1100             int maxMeasuredRed = 0;
1101
1102             // Measure red value for 10 seconds
1103             while (millis() - startTime < 10000) {
1104                 if (APDS.colorAvailable()) {
1105                     APDS.readColor(r, g, b, a);
1106                     if (r > maxMeasuredRed) {
1107                         maxMeasuredRed = r; // Update max red value if current is
1108                         higher
1109                     }
1110                 }
1111                 delay(100); // Avoid sensor read flooding
1112             }
1113
1114             maxRed = maxMeasuredRed; // Set max red after calibration
1115             redCalibrated = true;
1116             readyForGreen = true;
1117             Serial.println("Red calibration completed.");
1118             Serial.println("Place the green color chart in front of the
1119                 sensor and type 'OK' when ready.");
1120         }
1121     }
1122     return; // Exit loop until red calibration is confirmed
1123 }
1124
1125 // Green calibration step
1126 if (readyForGreen && !greenCalibrated) {
1127     if (Serial.available() > 0) {
1128         String input = Serial.readStringUntil('\n');
1129         input.trim(); // Remove whitespace
1130         if (input.equalsIgnoreCase("OK")) {
1131             readyForGreen = false;
1132             Serial.println("Measuring green value for 10 seconds...");
1133
1134             unsigned long startTime = millis(); // Start time for 10-second
1135             calibration
1136             int maxMeasuredGreen = 0;
1137
1138             // Measure green value for 10 seconds
1139             while (millis() - startTime < 10000) {
1140                 if (APDS.colorAvailable()) {
1141                     APDS.readColor(r, g, b, a);
1142                     if (g > maxMeasuredGreen) {

```

```

1138         maxMeasuredGreen = g; // Update max green if current is
higher
    }
1140     }
    delay(100); // Avoid sensor read flooding
1142 }

    maxGreen = maxMeasuredGreen; // Set max green after calibration
    greenCalibrated = true;
1146     readyForBlue = true;
    Serial.println("Green calibration completed.");
1148     Serial.println("Place the blue color chart in front of the
sensor and type 'OK' when ready.");
    }
1150 }
    return; // Exit loop until green calibration is confirmed
1152 }

// Blue calibration step
if (readyForBlue && !blueCalibrated) {
1156     if (Serial.available() > 0) {
        String input = Serial.readStringUntil('\n');
1158         input.trim(); // Remove whitespace
        if (input.equalsIgnoreCase("OK")) {
1160             readyForBlue = false;
            Serial.println("Measuring blue value for 10 seconds...");

1162             unsigned long startTime = millis(); // Start time for 10-second
calibration
            int maxMeasuredBlue = 0;

1166             // Measure blue value for 10 seconds
            while (millis() - startTime < 10000) {
1168                 if (APDS.colorAvailable()) {
                    APDS.readColor(r, g, b, a);
1170                     if (b > maxMeasuredBlue) {
higher
                        maxMeasuredBlue = b; // Update max blue if current is
1172                     }
                    }
1174                 delay(100); // Avoid sensor read flooding
            }

1176             maxBlue = maxMeasuredBlue; // Set max blue after calibration
            blueCalibrated = true;
1178             Serial.println("Blue calibration completed.");
            Serial.println("Calibration complete.");

1182             // Display final calibration results
            Serial.println("Calibration results:");
1184             Serial.print("maxRed = ");
            Serial.println(maxRed);
1186             Serial.print("maxGreen = ");
            Serial.println(maxGreen);
1188             Serial.print("maxBlue = ");
            Serial.println(maxBlue);
1190         }
    }
1192     return; // Exit loop until blue calibration is confirmed
1194 }

```

../Code/Nano33BLESense/APDS9960/APDS9960Calibration/APDS9960Calibration.ino

9.6.6. Step 4: Validating the Calibration

To ensure the calibration is successful, the sensor readings should be tested again using the standard color chart. The calibrated values for red, green, and blue should be close to 255 for the respective pure colors on the chart. If the readings deviate significantly, further adjustment of the maximum reference values may be necessary.

The calibration of the sensor APDS-9960 is essential for accurate RGB detection. By using a standard color chart and constant lighting conditions, the sensor's raw values can be normalized to provide consistent and reliable color readings. This process can be further refined by adjusting for specific environmental factors or sensor placement.

9.7. Tests

9.7.1. Simple Function Test

9.7.2. Test all Functions

9.8. Simple Application

Similarly, by following all the steps for uploading and compiling the sketch we can see the results of sensor APDS-9960 on Serial Monitor, too. For seeing the different output, we can change the input for the sensor too, e.g: for color detection we can switch the colors, for gesture detection we can also switch the gestures, and for proximity also do the same. The resulted output as shown in the figure. 9.3

WS:rewrite and extend

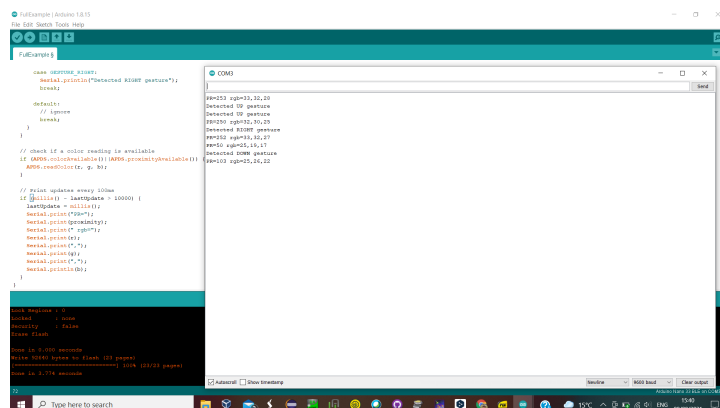


Figure 9.3.: Gesture, Proximity, Color Sensor Output Window

We can also run the single functionality of this sensor too, e.g; if we just need to capture the color of product, we can also run the color detection program. It depends upon the application and we can implement our application and modify the code as per our desire results.

Listing 9.5.: Simple sketch using the sensor APDS9960

```

1000 /**
1001  * @file TestAPDS9960.ino
1002  *
1003  * @brief Arduino sketch for color sensor APDS9960 and SSD1306 OLED
1004  *       display
1005  *
1006  * @author Elmar Wings
1007  * @version 1.0
1008  */

```

```

1008 // Library for sensor APDS9960
1010 #include <Arduino_APDS9960.h>

1012 // Library SSD1306 for text-only output and connection
// with cable via the I2C protocol
1014 #include <SSD1306AsciiWire.h>

1016 #define I2C_ADDRESS 0x3C /**< Enter the address of the OLED */
1018
1020 SSD1306AsciiWire oled; /**< Set the name of the OLED */

1022 // Initialization of the measurement button
const int TASTER_PIN = 11; /**< Pin number for measurement button */
1024
// Initialization of the LED pin
1026 const int LED_PIN = 12; /**< Pin number for LED */

1028 /**
 * @brief Setup function for Arduino
 */
1030 void setup() {
1032     Serial.begin(9600); /**< Initialize serial communication at 9600 baud
 */

1034     // Define pin as INPUT and activate internal pull-up resistor
pinMode(TASTER_PIN, INPUT);
1036     // Define pin as OUTPUT
pinMode(LED_PIN, OUTPUT);
1038
1040     /**
 * @brief Initialize APDS9960 sensor
 */
1042     if (!APDS.begin()) {
Serial.println("Error initializing APDS9960 sensor.");
1044     }

1046     // Arduino is hereby registered in the I2C bus,
// as it is to be registered as a master,
1048     // the address SEARCH SOURCE!!!! is omitted.
Wire.begin();
1050     // Clock frequency in Hertz,
// Standard: 100,000
1052     // Fast mode: 400,000
Wire.setClock(400000L);
1054     // first screen size, then reference to I2C address
oled.begin(&Adafruit128x64, I2C_ADDRESS);
1056     oled.setFont(System5x7);
oled.setCursor(0, 40);
1058     oled.println("INITIALISIERUNG!\n");

1060     /**
 * @brief Flash LED three times during initialization
 */
1062     for (int zaehler = 1; zaehler < 4; zaehler = zaehler + 1) {
1064         delay(2000);
digitalWrite(LED_PIN, HIGH);
1066         delay(200);
digitalWrite(LED_PIN, LOW);
1068         delay(200);
}
1070     oled.clear();
}

1072 /**
 * @brief Main loop function for Arduino
 */

```

```

1076 void loop() {
1077     oled.println("READY!");
1078
1079     /**
1080      * @brief Check whether color detection is available
1081      */
1082     while (!APDS.colorAvailable()) {
1083         delay(5);
1084     }
1085
1086     /**
1087      * @brief Check whether distance measurement is available
1088      */
1089     while (!APDS.proximityAvailable()) {
1090         delay(5);
1091     }
1092
1093     /**
1094      * @brief Create integer variables to save
1095      *         the measured values for red, green and blue
1096      */
1097     int r, g, b, tasterCount{ 0 };
1098
1099     /**
1100      * @brief Status query of the measurement button
1101      */
1102     bool zustandTaster = digitalRead(TASTER_PIN);
1103
1104     if (zustandTaster == 0) {
1105         tasterCount = 1;
1106     }
1107
1108     /**
1109      * @brief If clause switch to measuring mode when the button is
1110      *         pressed
1111      */
1112     if (tasterCount == 1) {
1113         int proximity = APDS.readProximity();
1114
1115         if (proximity != -1) {
1116             Serial.print("Abstand: ");
1117             Serial.println(proximity);
1118
1119             if (proximity < 20) {
1120                 oled.clear();
1121                 digitalWrite(LED_PIN, HIGH);
1122
1123                 /**
1124                  * @brief 3x Color scan and save in the 3 variables r, g and b
1125                  */
1126                 for (int i = 0; i < 3; i++) {
1127                     APDS.readColor(r, g, b);
1128                     delay(1500);
1129                 }
1130
1131                 /**
1132                  * @brief Display largest value via integrated RGB LED,
1133                  *         option for troubleshooting
1134                  */
1135                 if (r > g & r > b) {
1136                     digitalWrite(LED_R, LOW);
1137                     digitalWrite(LED_G, HIGH);
1138                     digitalWrite(LED_B, HIGH);
1139                 } else if (g > r & g > b) {
1140                     digitalWrite(LED_R, LOW);
1141                     digitalWrite(LED_G, HIGH);
1142                     digitalWrite(LED_B, HIGH);
1143                 } else if (b > g & b > r) {
1144                     digitalWrite(LED_R, LOW);
1145                     digitalWrite(LED_G, LOW);
1146                     digitalWrite(LED_B, HIGH);
1147                 }
1148             }
1149         }
1150     }
1151 }

```

```

1144     digitalWrite(LED_R, HIGH);
1145     digitalWrite(LED_G, HIGH);
1146 } else {
1147     digitalWrite(LED_R, HIGH);
1148     digitalWrite(LED_G, HIGH);
1149     digitalWrite(LED_B, HIGH);
1150 }
1151
1152 /**
1153  * @brief Output values in the serial monitor
1154  *       for testing and error analysis
1155  */
1156 Serial.print("Red light = ");
1157 Serial.println(r);
1158 Serial.print("Green light = ");
1159 Serial.println(g);
1160 Serial.print("Blue light = ");
1161 Serial.println(b);
1162 Serial.println();
1163
1164 /**
1165  * @brief Display color on OLED display
1166  */
1167 if (r > g & r > b) {
1168     oled.println("Das Objekt ist rot!");
1169 } else if (g > r & g > b) {
1170     oled.println("Das Objekt ist gruen!");
1171 } else if (b > g & b > r) {
1172     oled.println("Das Objekt ist blau!");
1173 }
1174
1175 delay(4000);
1176 oled.clear();
1177 } else {
1178     oled.clear();
1179     oled.println("Kein Objekt \nvorhanden!\n");
1180     oled.println("Halten Sie ein\nObjekt vor den \nSensor \noder
1181     veraendern Sie\ndie Position!");
1182     digitalWrite(LED_R, HIGH);
1183     digitalWrite(LED_G, HIGH);
1184     digitalWrite(LED_B, HIGH);
1185     delay(4000);
1186     oled.clear();
1187 }
1188 } else {
1189     oled.clear();
1190     oled.println("Abstandsmessung fehlgeschlagen!");
1191 }
1192 } else {
1193     // Switch off LED if not pressed
1194     digitalWrite(LED_PIN, LOW);
1195
1196     oled.setFont(System5x7);
1197     oled.setCursor(0, 40);
1198     oled.println("READY!");
1199
1200     digitalWrite(LED_R, HIGH);
1201     digitalWrite(LED_G, HIGH);
1202     digitalWrite(LED_B, HIGH);
1203 }
1204
1205 // Set button to 0
1206 zustandTaster = 0;
1207 }

```

../Code/Nano33BLESense/APDS9960/TestAPDS9960/TestAPDS9960.ino

9.9. Further Readings

- SparkFun Electronics, “APDS-9960 RGB and Gesture Sensor Hookup Guide”. A comprehensive guide that explains the functionality of the APDS-9960 and provides step-by-step instructions for implementation. Available at: <https://learn.sparkfun.com/tutorials/apds-9960-rgb-and-gesture-sensor-hookup-guide/all>, [Hym24]
- Maker Guides, “APDS-9960 Gesture and Color Sensor with Arduino”. This tutorial provides a hands-on introduction to using the APDS-9960 sensor with Arduino, including gesture and color recognition. Available at: <https://www.makerguides.com/apds-9960-gesture-and-color-sensor-with-arduino/>, [Mae24]

10. Sensor APDS 9960 Gesture

Introduction

WS:cite books,
applications, board

10.0.1. Sensormodul APDS 9960 for Gesture

Function of the Gesture Detection Sensor

The gesture recognition of the APDS-9960 is based on four additional angled photodiodes

- Up – Hand movement upwards.
- Down – Hand movement downwards.
- Left – Hand movement to the left.
- Right – Hand movement to the right.

which detect reflected infrared light (IR) from the integrated IR-LED. By analyzing the reflection, which measures direction-dependent changes using the photodiodes, the sensor can recognize movements and gestures such as swiping motions in different directions (e.g., up, down, left, right). The sensor converts this information into digital data, such as speed, direction, and distance of the movement. The 32-record FIFO register allows the sensor to temporarily store motion data, ensuring continuous processing. [Ava15]

10.1. General

General description
cite books

10.2. Specific Sensor

cite board

10.3. Specification

- cite data sheet
- Circuit Diagram

10.4. Bibliothek

10.4.1. Description

10.4.2. Installation

10.4.3. Functions

10.4.4. Example - Manual

10.4.5. Example

10.4.6. Example - Code

10.4.7. Example - Files

10.5. Calibration

`cite method`

10.6. Simple Code

10.7. Simple Application

10.8. Tests

10.8.1. Simple Function Test

10.8.2. Test all Functions

10.9. Simple Application

10.10. Further Readings

11. Sensor APDS 9960 Color

The general functionality of a color sensor is based on the reflection of white light from the object to be measured. White light contains wavelengths that cover the entire visible spectrum (p. 189 [Ryb13]). Due to the molecular structure of the object or chemical processes, such as photosynthesis, certain wavelengths are absorbed while others are reflected. The reflected light rays that reach the color sensor are spectrally selected by color filters or a diffraction grating [HS23].

11.1. General

Color filters: By using parallel red, green, and blue color filters, the intensities of the RGB components are evaluated by three photodiodes behind the filters [HS23].

Diffraction grating: As light passes through a diffraction grating, the physical diffraction effect occurs. This effect, due to the varying wavelengths of light, produces an interference pattern that enables the spectral decomposition of the light. The spectrum of the incoming light is spatially dispersed, similar to the refraction in a prism (p. 208 ff. [Ryb13]). The intensity of the color components can be measured by an array of spatially arranged photodiodes. Diffraction gratings offer higher resolution than color filters [HS23].

Photodiodes are semiconductors that generate electrical energy through the internal photoelectric effect (p. 220 [Ryb13]). The signal from the photodiodes is amplified and stored in a data register through analog-to-digital conversion. The measurement is taken over a specific period. By accumulating the RGB data, the measured color can finally be determined [Ava15].

Color sensors are typically used in image processing and quality control applications [Ava15].

As shown in Figure 11.1, the incoming light first passes through UV and IR filters. It is then separated into its red, green, blue, and unfiltered components by color filters. These filtered components are subsequently converted from analog to digital units, as described below.

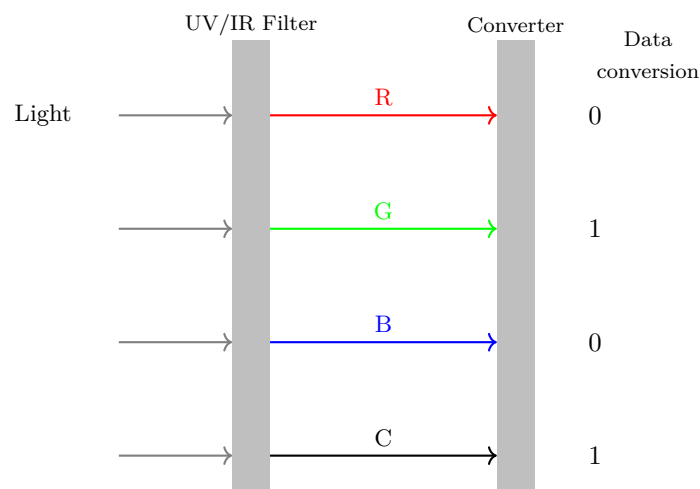


Figure 11.1.: Schematic of the main function of a color sensor

The block diagram 11.2 shows the processing flow of the APDS-9660 from light detection to communication via the I²C protocol with the Arduino. The individual steps are explained as follows:

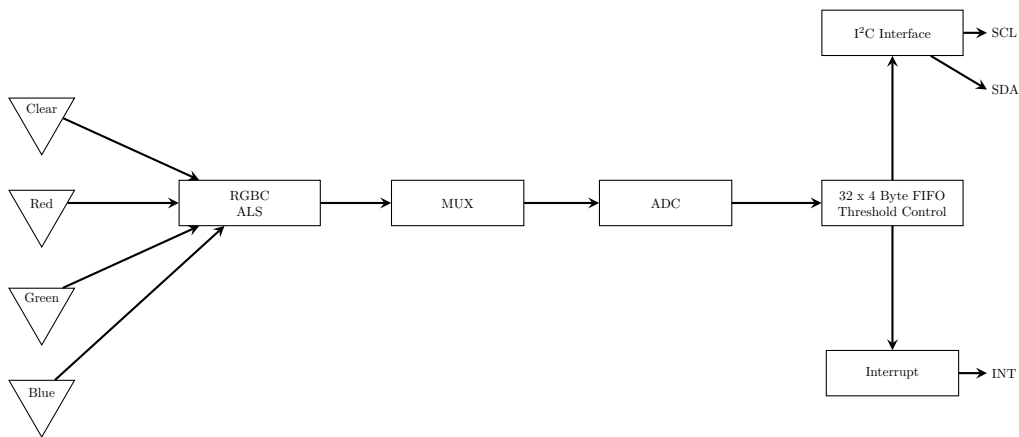


Figure 11.2.: Functional Block Diagram

WS: The graphic is not optimal. Description of the individual blocks is not clear. What does RGBC and ALS mean?

The photodiodes convert the intensities of the color components of the filtered light and the total light into electrical signals. These RGBC (Red, Green, Blue, Clear) and ALS (Ambient Light Sensor) signals are forwarded to the next processing stage using the multiplexer (MUX).

The MUX allows for the sequential processing of the color signals, as the ADC (Analog-to-Digital Converter) can only process one signal at a time. The ADC converts the analog signal selected by the MUX into a digital value. This conversion is necessary to make the signal suitable for digital processing and transmission.

The digital value is stored in a FIFO (First In, First Out) data register, which can store up to 32 data values (4 bytes each). A threshold controller monitors the data and triggers an interrupt when defined limits are reached. The detailed process is explained in Chapter Data Processing 11.1.

The stored data is transmitted to the Arduino via an I²C bus. The I²C bus uses two lines: the SCL (Serial Clock Line) for the clock and the SDA (Serial Data Line) for data transmission. The detailed functioning of the I²C bus is explained in Chapter ?? Protokoll I²C. [Ava15].

Data Processing

The color detection begins at the photodiodes with RGBC detection and ends with 16-bit values stored in the RGBC data registers. The signal from the photodiode array is accumulated over a period defined by the value in the ATIME register (ALS ADC Integration Time) before the results are transferred to the RGBCDATA registers. The gain factor can be set from 1x to 64x, which is controlled via the CONTROL<AGAIN> bit (ALS Gain Control). Performance parameters such as accuracy, resolution, conversion speed, and energy consumption can be adjusted to meet the application requirements.

Between measurements, a customizable, energy-efficient waiting period is maintained, the duration of which is determined by the control bits WEN (Wait Enable), WTIME (Wait Time), and WLONG (Wait Long Enable). The waiting time can range from 0 seconds to a maximum of 8.54 seconds.

An interrupt is triggered when the clear channel values exceed the thresholds defined in the AILTL/AIHTL (ALS low threshold, lower byte/ALS high threshold, lower byte) or AILTH/AIHTH (ALS low threshold, upper byte/ALS high threshold, upper

byte) threshold registers. To avoid unwanted or false interrupts, a persistence filter is integrated, ensuring that an interrupt is triggered only when a defined number of consecutive values fall outside the thresholds. This threshold is set in the APERS register (ALS Interrupt Persistence). If a value is within the thresholds, the persistence counter is reset. If the analog circuit is saturated, the ASAT bit is set, indicating potentially inaccurate RGBCDATA results. The AINT (ALS Interrupt) and CPSAT (Clear Diode Saturation) bits can be queried at any time via I²C. However, for AINT to trigger a hardware interrupt on the INT pin, the AIEN bit (ALS Interrupt Enable) must be set. Saturation of the analog-to-digital converter can be detected via the CPSAT bit; to enable this function, the CPSIEN bit (Clear Diode Saturation Interrupt Enable) must be set. The AVALID bit (ALS Valid) is reset by reading the RGBCDATA. ASAT and AINT can be reset via the CICLEAR (Clear Channel Interrupt Clear) or AICLEAR (All Non-Gesture Interrupt Clear) bits.

The RGBC results can be used to determine the color temperature in Kelvin or the ambient light intensity in Lux.

[Ava15]

11.2. Specific Sensor

cite board

11.3. Specification

- cite data sheet
- Circuit Diagram

11.4. Bibliothek

11.4.1. Description

11.4.2. Installation

11.4.3. Functions

11.4.4. Example - Manual

11.4.5. Example

11.4.6. Example - Code

11.4.7. Example - Files

11.5. Calibration

cite method

11.6. Simple Code**11.7. Simple Application****11.8. Tests****11.8.1. Simple Function Test****11.8.2. Test all Functions****11.9. Simple Application****11.10. Further Readings**

12. Sensor APDS 9960 Proximity

The proximity sensor is part of the gesture detection sensor and detects top-bottom gestures based on the temporal change in the measured distance. The proximity detection measures the distance between the sensor and an object by capturing the reflected IR light. The integrated IR LED emits light, which is measured by the same photodiodes as in gesture detection. The stronger the reflection, the closer the object is. The proximity sensor can also detect events such as the approach or withdrawal of an object and trigger corresponding interrupts when predefined thresholds are exceeded. To ensure reliable measurements, the sensor compensates for unwanted reflections by adjusting offset values and suppresses interference from ambient light. [Ava15]

12.1. General

General description
cite books

12.2. Specific Sensor

cite board

12.3. Specification

- cite data sheet
- Circuit Diagram

12.4. Bibliothek

12.4.1. Description

12.4.2. Installation

12.4.3. Functions

12.4.4. Example - Manual

12.4.5. Example

12.4.6. Example - Code

12.4.7. Example - Files

12.5. Calibration

cite method

12.6. Simple Code**12.7. Simple Application****12.8. Tests****12.8.1. Simple Function Test****12.8.2. Test all Functions****12.9. Simple Application****12.10. Further Readings**

13. Sensor Ambient Light

The ambient light sensor (ALS) of the APDS-9960 works with the same photodiode array and measures the intensity of the ambient light. For this purpose, the UV and IR light filters are installed in front of the photodiodes to ensure accurate measurements of visible light. The sensor converts the measured light intensity into 16-bit data, which enables high resolution and accuracy. A function of programmable gain and customizable integration time ensures that the sensor can operate accurately in different lighting conditions, such as dim or very bright light. Another important feature of the

sensor is its ability to correctly detect the intensity of ambient light even behind dark glass, which is particularly useful for devices with tinted covers. [Ava15]

13.1. General

General description
cite books

13.2. Specific Sensor

cite board

13.3. Specification

- cite data sheet
- Circuit Diagram

13.4. Bibliothek

13.4.1. Description

13.4.2. Installation

13.4.3. Functions

13.4.4. Example - Manual

13.4.5. Example

13.4.6. Example - Code

13.4.7. Example - Files

13.5. Calibration

cite method

13.6. Simple Code**13.7. Simple Application****13.8. Tests****13.8.1. Simple Function Test****13.8.2. Test all Functions****13.9. Simple Application****13.10. Further Readings**

14. Application of the Color Sensor with OLED Display

The color detection machine utilizes an Arduino Nano 33 BLE Sense with an APDS-9960 sensor for reliable recognition of red, green, and blue colors, making it suitable for various industrial automation processes. Every 750 ms, the machine reads the color of an object and displays the result on an OLED screen. Before each color reading, the proximity sensor checks if an object is close to the sensor, if so, the white LED activates to improve sensor accuracy in low-light conditions.

14.1. Manual

The color detection machine consists of the Arduino Nano 33 BLE Sense and a 1.30" IIC OLED display, which need to be connected according to the circuit diagram in Figure 14.1.

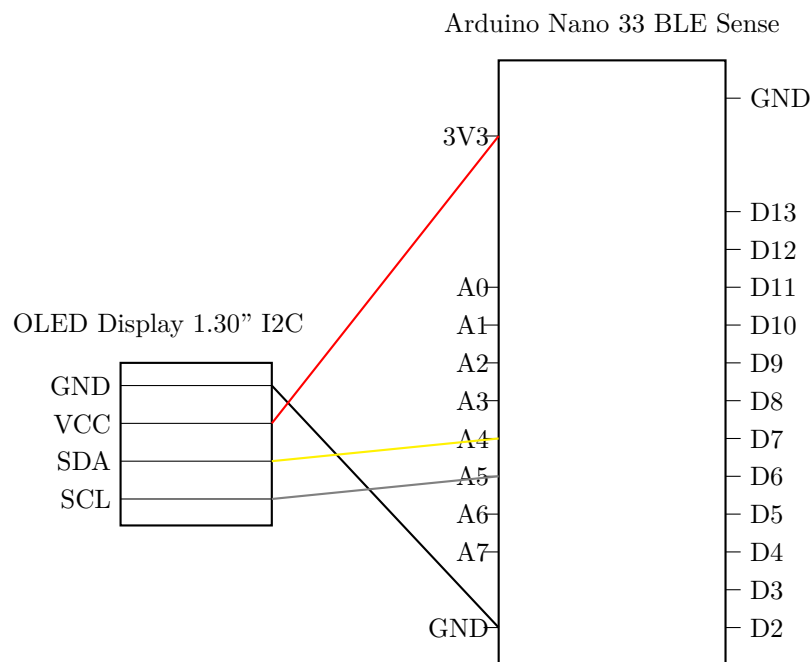


Figure 14.1.: Circuit diagram for connecting the Arduino Nano 33 BLE Sense with a 1.30" IIC OLED display.

Use a laptop or computer to open the `ApplicationAPDS9960` code in the Arduino IDE. Before uploading the code, enter the `maxRed`, `maxGreen`, and `maxBlue` calibration values to ensure accurate color detection by the machine.

Once the Arduino is programmed, the color detection machine can start. Place an object approximately one centimeter in front of the APDS-9960 sensor on the Arduino. The OLED display will show important instructions and display detected colors. Each time an object is removed and replaced, the sensor will read the color in its usual

interval. Ensure proper lighting, and calibrate the sensor under the same conditions in which it will operate.

14.2. Code Explanation

The following section provides an explanation of the application code:

The first part of the code initializes the Arduino Nano 33 BLE Sense with the APDS-9960 sensor and the 1.30" IIC OLED display. It includes pre-determined calibration values for red, green, and blue (maxRed, maxGreen, maxBlue) as integer values. In the `setup()` function, it starts serial communication, sets the LED pin as an output, and initializes the APDS-9960 sensor, printing an error message if the initialization fails. It also sets up the OLED display to show an "INITIALIZING!" message. Afterward, the LED blinks three times to indicate that the setup is complete, and the display is cleared.

Listing 14.1.: Application of the sensor APDS9960: Setup

```

1000 // Code for using the APDS-9960 sensor as a color recognition machine.
1002 // An Arduino Nano 33 BLE Sense and a 1.30" IIC OLED display are
      required.
      // In addition, the calibration factors maxRed, maxGreen, and maxBlue
      should
1004 // be determined in advance using APDS9960Calibration.ino.
1006 // file: ApplicationAPDS9960.ino
1008 #include <Arduino.h>
      #include <U8g2lib.h>
1010 #include <Wire.h>
      #include <Arduino_APDS9960.h>
1012
      #define I2C_ADDRESS 0x3C // I2C address for the OLED display
1014 const int LedPin = 12; // LED pin number
1016
      // Calibration values obtained from the calibration program
      const int maxRed = 31; // Maximum calibrated red value
1018 const int maxGreen = 15; // Maximum calibrated green value
      const int maxBlue = 13; // Maximum calibrated blue value
1020
      // Initialize display object for a 128x64 OLED with I2C interface
1022 U8G2_SH1106_128X64_NONAME_F_HW_I2C oled(U8G2_R0, /* reset= */
      U8X8_PIN_NONE);
1024
      void setup()
      {
1026 Serial.begin(9600); // Start serial communication
      pinMode(LedPin, OUTPUT); // Set the LED pin as an output
1028
      // Initialize APDS9960 sensor, log error if initialization fails
1030 if (!APDS.begin()) {
      Serial.println("Error initializing APDS9960 sensor.");
1032 }
1034
      // Initialize OLED display and display an "INITIALIZING" message
      oled.begin();
1036 oled.clearBuffer();
      oled.setFont(u8g2_font_ncenB08_tr); // Set display font
1038 oled.drawStr(10, 10, "INITIALIZING!");
      oled.sendBuffer(); // Send the buffer to display
1040
      // Blink LED three times as a visual indication that setup is complete
1042 for (int counter = 0; counter < 3; counter++) {
      digitalWrite(LedPin, HIGH);

```

```

1044     delay(200);
        digitalWrite(LedPin, LOW);
1046     delay(200);
    }
1048     oled.clearBuffer(); // Clear the display buffer after setup
        oled.sendBuffer();
1050 }

```

../Code/Nano33BLESense/APDS9960/ApplicationAPDS9960/ApplicationAPDS9960.ino

In the following `loop()` function, the code displays a "READY" message on the OLED display and then sets a timeout of 1000 ms to wait for color data to become available, decrementing the timeout in steps of 5 ms. If color data is not available within the timeout, a "Color measurement timeout" message is printed in the serial monitor, and the function returns. The code then resets the timeout to 1000 ms and similarly waits for proximity data. If proximity data does not become available within the timeout, a "Proximity measurement timeout" message is printed, and the function returns.

Listing 14.2.: Application of the sensor APDS9960: Loop

```

1000 void loop()
    {
1002     // Display "READY" message
        oled.clearBuffer();
1004     oled.setCursor(10, 20);
        oled.print("READY");
1006     oled.sendBuffer();

1008     int timeout = 1000; // Timeout value for waiting for color data
        // Wait until color data is available or timeout occurs
1010     while (!APDS.colorAvailable() && timeout > 0) {
        delay(5);
        timeout -= 5;
        }
1014     if (timeout <= 0) {
        Serial.println("Color measurement timeout");
1016     return;
    }

1018     timeout = 1000; // Reset timeout for proximity data
        // Wait until proximity data is available or timeout occurs
1020     while (!APDS.proximityAvailable() && timeout > 0) {
        delay(5);
        timeout -= 5;
1024     }
        if (timeout <= 0) {
1026     Serial.println("Proximity measurement timeout");
        return;
1028     }
    }

```

../Code/Nano33BLESense/APDS9960/ApplicationAPDS9960/ApplicationAPDS9960.ino

In the next section of the `loop()` function, the code reads the proximity value using `APDS.readProximity()`. If a valid proximity value is detected, it prints the proximity value to the serial monitor. If the proximity value is below a specified threshold (indicating an object is close), the OLED display is cleared, and the LED is turned on as a visual indication. To capture accurate color data, all LEDs are turned on to emit white light, and multiple color readings are averaged.

The red, green, and blue color values are then mapped to a 0-255 range using predefined calibration values (`maxRed`, `maxGreen`, `maxBlue`) and are constrained within the 0-255 range. The dominant color is determined based on the highest value among red, green, and blue, and the result is displayed on the OLED. If no object is detected within the proximity threshold, a message prompts the user to hold an object near the sensor or change its position.

If the proximity reading fails, an error message is displayed on the OLED. After processing, the code turns off the LED and resets the RGB LEDs to an off state.

Listing 14.3.: Application of the sensor APDS9960: Main function

```

1000 int r, g, b;
1001 int proximity = APDS.readProximity(); // Read proximity value
1002
1003 if (proximity != -1) {
1004     Serial.print("Proximity: ");
1005     Serial.println(proximity);
1006
1007     if (proximity < 20) { // If proximity is below threshold
1008         oled.clearBuffer();
1009         digitalWrite(LedPin, HIGH); // Turn on LED as visual indication
1010
1011         // Turn on all LEDs (for white light) to capture accurate color
1012         digitalWrite(LED_R, LOW);
1013         digitalWrite(LED_G, LOW);
1014         digitalWrite(LED_B, LOW);
1015
1016         // Average multiple color readings for accuracy
1017         int r_total = 0, g_total = 0, b_total = 0;
1018         for (int i = 0; i < 5; i++) {
1019             APDS.readColor(r, g, b);
1020             r_total += r;
1021             g_total += g;
1022             b_total += b;
1023             delay(750); // Delay between readings
1024         }
1025         r = r_total / 5;
1026         g = g_total / 5;
1027         b = b_total / 5;
1028
1029         // Map color values to a 0-255 range using calibration values
1030         r = map(r, 0, maxRed, 0, 255);
1031         g = map(g, 0, maxGreen, 0, 255);
1032         b = map(b, 0, maxBlue, 0, 255);
1033
1034         // Constrain values to the 0-255 range
1035         r = constrain(r, 0, 255);
1036         g = constrain(g, 0, 255);
1037         b = constrain(b, 0, 255);
1038
1039         // Print color values for debugging
1040         Serial.print("Red light = ");
1041         Serial.println(r);
1042         Serial.print("Green light = ");
1043         Serial.println(g);
1044         Serial.print("Blue light = ");
1045         Serial.println(b);
1046         Serial.println();
1047
1048         // Determine the dominant color and display result on OLED
1049         oled.setCursor(10, 20);
1050         if (r > g && r > b) {
1051             oled.print("The object is red.");
1052         } else if (g > r && g > b) {
1053             oled.print("The object is green.");
1054         } else if (b > g && b > r) {
1055             oled.print("The object is blue.");
1056         }
1057
1058         oled.sendBuffer(); // Display the color information
1059         delay(4000);
1060         oled.clearBuffer();
1061         oled.sendBuffer();
1062     } else {
1063         // Display message if no object is found within proximity

```

```

threshold
1064   oled.clearBuffer();
      oled.setCursor(10, 20);
1066   oled.print("No object found.");
      oled.setCursor(10, 30);
1068   oled.print("Hold an object");
      oled.setCursor(10, 40);
1070   oled.print("in front of sensor");
      oled.setCursor(10, 50);
1072   oled.print("or change position.");
      oled.sendBuffer();
1074   delay(4000);
      oled.clearBuffer();
1076   oled.sendBuffer();
    }
1078 } else {
      // Display error if proximity reading fails
1080   oled.clearBuffer();
      oled.setCursor(10, 20);
1082   oled.print("Proximity failed!");
      oled.sendBuffer();
1084 }

1086 // Turn off LED and set RGB LEDs to off state after processing
      digitalWrite(LedPin, LOW);
1088 digitalWrite(LED_R, HIGH);
      digitalWrite(LED_G, HIGH);
1090 digitalWrite(LED_B, HIGH);
    }

```

../Code/Nano33BLESense/APDS9960/ApplicationAPDS9960/ApplicationAPDS9960.ino

14.3. Application Test

The color recognition machine reliably identifies the colors red, green, and blue from the standardized color chart. After calibration, it can also recognize other colors like light blue, yellow-green, or orange as red, green, and blue. This flexibility ensures that objects do not need to be perfectly red, green, or blue to be detected. Activating the white LED when an object approaches proves to be very practical, as it serves as a feedback mechanism alongside the OLED display, indicating to the user that color recognition is now in progress.

14.4. Improvement Suggestions

A key improvement would be to integrate the color recognition machine into a stable, standalone setup, so it does not need to be handheld, allowing objects to be fixed in place in front of the sensor. This would eliminate potential issues like hand movement artifacts during detection.

Another enhancement would be to equip the device with a battery, enabling portable, laptop-free operation. To conserve battery life, the white LED is deliberately programmed to illuminate only during color measurement.

In an industrial application, adequate lighting should be ensured, potentially by adding external lighting that could be integrated into the code or kept continuously on. For use in dusty or humid environments, a thin protective cover over the sensor could prevent damage or contamination, allowing for easy cleaning or replacement as needed.

15. Mikrophone MP34DT05-A

- Allgemeine Information über Mikrophone
- Spezielle Betrachtung des Mikrophons
- Kalibrierung und Test des Mikrophons
- Algorithmen zur Auswertung eines Mikrophons
- Auswertung des speziellen Mikrophons
- Programmierung des Mikrophons

Some Arduino boards have a microphone on board which can use for the application. Das Mikrofon des Arduino Nano 33 BLE Sense ist ein ultrakompaktes Modul, das den MP34DT05-Sensor verwendet. Es wurde speziell für den Arduino Nano 33 BLE Sense entwickelt. Dieser Sensor nutzt die Puls-Dichtemodulation (PDM), um ein analoges Signal in ein binäres Signal umzuwandeln. Durch PDM wird ein analoges Signal in binäre Werte umgewandelt. [STM21]

WS:cite books,
applications, board

15.0.1. Ton und Frequenzen

Ton, auch bekannt als Schall, ist genauer eine einzige Schallfrequenz, die eine mechanische Deformation in dem Medium Luft verursacht, welche sich als Longitudinalwelle ausbreitet.

Die Frequenz f , gemessen in Hertz (Hz), stellt die Schwingungen über Luftdruckschwankung pro Sekunde in einer Schallwelle dar. Beispielsweise kann der Mensch einen Frequenzbereich von etwa 20 Hz bis 20 000 Hz wahrnehmen. Es gelten folgende Formeln für die Beschreibung von Schall:

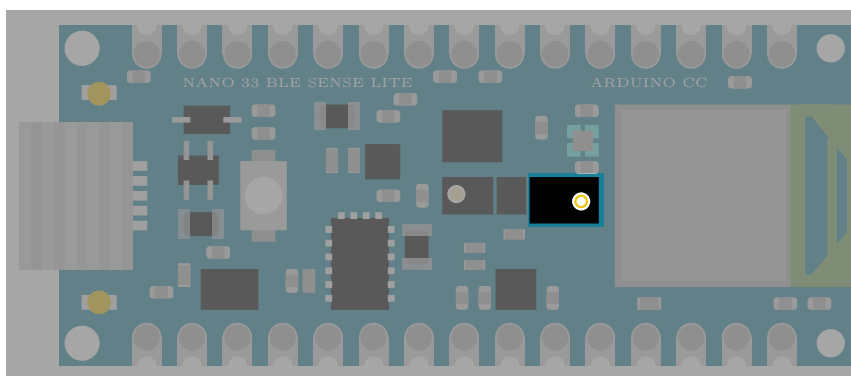


Figure 15.1.: Arduino Nano 33 BLE Sense's RGB LED with Pin 22, Pin 23, and Pin 24

- Schalldruck p :

$$p(t) = \hat{p} \cdot \cos(\omega t - \phi) \quad (15.1)$$

mit $\hat{p}$: Amplitude des Schalldrucks,

$\omega = 2 \cdot \pi \cdot f$: Kreisfrequenz,

ϕ : Phasenverschiebung

- Schallschnelle ν :

$$\nu(t) = \hat{\nu} \cdot \cos(\omega \cdot t) \quad (15.2)$$

mit $\hat{\nu}$: Amplitude des Schalldrucks

- Schallgeschwindigkeit c (für Gase):

$$c = \sqrt{(\kappa \rho) / \rho} \quad (15.3)$$

Für Luft beträgt die Schallgeschwindigkeit $c_{Luft} = 331,2 \text{ m/s}$.

Je schneller die Abfolge der Schwingungen, desto höher ist die Frequenz bzw. der Ton. Je größer die Amplitude ist, desto lauter ist der Ton.

Die Lautstärke wird durch die Amplitude beschrieben, die durch den Schalldruck erzeugt wird. Die Einheit für die Lautstärke wird in Dezibel (dB) angegeben. Hierfür wird der Abstand der Auslenkung zum Nullpunkt gemessen. Als Nullpunkt wurde, mit $P_{ref} = 2 \cdot 10^{-5} \text{ Pa}$ die menschliche Hörschwelle festgelegt. Ein negativer dB-Wert ist somit nicht mehr für den Menschen hörbar. ([Poh17])

In der Tonverarbeitung werden diese Schwingungen mechanisch, meist über Mikrofone aufgenommen.

15.0.2. Unterschied Ton, Klang und Geräusch

Ein Ton ist beschreibbar durch eine harmonische Funktion mit unveränderlicher Frequenz. Ein Klang ist hingegen ein komplexeres, aber periodisches Schallereignis. Wesentlicher Bestandteil der Charakterisierung eines Klanges ist die Überlagerung eines Grundtons mit seinen Oberschwingungen. Ein Geräusch ist generell nicht periodisch und zeigt veränderliche Strukturen.

WS:cite

15.0.3. Mikrofone

Die prinzipiellen Funktionsweisen verschiedener Mikrofonarten sind sehr ähnlich zueinander. Die Schallwellen versetzen eine Membran in Schwingung, diese wird durch verschiedene Methoden in elektrische Signale umgewandelt (siehe Abbildung 15.2).

Da Computerprozessoren mit diesen analogen elektrischen Signalen nicht rechnen können, werden sie durch elektronische Wandler in digitale Signale umgewandelt. Zunächst werden einige Methoden, die geeignet sind, Schwingungen der Membran in elektrische Signale umzuwandeln, betrachtet. Dies kann durch physikalische Prinzipien wie elektrodynamische Induktion, Kapazitätsänderungen, Piezoelektrische-Elemente, durch die Änderung des elektrischen Widerstands (Kohlemikrofone) usw. umgesetzt

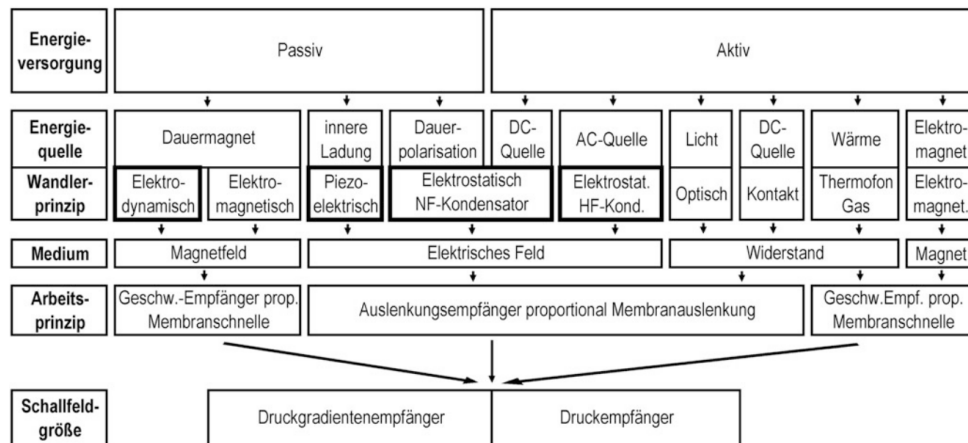


Figure 15.2.: Liste verschiedener Mikrofonarten mit Funktionsprinzip

werden. Neben diesen heute üblichen Methoden der Schallwandlungen gibt es auch noch eine Reihe anderer Methoden.

In diesem Fall wird das Mikrofon Thomann t.bone SC 420 benutzt. Dem Datenblatt des Mikrofons 15.3 sind folgende Angaben zu entnehmen:

- Beim Mikrofon handelt es sich um ein Kondensatormikrofon.
- Der nutzbare Frequenzbereich liegt bei 80 Hz - 18000 Hz.
- Das digitale Signal hat eine Auflösung von 16 bit (mono) bei einer Abtastfrequenz von 48 kHz.
- Es hat eine supernierenförmige Richtcharakteristik.
- Die Arbeitstemperatur liegt zwischen 0°C und 40°C, die erlaubte relative Luftfeuchtigkeit zwischen 20% und 80%.

Technical specifications

■ Connection/power supply	USB
■ Transducer principle	Condenser
■ Polar pattern	Supercardioid
■ Frequency range (-10 dB)	80 – 18,000 Hz
■ Sampling rate	16 Bit/48 kHz
■ Current consumption (mA)	150
■ Dimensions (Ø × D), without shock mount	51 mm × 187 mm
■ Thread	5/8-inch
■ Weight	285 g
■ Ambient conditions	Temperature range 0 °C...40 °C
	Relative humidity 20%...80% (non-condensing)

Figure 15.3.: Datenblatt des betrachteten Mikrofons

Kondensatormikrofon

Beim Kondensatormikrofon (siehe Abbildung 15.4) bildet die Membran eine Elektrode eines Kondensators. Durch die Bewegung der Membran ändert sich der Abstand zu der Gegenelektrode und damit die Kapazität des Kondensators. Der Kondensator wird über einen hochohmigen Widerstand geladen. Die Kapazitätsänderung führt dann zu einer Änderungen der am Kondensator anliegenden Spannung. Diese Spannungsänderung kann mit einem geeigneten Spannungsmesser gemessen werden.

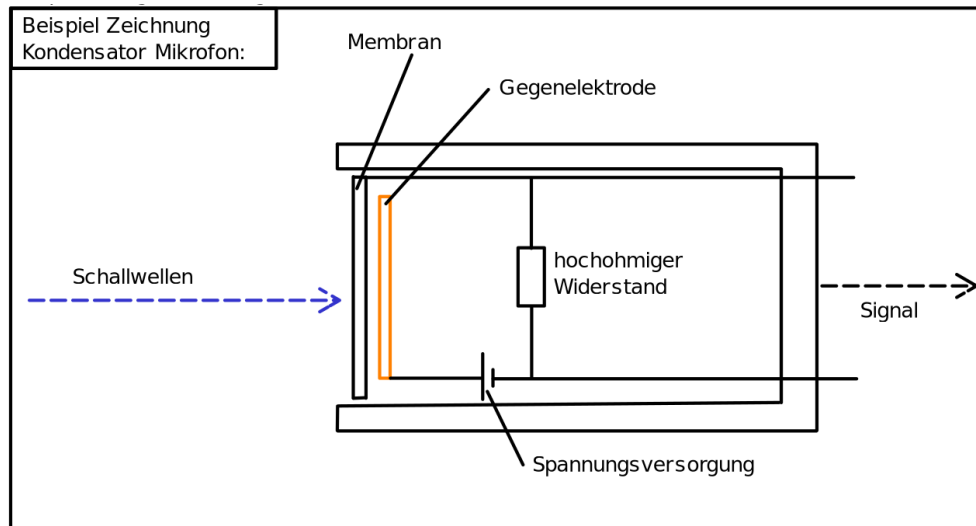


Figure 15.4.: Aufbau eines Kondensator Mikrofons

Anschließend muss dieses analoge, elektrische Signal noch in ein digitales Signal umgewandelt werden (siehe Abbildung 15.5). Dies ist notwendig, da Computerprozessoren nur mit digitalen Signalen arbeiten können. Dazu werden AD-Wandler (Analog zu Digital Wandler) genutzt. Die AD-Wandler können entweder im Mikrofon selbst eingebaut oder vom Mikrofon getrennt sein. AD-Wandler sind in heutigen Computern selbst eingebaut. Im professionellen Bereich werden aber meist hochwertige externe AD-Wandler eingesetzt. In diesem Projekt wird ein Mikrofon benutzt, bei dem der AD-Wandler direkt im Mikrofon verbaut ist. Das digitale Signal wird als binäres Signal über eine USB Schnittstelle an den Computer übermittelt.

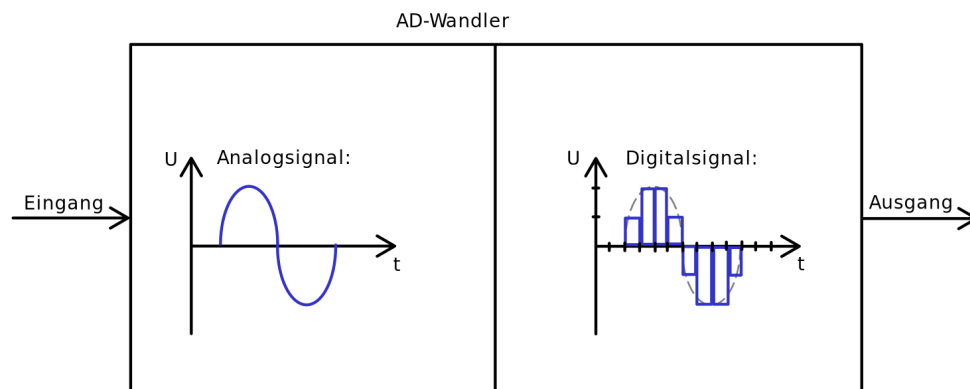


Figure 15.5.: Funktionsprinzip eines Analog zu Digital Wandlers

([Wei20])

15.0.4. Wavedatei

Unkomprimierte Audiodaten werden üblicherweise im Wave-Dateiformat gespeichert (siehe Abbildung 15.7). Dieses Dateiformat ist ein von Microsoft und IBM, im Jahr 1991 veröffentlichtes Format zur digitalen Speicherung von Audiodaten. Es setzt auf dem RIFF (Resource Interchange File Format) Dateiformat auf. Das besondere beim RIFF-Datenformat ist, dass in diesen Dateien Informationen über das Format des

Dateiinhalts am Anfang der Datei gespeichert sind. Bei WAV-Audiodateien handelt es sich um eine RIFF Datei mit einem WAVE Container. Normalerweise sind die Audiodateien als PCM (Pulse-Code-Modulation)-Rohdaten enthalten, wobei PCM das Pulsmodulationsverfahren ist, mit dem die analogen Signale in digitale Signale umgesetzt werden. Meist enthält eine RIFF Audiodatei einen WAVE Container, welcher zwei Untercontainer enthält: Den fmt (Format) Container, der das Format der Audiodaten und deren Speicherung beschreibt und einen data Container, welcher die eigentlichen Audiodaten in dem Format enthält, das im fmt Container beschrieben wird.

Eine Wave-Datei enthält in den ersten 4 Byte das Wort RIFF im ASCII-Format. Die folgenden vier Byte enthalten Größen der noch folgenden Daten im little-endian-Format. Der letzte Teil der Beschreibung sind noch einmal 4 Byte, die den Dateityp angeben. In unserem Fall also immer das Wort WAVE im ASCII-Format.

ASCII steht für „American Standard Code for Information Interchange“ und ist einer 7-Bit Zeichenkodierung, die der ISO-646 entspricht.

Die Zeichenkodierung besteht aus 128 Zeichen wovon 95 druckbar und 33 nicht druckbar sind. Einige hiervon sind:

- Buchstaben: A, B, C, ..., Z, a, b, c, ..., z
- Ziffern: 0, 1, 2, ..., 9
- Interpunktionszeichen: !, ", #, \$, %, &, ' , (,) , * , + , , - , . , / , : , ; , < , = , > , ? , [, \ ,] , ^ , _ , ` , { , | , } , ~

Eine ausführliche Darstellung ist in Abbildung 15.6.

Wichtig anzumerken ist, dass jedes Zeichen einem definiertem Zahlenwert entspricht.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	0x00	0x01	0x02	0x03	0x04	0x05	0x06	0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D	0x0E	0x0F
0	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
16	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
32	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
48	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
64	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
80	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
96	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
112	p	q	r	s	t	u	v	w	x	y	z	{		}	~	

Figure 15.6.: Symbole und numerische Werte des ASCII Formats

Die Zusatzwerte little endian(LE)/big endian(BE) geben an, von welcher Reihenfolge die Zahlenwerte der Bytes betrachtet werden.

- BE, auch als big-endian bekannt, speichert das höchstwertige Byte zuerst. Wenn mehrere Bytes gelesen werden, ist das erste Byte (oder die niedrigste Speicheradresse) das größte.
- LE, auch als little-endian bekannt, speichert das niederwertigste Byte zuerst. Wenn mehrere Bytes gelesen werden, ist das erste Byte (oder die niedrigste Speicheradresse) das kleinste.

In dem Container “fmt“ (Format) sind sämtliche Angaben zum Aufbau der nachfolgenden Audiodatei spezifiziert.

- Die ersten 4 Bytes leiten das Format Container mit dem Wort “fmt“ ein. Also die Buchstaben f, m, t und ein Leerzeichen. (ASCII) (big endian).
- (Subchunk1Size) Die nächsten 4 Bytes geben an, wie groß der restliche fmt Container noch ist. (little endian).
- (AudioFormat) Die nächsten zwei Bytes geben die Art an, wie die Audiodaten gespeichert sind. (üblicherweise 1 = PCM unkomprimiert) (little endian).
- (NumChannels) In den nächsten zwei Bytes steht, wie viele Kanäle in der Audiodatei enthalten sind (mono = 1, stereo = 2, ...).
- (SampleRate) Die nächsten 4 Bytes geben die Abtastrate des Signals in Hertz an.
- (ByteRate) In den nächsten 4 Bytes ist die Bandbreite des Audiosignals angegeben. Diese wird in Bytes pro Sekunde (Bytes/s) angegeben.
- (BlockAlign) In den nächsten zwei Bytes wird angegeben, wie viele Byte ein Block enthält. Ein Block entspricht einem Sample mal die Anzahl der Kanäle. (Bits pro Audiowert * Anzahl der Kanäle / 8).
- (BitsPerSample) In den hier letzten beiden Bytes des Beispiels wird angegeben, wie viele Bit ein Audiowert enthält. 8 Bit entsprechen einem Byte.

Würde bei den Audiodaten etwas Anderes als PCM verwendet werden, würde nach dem jetzigem Abschnitt noch ein Abschnitt folgen, der dieses Format spezifiziert. Als letztes sind im data-Container die wirklichen Audiodaten gespeichert, wie oben spezifiziert wurde. Eingeleitet wird dieser Container mit dem Wort „data“ in den ersten 4 Bytes (ASCII). In den darauf folgenden 4 Bytes wird die Größe der tatsächlichen Audiodaten in Byte (little Endian) angegeben.

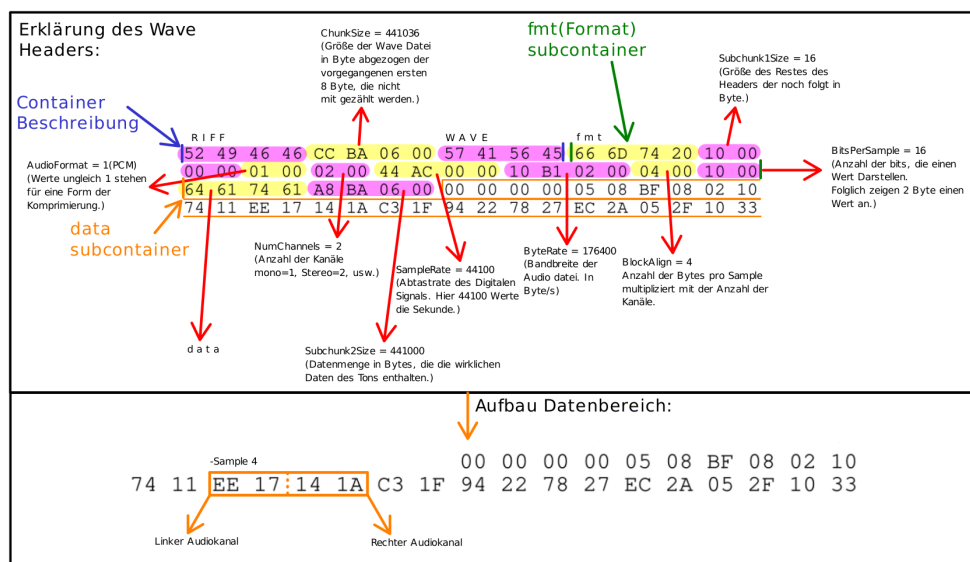


Figure 15.7.: Aufbau des Wave Dateiformats mit Erklärung [Wil03]

15.0.5. Package **PyAudio** Version 0.2.14

Die Bibliothek PyAudio ermöglicht den Zugriff auf ein Mikrofon. In diesem Fall wird das im Computer verbaute Mikrofon verwendet.

Zur Erläuterung der Einbindung der Bibliothek in einen Code wird ein Sample zum Abspielen einer Wave-Datei gezeigt.

```
import wave
import sys

import pyaudio

CHUNK = 1024
FORMAT = pyaudio.paInt16
CHANNELS = 1 if sys.platform == 'darwin' else 2
RATE = 44100
RECORD_SECONDS = 5

with wave.open('output.wav', 'wb') as wf:
    p = pyaudio.PyAudio()
    wf.setchannels(CHANNELS)
    wf.setsampwidth(p.get_sample_size(FORMAT))
    wf.setframerate(RATE)

    stream = p.open(format=FORMAT, channels=CHANNELS,
                    rate=RATE, input=True)

    print('Recording...')
    for _ in range(0, RATE // CHUNK * RECORD_SECONDS):
        wf.writeframes(stream.read(CHUNK))
    print('Done')

    stream.close()
    p.terminate()
```

Es ist zu sehen, dass zunächst durch den Befehl „import pyaudio“ die Bibliothek importiert wird. Mit dem Befehl `pyaudio.PyAudio()` kann nun auf die Bibliothek zugegriffen werden. `pyaudio.PyAudio.open()` öffnet als Nächstes einen Stream, das bedeutet, dass ein kontinuierlich bestehender Datenfluss ermöglicht wird, aus dem Audiodaten gelesen werden können mit `pyaudio.PyAudio.Stream.read()`. Um nun den Stream, also den kontinuierlichen Audiodatenfluss, zu beenden, wird `stream.close()` verwendet. Um den Zugriff auf das Mikrofon abubrechen, verwendet man `p.terminate()`. (vgl. PyAudio Documentation — PyAudio 0.2.14 documentation (mit.edu))

Die Bibliothek PyAudio besitzt jedoch noch weitere Funktionen. Die für dieses Projekt wichtigste Funktion ist jedoch der Zugriff auf Lautsprecher und Mikrofon, sowie der Aufbau eines Streams für einen kontinuierlichen Audiodatenfluss. [Pha06]

15.0.6. Package **Wave** Version 3.12

Die Bibliothek Wave ist bereits in Python integriert und ermöglicht das Lesen von Wave-Dateien.

Zur Erläuterung ist nachfolgend ein Sample zu sehen.

```
import numpy as np
import wave
```

```
FRAMES_PER_SECOND = 44100

def sound_wave(frequency, num_seconds):
    time = np.arange(0, num_seconds, 1 / FRAMES_PER_SECOND)
    amplitude = np.sin(2 * np.pi * frequency * time)
    return np.clip(
        np.round(amplitude * 32768),
        -32768,
        32767,
    ).astype("<h")

left_channel = sound_wave(440, 2.5)
right_channel = sound_wave(480, 2.5)
stereo_frames = np.dstack((left_channel, right_channel)).flatten()

with wave.open("output.wav", mode="wb") as wav_file:
    wav_file.setnchannels(2)
    wav_file.setsampwidth(2)
    wav_file.setframerate(FRAMES_PER_SECOND)
    wav_file.writeframes(stereo_frames)
```

Das oben gezeigte Sample erstellt eine Audiodatei mit einer Frequenz von 440 Hz, die über den linken Lautsprecher ausgegeben wird und einer Frequenz von 480 Hz, die über den rechten Lautsprecher ausgegeben wird. Beide Frequenzen werden zeitgleich für eine Dauer von 2,5 Sekunden ausgegeben.

Die Nutzung der Bibliothek `wave` beginnt mit dem Import mittels `import wave`. Danach wird mithilfe von `sound_wave` eine Sinuswelle erzeugt. Nun wird die Frequenz definiert, die über den linken und den rechten Lautsprecher ausgegeben wird, mit der Funktion `frequency`. Mit `num_seconds` wird jetzt noch die Dauer in Sekunden für den linken und rechten Lautsprecher festgelegt. In Zeile 17 werden nun die Töne, die durch `sound_wave(frequency, num_seconds)` definiert wurden, zusammengefügt, sodass sie durch `with wave.open(„output.wav“, mode=„wb“) as wav_file:` in eine neu erstellte Wave-Datei mit Namen `output.wav` geschrieben werden können. [Fou24b]

15.0.7. Package `NumPy` Version 1.26

NumPy ist eine sehr große und umfassende Bibliothek. Die Funktion, die für die erläuterte Aufgabenstellung von Interesse ist, ist die schnelle Verarbeitung von Daten, daher wird auch nur diese Funktion beschrieben.

```
import sys
import numpy as np
import pyaudio

CHUNK = 1024
FORMAT = pyaudio.paInt16
CHANNELS = 1 if sys.platform == 'darwin' else 2
RATE = 44100
RECORD_SECONDS = 5

p = pyaudio.PyAudio()

stream = p.open(format=FORMAT, channels=CHANNELS,
```



```

rate=RATE, input=TRUE)

data = np.array([])

print(`Recording...`)
for _ in range(0, RATE // CHUNK * RECORD_SECONDS):
    in_data = stream.read(CHUNK)
    in_data = np.frombuffer(in_data,np.int16)
    data = np.concatenate((data,in_data))
print(`Done`)

MAX_LENGTH = 1024
data = data[0::2]
data = data[-data.size//1024*1024:]

data = data.reshape(-1,data.size//1024)
data = np.mean(data,axis=1)

print(data)

stream.close()
p.terminate()

```

Das hier gezeigte Sample wurde inspiriert durch das Sample, das bereits zur Erklärung der Bibliothek [PyAudio](#) verwendet wurde. Dieses wurde um ein paar Zeilen zur Bibliothek [NumPy](#) ergänzt, um die hier relevante Funktion der Bibliothek erklären zu können.

Gestartet wird mit dem Befehl `import numpy as np`. Zudem wird in dieser Zeile der Bibliothek der Alias `np` zugeordnet, wodurch im Folgendem Code die Funktionen von [NumPy](#) durch die Verwendung des Alias aufgerufen werden können.

Die Bibliothek [PyAudio](#) stellt die Audiodaten in binärer Form dar. Da die Daten in dieser Form noch nicht ausgewertet werden können, ist es notwendig diese durch den Befehl `np.frombuffer(in_data,np.int16)` in einem Array darzustellen. Mit dem Befehl `np.concatenate((data,in_data))` können nun die gerade erstellten Daten und die ursprünglichen Daten zusammengefügt werden. Von Zeile 24 bis 26 werden alle Daten, die von der rechten Seite ausgegeben werden, herausgefiltert, sodass nur die Daten der linken Seite übrigbleiben. In Zeile 26 wird nun noch die Länge des Arrays auf eine konstante Ziellänge gebracht. Dabei muss die Ziellänge durch 1024 teilbar sein, was durch `MAX_LENGTH` definiert wird. Nun wird in der Zeile 28 das Array in ein zweidimensionales Array mit der Ziellänge 1024 umgeformt. Zeile 29 fasst die Arrays der zweiten Dimension in ihr jeweiliges arithmetisches Mittel zusammen. Nun muss das Ganze nur noch mit dem Befehl `print(data)` ausgegeben, und mit `stream.close()` geschlossen werden. [The22]

15.0.8. Package [Guizero](#) Version 1.5.0

[Guizero](#) ist eine Bibliothek, die auf der in Python enthaltenden Bibliothek Tkinter aufbaut. Die Bibliothek bietet die Möglichkeit, ein Dashboard zu erstellen, und bietet dazu einige Widges und Funktionen, die die Gestaltung des Dashboards erleichtern. Im Vergleich zu [Tkinter](#) ermöglicht [Guizero](#) eine einfachere Bedienung.

```

from guizero import App, MenuBar
def file_function():
    print("File option")

```

```

def edit_function():
    print("Edit option")

app = App()
menubar = MenuBar(app,
    toplevel=["File", "Edit"],
    options=[
        ["File option 1", file_function], ["File option 2",
        file_function]],
        ["Edit option 1", edit_function], ["Edit option 2",
        edit_function])
app.display()

```

Das Sample zeigt die Verwendung der Funktion zur Erstellung einer MenuBar mithilfe der Bibliothek [Guizero](#). Zunächst erfolgt wieder das Importieren der gewünschten Klassen [App](#) und [MenuBar](#) des Moduls [Guizero](#) mit dem Befehl `from guizero import App, MenuBar`. Als allererstes erstellt die Funktion [App](#) ein Fenster. [MenuBar](#) kann daraufhin eine Menüleiste erstellen mit den Oberpunkten [File](#) und [Edit](#) und den Unterpunkten [File option 1](#), [File option 2](#), [Edit option 1](#) und [Edit option 2](#). Dabei werden den Unterpunkten auch noch Funktionen zugewiesen, hier [file_function](#) und [edit_function](#). [SO20]

15.0.9. Package [Threading](#) (Version 3.7)

[Threading](#) ist eine Bibliothek, die eine Parallelisierung arbeitsintensiver Aufgaben ermöglicht.

```

import logging
import threading
import time

def thread_function(name):
    logging.info("Thread %s: starting", name)

if __name__ == "__main__":
    format = "%(asctime)s: %(message)s"
    logging.basicConfig(format=format, level=logging.INFO,
        datefmt="%H:%M:%S")

    threads = list()
    for index in range(3):
        logging.info("Main : create and start thread %d.", index)
        x = threading.Thread(target=thread_function, args=(index,))
        threads.append(x)
        x.start()

    for index, thread in enumerate(threads):
        logging.info("Main : before joining thread %d.", index)
        thread.join()
        logging.info("Main : thread %d done", index)

```

Das Sample dient zur Erläuterung der Bibliothek, bzw. der hier benötigten Funktion. Es beginnt wieder mit dem Importieren der Bibliothek mit `import threading`.

Der Code erstellt insgesamt drei parallel laufende Programme. In jedem Programm oder auch Thread genannt wird die Funktion `thread_function(name)` ausgeführt, dabei wird der Thread zwei Sekunden pausiert durch `time.sleep(2)`. Als Letztes wird nun noch eine weitere for-Schleife gestartet, die auf die Beendigung der Threads wartet, woraufhin die Nachricht `Main : thread %d done` protokolliert wird. Das `%d` wird hierbei durch den jeweiligen Namen des Threads ersetzt. [Fou24a]

15.0.10. Package **Librosa** Version 0.10.2

Librosa ist eine Bibliothek, die vielzahlige Audiotbearbeitungsfunktionen anbietet. Für die erläuterte Aufgabenstellung ist lediglich der Teilbereich `librosa.effects` von Interesse.

```
#Compress to be twice as fast
>>> y, sr = librosa.load(librosa.ex('choice'))
>>> y_fast = librosa.effects.time_strech(y, rate=0.2)
#Or half the original speed
>>> y_slow = librosa.effects.time_strechy(y, rate=0.5)
```

Das obige Sample zeigt, wie eine Audiodatei über Librosa geöffnet wird, was in diesem speziellen Fall nicht benötigt wird, da die Audiodatei über einen anderen Weg gewonnen wird, und anschließend die Geschwindigkeit verdoppelt und in einem zweiten Beispiel halbiert wird über die Funktion `time_stretch`.

```
#Shift up a major third (four steps if bins_per_octave is 12)
>>> y, sr = librosa.load(librosa.ex('choice'))
>>> y_third = librosa.effects.time_shift(y, sr=sr, n_steps=4)
#Shift down by tritone (six steps if bins_per_octave ist 12)
>>> y_tritone = librosa.effects.pitch_shift(y, sr=sr, n_steps=-
6)
Shift up by 3 quarter-tones
>>> y_three_qt = librosa.effects.pitch_shift(y, sr=sr, n_steps=3,
...                                         bins_per_octave=24)
```

Das obige Sample zeigt nun eine weitere Funktion des Teilbereichs `librosa.effects`. Im ersten Beispiel wird die Tonhöhe der Tonspur um vier Halbtöne erhöht, im zweiten Beispiel um sechs Halbtöne verringert und im dritten Beispiel um drei Vierteltöne erhöht. [McF+23]

15.1. Funktionen

15.1.1. Laden

```
import wave
dateipfad = "beispiel.wav" #Dateipfad zur Wave-Datei angeben

#Wavedatei oeffnen
with wave.open(dateipfad, 'r') as wave_datei:
#informationen ueber die Wavedatei ausgeben
print("Anzahl der Kanale:", wave_datei.getnchannels())
print("Abtastrate:", wave_datei.getframerate())
print("Anzahl der Frames:", wave_datei.getnframes())
print("Samplebreite in Bytes:", wave_datei.getsampwidth())
```

Dieses Sample zeigt, wie eine Wave-Datei geladen werden kann, dabei werden Kennzahlen, wie Anzahl der Kanäle, Abtastrate, Anzahl der Frames und Samplebreite ausgegeben. Voraussetzung für das erfolgreiche Laden ist, dass der Dateipfad entsprechend des Speicherortes der Beispieldatei festgelegt wird. Dieser Pfad muss im Befehl `wave.open` eingefügt werden.

15.1.2. Abspielen

```
import wave
import pyaudio

wf = wave.open('audio.wav', 'rb') # oeffnen der Wave-Datei

p = pyaudio.PyAudio() # Initialisierung des PyAudio-Objekts

# oeffnen des Audio-Streams
stream = p.open(format=p.get_format_from_width(wf.getsampwidth()),
               channels=wf.getnchannels(),
               rate=wf.getframerate(),
               output=True)

# Lesen der Daten aus der Wavedatei
data = wf.readframes(1024)

# Abspielen der Wavedatei
while data:
    stream.write(data)
    data = wf.readframes(1024)

# Beenden des Audio-Streams und des PyAudio-Objekts
stream.stop_stream()
stream.close()
p.terminate()
```

Es gelten für das Sample „Abspielen“ dieselben Voraussetzungen, wie für das Laden einer Wave-Datei, nämlich dass der Dateipfad für die abzuspielende Datei im Programm definiert werden muss. Das Sample öffnet die Datei `audio.wav` im Lesemodus und initialisiert den Audio-Stream mit den entsprechenden Parametern aus der Wave-Datei. Dafür wird der Befehl „stream“ aus der Bibliothek `PyAudio` verwendet. Es wird so lange „Abspielen“ ausgeführt, bis alle Daten aus der Wave-Datei abgespielt wurden. Zuletzt wird der Audio-Stream beendet mit dem Befehl „`stream.stop_stream`“.

15.1.3. Speichern

Voraussetzung des „Speicher“-Vorgangs ist immer, dass eine Wave-Datei bereits geöffnet ist. Dieser Schritt ist in den folgenden Samples mit enthalten. Beim erstmaligen Speichern öffnet sich automatisch die „Speichern unter“-Funktion. Hier müssen das Dateiformat und der Zielordner definiert werden. Für das Dateiformat bietet sich der Wave-Typ an, da dieser mit der Anwendung weiter bearbeitet werden kann und es sich um den Standard für nicht komprimierte Audio-Dateien handelt. Im weiteren Verlauf der Bearbeitung kann auch der Befehl „Speichern“ verwendet werden, unter der Bedingung, dass die zu bearbeitende Audio-Datei bereits einmal gespeichert wurde.

```
from guizero import App, FileSaveDialog
import wave
```

```

def save_wave_file():
    # Erstelle eine App-Instanz
    app = App(visible=False) # App im unsichtbaren Modus starten

    # Oeffne den Dateieexplorer, um den Zielordner auszuwaehlen
    file_path = FileSaveDialog(
        app, title="Speichern unter", filetypes=[(
            "Wave files", "*.wav")], initialfilename="*.wav").value

    if file_path:
        # oeffne die ausgewählte Datei im Schreibmodus
        with wave.open(file_path, 'w') as wf:
            wf.setnchannels(2) # Zwei Kanäle (Stereo)
            wf.setsampwidth(2) # 2 Bytes pro Sample
            wf.setframerate(44100) # Abtaste von 44100 Hz
            wf.setnframes(0) # Anfangs keine Frames

            # Schreibe Daten in die Datei
            # (hier müssten die Audiodaten eingefügt werden)
            # Beispiel: wf.writeframes(b'audiodaten')

    print(
        "Die Datei wurde erfolgreich gespeichert unter:", file_path)
    else:
        print("Speichern abgebrochen.")

    # Schliesse die App
    app.destroy()

save_wave_file()

```

In diesem Sample wird die Funktion „Speichern“ unter definiert, die es ermöglicht, eine Wave-Datei zu speichern. Zuerst wird ein Dateieexplorer geöffnet, um den Zielordner auszuwählen und die Audio-Datei kann im festgelegten Dateiformat (Wave) gespeichert werden. Anschließend wird die ausgewählte Wave-Datei im Schreibmodus geöffnet und die Parameter der Wave-Datei (Anzahl der Kanäle, Abtaste, etc.) festgelegt. Es wird eine Rückmeldung ausgegeben, ob die Datei erfolgreich gespeichert oder der Vorgang abgebrochen wird. Bei diesem Sample ist zu beachten, dass eine Wave-Datei ausgewählt werden muss, was in diesem Code nicht dargestellt wird.

```

import wave

input_file = 'input.wav'
wav_file = wave.open(input_file, 'r')

nchannels = wav_file.getnchannels()
sample_width = wav_file.getsampwidth()
framerate = wav_file.getframerate()
nframes = wav_file.getnframes()

audio_data = wav_file.readframes(nframes)

```

```

#erstellen der neuen Wavedatei im Schreibmodus
#setzen der Parameter basierend aus gelesenen Werten
output_file = 'output.wav'
wav_output = wave.open(output_file, 'w')
wav_output.setparams((nchannels, sample_width, framerate, nframes,
'NONE', 'not compressed'))

#schreiben von gelesenen Audiodaten in die neue Wavedatei
wav_output.writeframes(audio_data)

#schliessen sowohl die Eingabe- als auch die Ausgabedatei
wav_file.close()
wav_output.close()

```

Im Sample Speichern wird eine vorhandene Wave-Datei überschrieben mit den vom Nutzer festgelegten Parametern. Zunächst wird die Ursprungsdatei eingelesen und anschließend werden neue Parameter gesetzt. Zuletzt wird die Ursprungsdatei mit überschrieben.

15.1.4. Anzeigen von zwei Audio-Dateien

Um zwei Audio-Dateien in einem Diagramm mit Matplotlib darzustellen, kann die Wave Bibliothek verwendet werden, um die Audio-Dateien zu laden und mit der Matplotlib geplottet zu werden.

Begonnen wird mit dem Importieren der genutzten Bibliotheken.

Danach werden die Audiodaten aus den beiden Audio-Dateien geladen und mithilfe von der Bibliothek NumPy in Arrays umgewandelt. Anschließend erstellt Matplotlib ein Diagramm und zeichnet die Audiodaten aus beiden Audio-Dateien in dasselbe Diagramm. Die Label-Parameter in den Plot-Funktionen werden verwendet, um die Legende zu erstellen, sodass sichtbar ist, welche Linie zu welcher Audio-Datei gehört. Wichtig ist, dass die richtigen Pfade zu den Audio-Dateien angegeben sein müssen, wenn die Audio-Dateien nicht im selben Verzeichnis wie das Python-Skript liegen, muss zum Beispiel 'C:/Benutzer/IhrName/Dokumente/audio1.wav' anstelle von 'audio1.wav' verwendet werden.

Wichtig ist, dass die erforderlichen Bibliotheken installiert wurden, bevor dieser Code ausgeführt wird.

```

import matplotlib.pyplot as plt
import numpy as np
import wave

# Laden der Audiodateien
with wave.open('audio1.wav', 'rb') as wf1:
    y1 = wf1.readframes(-1)
    sr1 = wf1.getframerate()

with wave.open('inputAudio2.wav', 'rb') as wf2:
    y2 = wf2.readframes(-1)
    sr2 = wf2.getframerate()

# Umwandeln der Audio Dateien in numpy Arrays
y1 = np.frombuffer(y1, dtype=np.int16)
y2 = np.frombuffer(y2, dtype=np.int16)

```

```
# Achsenbestimmung des Diagramms
x1 = np.linspace(0, len(y1) / sr1, len(y1))
x2 = np.linspace(0, len(y2) / sr2, len(y2))

# Plotten der Daten
plt.figure(figsize=(10, 4))
plt.plot(x1, y1, label='Audio 1')
plt.plot(x2, y2, label='Audio 2')
plt.xlabel('Time (seconds)')
plt.ylabel('Amplitude')
plt.title('Audio Waveforms')
plt.legend()
plt.grid(True)
plt.show()
```

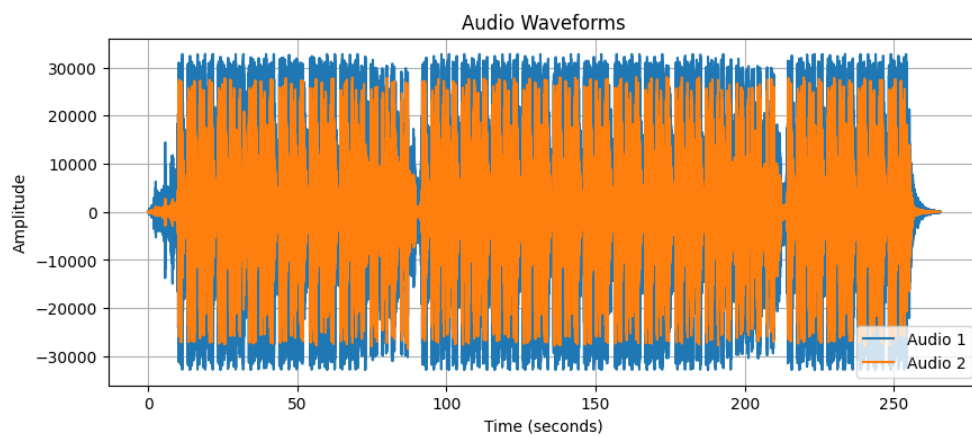


Figure 15.8.: Beispiel Anzeige von zwei Audio-Dateien

15.1.5. Glättung von Audio-Dateien

Zur Glättung der Audio-Datei wird die exponentielle Glättung verwendet. Die exponentielle Glättung ist einer der grundlegenden Algorithmen zur Glättung von Zeitreihen. Bei der exponentiellen Glättung wird die Wichtigkeit der vorhergegangenen Daten mit α gewichtet, je kleiner α , desto wichtiger sind die vorherigen Werte. Bei starkem Rauschen kann ein niedriger α gewählt werden, soll die Audio-Datei möglichst wenig verändert werden muss der α groß gewählt sein. Der mögliche nutzbare Zahlenbereich ist $1 \geq \alpha \geq 0$. Die genutzte Formel für die exponentielle Glättung sieht so aus.

$$S_i = \alpha * data_i + (1 - \alpha) * S_{i-1}$$

S_i = geglättet Daten zum Punkt i

α = alpha

$data_i$ = Daten zum Punkt i

S_{i-1} = geglättet Daten zum punkt i-1

Die benötigten Module sind `wave` und `numpy`. Für die Verwendung des Programms muss 'inputAudio.wav' durch den Pfad der zu glättenden Wave-Datei ersetzt werden. Der Ausgabebereich kann durch Ändern von `smoothedAudio.wav` definiert werden.

```

import wave
import numpy as np
# Definition von der Variable alpha
alpha = 0.01

# Definition der Glättungsfunktion
def exponential_smoothing(data):
    smoothed_data = np.zeros_like(data)
    smoothed_data[0] = data[0]
    for i in range(1, len(data)):
        smoothed_data[i] = alpha * data[i] + (1 - alpha) *
        smoothed_data[i - 1]
    return smoothed_data

def smooth_wave_file(input_file, output_file):
    with wave.open(input_file, 'rb') as wav_in:
        params = wav_in.getparams()
        data = np.frombuffer(wav_in.readframes(params.nframes),
            dtype=np.int16)
        smoothed_data = exponential_smoothing(data)
        with wave.open(output_file, 'wb') as wav_out:
            wav_out.setparams(params)
            wav_out.writeframes(smoothed_data.tobytes())

# Beispielaufruf:
input_wave_file = 'input.wav'
output_wave_file = 'smoothed_audio.wav'
smooth_wave_file(input_wave_file, output_wave_file)

```

Der Unterschied einer geglätteten Audio-Datei ist in Abbildung 15.8 gut zu beobachten. Dort wird die Ursprungsaudiodatei (Audio 1) und die Glättung dieser (Audio 2) mit dem $\alpha = 0.01$ dargestellt.

15.1.6. Ausschnitt

Zum Abspielen eines Abschnittes einer Wave-Datei in Python können die Bibliotheken `wave` und `pyaudio` verwendet werden. Die `wave` Bibliothek wird benötigt, damit Python das genutzte `wave` audio Format überhaupt einlesen kann. `Pyaudio` wird genutzt um die geladenen Audio-Datei anschließend auch noch abspielen zu können. Um jetzt das Programm zu nutzen, muss erst die richtige Wave-Datei geöffnet werden, wozu der Pfad zur Audio-Datei angegeben werden muss. Danach kann die Startdauer in Sekunden und die Dauer des Ausschnitts ebenfalls in Sekunden, unter Setzen der gewünschten Werte, angegeben werden.

```

import pyaudio
import wave

# Setzen der gewuenschten Werte
start = 4 # Startzeitpunkt in Sekunden
ende = 9 # Endzeitpunkt in Sekunden
dauer = ende - start # Dauer des Ausschnitts in Sekunden

# oeffne die WAV-Datei

```



```

#Durch ändern des Dateipfades anpassen der genutzen Wave Datei
wave_datei = wave.open('asdf.wav', 'rb')
# Erstelle ein PyAudio-Objekt
p = pyaudio.PyAudio()

# oeffne den Stream basierend auf dem Wave-Objekt
stream = p.open(
    format=p.get_format_from_width(wave_datei.getsampwidth()),
    channels=wave_datei.getnchannels(),
    rate=wave_datei.getframerate(),
    output=True)

# Lese die Daten (basierend auf der Chunk-Groesse)
wave_datei.readframes(start * wave_datei.getframerate())
data = wave_datei.readframes(dauer * wave_datei.getframerate())

# Spiele den Stream (von Anfang bis zum Ende)
stream.write(data)

# Speichern in einer neuen Datei
#Dateipfad der Auschnitt datei
neue_wave_datei = wave.open('ausschnitt.wav', 'wb')
neue_wave_datei.setparams(wave_datei.getparams())
neue_wave_datei.writeframes(data)
neue_wave_datei.close()

# Aufräumen
wave_datei.close()
stream.close()
p.terminate()

```

15.1.7. Zoomen in einer Matplotlib

Um innerhalb der von Matplotlib dargestellten Wave-Datei zu zoomen, wird der Zeitbereich angepasst. Dies geschieht durch Änderung der Start- und Endzeit des darzustellenden Graphen in Matplotlib. In diesem Beispiel soll die Wave-Datei von Sekunde 0 bis Sekunde 2 dargestellt werden. Soll nun ein anderer Bereich dargestellt werden, muss der Wert von `start_time` bzw. `end_time` geändert werden.

```

import numpy as np
import matplotlib.pyplot as plt
import wave

# Pfad zur Wave-Datei
wave_file_path = 'asdf.wav'

# Wave-Datei oeffnen
wave_file = wave.open(wave_file_path, 'r')

# Abtastrate (Sampling Rate) der Wavedatei
fs = wave_file.getframerate()

# Anzahl der Frames in der Wave-Datei

```

```

data_size = wave_file.getnframes()

# Daten aus der Wave-Datei lesen
wave_data = wave_file.readframes(data_size)
signal = np.frombuffer(wave_data, dtype=np.int16)

# Einstellen des Zeitbereichs
start_time = 0 # Startzeitpunkt (in Sekunden)
end_time = 2 # Endzeitpunkt (in Sekunden)

# Indizes für den gewünschten Zeitbereich
start_index = int(start_time * fs)
end_index = int(end_time * fs)

# Zeitvektor zuschneiden
cropped_time = np.linspace(start_time, end_time,
end_index - start_index)
cropped_signal = signal[start_index:end_index]

# Plot erstellen
plt.figure(figsize=(10, 6))
plt.plot(cropped_time, cropped_signal, 'b')
plt.xlabel('Zeit (s)')
plt.ylabel('Amplitude')
plt.title('Wave-Audiovisualisierung (Ausschnitt)')
plt.grid(True)
plt.show()

# Wave-Datei schliessen
wave_file.close()

```

Mit `wave_file.close` kann der Plot anschließend geschlossen werden. Neben dem Zoomen per Eingabe der Sekunden ist es auch möglich, die Eingabe per Mausrad umzusetzen. Das folgende Sample zeigt dies, anhand eines Sinussignals als Beispieltonspur.

```

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.widgets import Slider

# Beispieldaten erzeugen (Sinuskurve als Tonspur)
sampling_rate = 44100 # Abtastrate
duration = 5 # Dauer in Sekunden
t = np.linspace(0, duration, int(sampling_rate * duration),
endpoint=False)
frequency = 440 # Frequenz in Hz (A4-Ton)
signal = 0.5 * np.sin(2 * np.pi * frequency * t)

# Plot initialisieren
fig, ax = plt.subplots()
plt.subplots_adjust(bottom=0.25)
l, = plt.plot(t, signal, lw=1)
ax.set_title('Tonspuranzeige')
ax.set_xlabel('Zeit [s]')

```

```

ax.set_ylabel('Amplitude')

# Achsen-Initialwerte
axcolor = 'lightgoldenrodyellow'
axpos = plt.axes([0.1, 0.1, 0.65, 0.03], facecolor=axcolor)
spos = Slider(axpos, 'Pos', 0.0, duration, valinit=0.0)

# Zoom- und Scroll-Funktion
def zoom(event):
    cur_xlim = ax.get_xlim()
    cur_ylim = ax.get_ylim()
    xdata = event.xdata # x-position of mouse
    ydata = event.ydata # y-position of mouse
    if event.button == 'up':
        # Zoom in
        scale_factor = 1 / 1.5
    elif event.button == 'down':
        # Zoom out
        scale_factor = 1.5
    else:
        # Nicht zoomen
        scale_factor = 1
    return

    new_width = (cur_xlim[1] - cur_xlim[0]) * scale_factor
    new_height = (cur_ylim[1] - cur_ylim[0]) * scale_factor
    relx = (cur_xlim[1] - xdata) / (cur_xlim[1] - cur_xlim[0])
    rely = (cur_ylim[1] - ydata) / (cur_ylim[1] - cur_ylim[0])

    ax.set_xlim([xdata - new_width * (1 - relx),
xdata + new_width * (relx)])
    ax.set_ylim([ydata - new_height * (1 - rely),
ydata + new_height * (rely)])
    fig.canvas.draw()

def update(val):
    pos = spos.val
    ax.set_xlim([pos, pos + (cur_xlim[1] - cur_xlim[0])])
    fig.canvas.draw_idle()

spos.on_changed(update)
fig.canvas.mpl_connect('scroll_event', zoom)
plt.show()

```

Als Plot-Oberfläche wird Matplotlib verwendet. Die Funktion „Zoom“ behandelt das Zoom-Ergebnis, welches durch das Scrollen mit der Maus ausgelöst wird. Außerdem ermöglicht der Codeabschnitt, der den Befehl „slider“ enthält, das Verschieben der Ansicht entlang der Zeitachse. Das interaktive Steuern des Zooms wird für diese Funktion durch den letzten Codeblock ermöglicht.

15.1.8. Änderung der Tonhöhe und Geschwindigkeit

Bei einer Tonhöhenänderung bzw. Frequenzskalierung wird das digitale Signal im Nachhinein korrigiert, ohne andere Aspekte der Tonfolge wie z.B. die Wiedergabegeschwindigkeit zu beeinflussen. Das Phänomen, bei dem sich durch die Frequen-

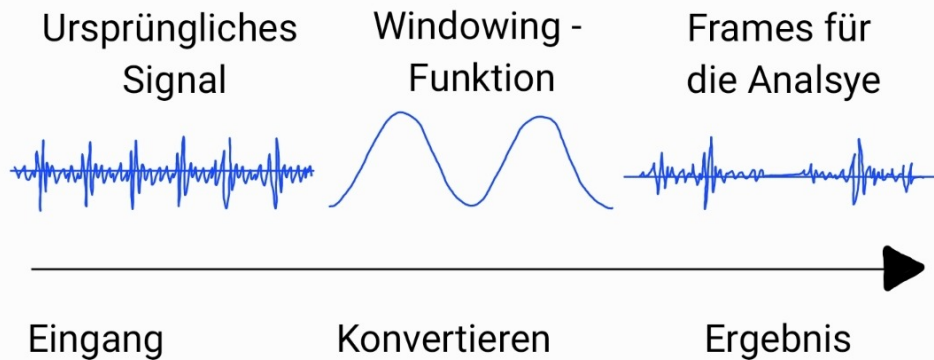


Figure 15.9.: Window-Funktion

zveränderung auch die Länge der Frequenz bzw. im übertragenen Sinne die Dauer der Audiodatei verändert, wird bei der Funktion umgangen (sog. Pitch-Shifting).

Eine ähnliche Situation tritt bei der Geschwindigkeitsveränderung auf. Das bloße Ändern der Wiedergabegeschwindigkeit führt zu Verzerrungen in den Tonhöhen. Das heißt, auch hier ist das Ziel, den Prozess der Komprimierung oder Streckung der Wiedergabedauer zu verändern, ohne den spektralen Inhalt zu beeinträchtigen (sog. Time-Stretching).

Folglich gilt für beide Funktionen: Um eine Änderung eines Parameters zu erreichen, muss die Abhängigkeit der Frequenz- und Zeitskalierung unterdrückt werden. Dafür wird „librosa.effects.pitch_shift“ bzw. „librosa.effects.time_stretch“ verwendet.

Das Verfahren basiert auf einem Phase Vocoder unter Anwendung der „Fast-Fourier Transformation“ (FFT) (siehe Kapitel 15.1.9). Im Folgenden wird der Algorithmus hinter dem Phase Vocoder verkürzt und vereinfacht erklärt.

Der erste Schritt bei einer Phase-Vocoder-Analyse ist die Windowing-Funktion. Dabei wird das digitale Samplesignal in zahlreiche Zeitblöcke unterteilt und diese Blöcke werden mit einer Hüllkurve multipliziert. Die dadurch entstehende Form der Window-Funktion hat Einfluss auf den bei diesem Prozess neu entstehenden Klang der Audio-Datei. Gängige Formen für einen Phase Vocoder sind meistens glockenförmig. Die Änderung des Signals wird in Abbildung 15.9 gezeigt.

Anschließend wird jedes Fenster einer Spektralanalyse unterzogen, die als STFT (Short-Time Fourier Transformation) bezeichnet wird. Der STFT-Algorithmus wendet nacheinander die diskrete Fourier-Transformation auf die gefensterten Abtastwerte an. Die STFT kann also als eine Folge von DFTs betrachtet werden. Eine DFT-Analyse gilt als zeitdiskret und diskret im Frequenzbereich, wodurch isolierte Spektrallinien dargestellt werden.

Jetzt muss die Anzahl der Frequenzbänder angegeben werden. Dies geschieht, indem für jedes positive Frequenzband ein gespiegeltes Frequenzband im negativen (imaginären) Bereich entsteht. Dabei sind beide Frequenzbänder Hälften der Amplitude des gemessenen realen Frequenzbands. Unter den verschiedenen Prozessen, die nach dieser Berechnung stattfinden, verwirft der Phasenvocoder die negativen Frequenzen und verdoppelt die Amplitude der positiven.

Das Ergebnis einer einzelnen FFT von fensterbehafteten Samples wird als Frame bezeichnet. Ein Frame besteht aus zwei Wertesätzen: den Magnituden (oder Amplituden) aller Frequenzbänder und den Anfangsphasen aller Frequenzbänder. Zusätzlich wird der Phasenunterschied zu dem vorherigen Frame bestimmt, was Hinweise darauf liefert, wo sich die Tonhöhe möglicherweise innerhalb des Frequenzbereichs dieses Bands befindet. Die Kombination aus Amplituden-/Phasen-unterschiedsdaten für jedes Band wird als Bin bezeichnet. Diese Paare von

Amplituden-/Phasendaten für jeden Bin werden als x-, y-Koordinaten ausgegeben. Zusätzlich muss noch der Analyseparameter Hop size oder Schrittweite bestimmt werden. Dieser Parameter gibt den Abstand zwischen den Anfangspunkten aufeinanderfolgender Fenster an. Mit ihm einher geht der Überlappungsfaktor, welcher die Menge an Informationen in der Analyse beeinflusst. Ein zu großer Überlappungsfaktor führt zu einem verschwommenen Klang.

Mit den aufgestellten Analysedaten wird jetzt die Resynthese des Klanges durchgeführt. Dabei wird durch eine inverse STFT eine laufende Aufzeichnung der Phasendifferenzen von Frame zu Frame in einem Prozess erstellt wird (Phase unwrapping).

Die Resynthese ist die Phase, in der sich das Vorgehen für Tonhöhenänderung und Geschwindigkeitsveränderung unterscheidet.

Für die Änderung der Tonhöhe werden in diesem Schritt die Frames in der Geschwindigkeit der Originalanalysedatei wiedergegeben. Die Frequenzen des gesamten Spektrums können jedoch proportional nach oben oder unten verschoben werden, was zu einer Änderung der Tonhöhe, aber nicht der Geschwindigkeit führt.

Für die Geschwindigkeitsveränderung wird die Rate, mit der die Frames neu erstellt werden, erhöht oder verringert, je nachdem, wie schnell oder langsam der ursprüngliche Ton abgespielt werden soll. Der Phase-Vocoder fügt zwischen den einzelnen Bildern neue Datenpunkte ein, um den Übergang zwischen ihnen sanfter zu gestalten (interpoliert). Da der Frequenzinhalt der einzelnen Frames nicht verändert wird, führen Änderungen der Geschwindigkeit nicht zu Änderungen der Tonhöhe.

WS:cite

Sample Tonhöhe ändern:

```
import librosa
import matplotlib.pyplot as plt
import soundfile as sf

# Pfad zur WAV-Datei
wav_datei = "input.wav"
# Einlesen der Datei
signal, abtastrate = librosa.load(wav_datei, sr=None, mono=False)
# Ändern der Tonhöhe durch Veränderung des n_steps= (Wertes) .
# Veränderung von 1 entspricht eines Halbtons.
höhen_änderung = librosa.effects.pitch_shift(signal,
n_steps=10,
sr=abtastrate,
bins_per_octave=12)

data = höhen_aenderung

# Ausgabe der Wave-Datei

sf.write('Höhenänderung.wav', data.T,
abtastrate, subtype='PCM_24', format='WAV')
```

Sample Geschwindigkeit ändern:

```
import librosa
import matplotlib.pyplot as plt
import soundfile as sf

# Pfad zur WAV-Datei
```

```

wav_datei = "input.wav"
# Einlesen der Datei
signal, abtastrate = librosa.load(wav_datei,
sr=None,
mono=False)

#Beschleunigen der Wave-Datei
beschleunigtes_Signal = librosa.effects.time_stretch(
signal, rate=1.5)

data = beschleunigtes_Signal

#Ausgabe der Wave-Datei

sf.write('Geschwindigkeit'aenderung.wav', data.T,
abtastrate, subtype='PCM_24', format='WAV')

```

15.1.9. Frequenzanalyse

Der Schall kann mit einem Mikrofon aufgezeichnet werden, wodurch die Spannungs-Zeit-Verläufe des Signals wiedergegeben werden. Allerdings ist es schwierig, die Signalkomponenten in dieser Form zu erkennen. Durch eine Frequenzanalyse können jedoch die Signaleigenschaften ermittelt und wichtige Muster erkannt werden. Eine Methode für die Frequenzanalyse ist die Spektralanalyse auf Basis einer Fast-Fourier-Transformation (FFT). FFT ist eine Spektralanalyse in der Audio- und Akustik-Messtechnik. Dabei wird ein zeitdiskretes Signal in seine Frequenzanteile zerlegt und dadurch analysiert. Eine Spektralanalyse ist ein grundlegendes Werkzeug für die Signalverarbeitung. Sie wird verwendet, um die Frequenzinhalte eines Signals zu untersuchen. Dabei wird ermittelt, welche Frequenzkomponenten im Originalsignal enthalten sind und wie sie verteilt sind.

WS:cite

FFT ist ein Algorithmus zur Berechnung der Fourier-Reihe, die auf diskrete Signale angewandt wird (DFT).

Für die diskrete Fourier Transformation gilt die Fourier Reihendarstellung (siehe Formel 15.1.9) mit dem diskreten Fourier-Koeffizienten (siehe Formel 15.1.9).

$$f_T(t) = \sum_{k=-\infty}^{\infty} \gamma_k e^{ik\omega t} \quad \text{mit } \omega = \frac{2\pi}{T} \quad (15.4)$$

$$\gamma_k \equiv \gamma_k(f) = \frac{1}{T} \cdot \int_{T/2}^{-T/2} f_T(\tau) e^{-ik\omega\tau} d\tau \quad \text{für } k = 0, \pm 1, \pm 2, \dots \quad (15.5)$$

Wobei:

f_T = Zeitkontinuierliches, T – periodisches Signal

$e^{-ik\omega\tau}$ = Grundschiwingung

$\omega = \frac{2\pi}{T}$ = Frequenz

γ_k = Verstärkungsfaktor der Grundschiwingung

Die Fourier Reihenformel beschreibt im Allgemeinen die Darstellung periodischer Signale im Frequenzbereich als Summe von Sinus- und Kosinusschwingungen unterschiedlicher Frequenzen. Bei der DFT werden aus den kontinuierlichen, periodischen Signale nicht-periodische, diskrete Signale.

WS:cite

Dementsprechend muss, um eine solche Rechenoperation durchzuführen, noch folgende Spektralanalyseparameter definiert werden:

- Anzahl der abgetasteten Punkte (NFFT)
- Abtastfrequenz

Durch diese wird die Frequenzauflösung der Analyse bestimmt.

Für das Sample einer Frequenzanalyse werden die Bibliotheken `matplotlib.pyplot` und `numpy` verwendet. Das Signal wird in dem Sample erzeugt und bezieht sich auf keine Audio-Datei.

```
#verwendete Bibliotheken
import matplotlib.pyplot as plt
import numpy as np

# Reproduzierbarkeit der Zufallszahlen
np.random.seed(19680801)

#Erzeugung von Zeit und Signalsample
dt = 0.0005
t = np.arange(0.0, 20.5, dt)

#Generierung von zwei Sinussignalen
s1 = np.sin(2 * np.pi * 100 * t)
s2 = 2 * np.sin(2 * np.pi * 400 * t)

# "chirp"-Signal für Frequenzveränderung über die Zeit
s2[t <= 10] = s2[12 <= t] = 0

# Rauschen hinzufügen für realistische Szenarien
nse = 0.01 * np.random.random(size=len(t))

x = s1 + s2 + nse # Signal mit Rauschen
NFFT = 1024 # Anzahl der abgetasteten Punkte
Fs = 1/dt # the Abtastgeschwindigkeit

#Plotten des Signals und Spektrogramms
fig, (ax1, ax2) = plt.subplots(nrows=2, sharex=True)
ax1.plot(t, x)
ax1.set_ylabel('Signal')

#Berechnung der Spektrogramms
Pxx, freqs, bins, im = ax2.specgram(x, NFFT=NFFT, Fs=Fs)
# Mit:
# - Pxx: Spektralleistung
# - freqs: Frequenzvektor
# - im: Bild der Spektrogrammdateien

#Achsenbeschriftung
ax2.set_xlabel('Time (s)')
ax2.set_ylabel('Frequency (Hz)')
```

```
ax2.set_xlim(0, 20)

#Plot anzeigen
plt.show()
```

Die Ausgabe des Codes sind zwei Diagramme. Das obere Diagramm zeigt das Signal und das untere die Frequenzanalyse auf Basis der FFT. (siehe Abbildung 15.10)

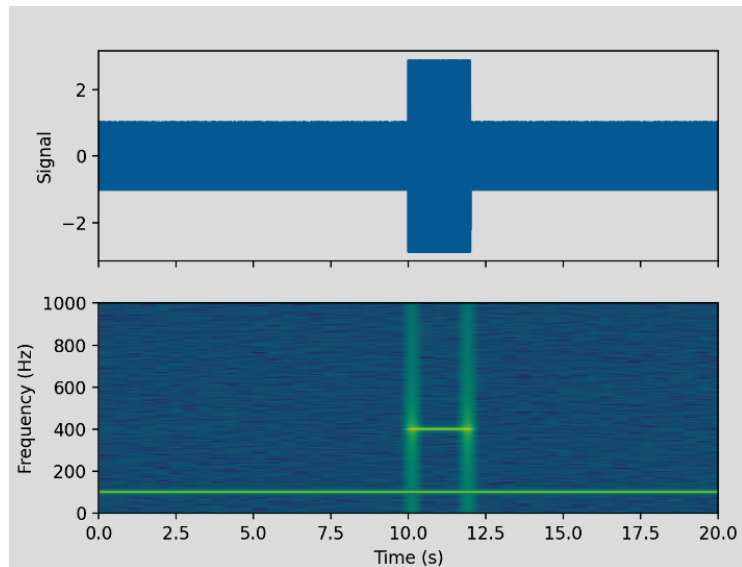


Figure 15.10.: Ausgabe Sample Frequenzanalyse

15.1.10. Overlay

Das folgende Sample beschränkt sich darauf, dass 3 Wave-Dateien gleichzeitig abgespielt werden.

```
import wave
import pyaudio
import threading

def play_sound(file):
    chunk = 1024
    wf = wave.open(file, 'rb')
    p = pyaudio.PyAudio()
    stream = p.open(format=p.get_format_from_width(
        wf.getsampwidth()),
        channels=wf.getnchannels(),
        rate=wf.getframerate(),
        output=True)
    data = wf.readframes(chunk)

    while data:
        stream.write(data)
        data = wf.readframes(chunk)

    stream.stop_stream()
    stream.close()
```



```

p.terminate()

files = ['sound1.wav', 'sound2.wav', 'sound3.wav']

threads = []
for file in files:
    thread = threading.Thread(target=play_sound, args=(file,))
    threads.append(thread)
    thread.start()

for thread in threads:
    thread.join()

```

Für das Sample müssen `sound1.wav`, `sound2.wav` und `sound3.wav` durch den gewünschten Dateipfad ersetzt werden. Die Gesamtlänge wird durch die Audio-Datei mit der längsten Dauer bestimmt, da der Code erst beendet wird, sobald alle Audio-Dateien komplett abgespielt wurden. Es werden in diesem Code zwei Schleifen verwendet, wobei die erste die Audio-Dateien in der Liste `files` durchläuft. Anschließend wird für jede Datei ein eigener Thread erstellt, welcher das synchrone Abspielen aller Dateien ermöglicht, indem die Funktion `playsound` mit entsprechenden Audio-Dateien als Argument aufgerufen werden. Parallel wartet die zweite Schleife darauf, dass alle Threads beendet sind, bevor das Programm zuletzt beendet wird.

15.2. General

General description

WS:cite books

15.2.1. Decibels

In electronics one often wants to represent the ratio of two signals on a logarithmic scale. For this we use the decibel, dB . If we have two powers P_1 and P_2

$$dB = 10 \log_{10} \left(\frac{P_2}{P_1} \right)$$

or equivalently – when comparing two signals with the same kind of waveform –

$$dB = 20 \log_{10} \left(\frac{V_2}{V P_1} \right).$$

Note dB is a relative unit of measurement, i.e. *This amplifier has a gain of 10 dB*. dB can be positive or negative. For example, $+10dB$ corresponds to P_2 greater than P_1 by a factor of 10, and $-3dB$ corresponds to P_2 less than P_1 by approximately a factor of 2.

For an absolute measure on a logarithmic scale there are a variety of other units. A common one is dBm .

$$dBm = 10 \log_{10}(P[mW])$$

Zero dBm corresponds to $1mW$ of power. And in a 50Ω system $0dBm$ corresponds to $220mV_{rms}$.

WS:cite books

15.2.2. Puls-Dichtemodulation

Pulsdichtemodulation (PDM) ist eine faszinierende Technik, die im Arduino Nano 33 BLE Sense verwendet wird, um Audiosignale zu digitalisieren. PDM wandelt analoge Audiosignale in digitale Daten um, indem es die Dichte der Impulse misst. Statt kontinuierlicher Werte verwendet PDM nur zwei Zustände: 1 (Impuls vorhanden) oder 0 (kein Impuls). Um das Mikrofon zu verwenden, steht die Bibliothek [pdm.h](#) zur Verfügung. PDM arbeitet mit einer hohen Samplingrate, um genaue Audioinformationen zu erfassen. Die Impulse werden als Bitstream übertragen, der später in Audiodaten umgewandelt wird.

Die Puls-Dichtemodulation (PDM) ist eine interessante Technik, die sowohl Vor- als auch Nachteile aufweist.

Die Puls-Dichtemodulation (PDM) hat folgende Vorteile:

- Einfachheit: PDM ist weniger komplex als einige andere Modulationsverfahren. Die Implementierung von Sendern und Empfängern für PDM ist vergleichsweise unkompliziert.
- Robustheit gegenüber Störungen: PDM ist widerstandsfähig gegenüber Rauschen und Interferenzen.
- Hohe Samplingrate: PDM arbeitet mit einer hohen Abtastrate, um genaue Audioinformationen zu erfassen.
- Geringe Verluste: Im Vergleich zu analogen Steuerungen verursacht PDM weniger Verluste, da es die Versorgungsspannung schnell ein- und ausschaltet.

Die Nachteile der Puls-Dichtemodulation (PDM) sind:

- Große Lasten: Die Steuerung großer Verbraucher erfordert hohe Spannungen und Ströme. Standard-Analogschaltungen und DACs sind hierbei weniger effizient.
- Wärmeentwicklung: Leistungstransistoren, die in PDM-Schaltungen verwendet werden, erzeugen Wärme und Verluste.

Insgesamt ist PDM eine leistungsstarke Technik, die in verschiedenen Anwendungen wie Spracherkennung, Klanganalyse und Datenübertragung eingesetzt wird. Es ist wichtig, die Vor- und Nachteile je nach Kontext zu berücksichtigen

WS:citations

15.3. Specific Sensor

cite board

15.4. Specification

- cite data sheet
- Circuit Diagram

Der Sensor hat folgende Spezifikationen:

- Signal-Rausch-Verhältnis: 64 dB
- Empfindlichkeit: -26 dBFS $\pm$ 3 dB
- Temperaturbereich: -40 bis 85 °C

Mikrofone werden in Mobilgeräten, Spracherkennungssystemen und sogar in Gaming- und Virtual-Reality-Eingabegeräten eingesetzt.

Mit dem eingebetteten MP34DT05-Sensor können Sie die Schallwerte Ihrer Umgebung messen und anzeigen.

WS:cite

15.5. Bibliothek `pdm.h`

15.5.1. Description

Die PDM-Bibliothek ermöglicht die einfache Verwendung des integrierten Mikrofons und ist auch über die Bibliothek `ArduinoSound` zugänglich.

15.5.2. Installation

15.5.3. Functions

Die PDM-Bibliothek für den Arduino Nano 33 BLE Sense ermöglicht die Verwendung von Puls-Dichtemodulation (PDM)-Mikrofonen, wie dem integrierten MP34DT05 auf dem Board. Hier sind die wichtigsten Funktionen, die diese Bibliothek bereitstellt:

`begin()` : Diese Funktion initialisiert die PDM-Schnittstelle. Sie nimmt zwei Parameter entgegen:

Parameter `channels` : Die Anzahl der Kanäle (1 für Mono, 2 für Stereo).

Parameter `sampleRate` : Die gewünschte Abtastrate in Hertz.

Beispiel:

```
1000 if (!PDM.begin(1, 16000)) {
      Serial.println("Fehler beim Starten der PDM!");
1002   while (1);
      }
1004
```

`end()` : Mit dieser Funktion wird die PDM-Schnittstelle deinitialisiert:

```
1000 PDM.end();
```

`available()` : Diese Funktion gibt die Anzahl der verfügbaren Bytes im PDM-Empfangspuffer zurück. Das sind bereits eingetroffene Daten, die darauf warten, gelesen zu werden:

```
1000 int bytesAvailable = PDM.available();
```

`read()` : Mit dieser Funktion liest man Daten aus dem PDM in einen angegebenen Puffer:

```
1000 short sampleBuffer[256]; // Puffer fuer Samples (16-Bit)
1002 int bytesRead = PDM.read(sampleBuffer, bytesAvailable);
```

onReceive() : Diese Funktion setzt die Callback-Funktion, die aufgerufen wird, wenn neue PDM-Daten zum Lesen bereit sind:

```

1000 void onPDMdata() {
      int bytesAvailable = PDM.available();
1002     // Weitere Verarbeitung der Daten...
      }
1004 PDM.onReceive(onPDMdata);

```

15.5.4. Example - Manual

15.5.5. Example

Der Code verwendet die folgenden Bibliotheken:

- **PDM.h**: Diese Bibliothek ermöglicht die Kommunikation mit dem Mikrofon zur Aufnahme von PDM-Signalen [Ardd]

15.5.6. Example - Code

Listing 15.1.: Simple sketch to control the built-in LED

```

1000 /**
      * @file TestLEDBuiltin.ino
1002  *
      * @brief Simple program for testing the built-in LED
1004  *
      *
1006  * Turns the built-in LED on for one second, then off for one second,
      * repeatedly.
      *
1008  * The LED is switched on for 1 second and switched off
      * for 1 second so that the LED flashes accordingly.
1010  *
      * On the Arduino Nano 33 BLE Sense, it is attached to digital pin 13
1012  *
      */
1014
1016 #include <../LEDs/BuiltinLED.h>
      #include <../LEDs/LED.h>
1018
1020 /**
      * @brief the setup function runs once when you press reset or power the
      * board
1022  *
      * Standard function of Arduino sketches
      *
1024  * Initialization of the pin LED_BUILTIN as output
      *
1026  * @param —
      *
1028  * @return void
      */
1030 void setup() {
      // Initialize the pin as an output
1032     BuiltinLEDinit();
      }
1034
1036 /**
      * @brief the loop function runs over and over again forever
      *

```

```

1038 * standard function of Arduino sketches
1040 * swichting the led on / off for 1sec.
1042 * @param ——
1044 * @return void
1046 */
1046 void loop() {
1048     // Turn the LED on
1048     BuiltinLED(SET_ON);
1050     // Wait for one second
1050     delay(1000);
1052     // Turn the LED off
1052     BuiltinLED(SET_OFF);
1054     // Wait for one second
1054     delay(1000);
1054 }

```

../Code/Nano33BLESense/Test/TestLEDBuiltin.ino

15.5.7. Example - Files

15.6. Calibration

cite method

15.7. Simple Code

15.8. Simple Application

15.8.1. Pin-Konfiguration

WS:Description

Die verwendeten Pins werden zu Beginn des Codes definiert:

- `microphoneDataPin`: Der Datenpin für das Mikrofon
- `microphoneClockPin`: Der Clock-Pin für das Mikrofon

15.8.2. Initialisierung und Setup

In der Funktion `setup()` werden die erforderlichen Initialisierungen durchgeführt:

- `PDM.onReceive(onPDMdata)`: Registriert den Empfang des Mikrofonsignals als PDM-Signal

15.8.3. Messungssteuerung

Die Funktion `onPDMdata()` wird aufgerufen, wenn Daten vom Mikrofon verfügbar sind. Dies liest die Daten in einen Array ein und zählt die Anzahl der gelesenen Samples.

Listing 15.2.: Einlesen der Daten

```

1000 const unsigned long averagingTime = 200; /*< Time [ms] to moving
      average the values */
      unsigned long averagingStartTime = 0; /*< Start time [ms] to moving
      average the values */
1002 int valueSum = 0; /*< sum of the values to
      moving average */

```

```
int valueCount = 0;           /*< number of the values to
    moving average */
```

```
../Code/Nano33BLESense/Test/TestMicrophone.ino
```

Die Hauptfunktion `loop()` enthält zwei Funktionen für die Messungssteuerung und die Benutzeroberfläche:

Listing 15.3.: Messungssteuerung

```
1000 int absoluteCount = 0;    /*< absolute number of the values to moving
    average */
1002 int redButtonResult = 0; /*< Button pressed? */
```

```
../Code/Nano33BLESense/Test/TestMicrophone.ino
```

Die Funktion `HandleInput()` überwacht den Zustand der beiden Taster und steuert somit den Messungsablauf:

Listing 15.4.: Überwachung

```
1000 /**
1002 * @brief the setup function runs once when you press reset or power the
    board
    *
1004 * standard function of Arduino sketches
    *
1006 * Initialization of
    * — the display
1008 * — the interrupt pin
    * — the microphone (PDM)
1010 *
1012 * @param —
    *
1014 * @return void
    */
1016 void setup() {
1018     display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
1020     display.clearDisplay();
1022     display.setTextColor(WHITE);
1024     display.setTextSize(2);

    pinMode(buttonPin, INPUT_PULLUP);
    Serial.begin(9600);

    PDM.onReceive(onPDMdata);
}
1026
1028 /**
    * @brief Interrupt service routine for the microphone
    *
1030 *
    * Attention: as short as possible
1032 *
    * If an interrupt is on, the function checks if there are samples,
1034 * copies them into the buffer and sets a flag
    *
1036 * @param —
    *
1038 * @return void
```

```
../Code/Nano33BLESense/Test/TestMicrophone.ino
```

- Wenn der blaue Taster gedrückt wird, wird die Messung gestartet, indem `isMeasuring` auf `true` gesetzt wird und die Funktion `StartSampling()` aufgerufen wird.

- Wenn der rote Taster gedrückt wird, wird die Messung beendet, indem `isMeasuring` auf `false` gesetzt wird und die Funktion `StopSampling()` aufgerufen wird. Je nach vorherigem Zustand des roten Tasters wird entweder der maximale Wert SPL in dB angezeigt (`ShowMaxdBspl()`) oder der durchschnittliche dB SPL-Wert (`ShowAveragedBspl()`).
- `isDelayOver` wird auf `true` gesetzt, wenn die Startverzögerung (definiert durch `measureThresholdMilliseconds`) von 1200ms abgelaufen ist.
- Es gibt eine kurze Verzögerung (`delay(50)`) zwischen den Schleifendurchläufen, um Tastenprellungen zu vermeiden.

15.8.4. Mikrofondatenverarbeitung

Die Funktion `StartSampling()` initialisiert die Datenverarbeitung:

Listing 15.5.: Initialisierung der Datenverarbeitung

```

1000 * @brief the loop function runs over and over again forever
1001 *
1002 * standard function of Arduino sketches
1003 *
1004 * The function reads the samples and sends the samples to the display
1005 *
1006 * @param —
1007 *
1008 * @return void
1009 */

```

../Code/Nano33BLESense/Test/TestMicrophone.ino

`PDM.begin(1, 16000)`: Initialisiert eine Abtastrate von 16.000 Hz. Die Funktion `StopSampling()` beendet die Aufnahme von Audiodaten.

15.8.5. Anzeige der Ergebnisse

Die Funktionen `ShowResult()` und `ShowMicrophoneValues()` sind für die Anzeige der Messergebnisse auf dem OLED-Display verantwortlich. `ShowResult()` zeigt den aktuellen SPL-Wert in dB auf dem Display an und aktualisiert den maximalen Wert SPL in dB, wenn nach der Startverzögerung ein neuer Höchstwert erreicht wird. `ShowMicrophoneValues()` verarbeitet die vom Mikrofon empfangenen Signale. Der maximale Samplewert wird ermittelt und verwendet, um den Wert SPL in dB zu berechnen. Die Funktion berechnet auch einen Durchschnittswert über eine bestimmte Zeit, `averagingTime = 200ms`, und zeigt das Ergebnis auf dem Display an. Die Funktionen `ShowMaxdBspl()` und `ShowAveragedBspl()` zeigen den maximalen bzw. den durchschnittlichen Wert SPL in dB auf dem Display an.

15.8.6. Umrechnung auf den Wert dB SPL

Die Umrechnung des Mikrofonsignals auf den dB SPL-Wert erfolgt in der Funktion `getDbValueFromPMC()`:

Listing 15.6.: Umrechnung des Mikrofonsignals

```

1000 bool blueButtonIsPressed = analogRead(blueButtonPin) == LOW;
1001 bool redButtonIsPressed = analogRead(redButtonPin) == LOW;
1002
1003 if (blueButtonIsPressed)
1004     isMeasuring = true;

```

../Code/Nano33BLESense/Test/TestMicrophone.ino

`MaxSampleVoltage` wird berechnet, indem der maximale Samplewert `maxSampleValue` mit der Referenzspannung des Mikrocontrollers („Vref = 3,3 V“) multipliziert und durch den maximalen Wert eines 16-Bit-Signed-Integers (32767) dividiert wird. 32767 ist der maximale Wert eines 16-Bit-Signed-Integers, der der maximalen Amplitude des Audiosignals entspricht. Der dB SPL-Wert („dBspl“) wird mit der Formel „ $20 * \log_{10}(\text{Voltage} / \text{Vrms}) + \text{sensitivity}$ “ berechnet [Quelle der Formel: PDF Decibels Formula https://physicscourses.colorado.edu/phys3330/phys3330_sp19/resources/Decibels_Phys_3330.pdf University of Colorado System], wobei „Voltage“ der berechnete „maxSampleVoltage“ ist, und „sensitivity“ die Empfindlichkeit des Mikrofons in dBV (Dezibel-Volt) ist. „Vrms“ ist die RMS-Spannung (Root Mean Square), die einem Schalldruckpegel von 94 dB SPL entspricht. Dieser Wert wurde experimentell ermittelt.

15.8.7. Benutzeroberfläche

Die Funktion `HandleUI()` aktualisiert OLED-Display Anzeige:

Listing 15.7.: Aktualisierung des Bildschirms

```

1000 * @param _____
1001 *
1002 * @return void
1003 */
1004 void StopSampling() {
1005     PDM.end();
1006 }

```

../Code/Nano33BLESense/Test/TestMicrophone.ino

Wenn eine Messung aktiv ist (`isMeasuring == true`), werden die aktuellen Messwerte auf dem Display angezeigt. Andernfalls wird der Displayinhalt aktualisiert, ohne neue Werte anzuzeigen.

15.9. Tests

The embed on-board MP34DT05 sensor in Arduino Nano 33 BLE Sense has the functionality to sense audio voice from the environment. There is build in Arduino library for this particular sensor, which is PDM as shown in the figure. 15.11

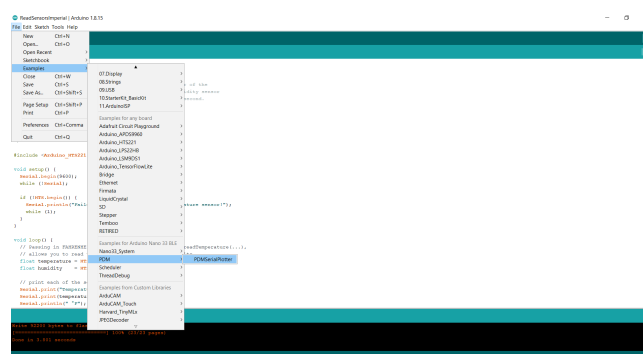


Figure 15.11.: MP34DT05, Digital Microphone

By following the same steps, for this sensor we can see the output the resulted frequency in the serial plotter instead of serial monitor as shown in figure. 15.12 It is more convenient to understand if we change the loudness of voice the plotter shows us a different frequency curve. MP34DT05 use to detect different voices or words too, with the help of these functionality we can easily make the valuable application with Arduino

15.13. Calibration

The microphone typically does not require calibration for standard applications. However, in advanced audio analysis or machine learning applications, calibrating for specific environments or sound pressure levels may be necessary.

15.14. Simple Code

Below is an example of code to read data from the microphone:

Listing 15.8.: PDM Microphone Example Code

```

1000 /**
1001  * @file PDM_Microphone.ino
1002  * @brief Example to capture and process audio data using PDM microphone.
1003  *
1004  * This example demonstrates how to initialize the PDM microphone,
1005  * read audio samples into a buffer, and print them to the Serial Monitor
1006  *
1007  * @note Ensure the PDM library is installed, and the board supports PDM
1008  *       input.
1009  *
1010  * @author Public Domain
1011  */
1012 #include <PDM.h> ///< Include the library for PDM microphone.
1013
1014 void setup() {
1015     Serial.begin(9600); ///< Start Serial communication at 9600 baud rate.
1016
1017     // Initialize the PDM microphone with 1 channel and 16 kHz sample rate
1018     if (!PDM.begin(1, 16000)) {
1019         Serial.println("Failed to start PDM!"); ///< Error message if
1020         initialization fails.
1021         while (1); ///< Halt the program if PDM initialization fails.
1022     }
1023
1024 void loop() {
1025     // Check how many bytes are available in the buffer.
1026     int bytesAvailable = PDM.available();
1027     if (bytesAvailable > 0) {
1028         short buffer[bytesAvailable]; ///< Declare a buffer to hold audio
1029         samples.
1030
1031         // Read the audio samples into the buffer.
1032         PDM.read(buffer, bytesAvailable);
1033
1034         // Print each sample to the Serial Monitor.
1035         for (int i = 0; i < bytesAvailable / 2; i++) {
1036             Serial.println(buffer[i]);
1037         }
1038     }
1039 }

```

../Code/Nano33BLESense/MikrophoneMP34DT05/MikrophoneMP34DT05/MikrophoneMP34DT05.ino

15.15. Simple Application

The microphone can be used in a sound level meter or voice-controlled interface. For instance:

- Detect claps for controlling lights.
- Capture audio for keyword spotting in machine learning applications.

15.16. Tests

- **Signal Test:** Verify that the microphone captures sound by observing output values on the serial monitor when exposed to different sound levels.
- **Frequency Response Test:** Use a frequency generator to check the microphone's response across its frequency range.
- **Noise Test:** Measure the baseline noise level in a quiet environment.

15.17. Further Readings

- Arduino Nano 33 BLE Sense Official Cheat Sheet: <https://docs.arduino.cc/tutorials/nano-33-ble-sense/cheat-sheet/>
- PDM Library Documentation: <https://www.arduino.cc/reference/en/libraries/pdm/>
- MEMS Microphone Datasheet: <https://www.st.com/resource/en/datasheet/mp34dt05-a.pdf>

WS:citations

16. Inertial Measurement Unit

16.1. Introduction

An Inertial Measurement Unit (IMU) is an electronic device that measures and reports a body's specific force, angular rate, and sometimes the orientation of the body. It consists of a combination of three sensors accelerometers, gyroscopes, and magnetometer that work together to provide information about an object's acceleration, velocity, orientation, and gravitational forces.[Sab11]

Arduino's library [Arduino_LSM9DS1](#) of the LSM9DS1 sensor contains three examples that allow the user to use one of the sensors with little effort.

WS:missing citation

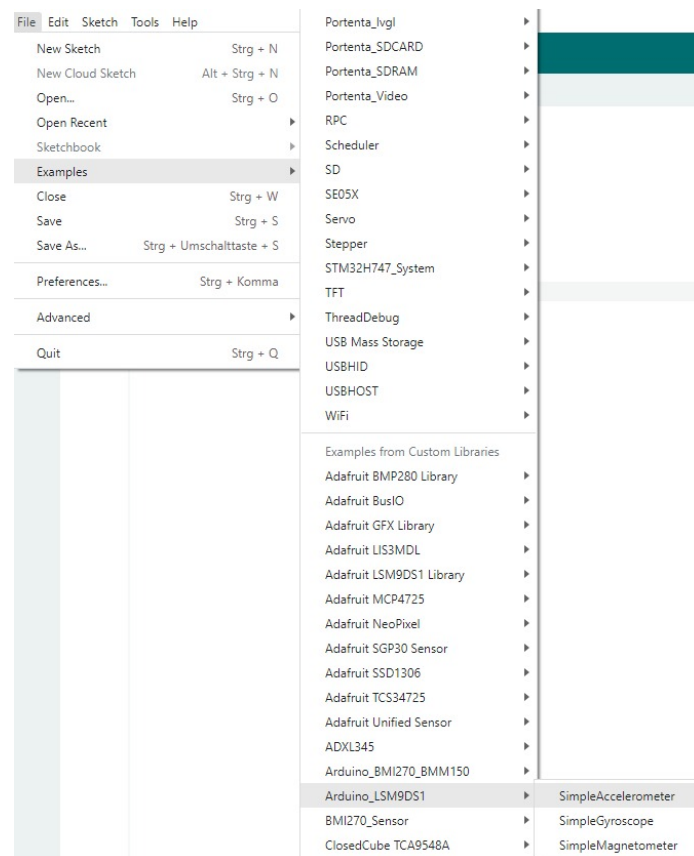


Figure 16.1.: Examples in the library [Arduino_LSM9DS1](#)

In addition to accelerometers and gyroscopes, IMUs may also include magnetometers, which measure the direction and strength of magnetic fields. These sensors can help provide additional information about the orientation and movement of an object in relation to Earth's magnetic field.[Wan+22]

Inertial Measurement Units (IMUs) are commonly used in various fields such as aerospace, robotics, and gaming. The device is made up of multiple sensors that can

measure the acceleration and angular rate of an object in three dimensions. The accelerometers measure linear motion in three axes (x, y, z), while the gyroscopes measure angular motion around these same axes.[Wah+11]

16.2. 6-Axis IMU LSM6DSOX

The LSM6DSOX is a 6-axis IMU developed by STMicroelectronics that features a high-performance 3-axis digital accelerometer and 3-axis digital gyroscope. The device is designed for accurate motion tracking and is commonly used in applications such as wearable devices and smart phones. It can measure linear and rotational motion simultaneously.[Stma] The device also includes a variety of other features such as a programmable digital signal processor (DSP), a configurable low-pass filter, and a built-in temperature sensor.

16.3. IMU LSM6DSOX Features

Features of the LSM6DSOX IMU, see [Stma], typically referring to the technical specifications and capabilities of the sensor.

Here are the features of LSM6DSOX IMU features:

- **Power consumption:**

Power consumption is an important factor when it comes to battery-powered devices like nicle vision and smart phones. These devices are designed to be portable and convenient, and they rely on batteries to power them. Therefore, the power consumption of each component is a critical consideration to extend battery life and to provide better user experience.

The LSM6DSOX has a power consumption of 0.55 mA in combo high-performance mode. This means that the sensor can operate at a low power consumption level while still delivering high-performance data. In this mode, the sensor can measure both acceleration and angular rate simultaneously, and provide accurate and reliable data with a high sampling rate.

The combo high-performance mode is an optimized mode that reduces power consumption without compromising on data accuracy and reliability. It achieves this by using advanced algorithms and signal processing techniques to filter out noise and unwanted signals, resulting in high-quality data with low power consumption.

- **Always-on:**

This means that the LSM6DSOX can be kept running all the time, even when a device is in sleep mode, without using up too much power. This is useful for applications that require continuous motion tracking, such as fitness tracking or navigation.

- **Smart FIFO:**

The Smart FIFO (First In First Out) is a buffer that can store up to 9 kilobytes of motion data. It is "smart" because it can be configured to store only the most important data, saving memory and reducing power consumption.[MAB22]

- **Android compatibility:**

The LSM6DSOX is compatible with the Android operating system, which is used in many smartphones and tablets.

- **Accelerometer full scale:**

The accelerometer in the LSM6DSOX can detect motion in up to four different full-scale ranges, from $\pm 2g$ to $\pm 16g$. full scale refers to the maximum range of acceleration that can be measured by the sensor without saturating or clipping the output signal.

For example, if an accelerometer has a full scale range of $\pm 2g$, [LAH22] it means that the sensor can accurately measure accelerations up to $2g$ in both the positive and negative directions. If the measured acceleration exceeds this range, the sensor output will saturate at the maximum value, which may cause distortion in the output signal.

- **Gyroscope full scale:**

The gyroscope in the LSM6DSOX can detect rotation in up to five different full-scale ranges, from ± 125 degrees per second (dps) to ± 2000 dps. Each range corresponds to a certain maximum angular velocity that the sensor can detect.

- **Analog supply voltage:**

1.71 V to 3.6 V: This is the voltage range that the LSM6DSOX can operate at.

- **Independent IO supply:**

The input/output connections on the LSM6DSOX can operate at a separate voltage of 1.62 V.

- **Compact footprint:**

2.5 mm x 3 mm x 0.83 mm: This is the size of the LSM6DSOX package, which is small enough to fit many types of electronic devices.

- **SPI / I²C & MIPI I3CSM serial interface with main processor data synchronization:**

These are three different types of communication interfaces that the LSM6DSOX supports. The main processor data synchronization means that the data collected by the sensor can be synchronized with other sensors or data sources.

- **Auxiliary SPI for OIS data output for gyroscope and accelerometer:**

This is an additional interface that can be used to output data from the gyroscope and accelerometer.

- **OIS configurable from Aux SPI, primary interface (SPI/I²C & MIPI I3CSM):**

The OIS (optical image stabilization) feature of the LSM6DSOX can be configured using either the auxiliary SPI or one of the other serial interfaces.

- **Advanced pedometer, step detector, and step counter:**

These features allow the LSM6DSOX to accurately track the number of steps taken by a person wearing a device that contains the sensor.

- **Significant Motion Detection, Tilt detection:**

The LSM6DSOX can detect significant motion events, such as a sudden acceleration or deceleration, as well as changes in orientation or tilt.

- **Standard interrupts:**

Free-fall, wakeup, 6D/4D orientation, click and double-click: These are pre-programmed events that can trigger.

16.4. IMU LSM6DSOX Data

The LSM6DSOX is a type of Inertial Measurement Unit (IMU) sensor that measures acceleration, angular speed and temperature. The data output from this sensor includes acceleration, gyroscope, and temperature measurements.

Accelerometer that measures the acceleration (A_x , A_y , A_z) the rate of change of acceleration over time. The acceleration in each axis is typically reported in units of meters per second squared (m/s^2). For example, if the LSM6DSOX measures an acceleration of $5 m/s^2$ in the x-axis, it means that the object being measured is increasing in velocity by 5 meters per second every second in the x-direction. See figure 16.2

Gyroscope that measures the angular speed (Ω_x , Ω_y , Ω_z) is a measure of the rate of change of the orientation of the object being measured with respect to each axis. The angular velocity in each axis is typically reported in units of degrees per second ($^\circ/s$). For example, if the LSM6DSOX measures an angular velocity of $100^\circ/s$ in the z-axis, it means that the object being measured is rotating at a rate of 100 degrees per second around the z-axis. See figure 16.3

Temperature sensor that measures the ambient temperature (T) in degrees celsius ($^\circ C$). The temperature sensor is located on the same chip as the accelerometer and gyroscope, and it operates by detecting changes in the voltage output of a diode that is sensitive to temperature. The temperature sensor can be used in a variety of applications, such as monitoring the temperature of electronic devices, measuring the temperature of an environment in which the sensor is placed, or compensating for temperature changes in the output of the accelerometer and gyroscope.

16.4.1. Accelerometer:

[Chi+06]

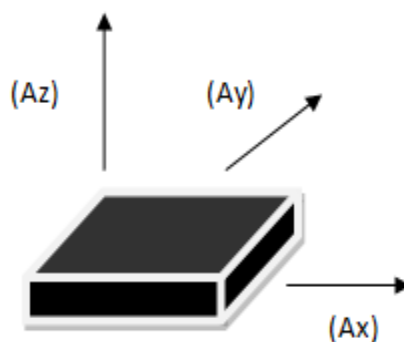


Figure 16.2.: *Axis Acceleration of Accelerometer Sensor*

WS:tikz!

- Acceleration in X-axis (A_x): meters per second squared (m/s^2)
- Acceleration in Y-axis (A_y): meters per second squared (m/s^2)
- Acceleration in Z-axis (A_z): meters per second squared (m/s^2)

16.4.2. Gyroscope

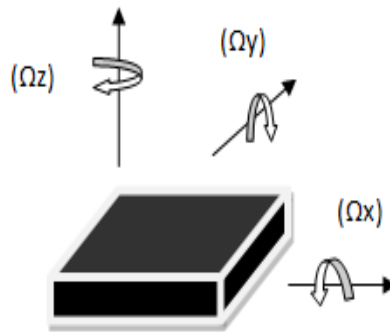


Figure 16.3.: Angular Speed of Gyroscope Sensor

- Angular speed in X-axis (Roll, Ω_x): degrees per second ($^{\circ}/s$)
- Angular speed in Y-axis (Pitch, Ω_y): degrees per second ($^{\circ}/s$)
- Angular speed in Z-axis (Yaw, Ω_z): degrees per second ($^{\circ}/s$)

16.5. Library setup in Arduino IDE

Install the LSM6DSOX Library:

- In the Arduino IDE, go to "Sketch" > "Include Library" > "Manage Libraries...".
- The Library Manager will open, providing a search bar. Type "LSM6DSOX" into the search bar. Look for the library called "LSM6DSOX" in the search results.
- Click on the library and then click the "Install" button to install the library.

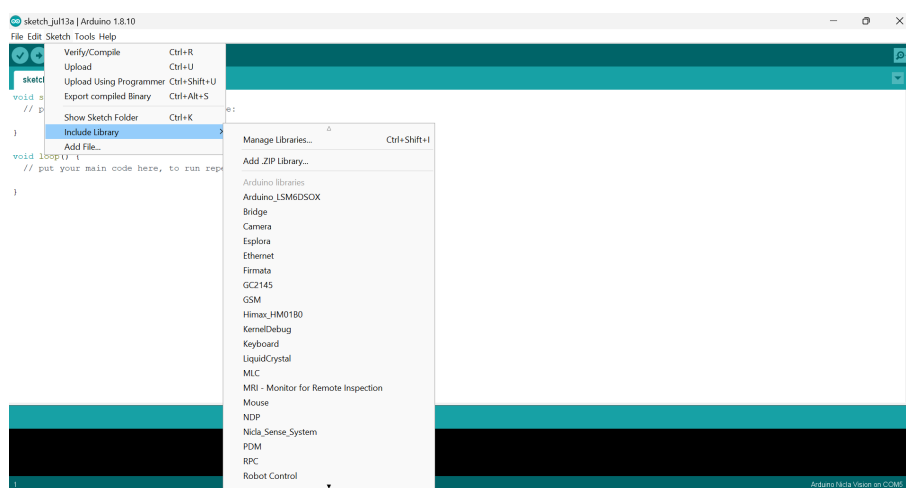


Figure 16.4.: Library setup in Arduino IDE

16.6. Applications

The LSM6DSOX 6-axis IMU (Inertial Measurement Unit) is a versatile sensor that combines a 3-axis accelerometer and a 3-axis gyroscope.

1. **Motion tracking and gesture detection:** The LSM6DSOX IMU can be used to track the motion of a device and detect gestures such as shaking, tilting, and rotating. This application is particularly useful in gaming, virtual reality, and augmented reality applications.
2. **Sensor hub:** The LSM6DSOX IMU can act as a sensor hub, collecting data from other sensors such as magnetometers, barometers, and GPS sensors. This enables the sensor to provide a more complete picture of the device's orientation and motion.[SDS17]
3. **Indoor navigation:** The LSM6DSOX IMU can be used for indoor navigation by tracking the device's motion and orientation. This application is useful in navigation and location-based services in indoor environments where GPS signals are not available.
4. **IoT and connected devices:** The LSM6DSOX IMU is an ideal sensor for IoT (Internet of Things) and connected devices. It can provide motion tracking data to a wide range of devices, such as smart homes, wearables, and industrial IoT devices.[Tri+22]
5. **Smart power saving for handheld devices:** The LSM6DSOX IMU can be used to optimize power consumption in handheld devices by detecting when the device is idle or in motion. This information can be used to adjust the device's power settings and conserve battery life.
6. **EIS and OIS for camera applications:** The LSM6DSOX IMU can be used to provide electronic image stabilization (EIS) and optical image stabilization (OIS) in camera applications. This allows for smoother video and reduces camera shake and blurring.
7. **Vibration monitoring and compensation:** The LSM6DSOX IMU can be used to monitor vibration levels in machinery and compensate for the effects of vibration. This is useful in industrial applications where vibration can cause damage or reduce the performance of machinery.

16.7. LSM9DS1(IMU)

16.8. Features

- 3 acceleration channels, 3 angular rate channels, 3 magnetic field channels.
- $\pm 2/\pm 4/\pm 8/\pm 16$ g linear acceleration full scale.
- $\pm 4/\pm 8/\pm 12/\pm 16$ gauss magnetic full scale.
- $\pm 245/\pm 500/\pm 2000$ dps angular rate full scale.
- 16-bit data output.

Applications

- Indoor navigation.
- Smart user interfaces.
- Advanced gesture recognition.
- Gaming and virtual reality input devices.
- Display/map orientation and browsing.

Description

The LSM9DS1 is a system-in-package featuring a 3D digital linear acceleration sensor, a 3D digital angular rate sensor, and a 3D digital magnetic sensor.

16.9. Pin description LSM9DS1

16.10. Pin connections LSM9DS1

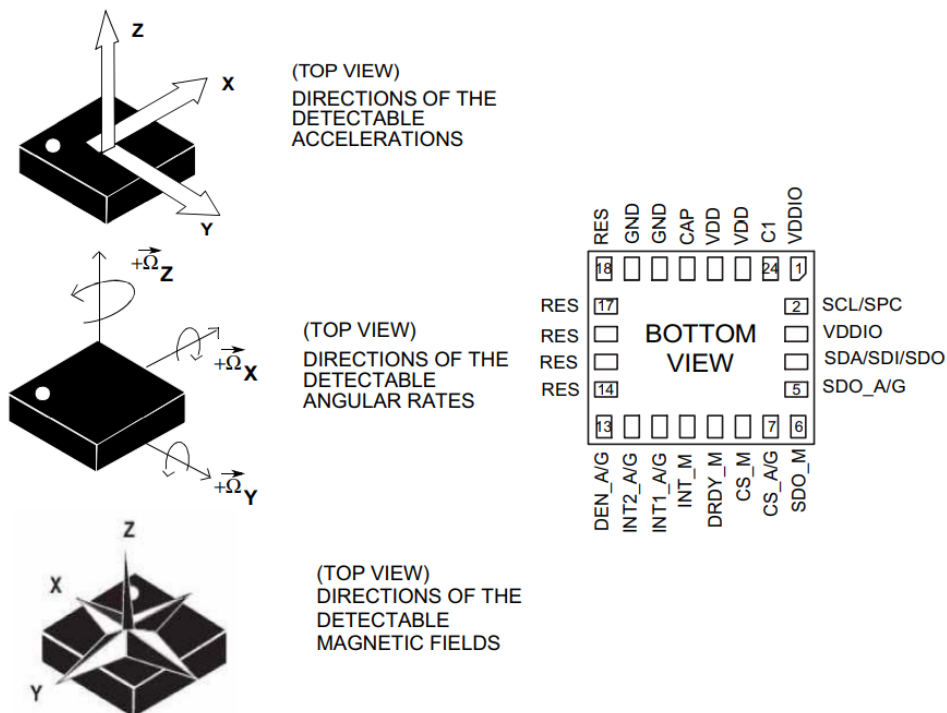


Figure 16.5.: **Pin connections LSM9DS1**

[Stmb]

Pin #	Name	Function
1	VDDIO ⁽¹⁾	Power supply for I/O pins
2	SCL/SPC	I ² C serial clock (SCL) / SPI serial port clock (SPC)
3	VDDIO ⁽²⁾	Power supply for I/O pins
4	SDA/SDI/SDO	I ² C serial data (SDA) SPI serial data input (SDI) 3-wire interface serial data output (SDO)
5	SDO_A/G	SPI serial data output (SDO) for the accelerometer and gyroscope I ² C least significant bit of the device address (SA0) for the accelerometer and gyroscope
6	SDO_M	SPI serial data output (SDO) for the magnetometer I ² C least significant bit of the device address (SA0) for the magnetometer
7	CS_A/G	SPI enable I ² C/SPI mode selection for the accelerometer and gyroscope (1: SPI idle mode / I ² C communication enabled; 0: SPI communication mode / I ² C disabled)
8	CS_M	SPI enable I ² C/SPI mode selection for the magnetometer (1: SPI idle mode / I ² C communication enabled; 0: SPI communication mode / I ² C disabled)
9	DRDY_M	Magnetic sensor data ready
10	INT_M	Magnetic sensor interrupt
11	INT1_A/G	Accelerometer and gyroscope interrupt 1
12	INT2_A/G	Accelerometer and gyroscope interrupt 2
13	DEN_A/G	Accelerometer and gyroscope data enable
14	RES	Reserved. Connected to GND.
15	RES	Reserved. Connected to GND.
16	RES	Reserved. Connected to GND.
17	RES	Reserved. Connected to GND.
18	RES	Reserved. Connected to GND.
19	GND	0 V supply
20	GND	0 V supply
21	CAP	Connected to GND with ceramic capacitor ⁽³⁾
22	VDD ⁽⁴⁾	Power supply
23	VDD ⁽⁵⁾	Power supply
24	C1	Capacitor connection (C1 = 100 nF)

Figure 16.6.: **Pin description LSM9DS1**
[Stmb]

Symbol	Parameter	Test conditions	Min.	Typ. ⁽¹⁾	Max.	Unit
LA_FS	Linear acceleration measurement range			±2		g
				±4		
				±8		
				±16		
M_FS	Magnetic measurement range			±4		gauss
				±8		
				±12		
				±16		
G_FS	Angular rate measurement range			±245		dps
				±500		
				±2000		
LA_So	Linear acceleration sensitivity	Linear acceleration FS = ±2 g		0.061		mg/LSB
		Linear acceleration FS = ±4 g		0.122		
		Linear acceleration FS = ±8 g		0.244		
		Linear acceleration FS = ±16 g		0.732		
M_GN	Magnetic sensitivity	Magnetic FS = ±4 gauss		0.14		mgauss/LSB
		Magnetic FS = ±8 gauss		0.29		
		Magnetic FS = ±12 gauss		0.43		
		Magnetic FS = ±16 gauss		0.58		
G_So	Angular rate sensitivity	Angular rate FS = ±245 dps		8.75		mdps/LSB
		Angular rate FS = ±500 dps		17.50		
		Angular rate FS = ±2000 dps		70		
LA_TyOff	Linear acceleration typical zero-g level offset accuracy ⁽²⁾	FS = ±8 g		±90		mg
M_TyOff	Zero-gauss level ⁽³⁾	FS = ±4 gauss		±1		gauss
G_TyOff	Angular rate typical zero-rate level ⁽⁴⁾	FS = ±2000 dps		±30		dps
M_DF	Magnetic disturbance field	Zero-gauss offset starts to degrade			50	gauss
Top	Operating temperature range		-40		+85	°C

Figure 16.7.: Sensor Characteristics

[Stmb]

Pin description LSM9DS1

16.10.1. Module specifications

Sensor characteristics

Temperature Sensor characteristics

Symbol	Parameter	Test condition	Min.	Typ. ⁽¹⁾	Max.	Unit
TODR	Temperature refresh rate	Gyro OFF ⁽²⁾		50		Hz
		Gyro ON		59.5		
TSen	Temperature sensitivity ⁽³⁾			16		LSB/°C
Top	Operating temperature range		-40		+85	°C

Figure 16.8.: Temperature Sensor Characteristics

[Stmb]

Absolute Maximum Ratings

Stresses above those listed as “Absolute maximum ratings” may cause permanent damage to the device. This is a stress rating only and functional operation of the

device under these conditions is not implied. Exposure to maximum rating conditions for extended periods may affect device reliability

Symbol	Ratings	Maximum value	Unit
V _{dd}	Supply voltage	-0.3 to 4.8	V
V _{dd_IO}	I/O pins supply voltage	-0.3 to 4.8	V
V _{in}	Input voltage on any control pin (including CS_A/G, CS_M, SCL/SPC, SDA/SDI/SDO, SDO_A/G, SDO_M)	0.3 to V _{dd_IO} +0.3	V
A _{UNP}	Acceleration (any axis)	3,000 for 0.5 ms	g
		10,000 for 0.1 ms	g
M _{EF}	Maximum exposed field	1000	gauss
ESD	Electrostatic discharge protection (HBM)	2	kV
T _{STG}	Storage temperature range	-40 to +125	°C

Figure 16.9.: Absolute Maximum Temperature
[Stmb]

16.10.2. Block Diagram

Accelerometer and gyroscope digital block diagram

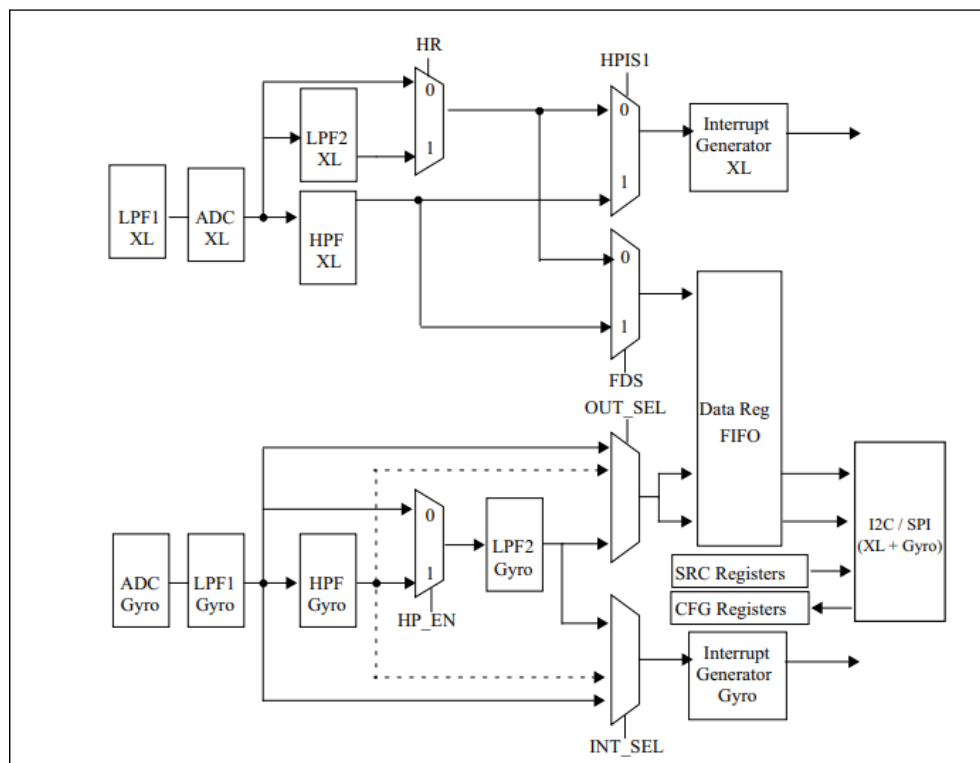


Figure 16.10.: Accelerometer and gyroscope block diagram
[Stmb]

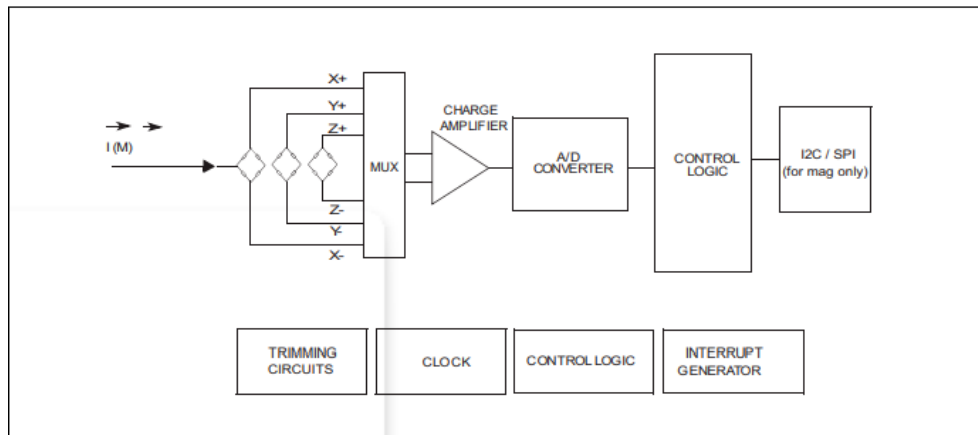
Magnetometer digital block diagram

Figure 16.11.: Magnetometer block diagram
[Stmb]

LSM9DS1 electrical connections

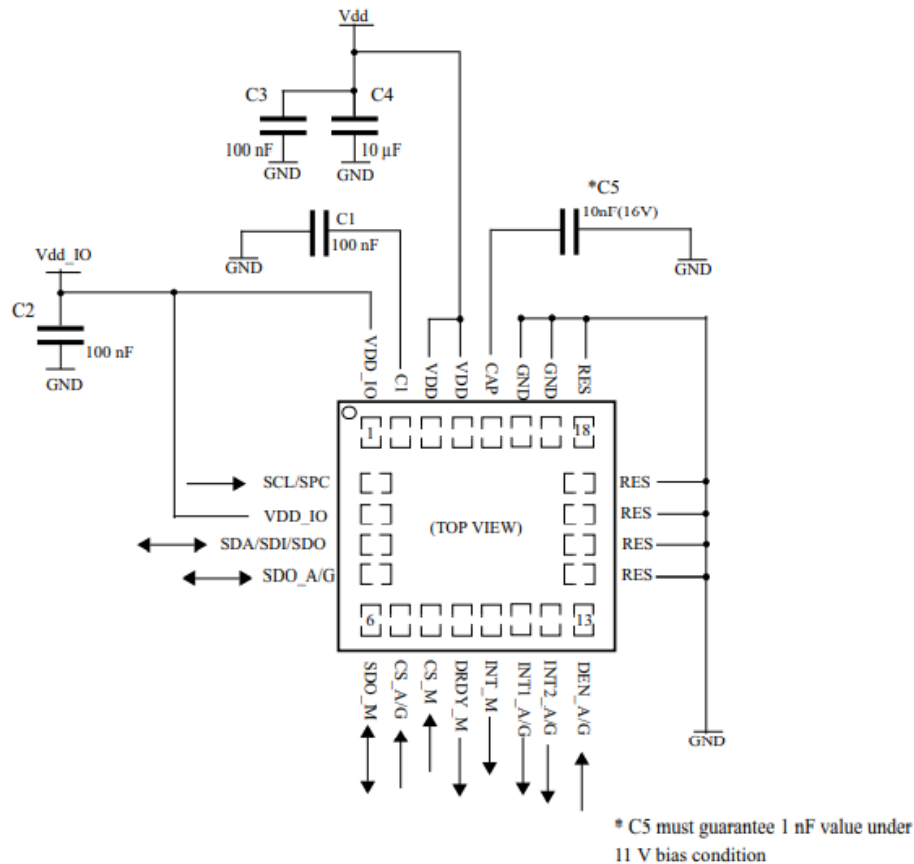


Figure 16.12.: LSM9DS1 electrical connections
[Stmb]

16.10.3. System Requirements

Operating System	Windows 7 or above, MacOS X or higher
CPU	Intel i3 or above
RAM	Min 2GB
Arduino IDE	V. 1.1819
Boards	Arduino mBED OS Nano
Libraries	LSM9DS1
Interface	USB port

16.10.4. Precautions to be taken

- This equipment complies with RF radiation exposure limits set forth for an uncontrolled environment.
- This Transmitter must not be co-located or operating in conjunction with any other antenna or transmitter.

- The operating temperature of the EUT can't exceed 85⁰C and shouldn't be lower than -40⁰C.
- The Frequency band should be in between 863-870Mhz.
- Arduino Nano 33 BLE only supports 3.3V I/Os and is NOT 5V tolerant so please make sure you are not directly connecting 5V signals to this board or it will be damaged.
- Keep away from water or fire.
- Hold it gently, do not harm the components and sensors present on the board.

16.11. Bibliotheken

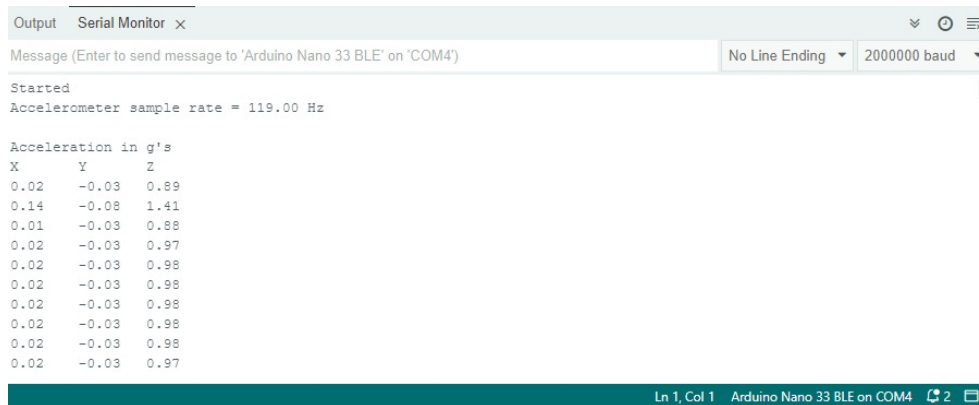
Eine Bibliothek ist eine Sammlung von Quelltext und Funktionen, die es dem Anwender ermöglicht, zum Beispiel jegliche Sensoren bedienen zu können, ohne alle Rohdaten selbst zu verarbeiten. Für dieses Projekt wird die Bibliothek [Arduino_LSM9DS1](#) des angesprochenen LSM9DS1 benötigt. In dieser Bibliothek sind die Funktionen enthalten, um die Bewegungen zu erfassen und in Winkel umzurechnen. Zusätzlich wird die Bibliothek [SSD1306Ascii](#) zur Zeichendarstellung für das OLED-Display benötigt.

WS:citations for the lib

16.12. Beispiele auf dem Mikrocontroller

16.12.1. Testen des Sensors LSM9DS1

Das Beispiel [SimpleAccelerometer](#) wurde geladen und der Sensor gibt erfolgreich die Daten der Beschleunigung aus.



```

Output Serial Monitor x
Message (Enter to send message to 'Arduino Nano 33 BLE' on 'COM4') No Line Ending 2000000 baud
Started
Accelerometer sample rate = 119.00 Hz

Acceleration in g's
X      Y      Z
0.02   -0.03   0.89
0.14   -0.08   1.41
0.01   -0.03   0.88
0.02   -0.03   0.97
0.02   -0.03   0.98
0.02   -0.03   0.98
0.02   -0.03   0.98
0.02   -0.03   0.98
0.02   -0.03   0.98
0.02   -0.03   0.98
0.02   -0.03   0.97
Ln 1, Col 1 Arduino Nano 33 BLE on COM4

```

Figure 16.13.: Output des Testprogramms

16.13. Programmierung

16.13.1. Programmablaufplan

16.13.2. Programmcode und Dokumentation

```

1000 #include <Arduino_LSM9DS1.h>
      #include <Wire.h>
1002 #include "SSD1306Ascii.h"
      #include "SSD1306AsciiWire.h"

```

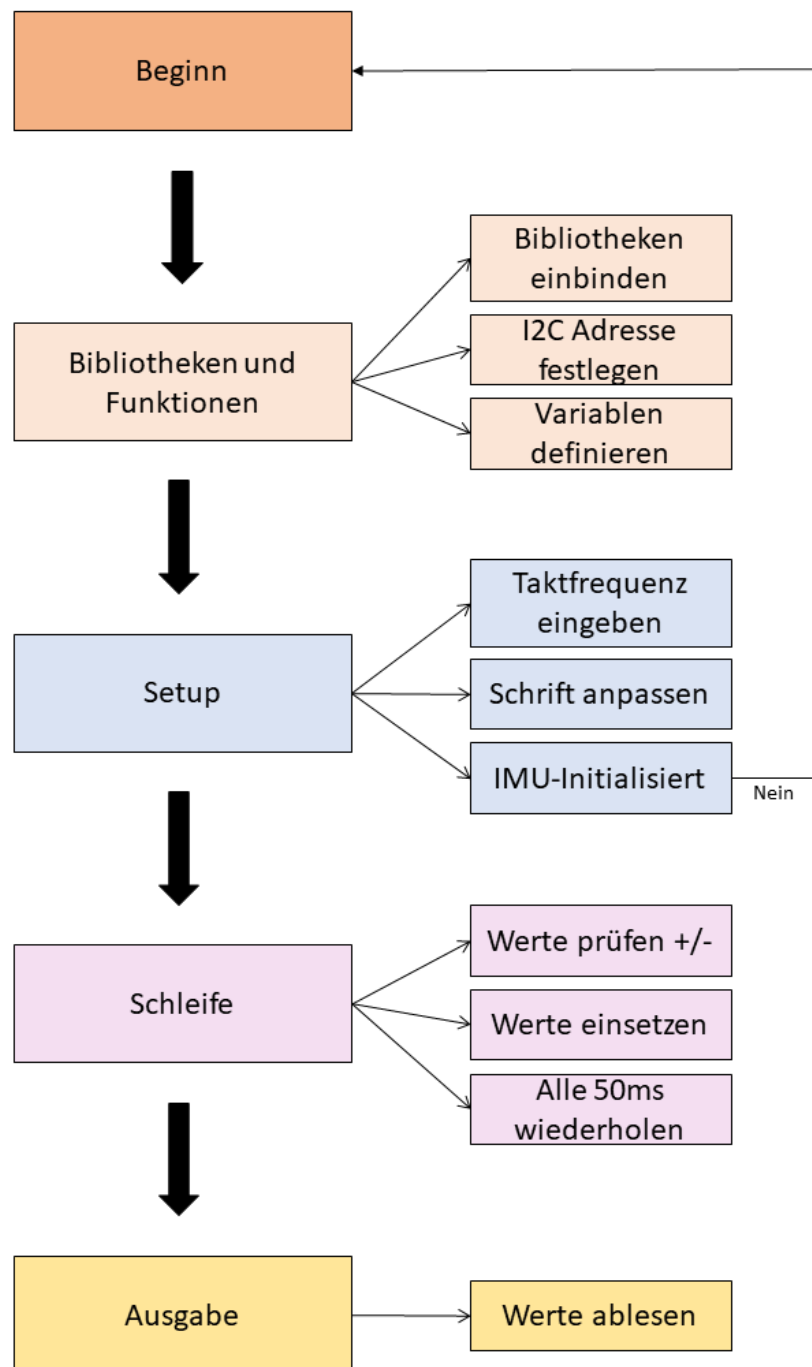


Figure 16.14.: Der Programmablaufplan

```

1004 SSD1306AsciiWire oled;
1006 #define I2C_ADDRESS 0x3C
1008 #define RST_PIN -1

```

Mit `#include` werden die Bibliotheken eingebunden. In diesem Fall werden die Bibliotheken des Sensors LSM9DS1 und die für das Display erforderliche `Wire.h` und `SSD1306Ascii.h` eingebunden. Über `#define` werden die Adressen festgelegt, damit eine Kommunikation stattfinden kann.

```

1000 float x, y, z;
1001 int degreesX = 0;
1002 int degreesY = 0;
String yPre, xPre;

```

Mit `float` (Gleitkommazahl / 4 Bytes) und `int` (Ganzzahl / 4 Bytes) werden numerische Variablen definiert. In diesem Fall haben wir `degreesX` und `degreesY`, in denen wir unsere jeweiligen X- und Y-Werte für die Ausgabe erhalten. Diese werden zu Beginn auf 0 gesetzt. Die Strings (String=Zeichenkette) `yPre` und `xPre` werden hier implementiert um später die Vorzeichen der X- und Y-Achse zwischen zu speichern.

Nun beginnt das Setup. Hier handelt es sich um eine Funktion, die nur ein einziges Mal aufgerufen wird, sobald der Arduino startet. Da keine Rückgabewerte aus der Methode benötigt werden, wird die Funktionstyp `void` verwendet. Die Funktion `Wire.setClock` definiert die Takt Frequenz für die I2C-Kommunikation.

```

1000 void setup()
1001 {
1002     Wire.begin();
1003     Wire.setClock(400000L);
1004
1005     #if RST_PIN >= 0
1006         oled.begin(&Adafruit128x64, I2C_ADDRESS, RST_PIN);
1007     #else // RST_PIN >= 0
1008         oled.begin(&Adafruit128x64, I2C_ADDRESS);
1009     #endif // RST_PIN >= 0
1010
1011     oled.setFont(TimesNewRoman16_bold);
1012     oled.clear();
1013     oled.println(" IMU Bereit");
1014
1015     if (!IMU.begin())
1016     {
1017         oled.clear();
1018         oled.println("Failed to initialize IMU!");
1019         while (1);
1020     }
1021
1022     xPre = " ";
1023     yPre = " ";
1024 }

```

Die Funktion `oled.setFont` wird verwendet, um die Schriftart und Größe zu ändern. Bei der verwendeten Schriftart handelt es sich um `TimesNewRoman`. Um das Ablesen der Werte zu erleichtern ist die Schriftgröße 16 eingestellt. Der Befehl `oled.println` gibt nach dem Einschalten des Arduinos auf dem Display "IMU Bereit" aus. So soll dem Anwender mitgeteilt werden, dass die Messungen gestartet werden können.

Falls die IMU nicht einsatzbereit sein sollte, wird über die `if`-Schleife `"Failed to initialize IMU!"` ausgegeben. Mit `xPre` und `yPre` werden aus optischen Gründen drei Leerzeichen definiert. So befinden sich auf der Anzeige die Messwerte geordnet übereinander.

`void loop` bezeichnet eine Funktion, die jedes mal wenn sie am Ende ist wieder von vorne beginnt. In dieser Schleife werden die Werte, die der Sensor ausgibt, immer wieder aufgerufen und in die entsprechenden Ausgaben eingefügt. Dies sorgt für die Aktualisierung auf dem OLED-Display.

In der ersten `if`-Abfrage wird abgefragt, ob der X-Wert größer ist als `0.1`. Wenn dies der Fall ist, merkt sich das Programm mit `xPre` das positive Vorzeichen. Die nächsten beiden `if`-Abfragen überprüfen, ob die entsprechenden Werte kleiner gleich 10 Grad sind (siehe Kap. 4.6). Wenn der Y-Wert kleiner gleich 10 Grad ist, gibt das Display die Ausgabe `" X 0 Grad"` aus. Sobald der Wert größer als 10 Grad ist, beginnt `else` und gibt dem Display die Anweisungen `oled.print(" X ");` für die Ausgabe des X-Wertes. Unmittelbar dahinter `oled.print(yPre);` für das gemerkte Vorzeichen von Y. Nun wird der vom Sensor gemessene Y-Wert für X eingesetzt `oled.print(degreesY);`. Zum Schluss wird mit `oled.print(" Grad");` Grad als Einheit hinter die Zeile gesetzt. Ausgaben wie `oled.print(" ");` beinhalten lediglich eine Leerzeile zur richtigen Ausrichtung der Werte untereinander. Die Funktion `map()` beschäftigt sich mit der Ganzzahlmathematik, dadurch werden selbst Brüche als ganze Zahlen weitergegeben.

Info: Aufgrund des aufgedruckten Koordinatensystems auf dem Gehäuse mussten die X- und Y-Werte getauscht werden, damit es für den Anwender bedienbar ist.

```

1000 void loop()
1001 {
1002     if (IMU.accelerationAvailable())
1003     {
1004         IMU.readAcceleration(x, y, z);
1005     }
1006     if (x > 0.1)
1007     {
1008         x = 100 * x;
1009         degreesX = map(x, 0, 97, 0, 90);
1010
1011         xPre = "+";
1012         oled.clear();
1013         oled.println();
1014
1015         if (degreesY <= 10)
1016         {
1017             oled.println(" X      0    Grad");
1018         }
1019         else
1020         {
1021             oled.print(" X ");
1022             oled.print(yPre);
1023             oled.print(" ");
1024             oled.print(degreesY);
1025             oled.print(" Grad");
1026             oled.println();
1027         }
1028         if (degreesX <= 10)
1029         {
1030             oled.println(" Y      0    Grad");
1031         }
1032         else
1033         {
1034             oled.print(" Y + ");
1035             oled.print(degreesX);
1036             oled.print(" Grad");
1037             oled.println();

```

```
1038 |     }
      | }

```

Ab der Zeile `if (degreesX <= 10)` erfolgt der gleiche Prozess wie oben beschrieben für den entsprechenden Y-Wert.

Im Folgenden Programmauszug wird die Schleife für die anderen Richtungen wiederholt. Für die X-Werte unter `-0,1` wird mit `xPre = " -"`; das Vorzeichen gemerkt und die Ausgaben wiederholen sich mit den entsprechenden Vorzeichen und Werten.

```
1000 |         if (x < -0.1)
      |         {
1002 |             x = 100 * x;
      |             degreesX = map(x, 0, -100, 0, 90);
1004 |
      |             xPre = " -";
1006 |             oled.clear();
      |             oled.println();
1008 |
      |             if (degreesY <= 10)
1010 |             {
      |                 oled.println(" X      0 Grad");
1012 |             }
      |             else
1014 |             {
      |                 oled.print(" X ");
1016 |                 oled.print(yPre);
      |                 oled.print(" ");
1018 |                 oled.print(degreesY);
      |                 oled.print(" Grad");
1020 |                 oled.println();
      |             }
1022 |
      |             if (degreesX <= 10)
1024 |             {
      |                 oled.println(" Y      0 Grad");
1026 |             }
      |             else
1028 |             {
      |                 oled.print(" Y - ");
1030 |                 oled.print(degreesX);
      |                 oled.print(" Grad");
1032 |                 oled.println();
      |             }
1034 |         }
      |
1036 |

```

Hier beginnt die Abfrage für die Y-Werte `if (y > 0.1)`. Sobald das Y positiv ist, wird sich mit `yPre = "+"`; das Positive Vorzeichen gemerkt und die Schleife läuft wie vorher schon bei den X-Werten beschrieben.

```
1000 |         if (y > 0.1)
      |         {
1002 |             y = 100 * y;
      |             degreesY = map(y, 0, 97, 0, 90);
1004 |
      |             yPre = "+";
1006 |             oled.clear();
      |             oled.println();
1008 |
      |             if (degreesY <= 10)
1010 |             {
      |                 oled.println(" X      0 Grad");
1012 |             }

```

```

else
1014 {
1015     oled.print(" X + ");
1016     oled.print(degreesY);
1017     oled.print(" Grad");
1018     oled.println();
1019 }
1020
1021 if (degreesX <= 10)
1022 {
1023     oled.println(" Y      0 Grad");
1024 }
1025 else
1026 {
1027     oled.print(" Y ");
1028     oled.print(xPre);
1029     oled.print(" ");
1030     oled.print(degreesX);
1031     oled.print(" Grad");
1032     oled.println();
1033 }
1034 }
1035 if (y < -0.1)
1036 {
1037     y = 100 * y;
1038     degreesY = map(y, 0, -100, 0, 90);
1039
1040     yPre = " -";
1041     oled.clear();
1042     oled.println();
1043
1044     if (degreesY <= 10)
1045     {
1046         oled.println(" X      0 Grad");
1047     }
1048     else
1049     {
1050         oled.print(" X - ");
1051         oled.print(degreesY);
1052         oled.print(" Grad");
1053         oled.println();
1054     }
1055
1056     if (degreesX <= 10)
1057     {
1058         oled.println(" Y      0 Grad");
1059     }
1060     else
1061     {
1062         oled.print(" Y ");
1063         oled.print(xPre);
1064         oled.print(" ");
1065         oled.print(degreesX);
1066         oled.print(" Grad");
1067         oled.println();
1068     }
1069 }
1070 delay(50);
}

```

Hier endet die Schleife. Mit `delay(50)` wird sie alle 50 Millisekunden wiederholt. So wird die Aktualisierung des Displays realisiert. Der Winkelmesser funktioniert also in Echtzeit.

VS: Der Begriff Echtzeit ist nicht bekannt. Hier muss gemessen werden!

16.13.3. Definition Echtzeit

Echtzeit bedeutet, dass ein System auf ein Ereignis innerhalb eines vorgegebenen Zeitrahmens zuverlässig reagieren kann. Das System ist in der Lage, alle Daten innerhalb einer Zykluszeit einzulesen und ausgeben. [Sch05]

Sobald das Programm hochgeladen wurde, gibt es zwei wesentliche Ausgaben auf dem OLED-Display, die der Anwender nun sehen sollte.

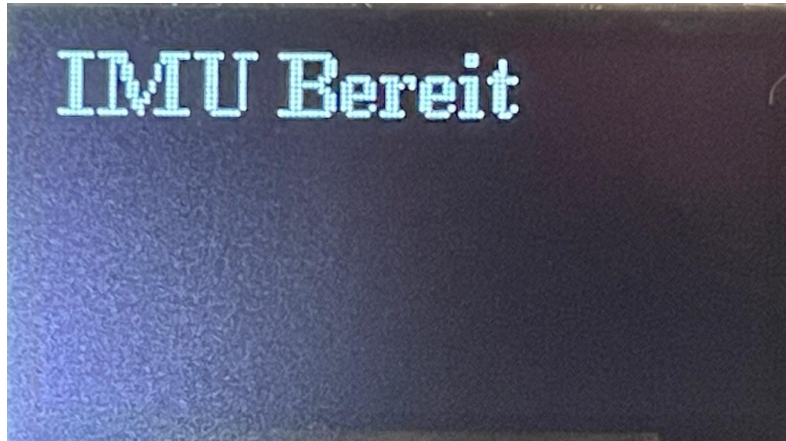


Figure 16.15.: Anzeige des Arduino OLED Displays sobald der Arduino eingeschaltet ist

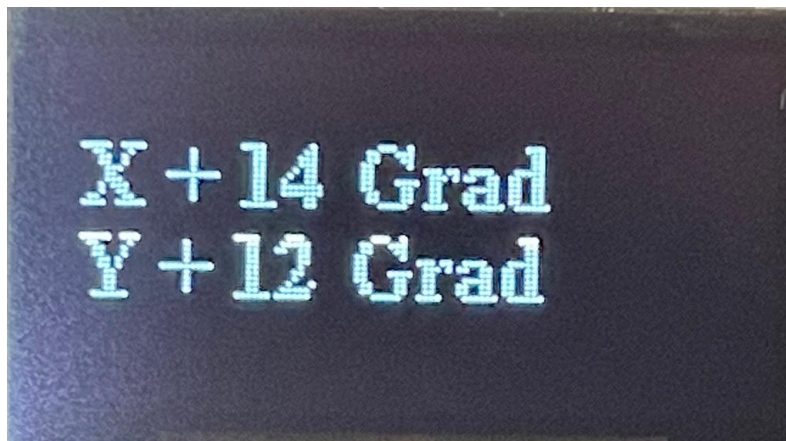


Figure 16.16.: Ausgabe von Messdaten

`X+ 14 Grad, Y+ 12 Grad` ist eine Ausgabe von einer Beispielbewegung des Arduinos und zeigt dem Benutzer eine Mischung aus einem positiven X- und Y-Wert.

16.14. Kalibrierung

Es war weder möglich eine fertige Kalibrierungssoftware von GitHub oder einen von einer KI erstellten Quelltext zu nutzen, um das Gerät zu kalibrieren. Der Arduino selbst zeigt präzise Winkel bis 90 Grad an, war aber nicht in der Lage Winkel kleiner als 10 Grad auszugeben. Dementsprechend wurde die if-Abfrage eingebaut, um Winkel die kleiner gleich 10 Grad sind auf dem Display als 0 Grad ausgegeben werden.

16.15. Probleme

Aufgrund mehrerer Fehler mit der seriellen Schnittstelle wurde zu Beginn angenommen, dass der Arduino einen Defekt hat. Nach weiteren Nachforschungen stellten wir allerdings fest, dass der Arduino sich ganz einfach zurücksetzen lässt. Mithilfe des kleinen Knopfs auf dem Arduino-Board, startet dieser in den Bootloader und lässt sich dann mit einer manuellen Änderung des COM-Ports wieder bespielen.

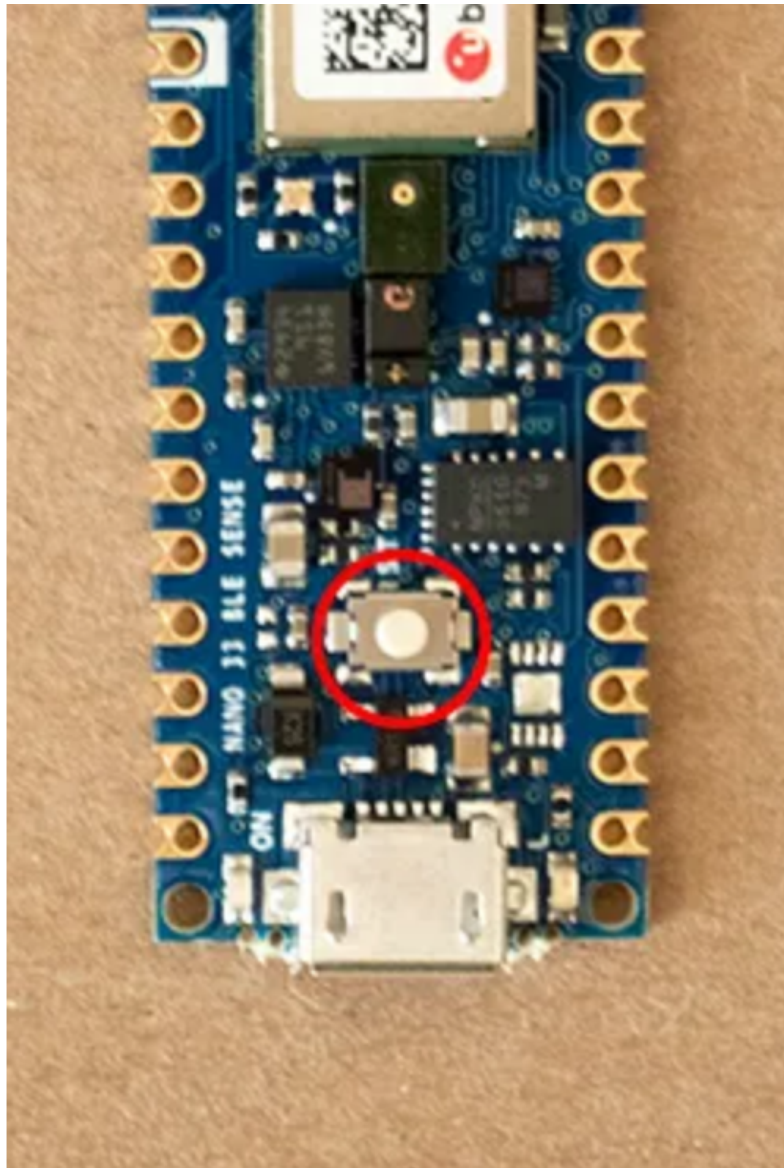


Figure 16.17.: Der Reset Button des Arduino

17. Calibration

17.1. Introduction

Calibration of an IMU (Inertial Measurement Unit) is the process of determining the bias, drift, and noise values of the sensors within the IMU. [Wah+11] This is done by measuring the output of the IMU in multiple positions and orientations and then using algorithms to determine the bias, drift, and noise values. The process is important for ensuring the accuracy of the IMU's measurements.[Vec]

17.2. Standard Operating Procedure

The standard operation procedure for calibrating LSM6DSOX IMU involves the following steps:

- Scope and Measurand(s) of Calibrations
- Description of the Item to be Calibrated
- Measurement Parameters, Quantities, and Ranges to be Determined
- Environmental Conditions and Stabilization Periods
- Procedure Include:
 - Handling, transporting, storing and preparation of items
 - Checks to be made before the work is started
 - Step by step process
- Handling, transporting, storing and preparation of items
- Checks to be made before the work is started
- Step by step process

Scope and Measurand(s) of Calibrations:

- The scope of the calibration process refers to the range of measurements or properties that are being assessed. In this case, the scope of calibration means determining the bias, drift, and noise values of the accelerometer and gyroscope readings. Bias refers to the systematic error in the sensor readings, while drift refers to the change in measurement over time due to factors such as temperature or humidity. Noise refers to the random fluctuations in the sensor readings.
- The measurands to be calibrated are the acceleration and angular velocity values of the sensor. Acceleration refers to the rate of change of velocity, and is typically measured in meters per second squared m/s^2 . Angular velocity refers to the rate of change of angular displacement, and is typically measured in degree per second ($^\circ/s$). Both acceleration and angular velocity are important measurements for applications such as robotics, navigation, and motion tracking.

- By calibrating the sensor, the accuracy of the acceleration and angular velocity measurements can be improved, leading to more reliable and precise data. This is especially important in applications where small errors in measurement can have significant consequences, such as in mobile robot.

Description of the Item to be Calibrated:

- The LSM6DSOX IMU sensor is a device that is designed to measure acceleration and angular velocity in three different axes. It is commonly used in applications where the measurement of motion or orientation is required, such as in drones, robotics, and virtual reality systems.
- The sensor uses a combination of accelerometers and gyroscopes to measure motion and orientation. The accelerometers measure changes in acceleration, while the gyroscopes measure changes in angular velocity. Together, these measurements can be used to calculate the orientation and movement of the sensor in three dimensions.
- The LSM6DSOX IMU sensor is a compact device that can be integrated into various systems and devices. It typically includes a microcontroller and communication interface, such as I2C or SPI, for sending the measured data to other devices or systems.

Measurement Parameters, Quantities, and Ranges to be Determined:

- The bias, drift, and noise are parameters that affect the accuracy and stability of the sensor readings. Bias refers to any systematic error in the sensor output that is not related to the input signal. It can be thought of as an offset that needs to be subtracted from the raw sensor output to get a more accurate measurement. Drift refers to the change in the sensor output over time, even when there is no change in the input signal. It can be caused by factors such as temperature changes or aging of the sensor components. Noise refers to the random fluctuations in the sensor output that can be caused by various sources, such as electrical interference or mechanical vibration.
- The quantities to be measured are the acceleration and angular velocity values in each axis of the sensor. Acceleration is measured in units of meters per second squared m/s^2 and angular velocity is measured in units of radians per second (rad/s).
- The range of the measurements will depend on the specifications of the sensor and the conditions in which it is being used. For example, the range of acceleration measurements might be $\pm 2g$, $\pm 4g$, $\pm 8g$, $\pm 16g$ (where g is the acceleration due to gravity) and the range of angular velocity measurements might be : ± 125 , ± 250 , ± 500 , ± 1000 , ± 2000 (degrees per second). The actual range of measurements will be determined by the sensitivity and resolution of the sensor and the specific application in which it is being used.

Environmental Conditions and Stabilization Periods:

- Environmental conditions play a critical role in ensuring accurate calibration of the sensor. Any significant vibration or movement in the environment can cause erroneous readings and introduce error into the calibration process. Therefore, it is essential to ensure that the environment in which the calibration is performed is stable and free from any such disturbances.
- The sensor also requires sufficient stabilization time to ensure that it is at a steady state before calibration. This stabilization period allows the sensor to adjust to

its environment and ensures that any initial drift is stabilized. The length of this stabilization period can vary depending on the sensor's specifications and the specific conditions in which the calibration is being performed. It is crucial to allow sufficient time for the sensor to stabilize before any measurements are taken to ensure accurate and reliable calibration results.

Procedure:

- **Handling, transporting, storing, and preparing the items:**

The sensor should be handled carefully to avoid any damage or contamination. It should be transported and stored in a clean and dry environment, away from any sources of electromagnetic interference. Before starting the calibration, the sensor should be mounted securely on a stable surface and connected to the calibration equipment.

- **Checks to be made before the work is started:**

Before starting the calibration, several checks should be made to ensure that the setup and environment are suitable for calibration. These checks may include verifying equipment is a flat surface, additional checks may include ensuring that the surface is level and stable, and checking that the sensor is securely mounted and positioned correctly on the surface, checking that the sensor is connected and communicating properly, and verifying that the environment is stable and free from any significant vibration or movement.

- **Step by step process:**

1. **Reading the accelerometer and gyroscope values from each axis:**

The sensor should be placed on a stable and flat surface to ensure accurate readings. The readings can be obtained using software designed to communicate with the sensor or a microcontroller. The readings are usually in the form of digital signals, which are then converted into acceleration and angular speed values.

2. **Updating the Kalman filter with these values:**

Once the readings from each axis are obtained, they are used to update the Kalman filter. The Kalman filter is a mathematical algorithm that estimates the bias, drift, and noise of each axis based on the readings obtained. The filter takes into account previous readings and estimates to provide a more accurate estimate of the current values. The Kalman filter is an essential part of the calibration process as it helps to correct for errors in the sensor readings.

3. **Storing these values:**

The estimated bias, drift, and noise values for each axis are stored in the sensor's memory or in a separate data storage device. These values can be used to correct for errors in subsequent measurements taken by the sensor. The storage location and format of the values depend on the sensor and the application. Some sensors have built-in memory, while others may require an external storage device. The values may be stored in a binary or text format depending on the application's requirements.

- **Measurement Assurance:**

- Measurement assurance is a critical aspect of any measurement process, which involves verifying the accuracy, precision, and reliability of the measurements. There are several ways to ensure measurement assurance, including comparing the measured values to known reference values, repeating the calibration process multiple times, and performing statistical analysis of the measurement data.

- Our approach is ensuring measurement assurance is to compare the estimated values to known reference values. This can involve using a reference standard or a calibration artefact with a known value, such as a calibrated weight or a temperature sensor with a known output. By comparing the measured values to the reference values, it is possible to determine the accuracy and precision of the measurement system and make any necessary adjustments to improve the measurement quality. For our LSM6DSOX IMU sensor measurement assurance is to compare all the accelerometer and gyroscope reading to zero when the IMU is in stationery over a flat surface and the temperature reading should be compared with external temperature sensor.

17.2.1. Low and high limit method

The low and high limit method involves recording minimum and maximum values on all three axes using a simple scratch to determine their absolute values. The sensor undergoes circular rotations along each axis multiple times[RS17]. The centre point is then identified between these extremes. Increasing the number of rotations enhances the likelihood of capturing the absolute peak. The center point will be close to zero if the sensor exhibits no offset. However, slight variations may indicate a hard iron offset attributed to distortion caused by the Earth's magnetic field[RS17]. This method assumes minimal soft iron distortion, evident from the rounded outlines in the graph. It is important to note that this method necessitates capturing values each time to prevent performance degradation due to component drift and aging sensors[RS17]. For devices relying on primary batteries, calibration becomes essential after each battery change, as the battery inevitably serves as the main source of magnetic disturbance, and new batteries may behave differently from their predecessors[RS17].

17.2.2. FreeIMU Calibration Application Magnetometer

In the FreeIMU Calibration Application Magnetometer method, raw magnetometer data undergoes pre-processing with axis-specific gain correction to convert the raw output into nanoTesla [RS17]:

- $X_{\text{m-nanoTesla}} = \text{rawCompass.m.x} * (100000.0 / 1100.0);$
- Gain X [LSB/Gauss] for selected input field range;
- $Y_{\text{m-nanoTesla}} = \text{rawCompass.m.y} * (100000.0 / 1100.0);$
- $Z_{\text{m-nanoTesla}} = \text{rawCompass.m.z} * (100000.0 / 980.0);$

The converted data is saved in the Mag-raw.txt file, which can be opened with the Magneto program. To implement this method, the scaling factors(e.g. 100000.0/1100.0) must be replaced with values specific to your sensor to convert the output into nanoTesla. Magneto generates twelve calibration values that correct for various errors, including bias, hard iron, scale factor, soft iron, and misalignment [RS17]. An additional benefit is that this method can be used to calibrate accelerometers by pre-processing raw accelerometer output, considering bit depth and G sensitivity, converting the data into milliGalileo. We can also enter a value of 1000 milliGalileo as the "norm" for the gravitational field [RS17].

17.2.3. Example

```
1000 #include <ArduinoSound.h>
1002 void setup() {
1004     Serial.begin(9600);
1006     Sound.begin();
1008 }
1010 void loop() {
1012     int micValue = Sound.read();
1014     Serial.println(micValue);
1016     delay(1000);
1018 }
```

Listing 17.1.: Example Microphone

```
1000 #include <Arduino_LSM9DS1.h>
1002 void setup() {
1004     Serial.begin(9600);
1006     if (!IMU.begin()) {
1008         Serial.println("Failed to initialize IMU!");
1010         while (1);
1012     }
1014 }
1016 void loop() {
1018     float x, y, z;
1020     if (IMU.accelerationAvailable()) {
1022         IMU.readAcceleration(x, y, z);
1024         Serial.print("AccX: "); Serial.print(x);
1026         Serial.print(", AccY: "); Serial.print(y);
1028         Serial.print(", AccZ: "); Serial.println(z);
1030     }
1032     if (IMU.gyroscopeAvailable()) {
1034         IMU.readGyroscope(x, y, z);
1036         Serial.print("GyroX: "); Serial.print(x);
1038         Serial.print(", GyroY: "); Serial.print(y);
1040         Serial.print(", GyroZ: "); Serial.println(z);
1042     }
1044     delay(1000);
1046 }
1048 \end{lstlisting}
```

Listing 17.2.: Example IMU (Accelerometer and Gyroscope)

```
1000 #include <Wire.h>
1001 #include <SparkFun_APDS9960.h>
1002
1003 // Object declaration
1004 SparkFun_APDS9960 apds;
1005
1006 void setup() {
1007     Serial.begin(9600);
1008     // Initialize sensor
1009     if (!apds.init()) {
1010         Serial.println("Failed to initialize sensor!");
1011         while (1);
1012     }
1013     // Enable proximity and gesture sensing
1014     apds.enableProximitySensor(true);
1015     apds.enableGestureSensor(true);
1016     Serial.println("Sensor initialized");
1017 }
1018
1019 void loop() {
1020     // Read proximity value
1021     if (apds.proximityAvailable()) {
1022         uint8_t proximity = apds.readProximity();
1023         Serial.print("Proximity: ");
1024         Serial.println(proximity);
1025     }
1026
1027     // Read gesture
1028     if (apds.isGestureAvailable()) {
1029         uint8_t gesture = apds.readGesture();
1030         Serial.print("Gesture: ");
1031         Serial.println(gesture);
1032     }
1033
1034     delay(1000);
1035 }
```

Listing 17.3.: Example ADPS-9960

18. Errors

18.1. Introduction

IMU (Inertial Measurement Unit) is a sensor that measures and reports the linear and rotational motion of an object. The error in IMU refers to any deviation or inaccuracy in the measurements reported by the sensor from the true or expected values.

1. **Bias:** Bias is the constant offset in the readings of the IMU. It can be caused by a variety of factors, including manufacturing defects, aging of the sensors, or changes in temperature. These errors can lead to systematic errors that can persist over time and can be difficult to detect. To correct for the bias error, the IMU must be calibrated periodically. Calibration involves comparing the IMU's measurements with a known reference, and then estimating and subtracting the bias from the measurements to obtain more accurate results.[Sab11]
2. **Drift:** Drift refers to the gradual change in bias over time. It can be caused by aging of the sensors, temperature changes, or electronic interference. Unlike the bias error, drift can vary over time, and it can be challenging to detect and correct. As a result, it is essential to calibrate the IMU regularly to correct for drift. More advanced techniques such as Kalman filtering can also be used to estimate and compensate for drift over time.[TPM14]
3. **Noise:** Noise refers to the random variations in the sensor readings that can affect the accuracy of the measurement. This error is especially pronounced when the IMU is stationary. The noise can be caused by various factors such as electronic interference, vibrations, or environmental conditions. To reduce the noise error, various techniques such as filtering or averaging can be used. Filtering techniques can help remove random variations in the sensor readings and provide more accurate measurements. Averaging can also be used to reduce noise by averaging out random variations in the measurements over time.[FH14]

18.2. Affected Parameters

Bias, drift, and noise errors will affect all the accelerometers and gyroscopes parameters. Here's how they impact each parameter:

18.2.1. Accelerometer:

- A_x , A_y , and A_z : The accelerometer measures linear acceleration in three directions, X, Y, and Z. If the accelerometer experiences bias, there will be a constant offset in the acceleration measurement in all three directions, even when there is no actual acceleration.
- Drift in an accelerometer can cause a slow, gradual change in the measured acceleration values over time. This means that even when the device is stationary, the accelerometer readings may drift away from the true acceleration values. For example, if the accelerometer is used to measure the tilt angle of a platform, drift can cause the tilt angle to slowly change even when the platform is not moving. Over time, this can result in significant errors in the measured tilt angle.
- Noise in an accelerometer can cause random fluctuations in the measured acceleration values. This means that even when the device is stationary, the accelerometer readings may vary randomly around the true acceleration values. For example, if the accelerometer is used to measure the vibration of a machine, noise can cause the measured vibration amplitude to fluctuate randomly around the true amplitude. This random fluctuation can make it difficult to accurately measure the machine's vibration characteristics.

18.2.2. Gyroscope:

- Ω_x , Ω_y , and Ω_z : The gyroscope measures the rate of change of angular velocity in three directions, Roll, Pitch, and Yaw. Bias in the gyroscope can cause a constant offset in the measured angular velocity values, even when there is no actual rotation. [NKG13]
- Drift in gyroscopes is caused by mechanical imperfections in the device, temperature changes, or other external factors that can lead to a gradual change in the measured angular velocity value over time, even when the device is not rotating. The drift error can accumulate over time and can significantly affect the accuracy of the gyroscope measurements. For example, a drone that uses a gyroscope for stabilization can experience drift error over time, causing it to drift off course and potentially crash.
- Noise in gyroscopes is caused by random fluctuations in the measured angular velocity values due to external disturbances such as vibrations or electromagnetic interference. The noise error can affect the precision of the gyroscope measurements and can lead to instability in the application. For example, in robotics, noise error can cause the robot to deviate from its intended path or make inaccurate movements.

To minimize these errors, calibration and filtering techniques can be used. Calibration can remove bias by using known reference values to adjust the sensor's measurements. Zero-g and zero-rate calibration techniques can be used to remove bias from accelerometers and gyroscopes, respectively. Filtering techniques can be used to remove noise and reduce drift. For example, a Kalman filter can be used to estimate the true acceleration or angular velocity values based on previous measurements and sensor models.[LBSK05]

19. IMU LSM6DSOX - Libraries and Functions

19.1. Libraries

A library refers to a collection of pre-written code that can be used by developers to perform specific tasks or functions without having to write the code from scratch. Libraries are designed to make the development process easier and more efficient by providing pre-built solutions to common programming challenges.

19.1.1. Wire.h

Wire.h is a library in Arduino that allows for communication between I2C devices. I2C stands for Inter-Integrated Circuit, which is a synchronous serial communication protocol used for connecting microcontrollers to peripheral devices. Wire.h provides functions for initializing the I2C bus, sending and receiving data over the bus, and managing multiple devices on the same bus. With this library, developers can easily connect multiple I2C devices, such as sensors or displays, to an Arduino board.[Ard19; Ari21]

19.1.2. Kalman.h

Kalman.h is a library that implements the Kalman filter algorithm. The Kalman filter is a mathematical technique used to estimate the state of a system based on incomplete measurements. It is commonly used in control systems, robotics, and navigation applications to improve the accuracy of sensor measurements and reduce errors. Kalman.h provides a simple interface for developers to implement the Kalman filter in their Arduino projects.[Ard19; Fet21]

19.1.3. Arduino_LSM6DSOX.h

Arduino_LSM6DSOX.h is a library that provides access to the LSM6DSOX accelerometer and gyroscope sensor on the Arduino Mbed OS Nucleo Board. The LSM6DSOX sensor is a 6-axis sensor that can measure both linear acceleration and angular velocity. Arduino_LSM6DSOX.h provides functions for initializing the sensor, reading data from the sensor, and configuring the sensor parameters. With this library, developers can easily integrate the LSM6DSOX sensor into their Arduino projects and use the sensor data for various applications, such as gesture recognition or orientation detection.[Lib21]

19.1.4. LSM6DSOXSensor.h

[LSM6DSOXSensor.h](#) is a library that provides an interface for interacting with the LSM6DSOX sensor. The LSM6DSOX is a 6-axis inertial measurement unit (IMU) sensor that combines a 3-axis accelerometer and a 3-axis gyroscope in a single chip. It is commonly used in applications that require motion sensing and orientation tracking, such as robotics, drones, wearable devices, and Internet of Things (IoT) devices. The LSM6DSOXSensor.h library allows developers to easily interact with the LSM6DSOX sensor by providing functions and classes for configuring the sensor, reading raw sensor

data, and performing sensor fusion to obtain calibrated accelerometer and gyroscope data, as well as derived data such as orientation, linear acceleration, and angular velocity. The library abstracts the low-level communication with the sensor, providing a higher-level API that simplifies the process of working with the sensor.

19.2. Functions

A function is a block of code that performs a specific task or set of tasks. Functions are designed to be reusable and modular, meaning they can be called and executed multiple times throughout a program without having to rewrite the code every time. Functions can take input parameters, perform operations on them, and return output values.

19.2.1. `setup()`

The `setup()` function is a special function in the Arduino programming language that is called once at the beginning of the program. The purpose of the `setup()` function is to initialize variables, pin modes, and other settings that are necessary for the program to run correctly. This function is typically used to set up hardware components, such as sensors or displays, and configure any necessary settings, such as communication protocols or data rates.^[Ardg]

19.2.2. `loop()`

The `loop()` function is another special function in the Arduino programming language that is called repeatedly after the `setup()` function. The purpose of the `loop()` function is to execute the main code of the program in a continuous loop until the program is terminated. This function is typically used to read sensor data, perform calculations, and control hardware components based on the input data.^[Ardf]

19.2.3. `IMU.begin()`

`IMU.begin()` is a function provided by the `Arduino_LSM6DSOX.h` library, which is used to initialize the LSM6DSOX accelerometer and gyroscope sensor on the Arduino Mbed OS Nicla Board. The purpose of the `IMU.begin()` function is to set up the communication between the Arduino board and the LSM6DSOX sensor and to configure the sensor settings to match the requirements of the program. This function is typically called in the `setup()` function of the program to initialize the sensor before reading data from it in the `loop()` function.

19.2.4. `IMU.setAccelerometerRange()`

The `IMU.setAccelerometerRange()` function is used to set the range of the accelerometer on the LSM6DSOX sensor. The accelerometer range determines the maximum acceleration that can be measured by the sensor. This function takes an argument that specifies the range in Gs (gravitational force) and can be set to 2G, 4G, 8G, or 16G depending on the application requirements.

19.2.5. `IMU.setGyroscopeRange()`

The `IMU.setGyroscopeRange()` function is used to set the range of the gyroscope on the LSM6DSOX sensor. The gyroscope range determines the maximum angular velocity that can be measured by the sensor. This function takes an argument that specifies the range in degrees per second (dps) and can be set to 125dps, 250dps, 500dps, 1000dps, or 2000dps depending on the application requirements.

19.2.6. IMU.setAccelerometerDataRate()

The `IMU.setAccelerometerDataRate()` function is used to set the data rate of the accelerometer on the LSM6DSOX sensor. The data rate determines how often the sensor samples the acceleration data and can be set to 12.5Hz, 26Hz, 52Hz, 104Hz, 208Hz, 416Hz, 833Hz, or 1660Hz depending on the application requirements.

19.2.7. IMU.setGyroscopeDataRate()

The `IMU.setGyroscopeDataRate()` function is used to set the data rate of the gyroscope on the LSM6DSOX sensor. The data rate determines how often the sensor samples the angular velocity data and can be set to 12.5Hz, 26Hz, 52Hz, 104Hz, 208Hz, 416Hz, 833Hz, or 1660Hz depending on the application requirements.

19.2.8. IMU.accelerationSampleRate()

The `IMU.accelerationSampleRate()` function is used to read the current sample rate of the accelerometer on the LSM6DSOX sensor. This function returns the current data rate value in Hz.

19.2.9. IMU.gyroscopeSampleRate()

The `IMU.gyroscopeSampleRate()` function is used to read the current sample rate of the gyroscope on the LSM6DSOX sensor. This function returns the current data rate value in Hz.

19.2.10. IMU.temperatureAvailable()

The `IMU.temperatureAvailable()` function is provided by the `Arduino_LSM6DSOX.h` library for the LSM6DSOX accelerometer and gyroscope sensor on the Arduino Mbed OS Nicla Board. This function is used to check if the temperature data is available to be read from the sensor. It returns a boolean value of true if the temperature data is available, and false if it is not.

19.2.11. IMU.readTemperature()

The `IMU.readTemperature()` function is provided by the `Arduino_LSM6DSOX.h` library and is used to read the temperature data from the LSM6DSOX sensor on the Arduino Mbed OS Nicla Board. This function returns the temperature in degrees Celsius as a floating-point value. Before calling this function, you should use the `temperatureAvailable()` function to check if the temperature data is available.

19.2.12. IMU.accelerationAvailable()

The `IMU.accelerationAvailable()` function is provided by the `Arduino_LSM6DSOX.h` library and is used to check if the acceleration data is available to be read from the LSM6DSOX sensor on the Arduino Mbed OS Nicla Board. It returns a boolean value of true if the acceleration data is available, and false if it is not.

19.2.13. IMU.readAcceleration()

The `IMU.readAcceleration()` function is provided by the `Arduino_LSM6DSOX.h` library and is used to read the acceleration data from the LSM6DSOX sensor on the Arduino Mbed OS Nicla Board. This function returns a 3D vector that represents the acceleration in units of Gs (gravitational force) along the x, y, and z axes. Before

calling this function, you should use the `accelerationAvailable()` function to check if the acceleration data is available.

19.2.14. IMU.gyroscopeAvailable()

The `IMU.gyroscopeAvailable()` function is provided by the `Arduino_LSM6DSOX.h` library and is used to check if the gyroscope data is available to be read from the LSM6DSOX sensor on the Arduino Mbed OS Nicla Board. It returns a boolean value of true if the gyroscope data is available, and false if it is not.

19.2.15. IMU.readGyroscope()

The `IMU.readGyroscope()` function is provided by the `Arduino_LSM6DSOX.h` library and is used to read the gyroscope data from the LSM6DSOX sensor on the Arduino Mbed OS Nicla Board. This function returns a 3D vector that represents the angular velocity in units of degrees per second (dps) along the x, y, and z axes. Before calling this function, you should use the `gyroscopeAvailable()` function to check if the gyroscope data is available.

19.2.16. randomGaussian()

The `randomGaussian()` function is a built-in function in the Arduino programming language. Gaussian distribution is also known as normal distribution, and it is a probability distribution that describes how values are distributed around the mean. This function takes two arguments: the mean and the standard deviation of the distribution. It returns a random floating-point number with a mean value equal to the first argument and a standard deviation equal to the second argument. This function is commonly used in statistical simulations and signal-processing application.

19.2.17. kalmanX.setQ(), kalmanY.setQ(), kalmanZ.setQ()

These functions set the process noise covariance for the Kalman filter in the X, Y, and Z axes respectively. The process noise covariance controls how much we trust our prediction of the state of the system at the next time step. A higher value of Q indicates that the system is more likely to deviate from the predicted state and a lower value indicates that the system is more likely to follow the predicted state.

19.2.18. kalmanX.setR(), kalmanY.setR(), kalmanZ.setR()

These functions set the measurement noise covariance for the Kalman filter in the X, Y, and Z axes respectively. The measurement noise covariance controls how much we trust the sensor measurement of the system. A higher value of R indicates that the sensor measurement is less reliable and a lower value indicates that the sensor measurement is more reliable.

19.2.19. kalmanX.update(), kalmanY.update(), kalmanZ.update()

These functions update the Kalman filter in the X, Y, and Z axes respectively. They take a measurement of the system and use it to update the state estimate of the system. The function returns the estimated bias of the measurement. The estimated bias is subtracted from the measurement to get a corrected measurement, which is used to update the state estimate.

19.2.20. `kalmanX.getSigma()`, `kalmanY.getSigma()`, `kalmanZ.getSigma()`

These functions return the standard deviation of the Kalman filter output in the X, Y, and Z axes respectively. The standard deviation is a measure of the noise in the output of the Kalman filter.

19.2.21. `delay()`

This function causes the program to pause execution for a specified number of milliseconds. In this code, it is used to wait for a short time before reading from the IMU again. This allows time for the IMU to take a new measurement and for the Kalman filter to update its estimates.

19.2.22. `lsm6dsoxSensor.Set_X_FS()`

`lsm6dsoxSensor.Set_X_FS()` is a function provided by the `LSM6DSOXSensor.h` library that is used to set the full-scale range of the accelerometer in the LSM6DSOX sensor. The accelerometer measures acceleration along three axes (X, Y, Z) and the full-scale range determines the maximum range of acceleration that the sensor can measure without saturation.

The function takes an argument that specifies the desired full-scale range for the accelerometer. The available options for the resolution range may vary depending on the specific sensor model, but commonly include $\pm 2g$, $\pm 4g$, $\pm 8g$, or $\pm 16g$. The full-scale range is typically expressed in units of acceleration (e.g., g) and represents the maximum acceleration that the sensor can accurately measure along each axis.

When `lsm6dsoxSensor.Set_X_FS()` is called with the desired resolution range as an argument, the function sends the appropriate configuration commands to the LSM6DSOX sensor to set the accelerometer to the specified full-scale range. This ensures that the sensor is configured to accurately measure accelerations within the desired range and prevents saturation of the sensor's output, which can result in inaccurate readings.

19.2.23. `lsm6dsoxSensor.Set_G_FS()`

`lsm6dsoxSensor.Set_G_FS()` is a method or function call that likely belongs to a software library or codebase that interfaces with an LSM6DSOX sensor. The LSM6DSOX is a type of inertial measurement unit (IMU) sensor that combines a 3-axis accelerometer and a 3-axis gyroscope in a single chip.

The `Set_G_FS()` function is likely used to configure the full-scale (FS) range or sensitivity of the gyroscope component of the LSM6DSOX sensor. Gyroscopes measure angular velocity or rotational motion, and the full-scale range determines the maximum angular velocity that the gyroscope can accurately measure.

The full-scale range is typically specified in units of degrees per second (dps) or radians per second (rad/s) and represents the maximum rate of rotation that the gyroscope can detect without saturating its output. A higher full-scale range allows the gyroscope to measure faster rotations, but it may result in reduced resolution for slower rotations.

The `Set_G_FS()` function likely takes an argument or parameter that specifies the desired full-scale range for the gyroscope, such as ± 125 dps, ± 250 dps, ± 500 dps, ± 1000 dps, or ± 2000 dps, depending on the capabilities of the LSM6DSOX sensor. The function would then configure the sensor to operate with the specified full-scale range for the gyroscope accordingly.

19.2.24. Initialize_IMU()

The `Initialize_IMU()` function is responsible for initializing and configuring the LSM6DSOX sensor for operation. It begins by establishing communication with the sensor using the `Wire.begin()` function, which is commonly used for I2C communication in Arduino-based projects. Next, it initializes the LSM6DSOX sensor using the `lsm6dsoxSensor.begin()` function, which sets up the sensor for operation.

The function then proceeds to set the full-scale range for the accelerometer and gyroscope using the `lsm6dsoxSensor.Set_X_FS()` and `lsm6dsoxSensor.Set_G_FS()` functions, respectively. In this case, the full-scale range is set to $\pm 4g$ for the accelerometer and ± 2000 deg/s for the gyroscope, which determines the maximum range of motion that the sensor can measure.

The sample rates for both the accelerometer and gyroscope are then set to 104 Hz using the `lsm6dsoxSensor.Set_X_ODR()` and `lsm6dsoxSensor.Se_G_ODR()` functions, respectively. The sample rate determines how frequently the sensor measures and reports data, with a higher sample rate resulting in more frequent updates but potentially higher power consumption.

19.2.25. Factors_Calculation()

The `Factors_Calculation()` function performs several steps. First, it checks if temperature data is available from the IMU sensor using the `IMU.temperatureAvailable()` function. If temperature data is available, it reads the temperature readings from the IMU using the `IMU.readTemperature()` function and stores the value in the temperature variable. Then, it calculates the temperature factor by multiplying the temperature sensitivity, temperature, and temperature tolerance, and dividing the result by 100.0 to convert the tolerance from percentage to decimal. Next, it calculates the linear acceleration factor by multiplying the linear acceleration sensitivity and linear acceleration tolerance. Finally, it calculates the angular rate factor by multiplying the angular rate sensitivity and angular rate tolerance.

19.2.26. Factors_Calculation()

The `Accelerometer_Gyroscope_Characteristics()` function performs several steps. First, it checks if accelerometer data is available from the IMU sensor using the `IMU.accelerationAvailable()` function. If accelerometer data is available, it reads the accelerometer readings for the X, Y, and Z axes from the IMU using the `IMU.readAcceleration()` function and stores the values in the variables `Ax`, `Ay`, and `Az`. Then, it calculates the acceleration in m/s^2 for each axis by multiplying the raw accelerometer readings with the linear acceleration factor, temperature factor, and gravity. The calculated values are stored in the variables `Ax_m_sec2`, `Ay_m_sec2`, and `Az_m_sec2`. Next, it adds noise characteristics to the accelerometer readings by calculating the noise values for each axis based on the accelerometer noise density and sample rate, and then adding them to the corresponding calculated acceleration values. It also checks if gyroscope data is available from the IMU sensor using the `IMU.gyroscopeAvailable()` function. If gyroscope data is available, it reads the gyroscope readings for the X, Y, and Z axes from the IMU using the `IMU.readGyroscope()` function and stores the values in the variables `Gx`, `Gy`, and `Gz`. Finally, it converts the gyroscope readings from LSB to deg/s by multiplying with the angular rate factor and temperature factor, and stores the calculated values in the variables `Gx_deg_sec`, `Gy_deg_sec`, and `Gz_deg_sec`.

20. Sensor Inertial Measure Unit

Inertial Measurement Units (IMU) are a component of many navigation and control systems. These devices integrate several sensors, including accelerometers and gyroscopes, to measure and follow an object's, linear and rotational, movements in three-dimensional space. IMUs are widely used in a variety of applications, such as autonomous vehicle navigation, virtual reality, image stabilization, and even in wearable devices for tracking physical activity. Their ability to provide precise data on linear and angular acceleration makes them indispensable tools for systems requiring precise control and orientation, and ongoing technological advances have reduced their size, power consumption and cost, widening their scope of application in many fields. [Stmb]

20.1. General Description

The Inertial Measure Unit is used to measure acceleration or rotation motion, this is an electronic device who used 3 types of sensors : magnetometer, accelerometer and gyroscope. This IMU used sensor LSM9DS1 and they give velocity, orientation and gravitational force.

The magnetometer can measure the strength of magnetic fields, this measure can give different information, if you pass a current through an electromagnet or in any other way.

20.1.1. Always On Mode

This feature implies that the LSM9DS1 can remain operational continuously, even when a device is in sleep mode, without excessively draining power. Such functionality proves invaluable for applications demanding uninterrupted motion tracking, like fitness monitoring or navigation systems.

The power consumption is an important factor when we have battery-powered device like smartphone or other portable application. This device is portable, so they need less battery than possible to do less weight than possible.

The LSM9DS1 boasts a power consumption of 0.55 mA when operating in combo high-performance mode. This signifies its ability to maintain a low power consumption level while delivering exceptional data quality. In this mode, the sensor excels in concurrently measuring acceleration and angular rate, ensuring precise and reliable data output even at a high sampling rate.

20.1.2. Tilt detection

The Tilt detection detects movement with accelerometer, if movement is doing so Tilt detects it. Tilt function with Ultra Low Power consumption.

For example, if you have your smartphone in your front pant pocket. Tilt detection can say if you standing to sitting or sitting to standing.

20.1.3. Significant Motion Detection

The Significant Motion Detection used in LSM9DS1 only accelerometer to detect big movement. It can be used for different usage :

- Waking up a device from sleep mode when the user starts moving.
- Tracking steps or activity changes in fitness applications.
- Triggering alarms or notifications when a significant movement is detected.

20.2. Specific Sensor

20.2.1. LSM9DS1

The sensor LSM9DS1, engineered by STMicroelectronics, is composed of a 3-axis Inertial Measurement Unit (IMU), which has a high-performance 3-axis digital accelerometer and 3-axis digital gyroscope. This device is meticulously crafted for precise motion follow up, finding widespread application in wearable devices and smartphones. Capable of concurrent measurement of both linear and rotational motion, it stands as a hallmark in motion-sensing technology.

The sensor LSM9DS1 was developed to detect movement, stationary/motion detection, tilt, pedometer functions, timestamping and to support the data and acquisition of an external magnetometer. This sensor gives hardware flexibility to connect the pins with different mode connections to external sensors to expand functionalities such as adding a sensor hub, auxiliary SPI.

20.2.2. Accelerometer

The STMicroelectronics LSM9DS1 accelerometer is a sensor designed to measure linear acceleration along the x, y, and z-axes. With a typical measurement range of $\pm 2g$, $\pm 4g$, $\pm 8g$, $\pm 16g$. It gives precision and reliability for a variety of applications. The accelerometer for LSM9DS1 has an acceleration sampling frequency that can be changed to the following values: 10 Hz, 50 Hz, 119 Hz, 238 Hz, 476 Hz, 952 Hz.

Its low power consumption makes it a great choice for battery-powered devices, with a typical operating current ranging from 0.55 mA to 1.25 mA. Additionally, its wide operating voltage range, from 1.71 V to 3.6 V, allows it to easily adapt to different system configurations.

Whether it's tracking movements in wearable devices, stabilizing drones, or measuring vibrations in structures, the accelerometer LSM9DS1 delivers outstanding performance in a compact and robust package.

The STMicroelectronics accelerometer LSM9DS1 is a preferred choice for engineers and designers seeking a precise and reliable solution for their projects.

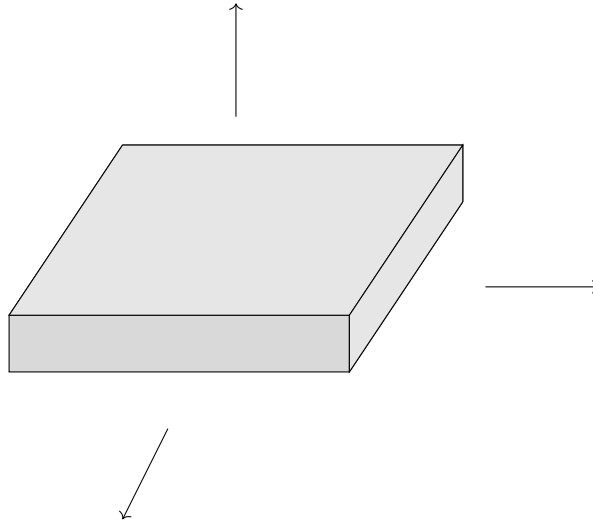


Figure 20.1.: LSM9DS1 axis on this card

20.2.3. Gyroscope

The STMicroelectronics LSM9DS1 gyroscope is a measuring instrument designed to assess rotation rates around the x, y, and z-axes. With a typical measurement range of ± 245 , ± 500 , ± 2000 dps, it offers adaptability to the specific needs of the application. The gyroscope of LSM9DS1 has an acceleration sampling rate that can be modified to take the following values: 14.9 Hz, 59.5 Hz, 119 Hz, 238 Hz, 476 Hz, 952 Hz.

This gyroscope operates with a voltage range of 1.71 V to 3.6 V, allowing it to easily integrate into different system configurations. Furthermore, its typical operating current ranges from 1 mA to 2 mA, ensuring optimal energy efficiency for battery-powered devices.

Whether it's for inertial navigation, drone stabilization, or other applications requiring precise motion measurement, the gyroscope of LSM9DS1 delivers reliable and accurate performance in a compact and robust form factor, meeting the highest requirements of engineers and designers.

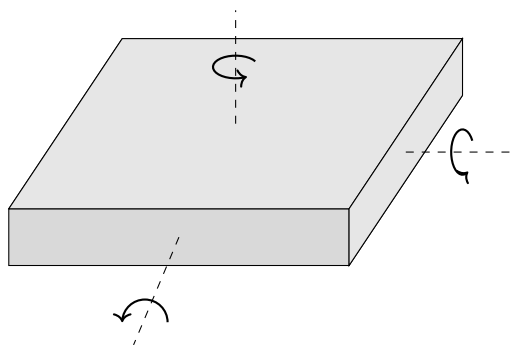


Figure 20.2.: LSM9DS1 axis on this card

20.3. Specification

20.3.1. Accelerometer Sensor

The accelerometer is composed by sensor type MEMS (Micro-Electro-Mechanical Systems) and they used microscopic structures like suspended beams or springs, which deform in response to acceleration. The deformation of these structures is measured using displacement sensors or changes in electrical resistance. MEMS accelerometers are commonly used due to their small size, low power consumption, and comparatively lower cost compared to other types of accelerometers.

The Accelerometer sensor function by :

- **Acceleration Integration for Velocity Estimation:** To estimate velocity, the acceleration measured by the accelerometer is integrated over time. Integrating acceleration over time yields velocity.
- **Error Compensation:** However, it's crucial to note that acceleration integration is susceptible to cumulative errors. Errors stemming from sensor noise, drift, and inaccuracies can accumulate over time, leading to inaccurate or biased velocity estimates.
- **Utilization of Filters and Sensor Fusion Techniques:** To compensate for these errors, advanced techniques such as Kalman filters, Complementary filters, or sensor fusion techniques can be employed. These techniques combine accelerometer data with other sensors such as gyroscopes to enhance velocity estimation accuracy.
- **Calibration and Adjustment:** It's also vital to calibrate and adjust the IMU to correct systematic errors and ensure precise measurements. This may involve steps such as compensating for Earth's gravity, correcting gyroscope drift, and reducing sensor noise.

20.3.2. PIN Connction

They are 4 ways to connect pin, ways depend of what they have after.

- **Mode 1:** You can utilize either the I²C / MIPI I3CSM slave interface or the SPI (3- and 4-wire) serial interface.
- **Mode 2:** This mode supports both the I²C / MIPI I3CSM slave interface or the SPI (3- and 4-wire) serial interface, along with an I²C interface master for external sensor connections.
- **Mode 3:** Here, you have the choice of employing either the I²C / MIPI I3CSM slave interface or the SPI (3- and 4-wire) serial interface for the application processor interface. Additionally, an auxiliary SPI (3- and 4-wire) serial interface is designated for external sensor connections, exclusively for the gyroscope.
- **Mode 4:** Similar to Mode 3, this mode offers the I²C / MIPI I3CSM slave interface or the SPI (3- and 4-wire) serial interface for the application processor interface. Additionally, it provides an auxiliary SPI (3- and 4-wire) serial interface for external sensor connections, catering to both the accelerometer and gyroscope.

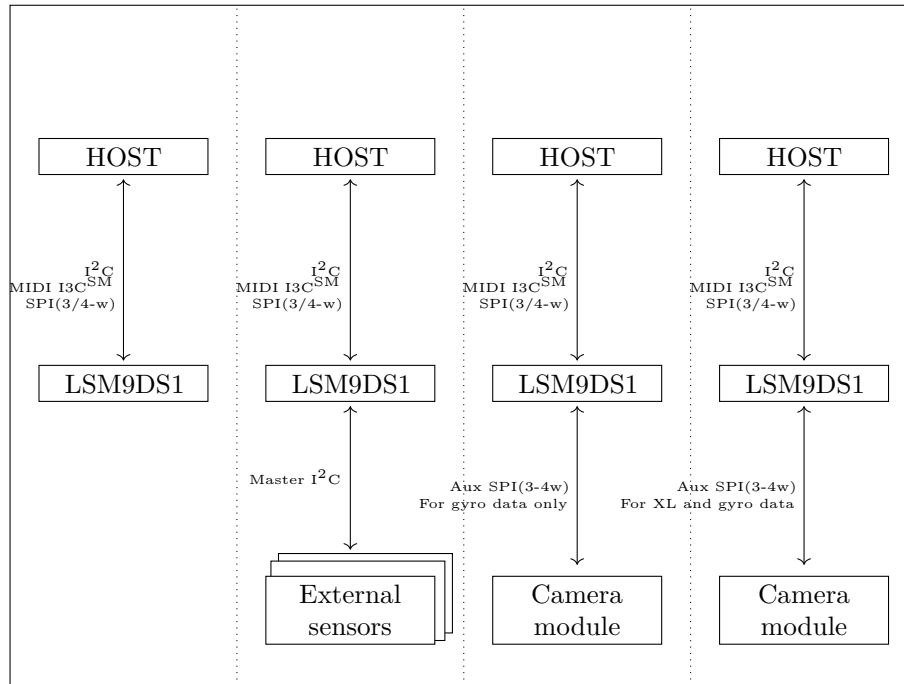


Figure 20.3.: Explication of pin mode

20.4. Library

20.4.1. Library Description

To support the project's requirements, we need 3 main libraries, each serving a specific need :

1. [Wire.h](#): This library is a standard Arduino library that facilitates communication between I^2C (Inter-Integrated Circuit) devices. I^2C is a synchronous serial communication protocol used to connect microcontrollers to external devices. [Wire.h](#) provides functions for initializing the I^2C bus, sending and receiving data on the bus, and managing multiple devices on the same bus. With this library, developers can easily connect multiple I^2C devices, such as sensors or displays, to an Arduino board.
2. [Kalman.h](#): This library implements the Kalman filter algorithm. The Kalman filter is a mathematical technique used to estimate the state of a system from incomplete measurements. It is commonly used in control systems, robotics and navigation applications to improve the accuracy of sensor measurements and reduce errors. [Kalman.h](#) provides a simple interface for developers to implement the Kalman filter in their Arduino projects.
3. [Arduino_LSM9DS1.h](#): The library [Arduino_LSM9DS1.h](#) is a software library designed to facilitate interaction with the sensor accelerometer LSM9DS1, developed by STMicroelectronics. This sensor combines an accelerometer, a gyroscope and a magnetometer in a single package. The library LSM9DS1 provides an interface for accessing the sensor's functionalities, such as reading acceleration, angular velocity and magnetic field data. It can be used to configure various sensor parameters, such as measurement scales, sampling rates and operating modes.

20.4.2. Installation

The library give some example of sketch, to use this sketch we need to download the library on ArduinoIDE. The library `Arduino LSM9DS1.h` and `Kalman.h` can be download directly on Arduino. The library `wire.h` is used if you input : `#include <wire.h>`

```
1  #include <Wire.h>
2
```

Figure 20.4.: Wire include

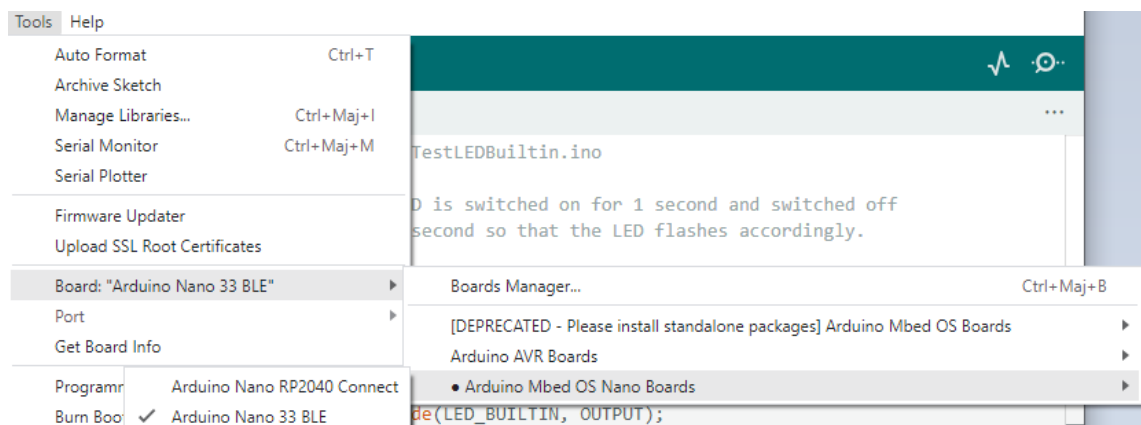


Figure 20.5.: Board card installation

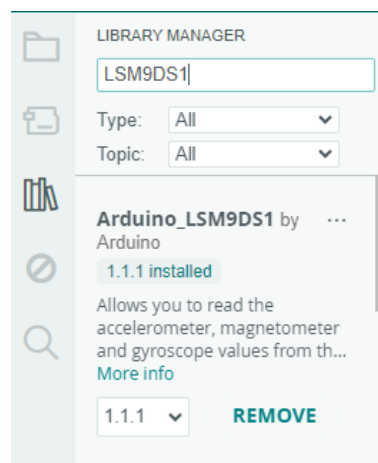


Figure 20.6.: Library choice

20.5. Function

The library LSM9DS1 on Arduino Nano 33 BLE Sense is composed of different function. The function is different if you was on I²C protocol or SPI protocol.

20.5.1. Library `Wire.h`

This library is used to develop communication between different components of an Arduino board, it's used for I²C. The library `Wire.h` is composed by :

- `Wire.begin()`: Initializes the library `Wire.h` and prepares the Arduino to act as master or slave on the bus I²C.
- `Wire.beginTransmission(address)`: Starts a transmission to a slave device with the specified address.
- `Wire.write(data)` : Sends data on the I2C bus during transmission.
- `Wire.endTransmission()` : Ends the current transmission and releases the I2C bus.
- `Wire.requestFrom(address, quantity)` : Requests data from a slave device with the specified address.
- `Wire.available()`: Checks whether data is available to be read after a request.
- `Wire.read()`: Reads data received from a slave device.

We can test the function with an example of the library `Wire.h`. This is an example to test the I²C bus on the board.

Listing 20.1.: Simple sketch using the sensor LSM9DS1 detection

```

1000 #include <Wire.h>
1002 #define LSM9DS1_M_ADDR 0x1E // I2C Address of the LSM9DS1 magnetometer
      module
1003 #define LSM9DS1_AG_ADDR 0x6B // I2C Address of the LSM9DS1 accelerometer
      -gyroscope module
1004
1005 void setup() {
1006     Serial.begin(9600); // Initialize serial communication
1007     Wire.begin(); // Initialize Wire library
1008     // Initialize LSM9DS1
1009     initLSM9DS1();
1010 }
1011
1012 void loop() {
1013     // Read LSM9DS1 data
1014     readLSM9DS1();
1015     delay(1000); // Pause for one second between readings
1016 }
1017
1018 void initLSM9DS1() {
1019     // Initialize magnetometer module
1020     Wire.beginTransmission(LSM9DS1_M_ADDR);
1021     Wire.write(0x20); // Control register address for mode
1022     Wire.write(0x1C); // Activate continuous mode at 10 Hz
1023     Wire.endTransmission();
1024
1025     // Initialize accelerometer-gyroscope module
1026     Wire.beginTransmission(LSM9DS1_AG_ADDR);
1027     Wire.write(0x10); // Control register address for gyro mode
1028     Wire.write(0x38); // Activate continuous m

```

../Code/Nano33BLESense/IMU/Test/WireEx.ino

20.5.2. Function `IMU.begin()`

The function to initialize the IMU is `IMU.begin()`, they start the IMU initialization process. This step is important to configure the IMU's basic parameters, set the communications with other devices, and prepares the unit for proper operation. Accurate initialization is essential to ensure the accuracy and reliability of the data provided by the IMU.

An error during the initialization can lead to inaccurate measurements or unpredictable behavior. For example, incorrectly configured measurement parameters or sampling rates can compromise data quality, affecting overall system performance.

```
1000     if (!IMU.begin()) {
1002         Serial.println("Failed to initialize IMU!");
        while(1) {}; }
```

20.5.3. Function `IMU.end()`

This function is composed without parameter. After do this function they return value of 1 if all is good and they return value of 0 if we had some problem.

This function is used to stop or deactivate the IMU once it is no longer required. It frees up the resources used by the IMU and ends its operation cleanly. When called, it ensures that all tasks associated with the IMU are properly finalized, avoiding memory leaks or other problems associated with incomplete de-initialization.

This step helps to ensure the stability and reliability of the system as a whole, by avoiding problems associated with incorrect IMU de-initialization.

The data was return at the end.

```
1000     if (!IMU.begin()) {
1002         Serial.println("Failed to initialize IMU!");
        while(1) {}; }
```

20.5.4. Function `IMU.readAcceleration(x, y, z)`

The `IMU.readAcceleration()` is used to read the IMU accelerometer and obtain the data, this is expressed in gravity (g). This function provides information on the movements detected by the IMU's accelerometer. The resulting acceleration data can be used for a variety of applications, such as shock or motion detection, or inertial navigation. By regularly interrogating the accelerometer and analyzing variations in acceleration, it is possible to understand and react to changes in motion with precision, which is essential in many modern embedded systems and electronic devices.

The parameters x, y and z are floats giving information on the x, y or z-axis

```
1000     float x;
1002     float y;
1004     float z;

1006     if (IMU.accelerationAvailable()) {
        IMU.readAcceleration(x, y, z);

1008         Serial.print(x);
        Serial.print('\t');
1010         Serial.print(y);
        Serial.print('\t');
1012         Serial.println(z); }
```

20.5.5. Function `IMU.readGyroscope()`

Retrieves data from the IMU's gyroscope and returns angular velocity in degrees per second (dps). This function provides information on the rotational movement detected by the IMU's gyroscope. The angular velocity data retrieved can be used in various applications such as robotics, motion tracking or navigation systems. By regularly interrogating the gyroscope and analyzing angular velocity variations, it becomes possible to understand and respond precisely to rotational movements, which is crucial in applications requiring precise orientation control or stabilization.

The parameters x, y and z are floats giving information on the x, y or z axis

```

1000     float x, y, z;
1002     if (IMU.gyroscopeAvailable()) {
1003         IMU.readGyroscope(x, y, z);
1004
1005         Serial.print(x);
1006         Serial.print('\t');
1007         Serial.print(y);
1008         Serial.print('\t');
1009         Serial.println(z); }

```

20.5.6. Function `IMU.accelerationAvailable()`

This function allows you to check the status of IMU acceleration data, indicating whether or not new data is ready to be retrieved.

In scenarios such as motion tracking, robotics or navigation systems, where the IMU continuously captures acceleration data, the availability of new data determines the system answers. Test the IMU at regular intervals is used for application and they can check for acceleration currently or non, ensuring that the system remains synchronized with object movements in real time.

When the function returns a value of 1, this means that new acceleration data is available, enabling the system to extract the data and process it further. A value of 0 indicates that there have been no recent acceleration updates.

```

1000     float x, y, z;
1002     if (IMU.accelerationAvailable()) {
1003         IMU.readAcceleration(x, y, z);
1004
1005         Serial.print(x);
1006         Serial.print('\t');
1007         Serial.print(y);
1008         Serial.print('\t');
1009         Serial.println(z); }

```

20.5.7. Function `IMU.gyroscopeAvailable()`

This function ask if new gyro data are available for the IMU and they returns a value 0 if no new gyro data is available, and 1 if new gyro data is ready to be retrieved.

In applications requiring real-time monitoring of rotational motion, such as drone stabilization, virtual reality systems or inertial navigation, the system can determine whether to wait for updated gyroscope readings or proceed to process available data. A value of 1 indicates that new gyro data is available, enabling the system to retrieve and use this information for adapted the orientation tracking or motion control. Conversely, a feedback value of 0 indicates that there have been no recent updates to

the gyroscope measurements, prompting the system to wait for new data to become available before proceeding.

```

1000     float x, y, z;
1002     if (IMU.gyroscopeAvailable()) {
1004         IMU.readGyroscope(x, y, z);
1006         Serial.print(x);
1008         Serial.print(' ');
1009         Serial.print(y);
1010         Serial.print(' ');
1011         Serial.print(z);
1012         Serial.println();
1013     }

```

20.5.8. Function `IMU.accelerationSampleRate()`

This function is designed to provide information on the sampling rate of the IMU's accelerometer. The function `IMU.accelerationSampleRate()` returns the accelerometer sampling rate, expressed in Hertz (Hz). This value represents the frequency at which the accelerometer takes measurements and provides acceleration data. It indicates how many times per second the accelerometer registers changes in acceleration along its axes.

In activities such as motion tracking, gesture recognition or vibration analysis, knowledge of the accelerometer's sampling rate ensures that the system can accurately capture and respond to changes in motion dynamics.

```

1000     Serial.print("Accelerometer sample rate = ");
1001     Serial.print(IMU.accelerationSampleRate());
1002     Serial.println(" Hz");
1003     Serial.println();
1004     Serial.println("Acceleration in g's");
1005     Serial.println("X \t Y \t Z");

```

20.5.9. Function `IMU.gyroscopeSampleRate()`

To recover and communicate the specific rate at which the gyroscope, integrated into the inertial measurement unit (IMU), collects data samples. This frequency is usually expressed in hertz (Hz), corresponding to the number of samples acquired per second. This is the frequency at which the gyroscope takes measurements, giving an idea of its operational efficiency and performance.

```

1000     Serial.print("Gyroscope sample rate = ");
1001     Serial.print(IMU.gyroscopeSampleRate());
1002     Serial.println(" Hz");
1003     Serial.println();
1004     Serial.println("Angular speed in degrees/second");
1005     Serial.println("X tY tZ");

```


21. Distance, Speed and Acceleration Detection Algorithms

21.1. Review of Distance Measurement Methods

Since Mitsubishi released the first cruise control with distance control in 1995, the vast majority of ACC functions have been based on Laser Radar or millimeter-wave radar (MWR).¹ But a few have opted to use a binocular camera as the basis for the technology, such as Subaru's EyeSight technology.² Each of these different sets of technical solutions has its own advantages and disadvantages:

Technology	Advantages	Disadvantages
Laser Radar (LiDAR)	- High accuracy and resolution - Capable of creating detailed 3D maps - Effective for object detection and classification	- Expensive - Affected by adverse weather conditions - High power consumption
Millimeter-Wave Radar	- Less affected by weather conditions - Long-range detection capabilities - Lower cost compared to LiDAR	- Lower resolution than LiDAR - Limited in detecting small or non-metallic objects - Can be affected by interference
Binocular Camera Systems	- Lower cost compared to LiDAR and radar - High resolution for nearby objects - Uses passive sensing (no emissions)	- Computationally intensive - Accuracy decreases with distance - Affected by lighting conditions

Table 21.1.: Comparison of distance measurement technologies: LiDAR, Millimeter-Wave Radar, and Binocular Cameras

For cars using Laser Radar, LiDAR operates by emitting laser pulses towards objects in front of the vehicle. When these pulses hit an object, they are reflected back to the sensor, which measures the time it takes for the pulses to return. This time-of-flight measurement allows the system to calculate the precise distance to the object, as well as its relative speed and position. For cars using millimeter-wave radar, the functional implementation is similar.

For systems that use binocular cameras, this process can be relatively more complex; essentially distance recognition for binocular camera-based systems utilizes bionics: Binocular disparity. This method involves using a pair of cameras positioned at a certain distance apart to capture two images with different viewing angles. By comparing the disparity between the two images (i.e., the difference in pixel positions).³

$$\text{depth} = \frac{f \times \text{baseline}}{\text{disparity}} \quad (21.1)$$

where depth represents the distance of an object from the camera, f denotes the focal length of the camera, baseline is the distance between the two cameras, and disparity is the difference in image location of the object in the two camera views.

¹MI-PILOT, Mitsubishi Motors, <https://www.mitsubishi-motors.com/en/brand/technology/mipilot2/index.html>

²EyeSight technology, Subaru, <https://www.subaru.com/eyesight.html>

³Mansour, M., Davidson, P., Stepanov, O. and Piché, R., 2019. Relative Importance of Binocular Disparity and Motion Parallax for Depth Estimation: A Computer Vision Approach. Remote Sensing, 11(17), p.1990. <https://doi.org/10.3390/rs11171990>

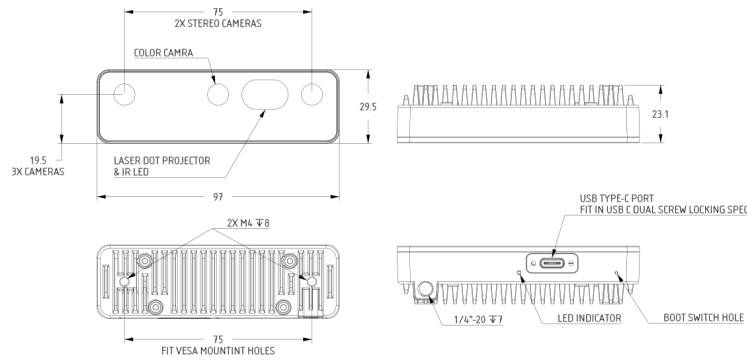


Figure 21.1.: Design of OAK-D Pro camera

For the OAK-D Pro camera, the baseline distance, which is the distance between the two monochrome cameras, is 7.5 cm.⁴

According to OAK's official documentation⁵, the OAK-D Pro can reach a theoretical maximum of 35m, but at this distance there is a very significant margin of error (about 33% of the theoretical error⁶) where the distance fluctuates very significantly. However, there are ways to improve the accuracy of distance detection, such as half-pixel mode to improve the accuracy of distance detection, and averaging distances over a period of time to calculate relative distances to improve the accuracy of relative speed calculations.

21.2. Review of Speed and Acceleration Detection Algorithms

For the measurement of relative velocity, the three schemes mentioned above will actually have two different principles and calculations.

Millimeter-wave radar primarily calculates the relative speed of objects using the Doppler effect. When the radar signal hits a moving object, the frequency of the reflected wave changes depending on the relative speed between the radar and the object. This frequency shift (Doppler shift) is directly proportional to the relative speed of the object.⁷

For LiDAR and Binocular Camera Systems, the relative speed of objects is calculated based on the change in distance over time. By continuously measuring the distance to an object and comparing it with previous measurements.

Each of these different sets of technical solutions has its own advantages and disadvantages:

⁴OAK-D Pro Camera Documentation, Luxonis, 2021

⁵OAK China official Website, OAK China, 2024

⁶Depth range enhancement, Luxonis Community, 2022

⁷Millimeter Wave Radar Sensors: Fundamentals, Texas Instruments, 2018

Technology	Advantages	Disadvantages
Laser Radar (LiDAR)	- High accuracy and resolution - Capable of creating detailed 3D maps - Effective for object detection and classification	- Expensive - Affected by adverse weather conditions - High power consumption
Millimeter-Wave Radar	- Less affected by weather conditions - Long-range detection capabilities - Lower cost compared to LiDAR	- Lower resolution than LiDAR - Limited in detecting small or non-metallic objects - Can be affected by interference
Binocular Camera Systems	- Lower cost compared to LiDAR and radar - High resolution for nearby objects - Uses passive sensing (no emissions)	- Computationally intensive - Accuracy decreases with distance - Affected by lighting conditions

Table 21.2.: Comparison of distance measurement technologies: LiDAR, Millimeter-Wave Radar, and Binocular Cameras

Part II.

Arduino Nano 33 BLE Sense - External Sensors and Actors

22. Tiny Machine Learning Kit

The Tiny Machine Learning Kit will equip you with all the tools which are needed to start with machine learning. [Ardh]

The kit consists of

- an Arduino Nano 33 BLE Sense Lite,
- a camera module OV7675,
- USB A to USB Micro B cable, and
- a custom Arduino shield to make it easy to attach components.



Figure 22.1.: Das Tiny Machine Learning Kit [Ardh]

22.1. Das Arduino Tiny Machine Learning Shield

Das Tiny Machine Learning Shield wird von Arduino für das Tiny Machine Learning Kit hergestellt, um das Verbinden von Komponenten, wie beispielsweise der Kamera, zu vereinfachen.

Bei dem Tiny Machine Learning Shield handelt es sich um ein Breadboard, was speziell für den Arduino Nano 33 BLE Sense ausgelegt ist. Komponenten können entweder wie die Kamera direkt in passende Slots gesteckt oder mit Grove-Kabeln verbunden werden. Außerdem verfügt das Tiny Machine Learning Shield über eine Anschlussklemme, um externe Spannungsquellen anzuschließen und über einen Taster.

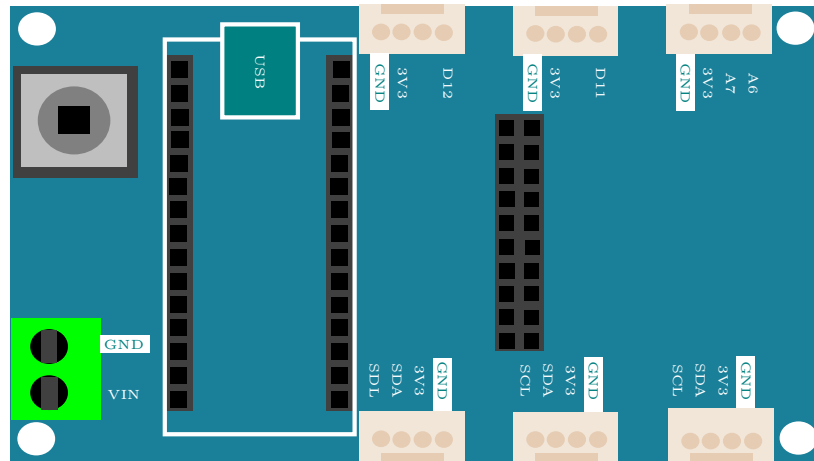


Figure 22.2.: Das Tiny Machine Learning Shield [Gua21]

Die Pinbelegung für die Grove-Schnittstellen sind auf dem Tiny Machine Learning Shield beschriftet. Die Belegung für den 10-poligen Steckplatz für die Kamera ist in der Grafik 22.3 dokumentiert.

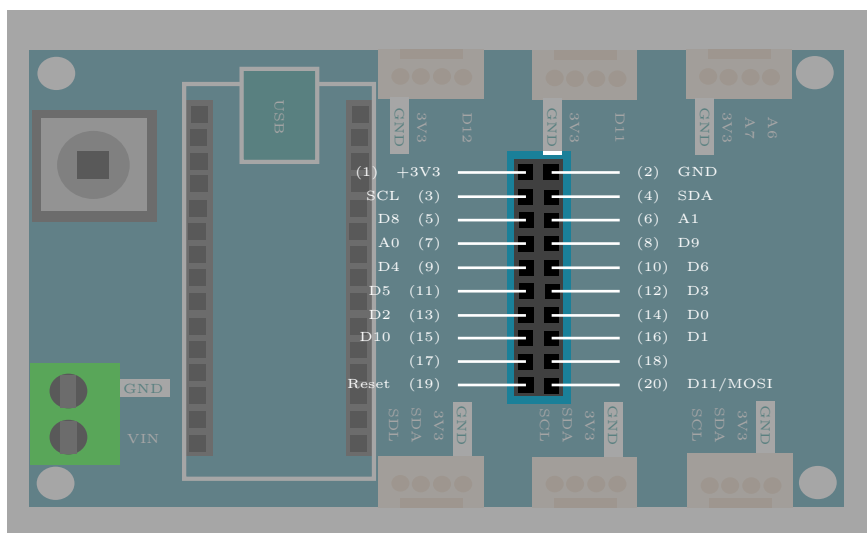


Figure 22.3.: Pin-Belegung des Tiny Machine Learning Shields [Gua21]

22.2. Die OV7675 Kamera

Da die auf dem Arduino fest verbauten Lichtsensoren nur Helligkeit und Farben messen können, ist in dem Kit eine Kamera enthalten. Diese kann verwendet werden, um Objekte zu erkennen oder einen Videostream auszugeben. In unserem Projekt findet die Kamera allerdings keine Verwendung.

Die Kamera kann direkt auf den mittleren Anschluss des Tiny Machine Learning Shields gesteckt werden. Es sind also keine zusätzlichen Kabel zum Verbinden notwendig.

WS:Quelle fehlt

23. Powering the TinyML Shield

Now that you have your Arduino development board up and running, let's talk about how you could deploy it independent of your computer! While some embedded systems call on AC-DC converters (or "wall warts," colloquially) to provide low voltage power to their electronics (the Google home speaker, for example), others are battery powered. Both of these paradigms are applicable to real-world deployment of tinyML and both are achievable using your TinyML kit.

WS:Komplettes Kapitel
muss überarbeitet werden
Sortierung, Quellen,
Diagramm,...

23.0.1. USB Power Delivery

To this point, we have provided power over USB to our microcontroller via the microB port on the Nano 33 BLE Sense. The 5V that USB carries is then down regulated on the development board to 3.3V, the logical reference for the MCU. While there's nothing wrong with this in development, a prototype of your application ought not depend on drawing power from your computer. Instead, you could call on an AC-DC converter with USB output. This has the added benefit of likely raising the current capacity of the 5V power rail, from at most 500 mA via a computer to whatever the specification happens to be for a given wall-bound converter. This could be meaningful for driving certain power hungry actuators, like speakers.

23.1. Battery

While the above solution removes the need for the computer, you're still tethered to a wall. To go fully mobile you'll want to call on a battery. So what are our options? The idea of using a voltage regulator (specifically, the MPM3610) to cut down an input voltage to a nice, stable 3.3V level applies here as well. If you were to take a closer look at the linked datasheet, you'd find that the MPM3610 accepts input voltages from 4.5V to 21V. The 5V delivered over USB is within this range, and any compatible battery will need to be as well. This unfortunately eliminates the possibility of calling on single cell 3.7V lithium batteries, but makes the selection of a 9V alkaline battery fairly obvious.

You might be wondering how any battery might connect to the boards in front of you, but never fear, we've got you covered. At one corner of the Tiny Machine Learning Shield you'll find a green terminal block with silkscreen labels that read, "VIN" and "GND," where GND is our reference voltage and as such should be connected to the negative terminal of any compatible battery. This green terminal block is where you'd want to screw in wires carrying 4.5V to 21V, and we'll add that 9V clip, like this, that terminates in pre-stripped hook-up wire makes this quite easy!

Assembly Steps

1. Screw down a wire leading from the negative battery terminal (black) to GND (Most < 3mm flat-head screwdrivers will suffice here).
2. Repeat this process for the positive battery terminal (red) to VIN. And that's it you're all set to power your Arduino from a battery!

Important Notes

1. While there is clever circuitry on board to handle such an exception, it is generally good practice to avoid having competing power sources, so we'd recommend that you unplug the Nano 33 BLE Sense from USB power before connecting a battery
2. With about 550 mAh capacity, a 9V battery can source 15 mA for about 37 hours before you will need to get a new one.

7S:tikz-Bild des Shields
fehlt.

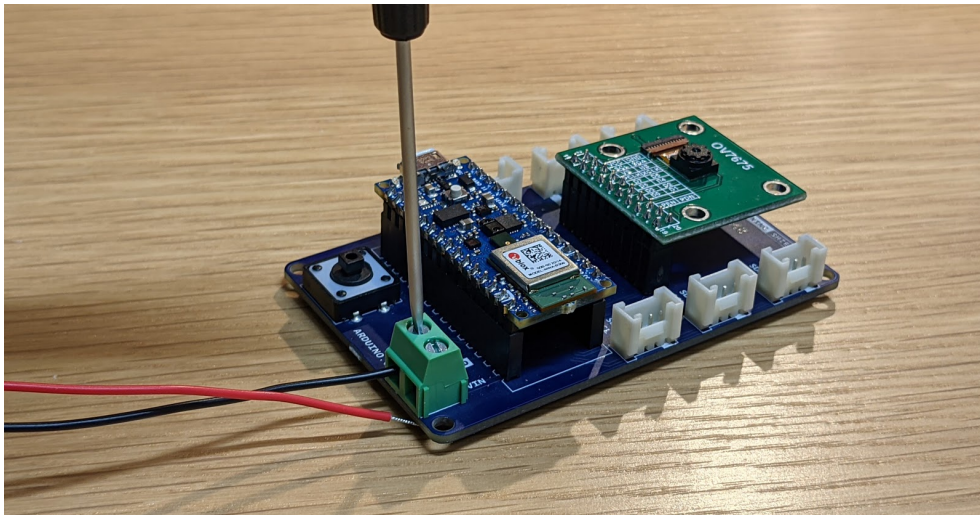


Figure 23.1.: Montage des Clips

An image of screwing down the black negative terminal to attach a wire.

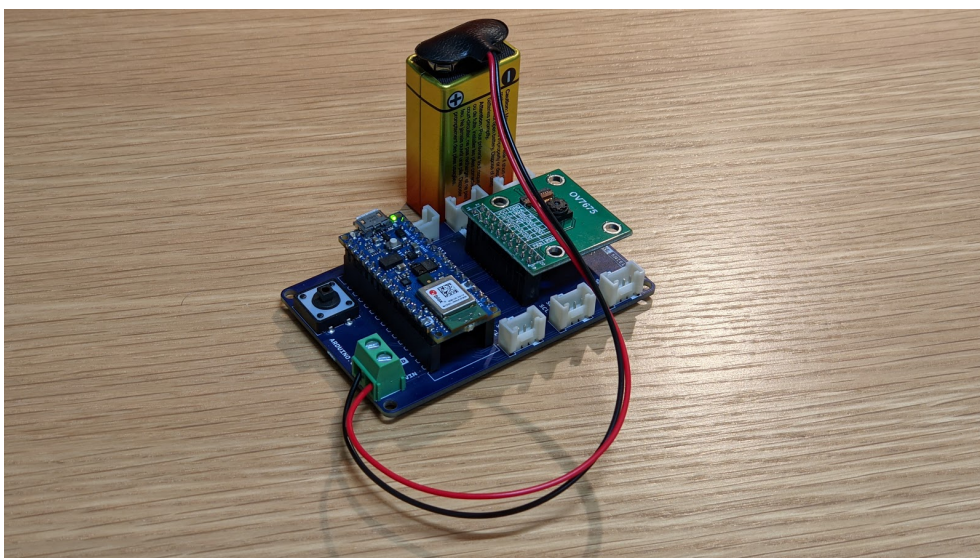


Figure 23.2.: Komplettes System mit Batterie

An image of the attached battery powering the device.

23.2. Checking the Battery Voltage

We use an analog input pin to read the voltage. As we are running from a 3.7V volt battery, we need to adjust the reference voltage used by the pin as otherwise it would

be comparing the voltage to itself. The statement `analogReference (INTERNAL)` sets the pin to compare the input voltage to a regulated 1.1V. We therefore need to reduce the voltage on the input pin to less than 1.1V for this to work. This is done by dividing the voltage using 2 resistors, 1m and 330k ohms. This divides the voltage by approximately 4 so when the battery is fully charged, which is 4.2V, the voltage at the pin input is $4.2/4 = 1.05V$.

```
// Battery Monitor
#define MONITOR_PIN A0           // Pin used to monitor supply voltage
const float voltageDivider = 4.0; // Used to calculate the actual voltage fRom
                                   // Using 1m and 330k ohm resistors divides th

voltage by approx 4

                                   // You may want to substitute
actual values of resistors in an equation (R1 + R2)/R2
                                   // E.g. (1000 + 330)/330 = 4.03
                                   // Alternatively take the voltage reading a
                                   // the 2 resistors to ground and divide one

// Read the monitor pin and calculate the voltage
float BatteryVoltage()
{
    float reading = analogRead(MONITOR_PIN);
    // Calculate voltage - reference voltage is 1.1v
    return 1.1 * (reading/1023) * voltageDivider;
}
```

The function `BatteryVoltage()`, reads the analog pin, which will range from 0 for 0V to 1,023 for 1.1V and using this reading calculates the actual voltage coming from the battery.

The function `DrawScreenSave()` function calls this then selects the appropriate bitmap to display based on the following:

- If voltage is greater than 3.6V - full
- Voltage between 3.5 and 3.6V - 3/4
- Voltage between 3.4 and 3.5V - half
- Voltage between 3.3 and 3.4V - 1/4
- Voltage < 3.3V - empty

23.3. Batterieclip

Der Batterieclip in Abb. 23.3, der vom Hersteller *reichelt* ist, kann vertikal an einen 9-Volt-Block angeschlossen werden. Die dazugehörigen Anschlussdrähte haben eine Länge von 150 mm. Der Anschlussclip ist in der I-Form ausgeführt, weshalb er sich platzsparend ins Gehäuse einbinden lässt [Rei].

23.4. Spannungssensor

Der Spannungssensor in Abb. ?? von dem Hersteller *Shenzhen Global Technology Co., Ltd* kann bei der Versorgungsspannung von 3,3 V Spannungen in dem Bereich von 0 V bis 16,5 V messen. Dieser wird genutzt, um den Ladestand der Batterie zu überwachen. Die analoge Auflösung des Sensors liegt bei 10 Bit. Damit kann bei dem angegebenen Messbereich die Spannung mit der Auflösung von 0,016 V gemessen werden. Zur



Figure 23.3.: Batterieclip für 9-Volt-Block[Rei24b]

Eingangsschnittstelle gehört der Voltage at the Common Collector (VCC)-Anschluss und der Ground (GND)-Anschluss [She]. Die Bauteilmaße betragen 13 mm x 27 mm.



Figure 23.4.: Voltage Sensor 170640 von dem Hersteller *Shenzhen Global Technology Co., Ltd*[She]

Mit dem Sketch `TestBattery.ino`, siehe ??TestBattery, soll der Spannungssensor getestet werden.

23.4.1. Durchführung

Für den Test werden die folgenden Hardware-Komponenten benötigt:

- Arduino Nano 33 BLE Sense Lite
- Tiny Machine Learning Shield
- USB-A auf USB-Mikro Verbindungskabel
- Grove Jumper zu Grove 4 Pin Kabel
- Spannungssensor
- Batterie
- Batterieclip

Listing 23.1.: Beispiel-Sketch zum Testen der Batterie

```

1000 /**
1001  * @file TestBattery.ino
1002  * @author Wings
1003  * @date 20.05.2024
1004  * @details The sketch is used to test the voltage sensor. For the test,
1005  *          the voltage is measured on a 9 V block battery and output on the
1006  *          serial monitor.
1007  *
1008  */
1009
1010 /** Pin A7 is referenced as the voltage pin for the voltage sensor. */
1011 #define VoltagePin A7
1012
1013 /** The parameter SignalEingang is later assigned the value read in from
1014     the VoltagePin pin. */
1015 long SignalEingang;
1016
1017 /** The parameter Spannung stands for the voltage that is measured with
1018     the voltage sensor and has the unit volt. */
1019 double Voltage;
1020
1021 /**
1022  * @brief Starting serial communication
1023  *
1024  * @details The function is executed when the programme is started. The
1025  *          serial interface to the computer is initialised at 9600 baud
1026  *          There is no return value.
1027  */
1028 void setup() {
1029     Serial.begin(9600);
1030 }
1031
1032 /**
1033  * @brief Display of the measured voltage
1034  *
1035  * @details The function is executed continuously. The signal at the pin
1036  *          VoltagePin is read out and a voltage value is converted.
1037  *          The value read in at the voltage pin is between 0 and 1023. The
1038  *          voltage measurement range is between 0 and 16.5 V. The read-in value
1039  *          can be converted into a voltage using the function map().
1040  *          The voltage is displayed in volts on the serial monitor. There is a 5-
1041  *          second wait after each cycle to avoid flooding the serial monitor.
1042  *          There is no return value.
1043  */
1044 void loop() {
1045     SignalEingang = analogRead(VoltagePin);
1046     Voltage = map(SignalEingang, 0, 1023, 0, 165); // Converting the
1047             input signal 165 for 16.5 V
1048     Voltage = Voltage/10; // Converting the input signal 165 for
1049             16.5 V
1050     Serial.print(Voltage);
1051     Serial.println(" V");
1052     delay(5000);
1053 }

```

../Code/Arduino/Battery/TestBattery.ino

Die Hardware-Komponenten werden zusammengebaut, aber die Batterie wird noch nicht angeschlossen. Dann wird der Arduino Nano 33 BLE Sense Lite mit einem Computer verbunden. Anschließend wird der Sketch `TestBattery.ino` auf den Arduino Nano 33 BLE Sense Lite geladen und der serielle Monitor in der Arduino IDE geöffnet. Die Batterie wird während des Tests an den Batterieclip angeschlossen und der gemessene Wert bei dem seriellen Monitor ausgelesen.

23.4.2. Ergebnisse

Zu Beginn des Tests zeigt der serielle Monitor die Spannung 0 V an. Nach dem Anschließen der Batterie an den Spannungssensor wird die Spannung 9,6 V angezeigt. Abbildung 23.5 zeigt den gemessenen Spannungsverlauf. Die angezeigte Spannung liegt in dem erwarteten Bereich einer 9 V Batterie.

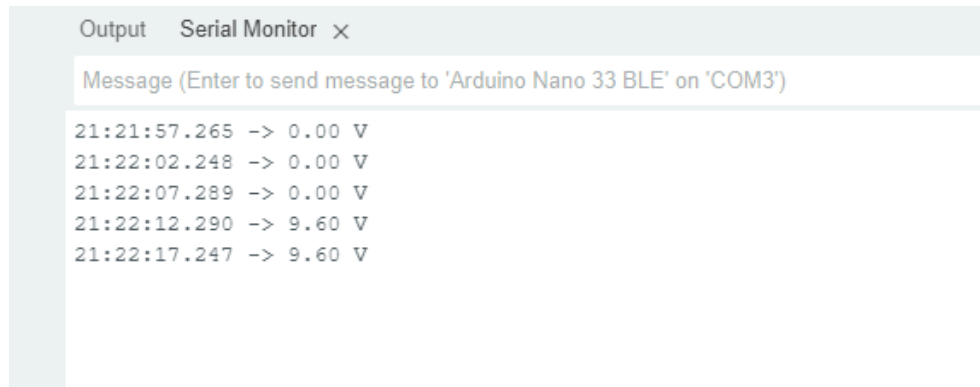


Figure 23.5.: Testoutput des Spannungssensors

24. Connectors of Type JST

24.1. Connector Series

JST connectors are electrical connectors manufactured to the design standards originally developed by J.S.T. Mfg. Co. (Japan Solderless Terminal). [Jsta] JST manufactures numerous series (families) and pitches (pin-to-pin distance) of connectors.

JST manufactures a large number of series (families) of connectors. The PCB (wire-to-board) connectors are available in top (vertical) or side (horizontal) entry, and through-hole or surface-mount. [Jstb]

This description refers exclusively to the mechanical structure and electrical behaviour. The pin assignment is not standardised.

Attention: The term “JST” is sometimes incorrectly used as a vernacular term meaning any small white electrical connector mounted on Printed Circuit Board (PCB)s.

24.2. JST VH

The technical data of the JST VH series is given in this section. They are taken from the data sheet for the series. [Jstd]

24.2.1. Product Profile

Series	VH connector
Category	Crimp Style Connectors (Wire-to-Board type)
Type	Crimp style, Compact type, With locking device Disconnectable type
Feature	With secure locking device

24.2.2. Specification

Pitch	3.96mm
Pin rows	1
Current rating	10A (AWG#16)
Voltage rating	250V
PC board mounting direction	Top entry, Side entry



Figure 24.1.: JST connector type VH <https://www.jst-mfg.com/product/index.php?series=262>

24.3. JST PH

The technical data of the JST PH series is given in this section. They are taken from the data sheet for the series. [Jstc]

24.3.1. Product Profile

Series	PH connector
Category	Crimp Style Connectors (Wire-to-Board type)
Type	Crimp style, Thin type Disconnectable type

24.3.2. Specification

Pitch	2.0mm
Current rating	2A (AWG#24)
Voltage rating	100V
PC board mounting direction	Top entry, Side entry

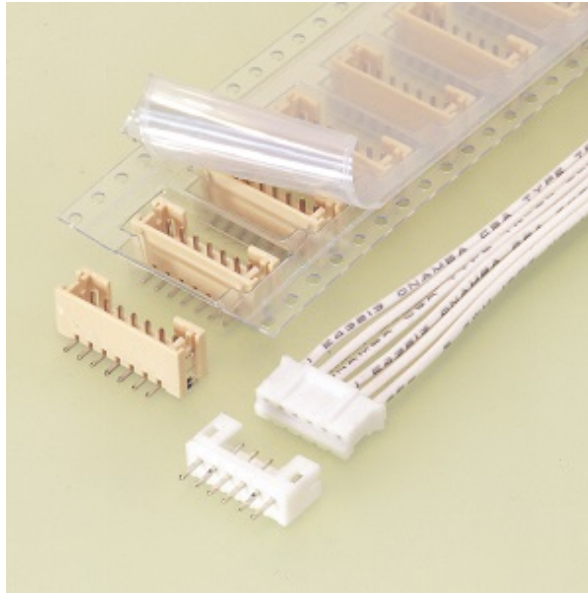


Figure 24.2.: JST connector type VH [Jstc]

24.3.3. JST PHR-2

One variant of the connector series is the JST PHR-2 type, which has 2 pins. Figure 24.3 shows a JST PHR-2 connector.



Figure 24.3.: JST connector type PHR-2 [Jstc]

25. Grove

Grove, ein Produkt von Seeed Studio, ist eine Hardware-Schnittstelle für Technikinteressierte. Es vereinfacht die Kommunikation zwischen Mikrocontrollern und Sensoren und reduziert das Löten. Diese Schnittstelle wurde im Jahr 2010 von Seeed Studio entwickelt, um die Zusammenarbeit zwischen verschiedenen Technologien zu vereinfachen und die Entwicklung neuer Projekte zu erleichtern.[See12; See13]

Mit der Verfügbarkeit eines Steckverbinder-Typs bietet diese Schnittstelle eine Vielzahl von Möglichkeiten für die Integration von Sensoren, Aktuatoren und Mikrocontrollern. Diese einfachen Kommunikationsverbindungen ermöglichen es den Nutzern, ihre Projekte ohne großen Zeitaufwand zu realisieren und zu testen. [See16] Mit jedem neuen Sensor, der mit der Grove-Familie auf dem Markt erscheint, wird die Flexibilität und kreative Möglichkeit für die Entwicklung von Projekten weiter gesteigert.

Die meisten Mikrocontroller verfügen nicht unmittelbar über die Grove-Schnittstelle. Damit Komponenten, die die Grove-Schnittstelle haben, werden häufiger Shields eingesetzt, die diese dann zur Verfügung stellt. Zum Beispiel verfügt der Arduino Nano 33 BLE Sense Lite über keine Grove-Anschlüsse. Das Tiny Machine Learning Shield ermöglicht die Verwendung von Grove-Komponenten für den Arduino Nano 33 BLE Sense, indem es als Schnittstelle zwischen dem Mikrocontroller und den Grove-Modulen fungiert. Der Arduino Nano 33 BLE Sense wird direkt auf das Shield gesteckt, welches über standardisierte Grove-Anschlüsse verfügt. Auf der Abbildung 25.1 ist dargestellt, wie der Mikrocontroller Arduino Nano 33 BLE Sense mit dem Shield verbunden ist.

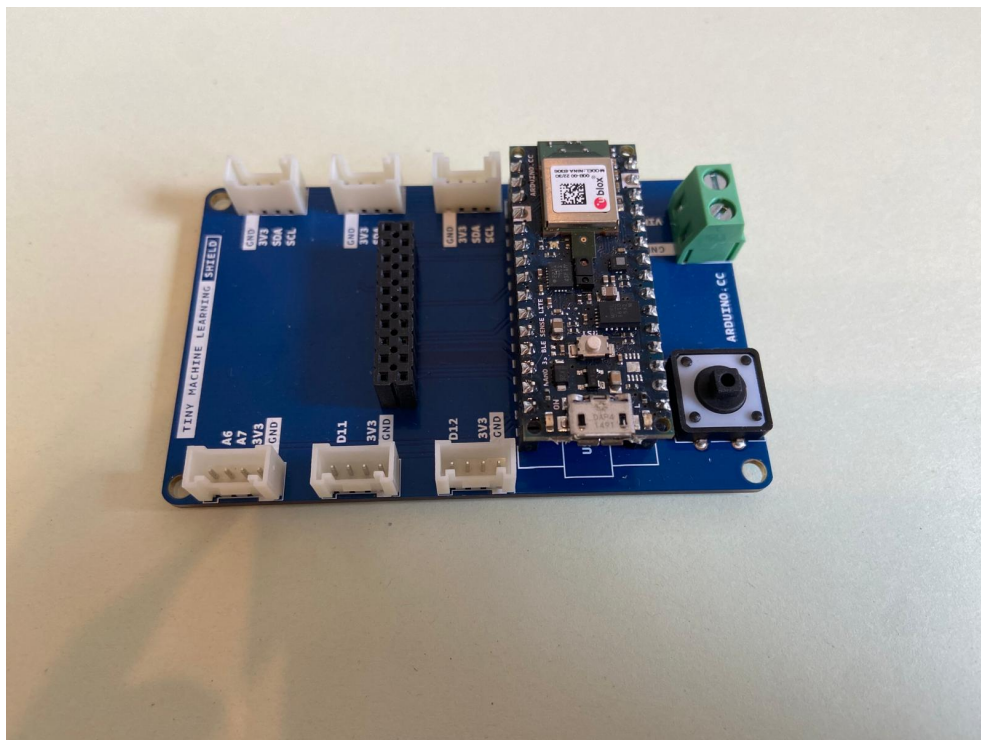


Figure 25.1.: Mikrocontroller gesteckt auf dem Shield

25.1. Grove-Universal-Kabel

Üblicherweise werden Sensoren und Mikrocontroller über ein Grove-Universal-Kabel verbunden. In der Abbildung 25.2, vom Hersteller *seeed studio* ermöglicht die Verbindung mit allen Modulen des Grove-Systems. Für ein Grove-System wird somit nur eine Kabelart benötigt. Die Schnittstellen des Kabels sind beidseitig Japan Solderless Terminal (JST)-VH-Stecker. In der Abbildung 25.2 beträgt beispielsweise die Kabellänge 50 mm [See16].

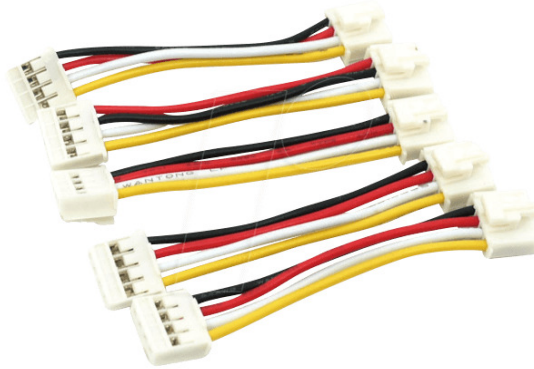


Figure 25.2.: Grove-Universal-Kabel[Rei24a]

25.2. Verbindung Grove-Schnittstelle zu 4-Pin-Lösungen

Da nicht alle Sensoren und Mikrocontroller über eine Grove-Schnittstelle verfügen, existieren Kabel, die einseitig die Grove-Schnittstelle haben. In der Abbildung 25.3 ist das Kabel dargestellt, das auf einer Seite die Grove-Schnittstelle zur Verfügung stellt und auf der anderen Seite 4 Pins. In diesem Beispiel hat das Kabel eine Länge von 30 cm [Rei24c].



Figure 25.3.: Grove Jumper zu Grove 4 Pin-Kabel[Rei24c]

25.3. Pinbelegung

Die Pinbelegung ist von *seeed studio* standardisiert. In der Tabelle 25.1 ist die Belegung erläutert. [See16]

Table 25.1.: Pinbelegung der Grove-Schnittstelle

Farbe	Pin	Funktion
Gelb	1	frei belegbar, D0 oder A0
Weiß	2	frei belegbar, D1 oder A1
Rot	3	VCC, 3,3V oder 5V
Schwarz	4	GND

Grove stellt über Pin 3 und 4 die Stromversorgung sicher. Der gelbe und der weiße Pin ist in Abhängigkeit vom Sensor belegt. Falls der Sensor nur einen Pin benötigt, so wird Pin 1 verwendet. Als Schnittstelle zum Mikrocontroller muss die Belegung für jede Anwendung definiert werden.

Für Protokolle, wie zum Beispiel I²C und Serial Peripheral Interface (SPI), sind die Belegung eindeutig festgelegt. Die Tabelle 25.2 listet die Pinbelegung der Grove-Schnittstelle für I²C-Grove-Stecker auf.

Table 25.2.: Pinbelegung der Grove-Schnittstelle für I²C-Grove-Stecker

Farbe	Pin	Funktion
Gelb	1	frei belegbar, zum Beispiel SCL auf I ² C-Grove-Steckern
Weiß	2	frei belegbar, zum Beispiel SDA auf I ² C-Grove-Steckern
Rot	3	VCC, 3,3V oder 5V
Schwarz	4	GND

Die Tabelle 25.3 enthält die Pinbelegung für die serielle Schnittstelle UART.

Table 25.3.: Pinbelegung der Grove-Schnittstelle für UART

Farbe	Pin	Funktion
Gelb	1	RX
Weiß	2	TX
Rot	3	VCC, 3,3V oder 5V
Schwarz	4	GND

26. BlueTooth

- <https://www.instructables.com/Arduino-NANO-33-Made-Easy-BLE-Sense-and-IoT/>
- <https://www.hackster.io/sridhar-rajagopal/control-arduino-nano-ble-with-blue>
- <https://docs.arduino.cc/tutorials/nano-33-ble-sense/ble-device-to-device>
- <https://projecthub.arduino.cc/8bitkick/sensor-data-streaming-with-arduino-a6>
- <https://www.okdo.com/getting-started/get-started-with-arduino-nano-33-sense/>
- <https://rootsaid.com/arduino-ble-example/>
- <https://learn.sparkfun.com/tutorials/bluetooth-basics>
- <https://ladvien.com/arduino-nano-33-bluetooth-low-energy-setup/>

26.1. Quick Start

This tutorial shows you how to use the free pfodDesignerV3 V3.0.3774+ Android app to create a general purpose Bluetooth Low Energy (BLE) and WiFi connection for Arduino NANO 33 boards without doing any programming. There are three (3) Arduino NANO 33 boards, the NANO 33 BLE and NANO 33 BLE Sense, which connect by BLE only and the NANO 33 IoT which can connect via BLE or WiFi. Connecting using pfodApp is the most flexible way to connect via BLE (or WiFi). See Using pfodApp to connect to the NANO 33 BLE (Step 4) and Using pfodApp to connect to the NANO 33 IoT via BLE (Step 7) and Using pfodApp to connect to the NANO 33 IoT via WiFi (Step 9) below. However simple sketches are also provided to send user defined command words via Telnet, for WiFi, or via the free Nordic nRF UART 2.0 for BLE. See Using the Nordic nRF UART 2.0 app to connect to NANO 33 BLE (Step 5) and Using the Nordic nRF UART 2.0 app to connect to the NANO 33 IoT via BLE (Step 8) and Using a Telnet terminal program to connect to the NANO 33 IoT via WiFi (Step 10) below

This tutorial is also available on-line at Arduino NANO 33 Made Easy.

26.2. Supplies

Arduino NANO 33 – either BLE or Sense or IoT
optionally pfodDesignerV3 and pfodApp

26.3. Step 1: Introduction

There are a number of problems with BLE. See this page for BLE problems and solutions and there are some BLE trouble shooting tips. The learning curve is steep and the specification has hundreds of specialise connect services each of which requires its own mobile application to connect to. This tutorial shows you how to generate

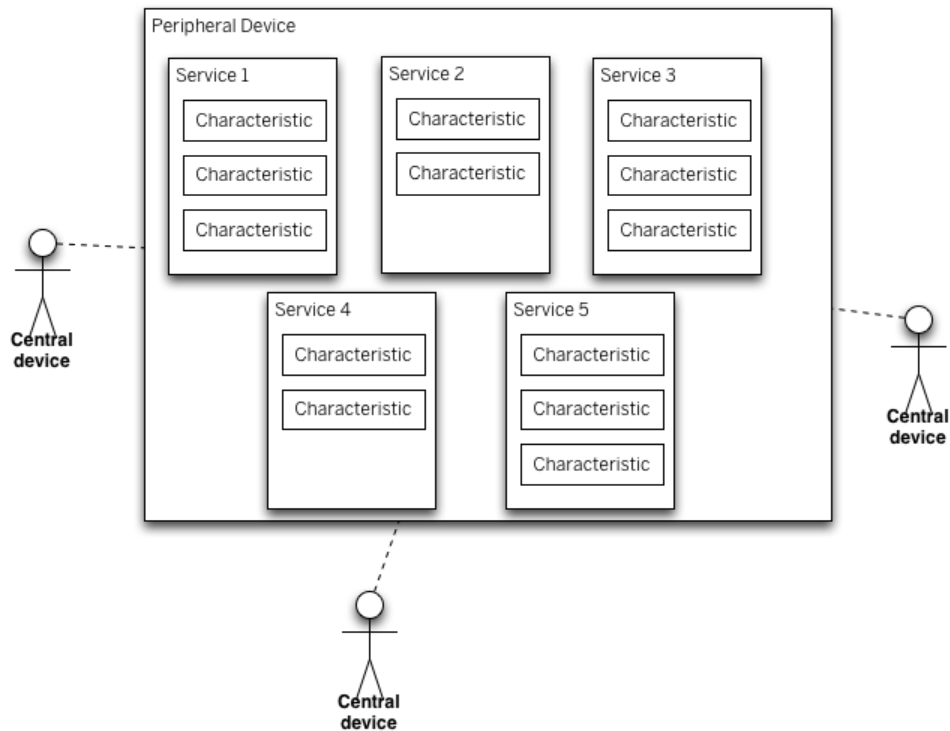


Figure 26.1.: BLE Devices – Peripheral and Central Devices

Arduino code for a general purpose Nordic UART BLE connection over which you can send and receive a stream of commands and data to a general purpose BLE UART mobile application. The free pfodDesignerV3 Android application is used to generate the Arduino code. The output is designed to connect to the paid pfodApp Android application which can display menus, send command, log data and show charts. No Android programming is required. The Arduino code has complete control over what is displayed by pfodApp.

For the NANO 33 IoT you can also connect via WiFi. Again the pfodDesignerV3 generates all the Arduino code and by default is designed to connect to pfodApp with optional 128bit security.

However you do not need to use pfodApp, you can connect to the generated code using the free Nordic nRF UART 2.0 or a Telnet programs (for the WiFi connection). Sketches are included which provide command words to control the boards.

The free pfodDesigner V3.0.3774+ will generate Arduino code for a wide range of boards and connection types including Serial connections, Bluetooth Low Energy (BLE), WiFi, SMS, Radio/LoRa, Bluetooth Classic and Ethernet. For examples Arduino code for of a wide range of BLE boards see Bluetooth Low Energy (BLE) made simple with pfodApp. Here we will be using a BLE connection for NANO 33 BLE, Sense and IoT and a WiFi connection for the IoT.

Each of the NANO 33 boards has extra sensor components that differ between the three (3) boards. The pfodDesignerV3 generates code to read/write the digital outputs and perform analogReads and analogWrites. In the examples below we will turn the board LED on and off and read the voltage at A0 and log and plot it. Once that sketch is running you can add each board's specialised sensor libraries and sent their data in place of the analogRead(A0).

pfodDesigner generates simple menus and charts, however you can also program custom graphical interfaces in your Arduino sketch. Above is an example of slider control

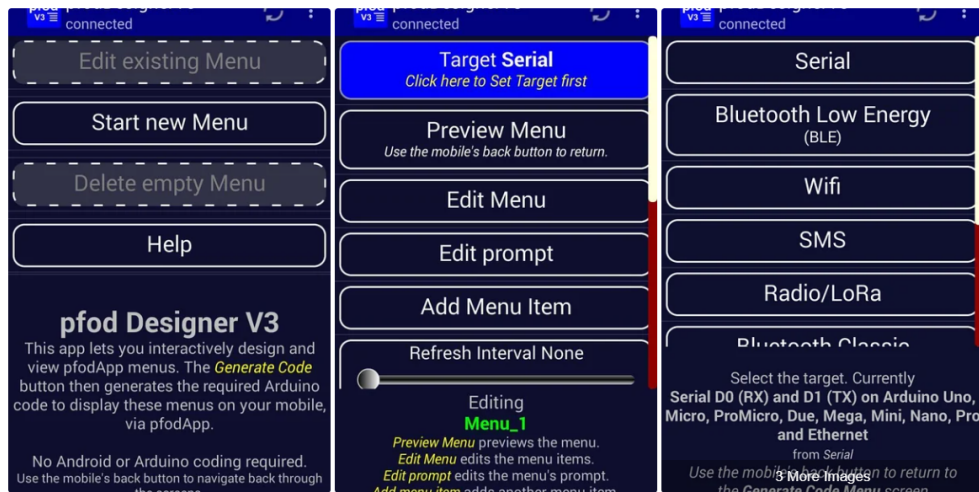


Figure 26.2.: Step 2: Creating the Custom Android Menus and Generating the Code

adjusting a gauge. All the code for this control is in your Arduino sketch. No Android programming necessary. See Custom Arduino Controls for more examples.

26.4. How pfodApp is optimised for short BLE style messages

Bluetooth Low Energy (BLE) or Bluetooth V4 is a completely different version of Bluetooth. BLE has been optimised for very low power consumption. pfodApp is a general purpose Android app whose screens, menus, buttons, sliders and plots are completely defined by the device you connect to.

BLE only sends 20 bytes in each message. Fortunately the pfod Specification was designed around very small messages. Almost all of pfod's commands are less than 20 bytes. The usual exception is the initial main menu message which specifies what text, menus, buttons, etc. pfodApp should display to the user, but the size of this message is completely controlled by you and you can use sub-menus to reduce the size of the main menu.

The pfod specification also has a number of features to reduce the message size. While the BLE device must respond to every command the pfodApp sends, the response can be as simple as (an empty response). If you need to update the menu the user is viewing in response to a command or due to a re-request, you need only send back the changes in the existing menu, rather than resending the entire menu. These features keep the almost all messages to less than 20 bytes. pfodApp caches menus across re-connections so that the whole menu only needs to be sent once. Thereafter short menu updates can be sent.

26.5. Step 2: Creating the Custom Android Menus and Generating the Code

Before looking at each of these BLE modules, pfodDesignerV3 will first be used to create a custom menu to turn a Led on and off and plot the voltage read at A0. pfodDesignerV3 can then generate code tailored to the particular hardware you select. You can skip over this step and come back to it later if you like. The section on each module, below, includes the completed code sketch for this example menu generated

for that module and also includes sketches that do not need pfodApp

The free pfodDesignerV3 is used to create the menu and show you an accurate preview of how the menu will look on your mobile. The pfodDesignerV3 allows you to create menus and sub-menus with buttons and sliders optionally connected to I/O pins and generate the sketch code for you (see the pfodDesigner example tutorials) but the pfodDesignerV3 does not cover all the features pfodApp supports. See the pfodSpecification.pdf for a complete list including data logging and plotting, multi- and single- selections screens, sliders, text input, etc.

Create the Custom menu to turn the Arduino LED on and off and Plot A0 Start a new menu and select as a target Bluetooth Low Energy (BLE) and then select NANO 33 BLE (and Sense). pfodDesignerV3.0.3770+ has support for NANO 33 boards.

Then follow the tutorial Design a Custom menu to turn the Arduino Led on and off for step by step instructions for creating a LED on/off menu using pfodDesignerV3.

If you don't like the colours of font sizes or the text, you can easily edit them in pfodDesignerV3 to whatever you want and see a WYSIWYG (What You See Is What You Get) display of the designed menu.

Now we will add a Chart button to display the A0 reading. The steps to does this in pfodDesignerV3 are shown in Adding a Chart and Logging Data The AtoD range is 0 to 1023 for 0 to 3.3V

Generating the code from pfodDesignerV3 gives the this sketch [Nano33BLE_Led_A0.ino](#)

26.6. BLE Mobile App “Nordic Semiconductors nRF Connect”

- On your mobile, install the App “Nordic Semiconductors nRF Connect”. There are Android and IOS versions.
- In the Arduino IDE open Serial Monitor by clicking on the magnifying glass icon on the top right.

The Nano will start sending BLE advertising packets and wait for a Central (Mobile Phone) to connect.

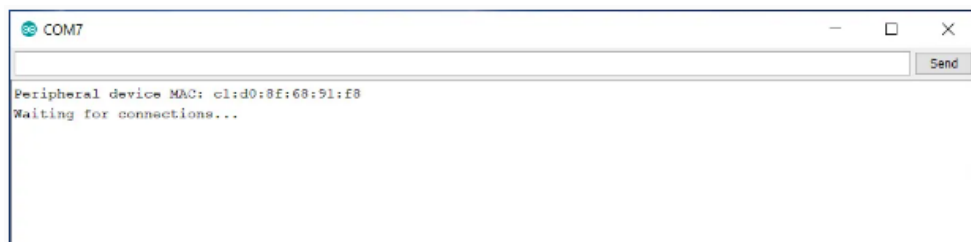


Figure 26.3.: App nRF is searching the Arduino Nano BLE Sense

In the App nRF on your mobile select **Scan** and you should see **Nano33BLE** as a device you can connect to.

Attention: BLE Peripherals can only connect to one device at a time – if the Nano33BLE is already connected it could be to another device nearby.

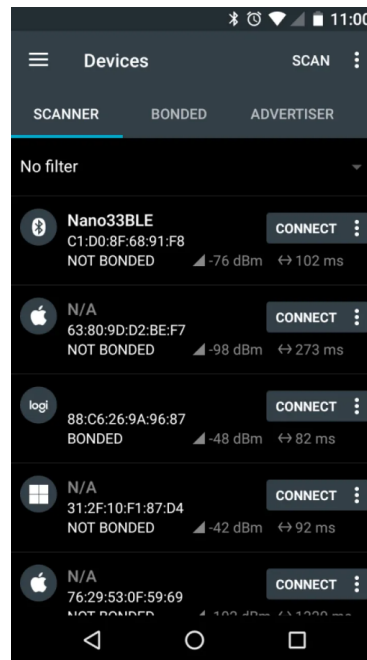


Figure 26.4.: App nRF is searching the Arduino Nano BLE Sense

Connect to the Nano and select the Unknown Service.

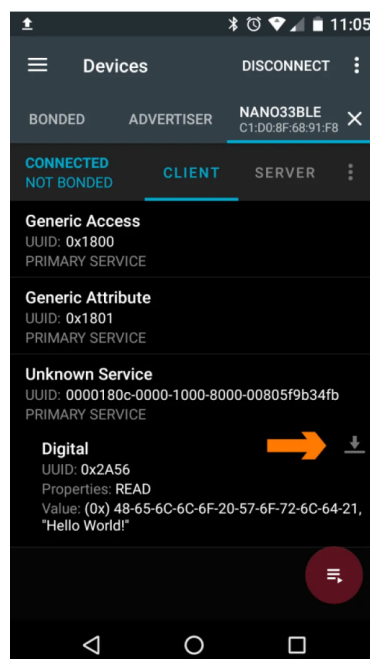


Figure 26.5.: App nRF is getting data the Arduino Nano BLE Sense

Touch the **down arrow** next to the characteristic **Digital** and it will read the message “Hello World!” being broadcast by the Nano33BLE.

26.7. Arduino BLE Example – Explained Step by Step

26.7.1. Arduino BLE Example Code Explained

In this tutorial series, I will give you a basic idea you need to know about Bluetooth Low Energy and I will show you how you can make Arduino BLE Chipset to send and receive data wirelessly from mobile phones and other Arduino boards. Let's Get Started.

Arduino Nano 33 BLE Sense, Nano with BLE connectivity focussing on IOT, which is packed with a wide variety of sensors such as 9 axis Inertial Measurement Unit, pressure, light, and even gestures sensors and a microphone.

It is powered by Nina B306 module that supports BLE as well as Bluetooth 5 connection. The inbuilt Bluetooth module consumes very low power and can be easily accessed using Arduino libraries. This makes it easier to program and enable wireless connectivity to any of your projects in no time. You won't have to use external Bluetooth modules to add Bluetooth capability to your project. Save space and power.

26.7.2. Arduino BLE – Bluetooth Low Energy Introduction

BLE is a version of Bluetooth which is optimized for very low power consuming situations with very low data rate. We can even operate these devices using a coin cell for weeks or even months.

WS:cite Arduino have a wonderful introduction to BLE but here in this post, I will give you a brief introduction for you to get started with BLE communication.

Basically, there are two types of devices when we consider a BLE communication block.

- The Peripheral Device
- The Central Device

Peripheral Device is like a Notice board, from where we can read data from various notices or pin new notices to the board. It posts data for all devices that needs this information.

Central Devices are like people who are reading notices from the notice board. Multiple users can read and get data from the notice board at the same time. Similarly multiple central devices can read data from the peripheral device at the same time.

The information that is given by the Peripheral devices are structured as Services. And These services are further divided into characteristics. Think of Services as different notices in the notice board and services as different paragraphs in each notice board. If Accelerometer is a service, then their values X, Y and Z can be three characteristics. Now let's take a look at a simple Arduino BLE example.

26.7.3. Arduino BLE Example 1 – Battery Level Indicator

In this example, I will explain how you can read the level of a battery connected to pin A0 of an Arduino using a smartphone via BLE. This is the code here. This is pretty much the same as that of the example code for Battery Monitor with minor changes. I will explain it for you.

First you have to install the library ArduinoBLE from the library manager.

Just go to [Sketch -> Include Library -> Manage Library](#) and Search for [ArduinoBLE](#) and simply install it.

```

1000 #include <ArduinoBLE.h>
1001 BLEService batteryService("1101");
1002 BLEUnsignedCharCharacteristic batteryLevelChar("2101", BLERead |
      BLENotify);

1004 void setup() {
1005     Serial.begin(9600);
1006     while (!Serial);

1008     pinMode(LED_BUILTIN, OUTPUT);
1009     if (!BLE.begin())
1010     {
1011         Serial.println("starting BLE failed!");
1012         while (1);
1013     }

1014     BLE.setLocalName("BatteryMonitor");
1015     BLE.setAdvertisedService(batteryService);
1016     batteryService.addCharacteristic(batteryLevelChar);
1017     BLE.addService(batteryService);

1018     BLE.advertise();
1019     Serial.println("Bluetooth device active, waiting for connections...");
1020 }

1022 }

1024 void loop()
1025 {
1026     BLEDevice central = BLE.central();

1028     if (central)
1029     {
1030         Serial.print("Connected to central: ");
1031         Serial.println(central.address());
1032         digitalWrite(LED_BUILTIN, HIGH);

1034         while (central.connected()) {

1036             int battery = analogRead(A0);
1037             int batteryLevel = map(battery, 0, 1023, 0, 100);
1038             Serial.print("Battery Level % is now: ");
1039             Serial.println(batteryLevel);
1040             batteryLevelChar.writeValue(batteryLevel);
1041             delay(200);

1042         }

1044     }
1045     digitalWrite(LED_BUILTIN, LOW);
1046     Serial.print("Disconnected from central: ");
1047     Serial.println(central.address());
1048 }

```

Listing 26.1.: Arduino BLE Tutorial Battery Level Indicator Code

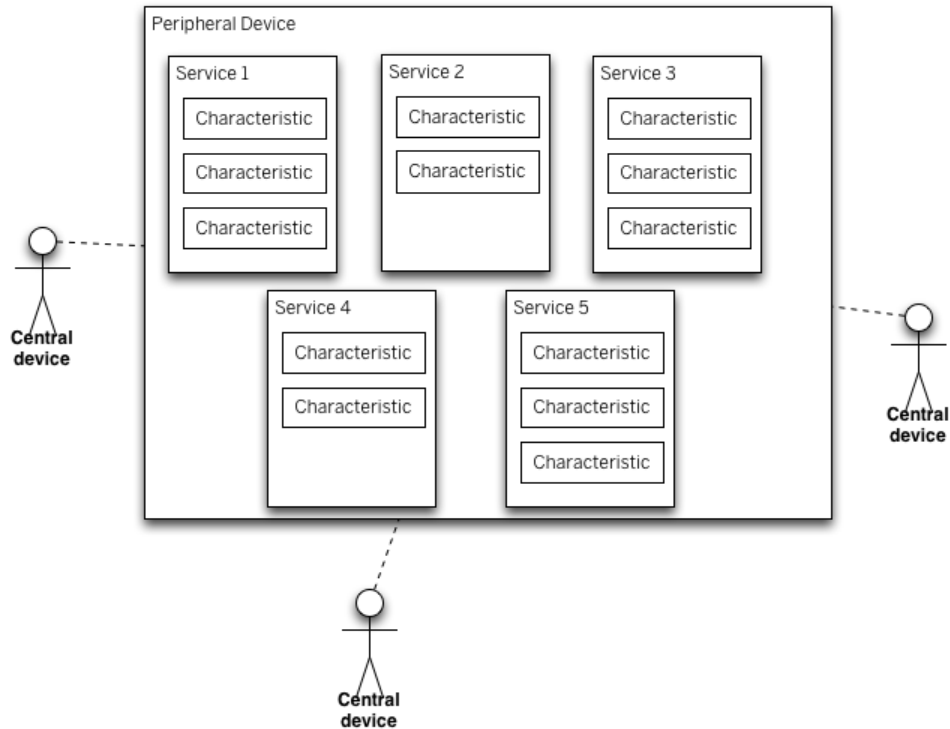


Figure 26.6.: BLE Devices – Peripheral and Central Devices

26.7.4. Arduino BLE Tutorial Battery Level Indicator Code

26.7.5. Arduino Bluetooth Battery Level Indicator Code Explained

```
#include <ArduinoBLE.h>
BLEService batteryService("1101");
BLEUnsignedCharCharacteristic batteryLevelChar("2101", BLERead | BLENotify);
```

The first line of the code is to include the file [ArduinoBLE.h](#). Then we will declare the Battery Service as well the battery level characteristics here. Here we will be giving two permissions – [BLERead](#) and [BLENotify](#).

[BLERead](#) will allow central devices (Mobile Phone) to read data from the Peripheral device (Arduino). And [BLENotify](#) allows remote clients to get notifications if this characteristic changes.

Now we will jump on to the Setup function.

```
Serial.begin(9600);
while (!Serial);
pinMode(LED_BUILTIN, OUTPUT);
if (!BLE.begin()) {
    Serial.println("starting BLE failed!");
    while (1);
}
```

Here it will initialize the Serial Communication and BLE and wait for serial monitor to open.

Set a local name for the BLE device. This name will appear in advertising packets and can be used by remote devices to identify this BLE device.

```
BLE.setLocalName("BatteryMonitor");
```



```
BLE.setAdvertisedService(batteryService);
batteryService.addCharacteristic(batteryLevelChar);
BLE.addService(batteryService);
```

Here we will add and set the value for the Service UUID and the Characteristic.

```
BLE.advertise();
Serial.println("Bluetooth device active, waiting for connections...");
```

And here, we will start advertising BLE. It will start continuously transmitting BLE advertising packets and will be visible to remote BLE central devices until it receives a new connection.

```
BLEDevice central = BLE.central();
if (central) {
    Serial.print("Connected to central: ");
    Serial.println(central.address());
    digitalWrite(LED_BUILTIN, HIGH);
}
```

And here, the loop function. Once everything is setup and have started advertising, the device will wait for any central device. Once it is connected, it will display the MAC address of the device and it will turn on the builtin LED.

```
while (central.connected()) {
    int battery = analogRead(A0);
    int batteryLevel = map(battery, 0, 1023, 0, 100);
    Serial.print("Battery Level % is now: ");
    Serial.println(batteryLevel);
    batteryLevelChar.writeValue(batteryLevel);
    delay(200);
}
```

Now, it will start to read analog voltage from A0, which will be a value in between 0 and 1023 and will map it within the 0 to 100 range. It will print out the battery level in the serial monitor and the value will be written for the batteryLevelChar characteristics and waits for 200 ms. After that the whole loop will be executed again as long as the central device is connected to this peripheral device.

```
digitalWrite(LED_BUILTIN, LOW);
Serial.print("Disconnected from central: ");
Serial.println(central.address());
```

Once it is disconnected, a message will be shown on the central device and LE

26.7.6. Installing the App for Android

In your Android smartphone, install the app “nRF Connect”. Open it and start the scanner. You will see the device “Battery Monitor” in the device list. Now tap on connect and a new tab will be opened.

Go to that and you will see the services and characteristics of the device. Tap on Battery service and you will see the battery levels being read from the Arduino.

26.7.7. Example 2 – Arduino BLE Accelerometer Tutorial

I showed you a very easy example to show you how you can send simple data via Bluetooth. If you are new to this, try this example first. It will help to give you a better understanding of the BLE.

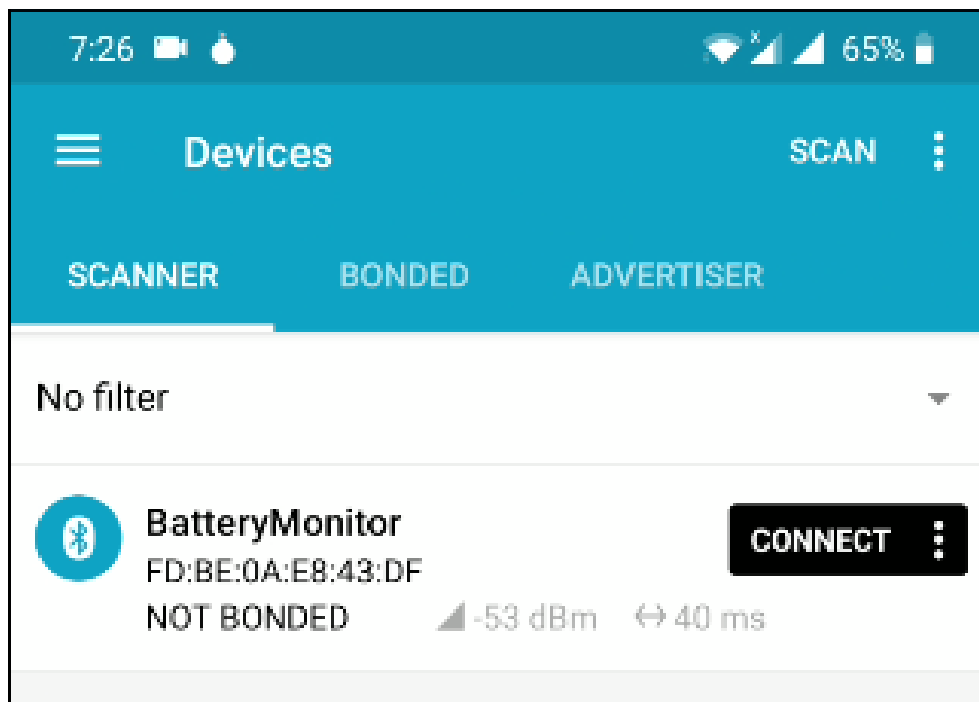


Figure 26.7.: Battery app “nRF Connect”

Step 1 – Installing Libraries

For this example, we will need two libraries

- [ArduinoBLE](#) – To Send Data via Bluetooth
- [LSM9DS1](#) – To Read data from inbuilt Accelerometer

Both these libraries are available in library manager. Simply search for that using the name and click on install.

Step 2 – Test Accelerometer Code (Optional)

Now we will try running the accelerometer code to make sure that these data are being read properly. For that, use the below code.

```
#include <Arduino_LSM9DS1.h>
```

```
void setup() {
  Serial.begin(9600);
  while (!Serial);
  Serial.println("Started");

  if (!IMU.begin()) {
    Serial.println("Failed to initialize IMU!");
    while (1);
  }

  Serial.print("Accelerometer sample rate = ");
  Serial.print(IMU.accelerationSampleRate());
  Serial.println(" Hz");
}
```

```

    Serial.println();
    Serial.println(" Acceleration in G's");
    Serial.println("XtYtZ");
}

void loop() {
    float x, y, z;

    if (IMU.accelerationAvailable()) {
        IMU.readAcceleration(x, y, z);

        Serial.print(x);
        Serial.print('t');
        Serial.print(y);
        Serial.print('t');
        Serial.println(z);
    }
}

```

Once uploaded, start the serial monitor. You will see the data being populated. Try tilting the board in all direction and you will see the value changes accordingly.

Step 3 – Upload the Code

Now its time to upload the complete code. Copy the code below and paste it in the IDE.

```

#include <ArduinoBLE.h>
#include <Arduino_LSM9DS1.h>

int accelX=1;
int accelY=1;
float x, y, z;

BLEService customService("1101");
BLEUnsignedIntCharacteristic customXChar("2101", BLERead | BLENotify);
BLEUnsignedIntCharacteristic customYChar("2102", BLERead | BLENotify);

void setup() {
    IMU.begin();
    Serial.begin(9600);
    while (!Serial);

    pinMode(LED_BUILTIN, OUTPUT);

    if (!BLE.begin()) {
        Serial.println("BLE failed to Initiate");
        delay(500);
        while (1);
    }

    BLE.setLocalName("Arduino Accelerometer");
    BLE.setAdvertisedService(customService);
    customService.addCharacteristic(customXChar);
    customService.addCharacteristic(customYChar);
    BLE.addService(customService);
}

```

```

    customXChar.writeValue(accelX);
    customYChar.writeValue(accelY);

    BLE.advertise();

    Serial.println("Bluetooth device is now active , waiting for connections ...");
}

void loop() {

    BLEDevice central = BLE.central();
    if (central) {
        Serial.print("Connected to central: ");
        Serial.println(central.address());
        digitalWrite(LED_BUILTIN, HIGH);
        while (central.connected()) {
            delay(200);
            read_Accel();

            customXChar.writeValue(accelX);
            customYChar.writeValue(accelY);

            Serial.print("At Main Function");
            Serial.println("");
            Serial.print(accelX);
            Serial.print(" ");
            Serial.println(accelY);
            Serial.println("");
            Serial.println("");
        }
        digitalWrite(LED_BUILTIN, LOW);
        Serial.print("Disconnected from central: ");
        Serial.println(central.address());
    }

    void read_Accel() {

        if (IMU.accelerationAvailable()) {
            IMU.readAcceleration(x, y, z);
            accelX = (1+x)*100;
            accelY = (1+y)*100;

        }
    }
}

```

Select the right port and board. Click on upload.

Step 4 – Testing Arduino BLE Accelerometer

In your Android smartphone, install the app “nRF Connect”. Open it and start the scanner. You will see the device “Arduino Accelerometer” in the device list. Now tap on connect and a new tab will be opened.

Go to that and you will see the services and characteristics of the device.

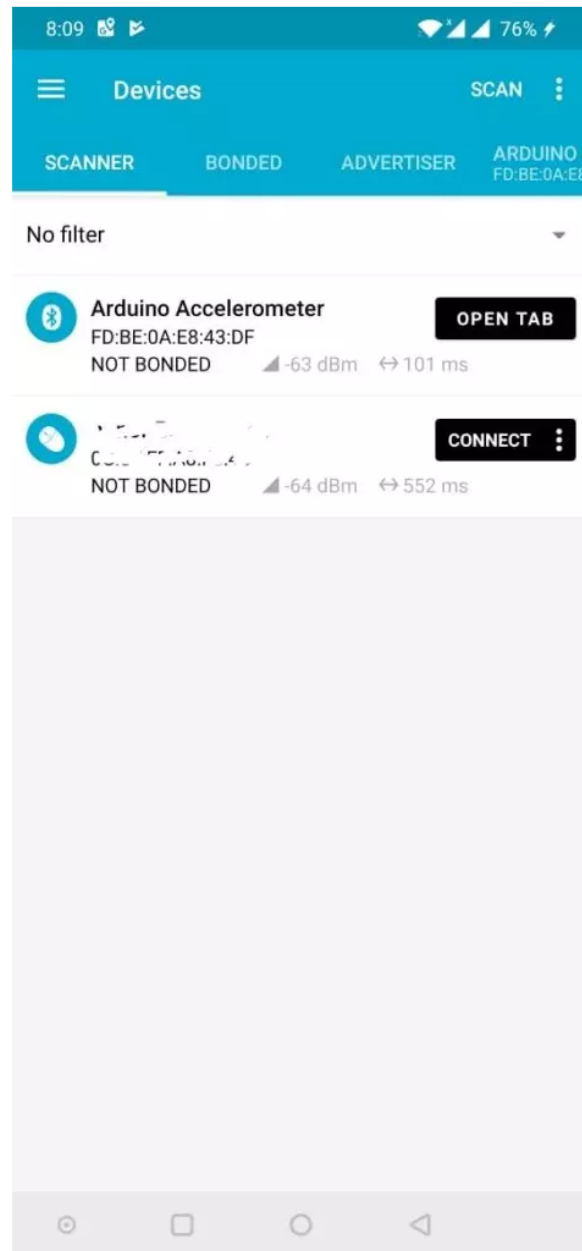


Figure 26.8.: nRF Connect Screen

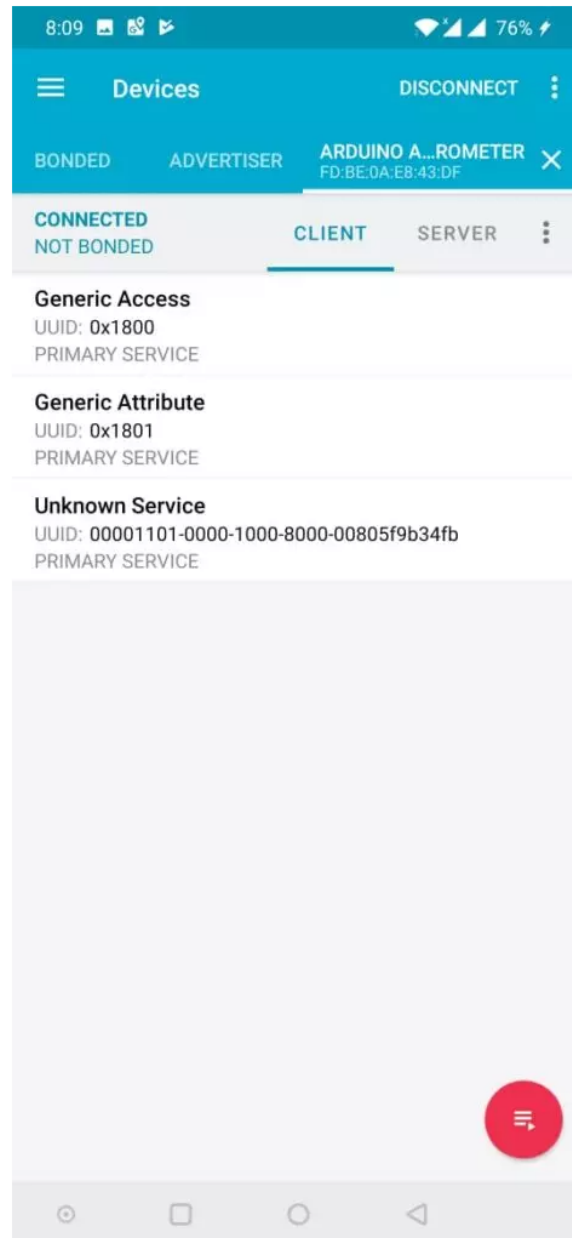


Figure 26.9.: Accelerometer Values Read from Arduino Using BLE

Tap on Unknown Service and you will see the accelerometer values being read from the Arduino.

In the next post, I will show you how you can send inbuilt sensor values such as accelerometer, gyroscope, color sensor and gesture sensor from the Arduino to your phone as well as another Arduino via BLE.

26.8. Arduino Library **BLEAK**

Low Energy (BLE) protocol. BLE allows you to exchange data between devices wirelessly and efficiently. You can use the bleak library in Python to interact with [BLE devices](#).

To get started, you need to install the bleak library using pip:

```
pip install bleak
```

Then, you need to write a Python program that can scan for BLE devices, connect to the Arduino Nano 33 BLE 33 Sense, and read or write data to its characteristics. Characteristics are the attributes that define the data and behavior of a BLE device. [For example, the Arduino Nano 33 BLE 33 Sense has a characteristic for each color of its RGB LED.](#)

You can find an example of a Python program that can control the RGB LED of the Arduino Nano 33 BLE 33 Sense using bleak [here](#)³. You can also find the corresponding Arduino sketch that sets up the BLE service and characteristics for the Nano 33 BLE 33 Sense [here](#)⁴.

26.9. Test

26.10. Test with Bluetooth Module Connection

The same procedure we need to follow for making the successful bluetooth connection, the one we follow for on-board sensors. Below figure 26.10 shows the ArduinoBLE library in the example section of Arduino IDE.

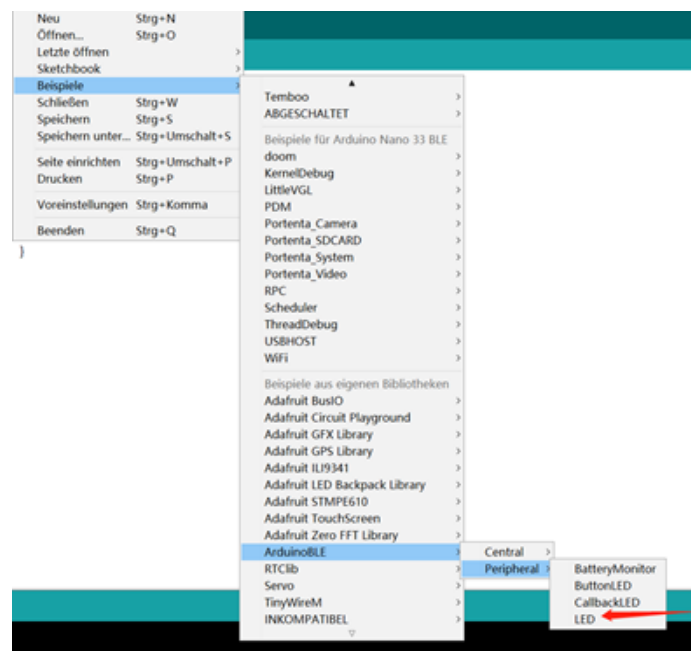
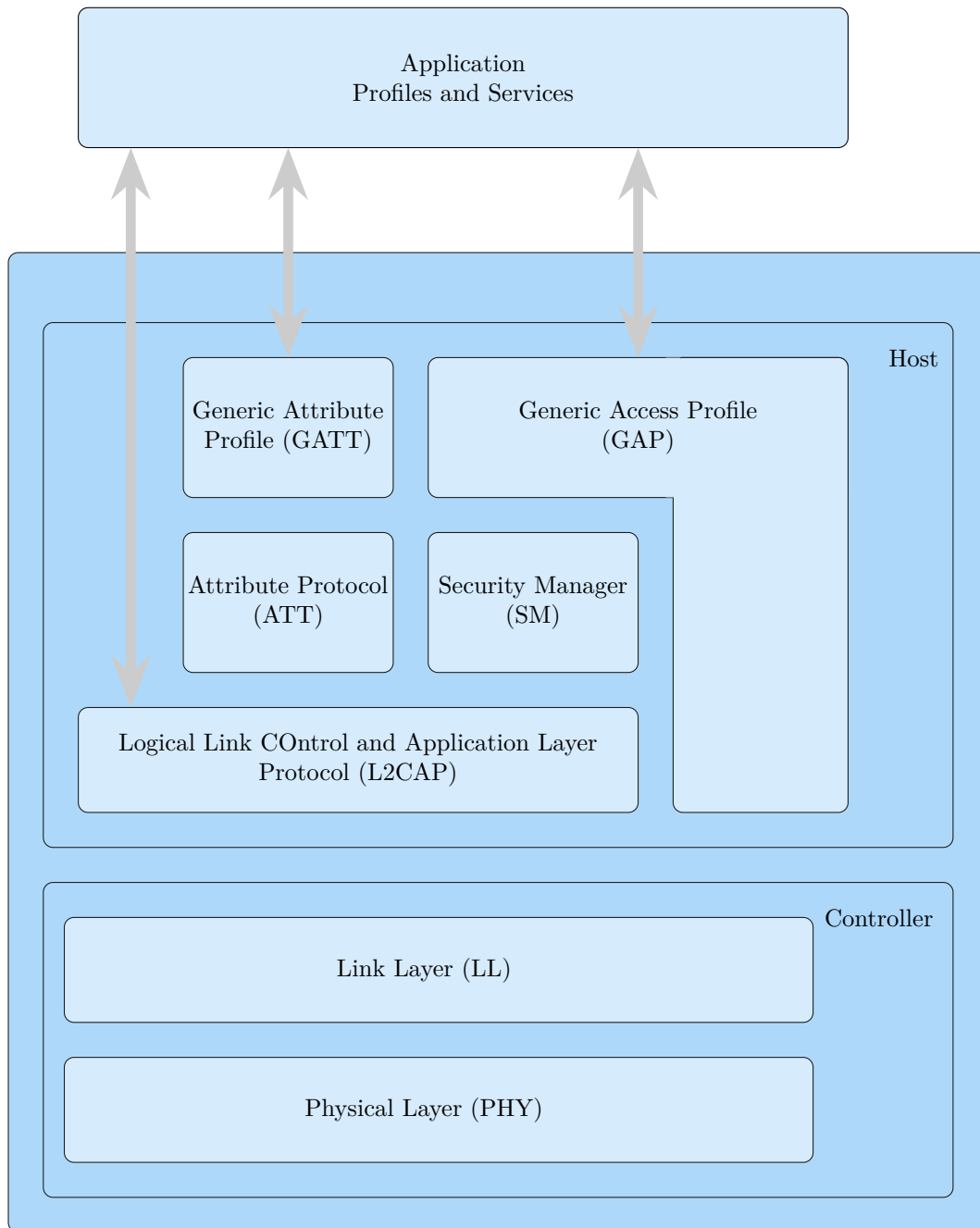


Figure 26.10.: Bluetooth Connection

Bluetooth wireless technology allows us to share the data, the voice, the music, the video, and a lot of information between paired devices, It is built into many products, from mobile phones, cars to medical devices and computers. It has lower power consumption. It is easily upgradeable. It has range better than Infrared communication. Bluetooth is used for voice and data transfer, we can communicate by receiving and sending the data with other bluetooth connected devices. It is also possible to output a value on the phone side or also on the laptop by making the proper pairing.

26.11. BLEStack



26.12. Control Arduino Nano BLE with Bluetooth & Python

<https://www.hackster.io/sridhar-rajagopal/control-arduino-nano-ble-with-bluetooth>
Demonstrated with the on-board RGB LED.

26.13. Peripheral Device aka Arduino Nano

The Generic Attribute Profile (or GATT) defines how Bluetooth LE devices can transfer data back and forth, using the Attribute Protocol (or ATT) which organizes

the data hierarchically into Service and Characteristics.

For our purpose, we define a top level service, as well as 3 characteristics, corresponding to each of the 3 colors of our RGB LED:

```
BLEService nanoService("13012F00-F8C3-4F4A-A8F4-15CD926DA146");
BLEByteCharacteristic redLEDCharacteristic("13012F01-F8C3-4F4A-A8F4-15CD926DA146");
BLEByteCharacteristic greenLEDCharacteristic("13012F02-F8C3-4F4A-A8F4-15CD926DA146");
BLEByteCharacteristic blueLEDCharacteristic("13012F03-F8C3-4F4A-A8F4-15CD926DA146");
```

The UUIDs used above have been randomly generated. They must match between the peripheral device (the Nano) and the central device (the Python program).

In our setup function, apart from setting up the pinMode for the RGB LEDs, we also setup our BLE service. Here, we set the local name for our BLE peripheral - central devices will see this when trying to connect to advertising services.

We add the service and characteristics we defined above, set their initial values, and start advertising. Here, at this point, central devices that are listening will see our advertising packets and know that we are ready to accept connections.

```
// set advertised local name and service UUID:
BLE.setLocalName("Arduino_Nano_33_BLE_Sense");
BLE.setAdvertisedService(nanoService);
// add the characteristic to the service
nanoService.addCharacteristic(redLEDCharacteristic);
nanoService.addCharacteristic(greenLEDCharacteristic);
nanoService.addCharacteristic(blueLEDCharacteristic);
// add service
BLE.addService(nanoService);
// set the initial value for the characteristic:
redLEDCharacteristic.writeValue(0);
greenLEDCharacteristic.writeValue(0);
blueLEDCharacteristic.writeValue(0);
// start advertising
BLE.advertise();
```

In our main loop(), we wait for a central device to connect. We then wait to see if any of the characteristics are written to, and the value of that characteristic. In our example, a non-zero value written to the redLEDCharacteristic, for example, will cause us to turn on the RED LED. When we encounter a zero value, we turn off the LED.

```
void loop() {
    BLEDevice central = BLE.central();
    if (central) {
        ...
        while (central.connected()) {
            if (redLEDCharacteristic.written()) {
                if (redLEDCharacteristic.value()) {
// any non-zero value
                    Serial.println("RED_LED_on");
                    digitalWrite(REDA_PIN, LOW); // LOW will turn on LED
                } else { // a zero value
                    Serial.println(F("RED_LED_off"));
                    digitalWrite(REDA_PIN, HIGH); // HIGH will turn off LED
                }
            }
            .... do similar stuff for GREEN and BLUE
        }
    }
}
```

```

        // when the central disconnects, print it out:
        Serial.print(F("Disconnected from central: "));
        Serial.println(central.address());
    }
}

```

Here we have to note that the RGB LEDs on the Arduino Nano 33 BLE Sense follow a reverse logic, so setting the pin LOW turns ON the LED and setting it HIGH turns it OFF.

The onboard RGB LED also does not support PWM, so the states of the LEDs can either be ON or OFF, giving us 7 different colors:

R	G	B	
OFF	OFF	OFF	— OFF
ON	OFF	OFF	— RED
OFF	ON	OFF	— GREEN
OFF	OFF	ON	— BLUE
ON	ON	OFF	— ORANGE
ON	OFF	ON	— PURPLE
OFF	ON	ON	— CYAN
ON	ON	ON	— WHITE

The Central device can also read the characteristic's value. This is automatically handled by the BLE library and we don't need to do anything more. We initialize the value appropriately on startup, and maintain the value, so it can be read anytime.

Let's dive into the code - Central Device aka Python On the Python side, we utilize the Python bleak library for BLE support - of course, the machine we are running it on needs to have Bluetooth support as well!

We also use the asyncio asynchronous framework for our program. Our main loop here is the run() function. It goes into discover mode and checks the local names of the Bluetooth devices that are advertising, and connects to the device with the local name of "Arduino Nano 33 BLE Sense" (the local name we chose).

It then reads the RED, GREEN and BLUE Characteristics to know the initial states of the LEDs, and then goes into the setColor() loop:

```

async def run():
    global RED, GREEN, BLUE
    print('ProtoStax_Arduino_Nano_BLE_LED_Peripheral_Central_Service')
    print('Looking for Arduino_Nano_33_BLE_Sense_Peripheral_Device...')
    found = False
    devices = await discover()
    for d in devices:
        if 'Arduino_Nano_33_BLE_Sense' in d.name:
            print('Found_Arduino_Nano_33_BLE_Sense_Peripheral')
            found = True
            async with BleakClient(d.address) as client:
                print(f'Connected to {d.address}')
                val = await client.read_gatt_char(RED_LED_UUID)
                if (val == on_value):
                    print('RED_ON')
                    RED = True
                else:
                    print('RED_OFF')
                    RED = False
            ... do similar stuff for GREEN and BLUE
    while True:
        await setColor(client)

```

```
if not found:
    print('Could not find Arduino Nano 33 BLE Sense Peripheral')
```

In `setColor()`, it prompts for user input, and checks the user input to see if the string has any 'r', 'g' or 'b' values in it. It then toggles the color and writes the appropriate value to the characteristic, and waits for further user input again. This continues until the user hits Control-C to stop the program.

```
async def setColor(client):
    global RED, GREEN, BLUE
    val = input('Enter rgb to toggle red, green and blue LEDs: ')
    print(val)
    if ('r' in val):
        RED = not RED
    await client.write_gatt_char(RED_LED_UUID, getValue(RED))
    ... do similar stuff for GREEN and BLUE
```

26.13.1. Taking the Project Further

The Arduino Nano has a whole bunch of sensors onboard. You can extend the project by adding characteristics for the other sensors - for example, reading the temperature, humidity and pressure values (there is no need to write these values, so the characteristics can be defined read-only). You'll also need to modify the program to handle the read command - polling the sensor and writing the data with the current value of the sensor.

On the central device (Python Program), you can choose to store the data along with the timestamp of when it was read, to a database or comma-separated file. This way, your central device can act like a data logger, and you can even utilize the data to perform actions, or send the data to a cloud service like AWS to process and run analytics on. Having the central device on Python allows you to utilize the numerous packages and libraries and capabilities of Python.

Happy Coding! If you have any questions, feel free to ask them in the comments below! We'll also be happy to hear how you have taken the project further and what wonderful creations you have made!

27. Ultraschallsensor HC-SR04

27.1. Einleitung

Im Rahmen des Projekts „Magic Faucet Fountain“, bei dem der Eindruck eines schwebenden, magisch fließenden Wasserhahns erzeugt wird, spielt die kontinuierliche Überwachung des Wasserstandes eine zentrale Rolle. Um sicherzustellen, dass die Pumpe stets ausreichend Wasser zur Verfügung hat und der Effekt der „schwebenden Fontäne“ stabil bleibt, ist eine zuverlässige Wasserstandsmessung erforderlich. Hierfür kommt der Ultraschallsensor HC-SR04 zum Einsatz. Er misst berührungslos den Abstand zwischen Sensor und Wasseroberfläche. Aus dieser Distanz kann der aktuelle Wasserfüllstand im Behälter berechnet werden. Der Sensor arbeitet nach dem Prinzip der Laufzeitmessung von Ultraschallwellen und bietet eine einfache, kostengünstige und präzise Möglichkeit zur Pegelüberwachung. So lässt sich frühzeitig erkennen, wenn Wasser nachgefüllt werden muss oder ein kritischer Mindeststand erreicht ist.

27.2. Domänenwissen

Ultraschallsensoren zur Abstandsmessung dienen der Ermittlung von Objektabständen über die Laufzeit von Ultraschallimpulsen und die Schallgeschwindigkeit. Da die Schallgeschwindigkeit temperatur- und luftdruckabhängig ist und zusätzlich durch die Luftzusammensetzung beeinflusst wird, ist bei der Auslegung auf solche Einflussfaktoren Rücksicht zu nehmen. Bei Nichtbeachtung sind Messungenauigkeiten in der Entfernungsmessung zu erwarten. (1) Umwelteinflüsse wie Feuchte, Staub und Rauch beeinflussen die Messgenauigkeit nicht. Einige Gase und Flüssigkeiten können die Sensorfunktion jedoch beeinträchtigen oder gar untüchtig machen. Beispiele hierfür sind CO₂ und Dämpfe von Lösungsmitteln. (2) Ultraschallsensoren können nach ihrem Sensorprinzip in Tastbetrieb und Schrankenbetrieb unterschieden werden. Der Tastbetrieb basiert dabei auf der Auswertung der Laufzeitmessung zwischen dem Senden und Empfangen des Schalls. Der Schrankenbetrieb dahingegen beruht auf der Kontrolle, ob das gesendete Signal empfangen wurde. (4) Der Ultraschallsensor zur Abstandsmessung beruht auf ersterem Prinzip. Im Folgenden soll die Funktionsweise näher erläutert werden. Zunächst aktiviert die Steuerelektronik den Leistungsverstärker, welcher den Schallwandler mit einer elektrischen Leistung und einer Sinusspannung ansteuert. Dadurch arbeitet der Schallwandler als Lautsprecher und sendet einen Ultraschallimpuls, auch Burst genannt, aus. Bis der Schallwandler ausgeschwungen ist, dauert es zwei- bis dreimal so lang wie die Sendezeit. Anschließend leitet die Steuerelektronik den Empfangsbetrieb ein und der Schallwandler dient nun als Mikrofon. Sollte sich in der Schallkeule ein Objekt befinden, wird der Ultraschallimpuls zum Sender zurückreflektiert. Der Schallwandler beginnt zu schwingen und erzeugt eine Sinusschwingung, die auf einen Verstärker übertragen wird. Durch Autokorrelation wird kontrolliert, ob es sich bei dem reflektierten Schall wirklich um das Echo der ausgesandten Ultraschallwellen handelt. (1) Eine ganze Bandbreite an Objekten unabhängig von der Oberflächenbeschaffenheit, Farbe, Transparenz (2) und Form (1) kann durch den Ultraschall erfasst werden. Der Ultraschall findet seine Anwendung in sehr vielen verschiedenen Bereichen (Kap.3), näher soll auf ihre Anwendung in den Bereichen der Medizin und Industrie eingegangen werden. In der Medizin gehört der Ultraschall mit zu den wichtigsten und häufigsten genutzten Diagnoseverfahren. Er lässt sich vielfältig einsetzen. So dient es zur Erkennung und Darstellung von Organen,

gefüllten Hohlräumen, Gefäßen, Knochen, Sehnen und anderen Objekten im menschlichen Körper. Kennzeichnend für die Anwendung des Ultraschalls in der Medizin ist außerdem sein geringes Gesundheitsrisiko für Patienten. In der Industrie gibt es vielfältige Einsatzbereiche von Ultraschallsensoren. Sie können der Überwachung von Stellplätzen in der Lagerhaltung, der Füllstandbestimmung von Schüttgütern (1) oder der Detektion von Objekten und Grenzflächen dienen. Beispiel für letztere Erwähnung ist die Rückfahrhilfe bei Fahrzeugen. Eine weitere sehr wichtige Anwendung findet sich in der zerstörungsfreien Prüfung von Werkstücken und Werkstoffen wieder. Vorrangig wird dabei das Materialgefüge auf Unebenheiten und Auffälligkeiten überprüft. Solche Untersuchungen werden häufig für sicherheitsrelevante Bauteile wie Eisenbahnräder oder Druckbehälter für Atomanlagen verwendet (3).

27.3. Grundlagen zur Verwendung eines Ultraschallsensors

Um den Ultraschallsensor HC-SR04 gezielt und effizient einsetzen zu können, ist ein grundlegendes Verständnis über die Funktionsweise, technischen Eigenschaften und Anwendungsvoraussetzungen abstandsmessender Ultraschallsensoren notwendig. Die folgenden Unterkapitel bieten eine strukturierte Einführung in die physikalischen und technischen Grundlagen des Sensors. Ziel ist es, ein tiefergehendes Verständnis für den Aufbau, die Signalverarbeitung und die praktische Handhabung des HC-SR04 zu vermitteln.

27.3.1. Grundlagen Schall

Als Schall bezeichnet man die Ausbreitung von lokalen Druckschwankungen in einem elastischen Medium. Die Schallwelle breitet sich dabei longitudinal aus, die Auslenkung der Moleküle erfolgt also in Ausbreitungsrichtung [TM24]. Die Einteilung der Schallbereiche erfolgt auf Grundlage der Frequenz:

- *Infraschall*: bis 16 Hz,
- *Hörbarer Schall*: von 16 Hz bis 20 kHz,
- *Ultraschall*: von 20 kHz bis 1 GHz,
- *Hyperschall*: über 1 GHz [HS23].

27.3.2. Schallgeschwindigkeit und Wellenlänge

Die Geschwindigkeit, mit der sich Schall in Luft ausbreitet, ist von der Lufttemperatur T und der Luftfeuchtigkeit φ abhängig. Da die Luftfeuchtigkeit einen erheblich geringeren Einfluss auf den Wert hat

$$\Delta v_{Schall,\varphi} \approx 1,2 \frac{\text{m}}{\text{s}}; 0 < \varphi < 1, T_C = 20^\circ\text{C}, \quad (27.1)$$

[HS23] lässt sich die Schallgeschwindigkeit mit ausreichender Genauigkeit mit der Formel 27.2

$$v_{Schall} = \sqrt{\kappa R T} \quad (27.2)$$

und der Annahme $\varphi = 0$ berechnen [TM24]. Dabei ist $\kappa = 1,4$ der Adiabatenexponent von Luft, $R = 287 \frac{\text{J}}{\text{kg K}}$ die spezifische Gaskonstante von Luft und T die absolute Temperatur in Kelvin [TM24; Wil22]. Für eine angenommene Temperatur von $T_C = 20^\circ\text{C}$ ergibt sich damit eine Geschwindigkeit von $v_{Schall} = 343,2 \frac{\text{m}}{\text{s}}$. In Abb. 27.1

ist die Schallgeschwindigkeit in Luft im Temperaturbereich von $T_C = -10^\circ\text{C}$ bis 60°C dargestellt.

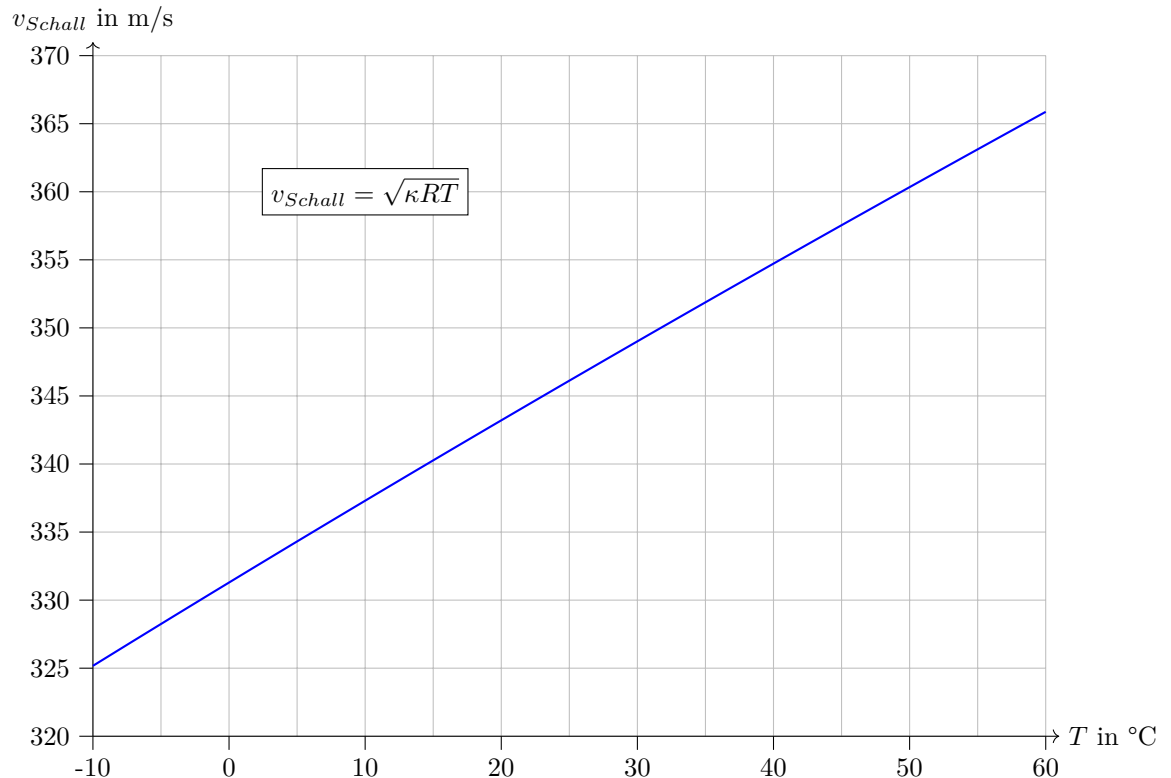


Figure 27.1.: Ausbreitungsgeschwindigkeit von Schall in Luft

Über den Formelzusammenhang 27.3

$$v_{Schall} = \lambda f \quad (27.3)$$

lässt sich bei gegebener Frequenz f und mit der berechneten Schallgeschwindigkeit v_{Schall} die Wellenlänge des Schalls berechnen [Wil22]. Diese beträgt bei einer angenommenen Frequenz von $f = 40\text{kHz}$ und der in Abschnitt 27.3.2 berechneten Schallgeschwindigkeit $\lambda = 8,58 \cdot 10^{-3} \text{ m}$.

27.3.3. Absorption und Reflexion

Absorption beschreibt die Abnahme der Intensität I einer Schallwelle, während sie Weg durch das sie leitenden Medium zurücklegt. Sie ist vom Quadrat der Frequenz f des Schalls und den Absorptionseigenschaften a des Ausbreitungsmediums abhängig und lässt sich über die Formel 27.4

$$I(x) = I_0 \times \exp(-\alpha x); \alpha = a f^2 \quad (27.4)$$

berechnen [HS23; HMS21]. In Abbildung 27.2 ist die Abhängigkeit des Verhältnisses der Intensität I zu I_0 von der Frequenz dargestellt. Die Wegstrecke von einem Meter und die Umgebungsbedingungen sind entsprechend konstant.

Für eine Frequenz von $f = 40 \text{ kHz}$ ergibt sich ein Verhältnis von 0,95.

Trifft die Schallwelle auf die Grenzschicht zweier Medien mit verschiedenen Schallkennimpedanzen Z_n , wird sie zum Teil reflektiert und zum anderen Teil transmittiert. Der Reflexionsgrad ist bei großen Differenzen zwischen den Impedanzen

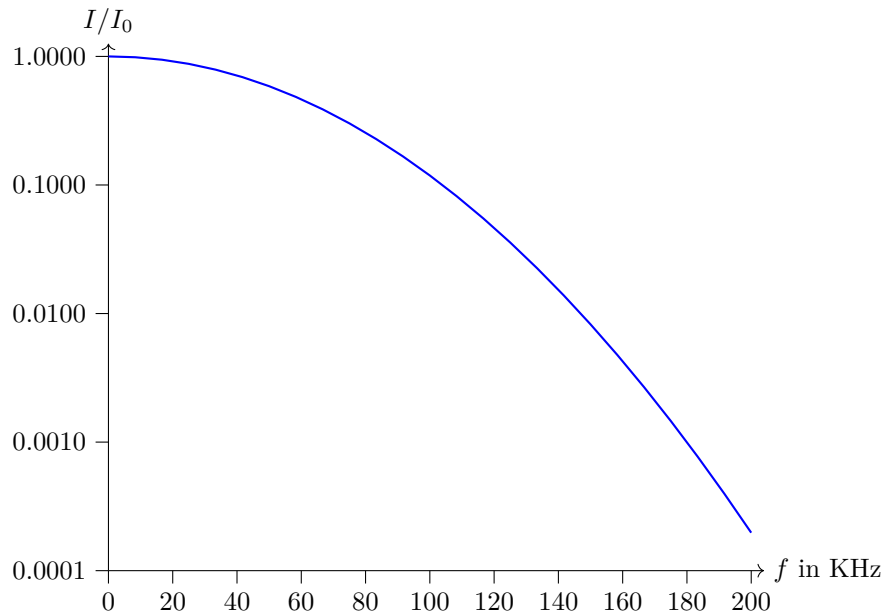


Figure 27.2.: Abnahme der Schallintensität in Abhängigkeit der Frequenz bei einem Meter, nach [HS23]

($Z_1 \ll Z_2$ oder $Z_1 \gg Z_2$) der Medien besonders hoch. Sind die Impedanzen hingegen ähnlich ($Z_1 \approx Z_2$) ist der Absorptionsgrad sehr hoch [HMS21]. Da viele Flüssigkeiten und feste Stoffe eine um den Faktor 10^4 bis 10^5 höhere Impedanz im Vergleich zu Luft vorweisen, ist der Reflexionsgrad in diesen Fällen $\gg 99\%$ [HMS21; HS23].

27.3.4. Weg- und Abstandsmessung mit Ultraschall

Um eine Entfernung oder einen Abstand eines Objekts zu dem Sensor zu ermitteln, wird der Sensor im sog. *Tastbetrieb mit Echo-Laufzeit-Messung*, wie in Abb. 27.3 zu sehen, verwendet. Dabei sendet der Sensor zunächst ein Schallpuls in Richtung des zu messenden Objekts aus (Lautsprecher), welche zum Teil von diesem reflektiert wird und somit zurück zum Sensor gelangt (Echo). Dieses Echo kann der Sensor detektieren (Mikrofon) und über die gemessene Laufzeit des Echos sowie die Schallgeschwindigkeit im dem den Sensor umgebenden Medium die Distanz berechnen [HS23].

In der Anwendung einköpfiger Sensoren ist für die Messung sehr kleiner Distanzen auf die Ausschwingzeit des Wandlers sowie die Umschaltzeit vom Lautsprecher- in den Mikrofonbetrieb zu achten, da in dieser Zeit kein Echosignal aufgenommen werden kann. Bei zweiköpfigen Systemen, also je einem dedizierten Lautsprecher und Mikrofon, sind die Öffnungswinkel der Schallkeulen in Verbindung mit dem radialen Abstand maßgeblich für die minimale Distanz.

Bei großen Distanzen sind sowohl die Genauigkeit der Ausrichtung des Sensors (bei kleinen Öffnungswinkeln der Schallkeulen) als auch die Absorption der Schallintensität in dem Ausbreitungsmedium relevant [HS23].

Ebenfalls störend auswirken können sich die Geometrie des zu messenden Objektes (z. B. bei stark konkaven oder konvexen Oberflächen), absorbierendes/diffus reflektierendes Material (Filz, Watte, Schaumstoff) oder Umwelteinflüsse orthogonal zur Messrichtung wirkender Wind. Verunreinigungen wie Staub und Rauch oder leichter Niederschlag in Form von Regen und Schnee beeinträchtigen die Messung hingegen nicht [HS23].

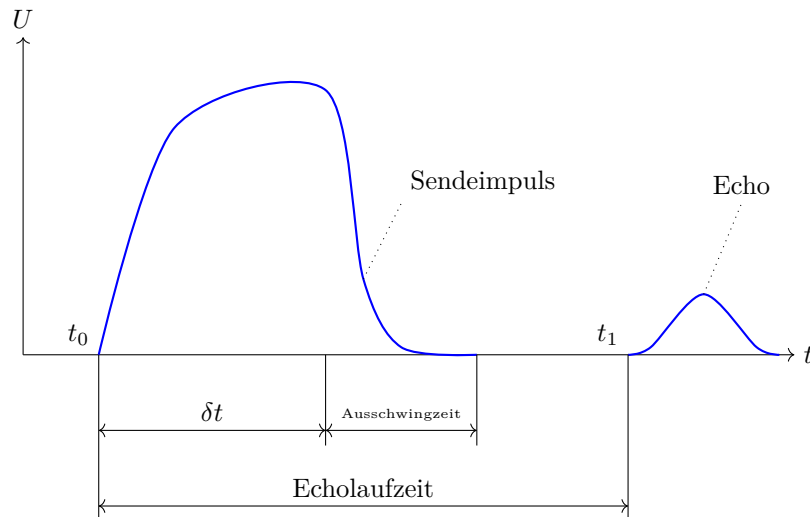


Figure 27.3.: Spannungsverlauf Schallwandler bei einer Echo-Laufzeit-Messung, nach [HS23]

27.3.5. Anwendungen

Ultraschallsensoren haben in der Praxis ein breites Einsatzgebiet [HS23]. Basierend auf dem Ausbreitungsmedium lassen sie sich in drei Kategorien einteilen.

Die erste Kategorie wird dem Luftschall zugeschrieben. Typische Anwendungsbeispiele sind Füllstandsensoren, Durchmessererfassung für Auf- und Abwickelsteuerung oder Kollisionsschutz/Objekterkennung [HS23].

Bei Sensoren der zweiten Kategorie breitet sich der Schall über Flüssigkeiten aus, wodurch diese vor allem im maritimen Bereich anzutreffen sind. Als Beispiele sind Sonarsysteme zum Orten von Unterseebooten oder Fischschwärmen und das Echolot zur Kartografie des Meeresbodens zu nennen [HMS21].

Die dritte Kategorie umfasst Körperschallsensoren. Viel verwendetes Beispiel in der Industrie sind Sensoren zur zerstörungsfreien Materialprüfung. In der Medizin ist die bildgebende Ultraschall Diagnostik ein wichtiges Werkzeug zur Überprüfung von Organen oder Föten [HMS21].

27.3.6. Herausforderungen

Dadurch, dass die Schallgeschwindigkeit von der Umgebungstemperatur abhängig ist, muss diese für die Berechnung der Distanz mit einbezogen werden, wenn der Sensor in einem größeren Temperaturbereich eingesetzt werden soll. In Abbildung 27.1 ist zu erkennen, dass die Differenz der Geschwindigkeit in dem Temperaturbereich, in dem alle Komponenten des Sensors betrieben werden könnten ($-10\text{ °C} \leq T_C \leq 55\text{ °C}$), etwa $38 \frac{\text{m}}{\text{s}}$ beträgt. Dies entspricht, ausgehend von dem Standardwert für die Schallgeschwindigkeit $v_{\text{Schall},20} = 343,2 \frac{\text{m}}{\text{s}}$, einer Streuung von $-5,2\%$ und $+5,8\%$. Hinzu kommt die Unsicherheit aufgrund der Luftfeuchtigkeit. Hier liegt die Streuung, bezogen auf die Schallgeschwindigkeiten bei $\varphi = 0$, in einem Bereich zwischen $+0,37\%$ für $T_C = -10\text{ °C}$ und $+0,33\%$ für $T_C = 55\text{ °C}$.

Physikalische Effekte wie die Absorption, welche unter Laborbedingungen maßgeblich die Reichweite des Sensors einschränkt, oder einen schlechten Reflexionsgrad bestimmter Materialien, schränken das System in seiner Vielseitigkeit ein. Selbiges gilt für Attribute des Sensors wie die Ausschwingzeit des Schallwandlers oder die

geometrische Anordnung der Schallkeulen bei zweiköpfigen Systemen. Auch die genannten Umwelteinflüsse können dafür sorgen, dass eine Messung fehlschlägt oder ein falsche Ergebnis liefert.

27.3.7. Lösungsansätze

Um große mögliche Messfehler aufgrund einer fest hinterlegten Schallgeschwindigkeit zu vermeiden, soll die Temperatur von dem internen Temperatursensor des Arduino Nano 33 BLE Sense Lite bei einer Abstandsmessung erfasst und auf dessen Grundlage die Schallgeschwindigkeit berechnet werden. Da der Sensor jedoch auf der Platine ist, diese sich bei Benutzung selbst erwärmt und zusätzlich in einem Gehäuse verbaut ist, dessen Material gute wärmeisolierende Eigenschaften hat, ist die von ihm gemessene Temperatur zeitabhängig. Daraus folgt, dass seine Messwerte nicht für die Berechnung der Schallgeschwindigkeit verwendet werden können. Die Verwendung eines zusätzlichen Temperatursensors wäre eine mögliche Lösung. Daher wird, um eine zu große Abweichung des angezeigten Messwertes zu verhindern, der für den Sensor zugelassene Temperaturbereich eingeschränkt. Ein möglicher Temperaturbereich ist zwischen 17 °C und 23 °C. In diesem Bereich liegt die Abweichung der Schallgeschwindigkeit bei unter 0,5%. Ob diese Genauigkeit ausreichend ist, ist von der Anwendung abhängig.

27.4. Ultraschall Abstandsensor

Der Ultraschallabstandssensor in Abbildung 27.4 vom Hersteller JOY-it enthält den Sensor HC-SR04. Er ist als zweiköpfiger Sensor ausgeführt, wodurch er über je einen Lautsprecher und ein Mikrofon verfügt.

und ist zur Erfassung von Abständen in einem Bereich von 3 bis 400 cm spezifiziert. Dafür nutzt er eine Schallfrequenz von 40 kHz. Er ist als zweiköpfiger Sensor ausgeführt, wodurch er über je einen Lautsprecher und ein Mikrofon verfügt. Die Abmessungen des Sensors betragen in der Breite 43 mm, in der Höhe 15 mm und in der Tiefe 20 mm. Die Auflösung des Sensors beträgt ± 1 mm. Angeschlossen wird der Abstandssensor über den VCC-Pin, Trig/Serail Clock (SCL)-Pin, Echo/Serail Data (SDA)-Pin und dem GND-Pin. Der Sensor kann über die drei Schnittstellen General Purpose Input Output (GPIO), Universal Asynchronous Receiver Transmitter (UART) und Inter-Integrated Circuit (I²C) verbunden werden [Simb].



Figure 27.4.: Ultraschallabstandssensor der Marke JOY-IT[Simb]

27.5. Specification

Der Sensor HC-SR04 ist zur Erfassung von Abständen in einem Bereich von 3 bis 400 cm mit einer Auflösung von ± 1 mm spezifiziert. Dafür nutzt er eine Schallfrequenz

von 40 kHz. Die Abmessungen des Sensors betragen in der Breite 43 mm, in der Höhe 15 mm und in der Tiefe 20 mm.

Angeschlossen wird der Abstandssensor über den SCL-Pin für das Triggersignal, den SDA-Pin fürs Echo, den VCC-Pin zur Stromversorgung und dem GND-Pin.

Der Sensor kann über die drei Schnittstellen GPIO, UART und I²C verbunden werden [Simb].

Bei den folgenden Beispielprogrammen wird der Arduino Nano 33 BLE Sense verwendet. In den Beispielen werden die Pins D6 und D9 für die Datenübertragung verwendet. In der Tabelle 27.1 ist dies für die Grove-Schnittstelle festgehalten.

Table 27.1.: Pinbelegung für den Ultraschallabstandssensor [Simb]

Arduino Pin	Ultraschall Abstandssensor
D9	Trig
D6	Echo
3V3	VCC
GND	GND

27.6. Bibliothek

27.6.1. Description

27.6.2. Installation

27.6.3. Functions

27.6.4. Example - Manual

27.6.5. Example

27.6.6. Example - Code

27.6.7. Example - Files

27.7. Calibration

27.8. Kalibrierung

Der Ultraschallabstandssensor wird für die Verwendung bei der Temperatur von 20 °C ausgelegt. Die berechnete Schallgeschwindigkeit $v_{Schall} = 343,2 \frac{\text{m}}{\text{s}}$ bei 20 °C wird für die Umrechnung der Schalldauer in die Distanz zur Reflexionsfläche verwendet.

WS:cite method

27.9. Simple Code

27.10. Einfacher Beispiel zur Verwendung des Ultraschallabstandssensors

Mit dem Sketch `TestHCSR04 UltraschallsensorTest.ino` wird die Funktionalität des Ultraschallsensors getestet.

27.10.1. Durchführung

Für den Test werden die folgenden Hardware-Komponenten benötigt:

```

1000 /**
1001  * @file UltraschallsensorTest.ino
1002  * @author Wings
1003  * @date 20.05.2024
1004  * @details The sketch is used to test the functionality of the
1005           ultrasonic sensor. The transit time of the ultrasonic signal is
1006           measured
1007           * The distance is then calculated from the transit time and the speed of
1008           sound and displayed on the serial monitor.
1009  * Projekt: Abstandssensor
1010  */
1011
1012 /** Pin D6 is referenced as an echo pin for the ultrasonic sensor. */
1013 #define Echopin D6
1014 /** Pin D9 is referenced as the trigger pin for the ultrasonic sensor.
1015     */
1016 #define Triggerpin D9
1017
1018 /** Der Parameter MaxReichweite definiert die obere Grenze des
1019     Messbereichs von dem Ultraschallsensor und ist hier in cm angegeben.
1020     */
1021 int MaxReichweite = 215;
1022 /** Der Parameter MinReichweite definiert die untere Grenze des
1023     Messbereichs von dem Ultraschallsensor und ist hier in cm angegeben.
1024     */
1025 int MinReichweite = 3;
1026 /** The parameter Anstand stands for the distance between a wall and
1027     the ultrasonic sensor and is shown on the display in cm. */
1028 double Abstand;
1029 /** The duration parameter stands for the runtime of the ultrasonic
1030     signal and is used here with the unit microseconds. */
1031 double Dauer;
1032 /** The parameter Schallgeschwindigkeit stands for the speed of sound
1033     in the medium air at a temperature of 20C and is specified in m/s.
1034     */
1035 double Schallgeschwindigkeit = 343.2;
1036
1037 /**
1038  * @brief Starten der seriellen Kommunikation und festlegen der Pinmodi
1039  * @details The function is executed when the programme is started. The
1040           serial interface to the computer is initialised at 9600 baud, the
1041           pin mode for the trigger pin is defined as OUTPUT
1042  * and the pin mode for the echo pin is defined as INPUT
1043  * There is no return value.
1044  */
1045 void setup() {
1046     Serial.begin(9600);
1047     pinMode(Triggerpin, OUTPUT);
1048     pinMode(Echopin, INPUT);
1049 }
1050
1051 /**
1052  * @brief Abstand messen und anzeigen
1053  * @details The function is performed continuously. An ultrasonic signal
1054           is sent out and the runtime is measured until the signal arrives
1055           back at the ultrasonic sensor.
1056  * The distance in cm is calculated using the duration of the runtime and
1057           the speed of sound and then displayed on the serial monitor.
1058  * There is no return value.
1059  */
1060 void loop() {
1061     digitalWrite(Triggerpin, LOW); // Der Triggerpin wird auf Low gestellt
1062     delayMicroseconds(2); // Es wird 2 Millisekunden gewartet
1063     digitalWrite(Triggerpin, HIGH); // Das Senden eines Ultraschallsignals
1064           wird initiiert
1065     delayMicroseconds(10); // Es wird 10 Millisekunden gewartet
1066     digitalWrite(Triggerpin, LOW); // Der Triggerpin wird auf Low gestellt
1067     Dauer = pulseIn(Echopin, HIGH); // Measure the time until the signal
1068           returns
1069     Abstand = 0.5 * Schallgeschwindigkeit * Dauer * pow(10,-6) * 100; //
1070           The distance to the reflective surface is calculated in cm
1071     Serial.print(Abstand);
1072     Serial.println(" cm");
1073     delay(5000);
1074 }

```

- Arduino Nano 33 BLE Sense Lite
- Tiny Machine Learning Shield
- USB-A auf USB-Mikro Verbindungskabel
- Grove Jumper zu Grove 4 Pin Kabel
- Ultraschallsensor

Die Hardware-Komponenten werden verbunden und anschließend der Arduino Nano 33 BLE Sense Lite mit einem Computer verbunden. Dann wird der Sketch [TestHCSR04.ino](#) auf den Arduino Nano 33 BLE Sense Lite geladen und der serielle Monitor in der Arduino IDE geöffnet. Ein Gegenstand wird ungefähr 20 cm orthogonal vor den Ultraschallsensor gehalten.

27.10.2. Ergebnisse

Der gemessene Abstand wird in Abbildung 27.5 dargestellt und liegt in dem erwarteten Bereich.



```
Message (Enter to send message to 'Arduino Nano 33 BLE' on 'COM3')
23:55:54.755 -> 21.14 cm
23:55:59.770 -> 21.16 cm
23:56:04.762 -> 21.19 cm
```

Figure 27.5.: Testoutput des Ultraschallsensors

27.11. Simple Application

27.12. Tests

27.12.1. Simple Function Test

27.12.2. Test all Functions

27.13. Simple Application

27.14. Further Readings

28. Entwicklerboard - Motorsteuerung DEBO MotoDriver2 L298N

Um die zwei verwendeten Motoren ansteuern zu können, wird die Erweiterungsplatine DEBO MotoDriver2 L298N benutzt. Diese erlaubt die Steuerung und Versorgung von bis zu zwei Gleichstrommotoren. Die Motoren können mit Spannungen zwischen 5V und 35V angetrieben werden. Der verwendete Chipsatz ist L298N. Es wird auf dem Logiklevel von 5V gearbeitet. Die Ausgangsleistung beträgt 25W bei einem maximalen Treiberstrom von bis zu 2A. Die Abmaße der Platine sind 43mm × 43mm × 27mm. In der folgenden Abbildung 28.1 ist die Erweiterungsplatine zu sehen. In der folgenden Abbildung 28.2 sind die Pins des Treibers beschriftet und in der Tabelle 28.1 ist die Pinbelegung dargestellt. [Sima]

PIN	Belegung
1	DC Motor 1 / Stepper Motor +
2	DC Motor 1 / Stepper Motor GND
3	12V Jumper
4	Stromversorgung +
5	Stromversorgung GND
6	5V Ausgang (wenn Jumper 3 gesetzt)
7	DC Motor 1 Jumper
8	Input 1
9	Input 2
10	Input 3
11	Input 4
12	DC Motor 2 Jumper
13	DC Motor 2 / Stepper Motor +
14	DC Motor 2 / Stepper Motor GND

Table 28.1.: Pinbelegung des Treibers

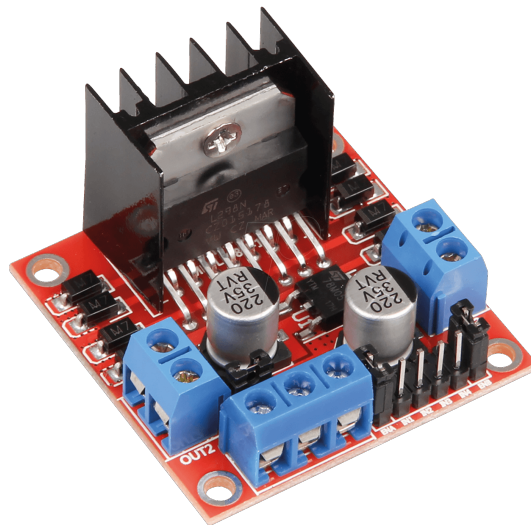


Figure 28.1.: Motorsteuerung DEBO MotoDriver2 L298N

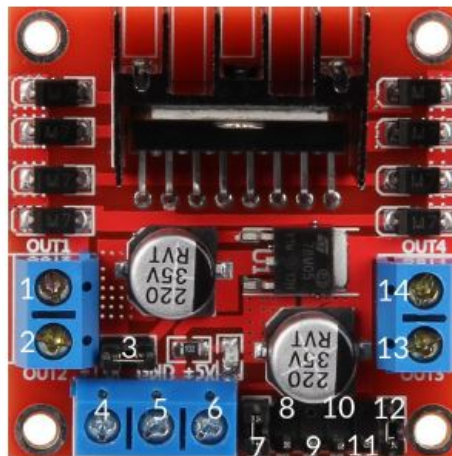
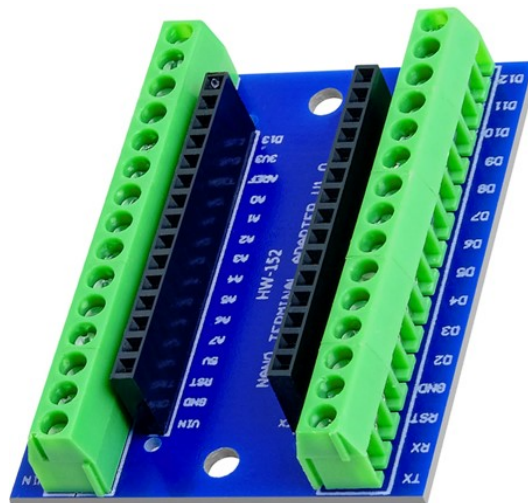


Figure 28.2.: Treiber mit Beschriftung

29. Terminal Adapter Board mit Schraubklemmen kompatibel mit Nano V3 und Arduino

Das verwendete Shield führt alle Anschlüsse des Arduinos auf außenliegende Schraubklemmen. Dies ermöglicht die Nutzung aller Anschlüsse durch das Verwenden von Jumper-Kabeln. Das Shield ist in der folgenden Abbildung 29.1 zu sehen. Die Abmessungen des Shields sind 54x43x13mm und es hat ein Gewicht von 21 Gramm. [Az]



30. Drehwinkel-Encoder

Der Demonstrator soll mehrere Programme fahren können, deswegen wurde ein Drehwinkel-Encoder der Steuerung hinzugefügt. Dieser wird über drei Pins am Arduino angeschlossen und über zwei weitere Pins wird er mit Spannung versorgt. Durch drehen des Drehschalters werden nacheinander drei Kontakte geschlossen oder geöffnet (ein Kontakt ist immer geschlossen). Dieser dadurch entstehende Signalfluss, bestehend aus zwei um 90 Grad versetzte Sinus bzw. Cosinusschwingungen werden ausgewertet. Daraus wird bestimmt, in welche Richtung (im oder gegen Uhrzeigersinn) und wie weit (inkrementell) gedreht wurde. Mithilfe dieser Logik kann durch ein Menü eine Bewegungsstufe ausgewählt werden, die der Schrittmotor fahren soll.[Bas16] Bei einer Drehung im Uhrzeigersinn wird im Menü eine Bewegungsstufe höher und bei einer Drehung gegen den Uhrzeigersinn eine Bewegungsstufe niedriger ausgewählt. Zusätzlich zum Drehwinkel-Encoder ist auch noch ein Schaltfunktion im Bauteil selbst integriert. Durch eindrücken des Encoders wird ein Taster betätigt, durch den der eingestellte Wert bestätigt und an den Arduino zur weiteren Verarbeitung weitergegeben wird. Zur Besseren Handhabung des Drehwinkel-Encoders wurde noch ein Drehgriff angefertigt und auf dem Drehgeber montiert. Weitere Details:

- **Abmessungen** ($b \times l \times h$): $18\text{ mm} \times 31\text{ mm} \times 30\text{ mm}$
- **Betriebsspannung**: 3,3 - 5 V [Sim19]

30.1. Encoder

Die Library Encoder ermöglicht das Zählen von Impulsen aus bestimmten Signalen, die häufig von Drehknöpfen, Motor- oder Wellensensoren sowie anderen Positionsgebern stammen. Diese Bibliothek ist für Arduino und Teensy Boards optimiert und bietet verschiedene Zählmodi, darunter 4X counting für präzise Positionserfassung. Die Verwendung erfolgt durch die Initialisierung eines Encoder-Objekts mit zwei Pins, wobei der erste Pin idealerweise über Interruptfähigkeit verfügt für optimale Leistung. Die Bibliothek unterstützt sowohl I2C- als auch SPI-Schnittstellen und bietet verschiedene Hardware- und Softwareoptionen zur Anpassung an die Anforderungen des Benutzers. Sie wird in diesem Projekt in der Version 1.4.4 verwendet.[Sto24]

30.2. Drehwinkel-Encoder

Um die Funktion und korrekte Ansteuerung des Drehwinkel-Encoders zu testen, wird das Testprogramm 30.1 verwendet. Das Programm wertet die Drehung des Encoders aus und gibt den neuen Zustand im seriellen Monitor der Arduino IDE aus [Sto24].

```
#include <Encoder.h>

const int CLK = 9;
const int DT = 2;
const int SW = 3;
long altePosition = -999;

Encoder meinEncoder(DT, CLK);

void setup()
{
    Serial.begin(9600);
    pinMode(SW, INPUT);
}

void loop()
{
    long neuePosition = meinEncoder.read();

    if (neuePosition != altePosition)
    {
        altePosition = neuePosition;
        Serial.println(neuePosition);
    }

    int StatusTaster = digitalRead(SW);
    if (StatusTaster == 0) {
        Serial.println("Switch_betaetigt");
        Serial.println("Switch_betaetigte");
    }
}
```

Listing 30.1.: Testprogramm für den Encoder

31. Getting Started

31.1. Download and Install the NRF Connect App on Your Mobile

To download and install the **nRF Connect** app on Android or iOS devices, follow these simple steps:

31.1.1. For Android

- Open the **Google Play Store** on your Android device.
- In the search bar, type "*nRF Connect for Mobile*".
- Select the app by **Nordic Semiconductor ASA**.
- Tap the button **“Install”** to download and install the app.
- Once installed, you can open the app from your home screen or app drawer.

31.1.2. For iOS

- Open the **App Store** on your iOS device.
- In the search bar, type "*nRF Connect for Mobile*".
- Select the app by **Nordic Semiconductor ASA**.
- Tap the button **“Get”** to download and install the app.
- Once installed, you can open the app from your home screen or app drawer.

The **nRF Connect** app allows you to interact with **Bluetooth Low Energy (BLE)** devices, such as the Arduino Nano 33 BLE, and monitor their services and data.

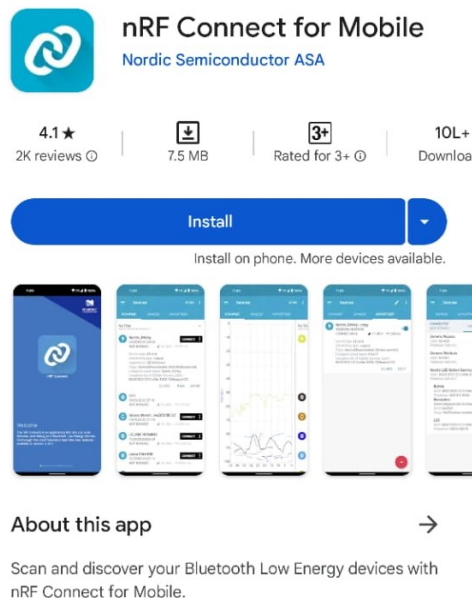


Figure 31.1.: NRF Connect Application

31.2. Power ON the Arduino Nano 33 BLE Sense Lite

To power on the Arduino Nano 33 BLE, connect it to your computer or a USB power adapter using the micro-USB cable provided in the box.

31.2.1. Connect to Arduino Nano 33 BLE Sense Lite through NRFconnect

31.2.2. For Android

- **Open the NRFconnect App:** Open the app and start scanning for nearby BLE devices.
- **Find The Arduino:** You should see a device named “MyProctName” in the list of available devices.
- **Connect:** Tap on the ArduinoNano33 device to connect. The Arduino Nano will now show “Connected” in the Mobile application.

31.2.3. For iOS

- **Open the NRFconnect App:** Open the app and start scanning for nearby BLE devices.
- **Find Your Arduino:** Look for “MyProctName” in the list of detected BLE devices.
- **Connect:** Tap the device name to connect. You will be able to see connection status in the Mobile application.

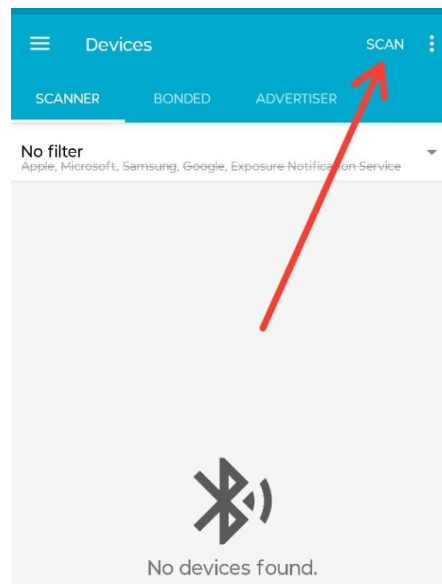


Figure 31.2.: Landing page

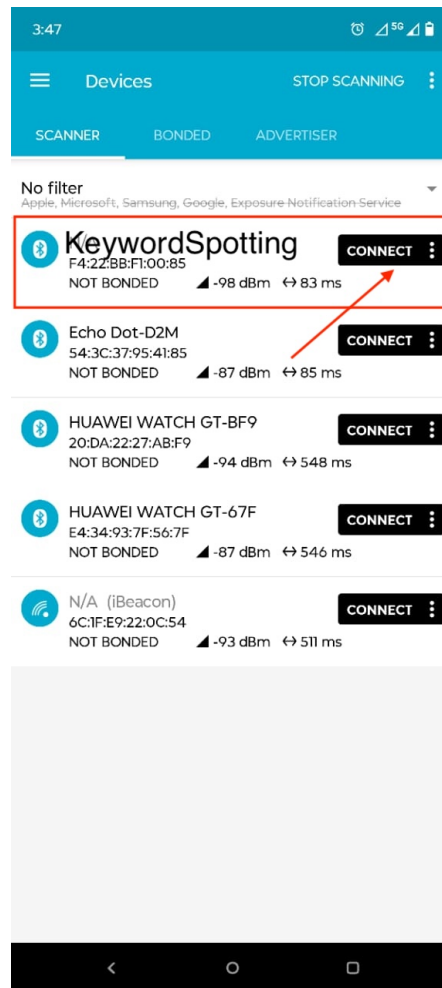


Figure 31.3.: List of devices

Part III.

Applikation <Title>

32. **Application**

33. Conclusion XY

34. To DoX Y

Part IV.

Templates

35. Sensor

Introduction

WS:cite books,
applications, board

35.1. General

General description
cite books

35.2. Specific Sensor

cite board

35.3. Specification

- cite data sheet
- Circuit Diagram

35.4. Library

35.4.1. Description

35.4.2. Installation

35.4.3. Functions

35.4.4. Example's Manual

35.4.5. Inside the Example

35.4.6. Example's Code

35.4.7. Example's Files

35.5. Calibration

cite method

35.6. Simple Code**35.7. Simple Application****35.8. Tests****35.8.1. Simple Function Test****35.8.2. Test all Functions****35.9. Simple Application****35.10. Further Readings**

36. Package Example

36.1. Introduction

36.2. Description

36.3. Installation

`pip packageExample`

36.4. Example - Manual

36.5. Example

36.6. Example - Code

36.7. Example - Files

`PackageExample.py`

36.8. Further Reading

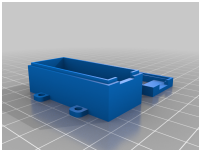
References

- [Aba+16] M. Abadi et al. “TensorFlow: A System for Large-Scale Machine Learning”. In: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. Savannah, GA: USENIX Association, Nov. 2016, pp. 265–283. ISBN: 978-1-931971-33-1. URL: <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi>.

Part V.

Anhang

37. Materialliste

Anzahl	Bezeichnung	Link	Preis
1	ARD NANO 33BLE H - Arduino Nano 33 BLE with header	www.reichelt.de - ARD NANO 33BLE H	29,50 €
			
1	ARD NANO 33BSH2 - Arduino Nano 33 BLE Sense Rev.2 with Header	www.reichelt.de - ARD NANO 33BSH2	44,80 €
			
1	ARD KIT TINYML - Arduino Lern-Kit Tiny Machine	www.reichelt.de - ARD Kit TinyML	49,24 €
			
1	Arducam B0226 Lens Calibration Tool	www.welectron.com - Arducam B0226 Lens Calibration Tool	11,90 €
			
1	3D Printed Case - A protective case for Arduino Nano 33 BLE Sense, customizable and available at Thingiverse	Thingiverse	./.
			

Stand: 03.02.2025

Part VI.

Hints - Examples for LaTeX - Read and Use

38. Hints

Please, use **biber.exe**.

If you want to create a symbol directory, call makeindex:

```
makeindex %.nlo -s nomenc1.ist
```

38.1. Allgemeine mathematische Beschreibung Bézier-Kurve

Bézier curves are formed with the help of Bernstein polynomials. If at least two points and two tangents are known, control points can be determined with which a control polygon is formed. The course of the curve is oriented to this control polygon. The number of control points depends on the degree of the Bézier curve. A Bézier curve of degree n has $n+1$ control points. The Bézier curve is calculated using the De Casteljau algorithm.

Definition. In the interval $[0; 1]$ the **Bernstein polynomial of n^{th} degree** is defined by:

$$b_{i,n}(\lambda) = \binom{n}{i} (1-\lambda)^{n-i} \lambda^i, \quad \lambda \in [0, 1], \quad i = 0, \dots, n \quad (38.1)$$

Definition. The binomial coefficient is defined by:

$$\binom{n}{i} = \frac{n!}{i!(n-i)!}, \quad i = 0, \dots, n \quad (38.2)$$

Here the Bernstein polynomials $b_{i,n}$ form a basis of the vector space $\mathbb{P}^n(I)$ of polynomials of degree at most n over I . Thus every polynomial of degree at most n can be written uniquely as a linear combination. [Far02]

Beispiel. Determination of Bernstein polynomials for $n = 3$ holds:

$$\begin{aligned} b_{3,0}(\lambda) &= \binom{3}{0} (1-\lambda)^{3-0} \lambda^0 = (1-\lambda)^3 \\ b_{3,1}(\lambda) &= \binom{3}{1} (1-\lambda)^{3-1} \lambda^1 = 3\lambda(1-\lambda)^2 \\ b_{3,2}(\lambda) &= \binom{3}{2} (1-\lambda)^{3-2} \lambda^2 = 3\lambda^2(1-\lambda) \\ b_{3,3}(\lambda) &= \binom{3}{3} (1-\lambda)^{3-3} \lambda^3 = \lambda^3 \end{aligned}$$

$b_{3,i}(\lambda)$ with $i = 0, 1, 2, 3$ are the cubic Bernstein polynomials of degree 3.

Definition. Given are the points

$$Q_i = \begin{pmatrix} x_i \\ y_i \end{pmatrix} \quad \text{mit} \quad Q_i \in \mathbb{R}^2, \quad i = 0, 1, \dots, n$$

A **Bézier curve** is then defined by

$$C(\lambda) = \sum_{i=0}^n Q_i \cdot b_{i,n}(\lambda) \quad (38.3)$$

The points $Q_i, i = 0, \dots, n$ are called **control points**.

The control points of a Bézier curve form the so-called control polygon.

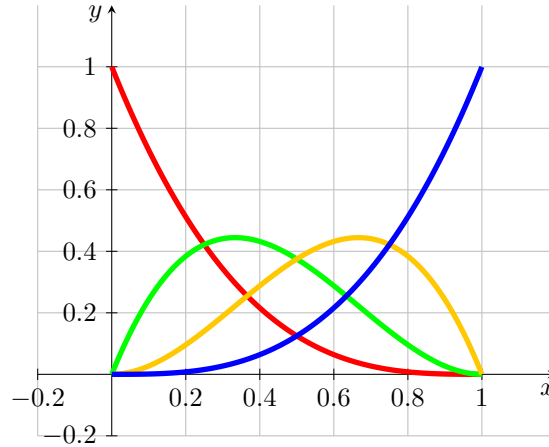


Figure 38.1.: Bernstein polynomial of degree 3

Bemerkung. Let the starting point P_0 and the end point P_1 , as well as the tangents $\vec{t}_0$ and $\vec{t}_1$ be given. The tangents are not necessarily normalised. The control points Q_0, Q_1, Q_2 and Q_3 of the associated Bézier curve are given by the following equations:

$$Q_0 = P_0, \quad Q_1 = P_0 + \lambda_0 \vec{t}_0, \quad Q_2 = P_1 - \lambda_1 \vec{t}_1, \quad Q_3 = P_1 \quad (38.4)$$

[Jak+10]

Beispiel. Given are

$$P_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad \vec{t}_0 = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad \text{und} \quad P_1 = \begin{pmatrix} 6 \\ 1 \end{pmatrix}, \quad \vec{t}_1 = \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$

Then, according to theorem ?? for control points of the associated Bézier curve results:

$$Q_0 = P_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad Q_1 = P_0 + \vec{t}_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 5 \end{pmatrix},$$

$$Q_2 = P_1 - \vec{t}_1 = \begin{pmatrix} 6 \\ 1 \end{pmatrix} - \begin{pmatrix} 1 \\ -2 \end{pmatrix} = \begin{pmatrix} 5 \\ 3 \end{pmatrix}, \quad Q_3 = P_1 = \begin{pmatrix} 6 \\ 1 \end{pmatrix}$$

The Bézier curve and its control points are shown in the figure ??.

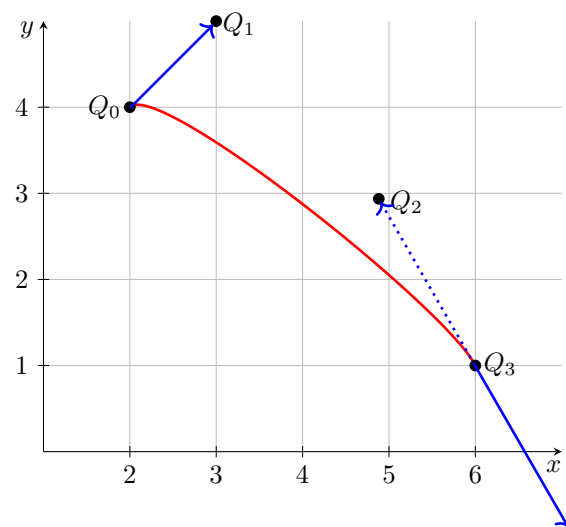


Figure 38.2.: Bézier curve for example 38.1

39. Verrundung mit einem Kreisbogen

39.1. Gleichungen

Blending with an arc is a simple variant of smoothing corners. This variant enables the processing and the generation of files according to DIN 66025. For smoothing, three points P_0 and S and P_1 are always considered, thus a symmetric Hermite problem results. According to theorem ?? it is assumed to be in the standard form $HP(L, \alpha)$.

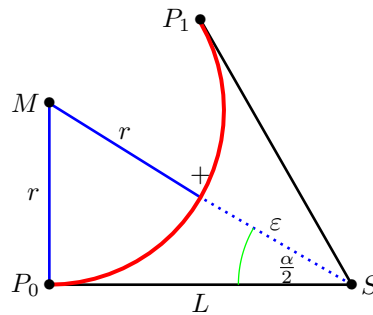


Figure 39.1.: Smoothing a corner with the help of an arc - triangle

The figure ?? represents the situation. The points P_0 , S and P_1 are given. The included angle $\alpha = \angle(P_0; S; P_1)$ as well as the distance $L = \|S - P_0\| = \|P_1 - S\|$ are shown in the graph. The red arc is the desired result. The distance of its centre M to S is then $r + \varepsilon$, where r is the radius of the arc and ε is the given tolerance. Thus we obtain a right triangle P_0SM whose edge lengths are L , $r + \varepsilon$ and r .

According to the definition of the sine and the tangent, it follows:

$$\sin\left(\frac{\alpha}{2}\right) = \frac{L}{r + \varepsilon} \quad \text{und} \quad \tan\left(\frac{\alpha}{2}\right) = \frac{L}{r}$$

$$\Leftrightarrow \varepsilon = \frac{L}{\sin\left(\frac{\alpha}{2}\right)} - r \quad \text{und} \quad r = \cot\left(\frac{\alpha}{2}\right) L$$

The two equations can be combined to give the following conditions:

$$\varepsilon = L \cdot \frac{1 - \cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} \quad \text{bzw.} \quad L = \varepsilon \cdot \frac{\sin\left(\frac{\alpha}{2}\right)}{1 - \cos\left(\frac{\alpha}{2}\right)}$$

This gives the equation for the radius:

$$r = L \cdot \cot\left(\frac{\alpha}{2}\right) = L \cdot \sqrt{\frac{1 - \cos(\alpha)}{1 + \cos(\alpha)}} = L \cdot \frac{\sin(\alpha)}{1 + \cos(\alpha)} = L \cdot \frac{P_{1,x}}{P_{1,y}}$$

The factor for converting L and ε can be simplified using trigonometric transformations.

$$\frac{1 - \cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} = \frac{1 - \sqrt{\frac{1 - \cos(\alpha)}{2}}}{\sqrt{\frac{1 + \cos(\alpha)}{2}}} = \frac{\sqrt{2} - \sqrt{1 - \cos(\alpha)}}{\sqrt{1 + \cos(\alpha)}} = \frac{\sqrt{1 + \cos(\alpha)} - \sin(\alpha)}{1 + \cos(\alpha)}$$

By comparing this with the symmetric Hermite problem in standard form, the following notation is then obtained:

$$\frac{1 - \cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} = \frac{\sqrt{P_{1,x}} - P_{1,y}}{P_{1,x}}$$

The previous considerations are now summarised in the following sentence.

Satz. Let a symmetric Hermite problem $(P_0, \vec{t}_0, P_1, \vec{t}_1, S, L)$ be given. Without restriction of generality, it is in the standard form

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} L + L \cdot \cos(\alpha) \\ L \cdot \sin(\alpha) \end{pmatrix}, \begin{pmatrix} \cos(\alpha) \\ \sin(\alpha) \end{pmatrix}, \begin{pmatrix} L \\ 0 \end{pmatrix}, L \right\}$$

with $L \in \mathbb{R}^{>0}$ and $\alpha \in (-\pi; \pi]$.

Then an arc can be found that connects the points P_0 and P_1 , has the same tangent directions at the two points.

For the circular arcs applies:

$$r = L \cdot \left| \frac{P_{1,x}}{P_{1,y}} \right|; \quad \phi_0 = -\text{sign}(\alpha) \cdot \frac{\pi}{2}; \quad \phi_1 - \phi_0 = \text{sign}(\alpha) \cdot \pi - \alpha = \beta;$$

$$M = P_0 + \text{sign}(\alpha) \cdot r \cdot \vec{t}_0^\perp = \text{sign}(\alpha) \cdot r \cdot \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

From the specification of the distance L , the maximum error can be calculated, as shown in theorem ???. According to the derivation, it is also possible to specify the maximum error ε and determine the maximum distance L from it.

Satz. Let a symmetric Hermite problem $(P_0, \vec{t}_0, P_1, \vec{t}_1, S, L)$ be given. Without restriction of generality, it is in the standard form

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} L + L \cdot \cos(\alpha) \\ L \cdot \sin(\alpha) \end{pmatrix}, \begin{pmatrix} \cos(\alpha) \\ \sin(\alpha) \end{pmatrix}, \begin{pmatrix} L \\ 0 \end{pmatrix}, L \right\}$$

with $L \in \mathbb{R}^{>0}$ and $\alpha \in (-\pi; \pi]$.

- a) Let the maximum error ε be given. The maximum distance L for which an arc exists according to the theorem ??? that takes the error into account is given by:

$$L(\varepsilon, \alpha) = \varepsilon \cdot \frac{P_{1,x}}{\sqrt{P_{1,x}} - P_{1,y}} = \varepsilon \cdot \frac{L + L \cdot \cos(\alpha)}{\sqrt{L + L \cdot \cos(\alpha)} - L + L \cdot \cos(\alpha)}$$

- b) When the distance L is specified, the following maximum error results:

$$\varepsilon(L, \alpha) = L \cdot \frac{\sqrt{P_{1,x}} - P_{1,y}}{P_{1,x}} = L \cdot \frac{\sqrt{L + L \cdot \cos(\alpha)} - L + L \cdot \cos(\alpha)}{L + L \cdot \cos(\alpha)}$$

These considerations result in limitations for the use of this strategy, which are summarised in the following comment.

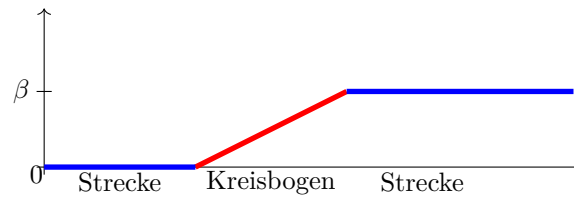


Figure 39.2.: Angular change with blending arc

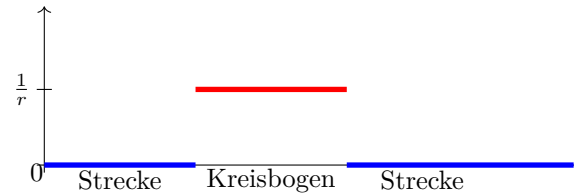


Figure 39.3.: Curvature progression for a blending arc

Bemerkung. The following conditions for applying the strategy of a circle must be fulfilled:

a)

$$P_0 \neq P_1$$

b)

$$\vec{t}_0 = -\vec{t}_1 \Leftrightarrow \alpha = 0$$

c)

$$\vec{t}_0 = \vec{t}_1 \Leftrightarrow \alpha = \pi$$

39.2. Bewertung

Rounding by means of a circular arc leads to a continuous course of the angle change. The figure ?? illustrates this.

Due to the rounding with a circular arc, the curvature of the curve is not continuous. The figure ?? shows that the curvature is constant in the range of distances 0 and on the circular arc with the value $\frac{1}{r}$, where r is the radius of the circular arc used. Due to the formula $a = \frac{v^2}{r}$ for a movement on a circular arc, the acceleration course exhibits continuity jumps at the transitions for a constant path velocity.

Depending on the angle change β at the corner S and the tolerance ε , the maximum length of the shortening of the lines can be calculated. The figure ?? shows the corresponding diagram, where the tolerance ε is set to the value 1.

The area provided can also be specified. Depending on the distance L from the corner point S , the deviation ε can be calculated. The figure ?? represents the function as a function of L and the angle α .

The figures ?? and ?? show the curves of the length as a function of the angle change for a fixed tolerance and the error as a function of the angle change for a given length. If the angle change is minimal, then the error or length L is extreme. In this case,

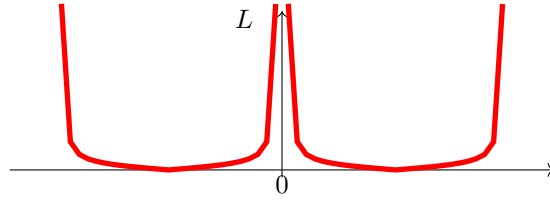


Figure 39.4.: Maximum range for a blending arc of a circle - $L(\varepsilon = \text{const}, \alpha)$

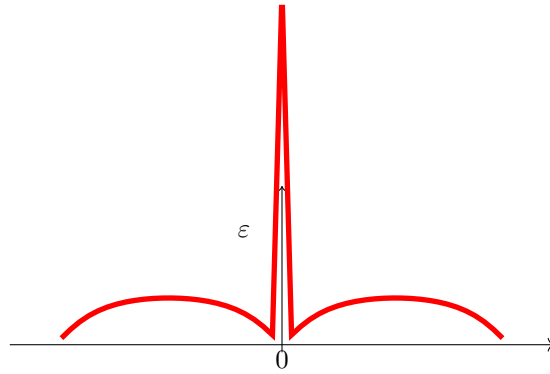


Figure 39.5.: Deviation when specifying the distance L when rounding with a circular arc

rounding by means of an arc is not possible. In order to develop a sensible strategy, a minimum angle change must be specified from which a blending arc is permitted. Likewise, a maximum angle change must also be defined. For an angular change of $\pm\pi$ is a bend. In this case, a blending is also not possible.

39.3. Examples

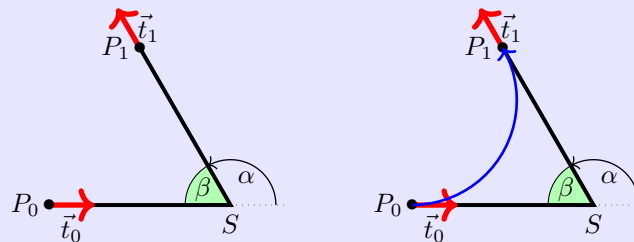
Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{2}{3}\pi\right) \\ 6.0 \cdot \sin\left(\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{2}{3}\pi\right) \\ \sin\left(\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = \frac{2}{3}\pi$.

For the blending arc follows:

$$M = \begin{pmatrix} 0 \\ 3,4641 \end{pmatrix}; \quad r = 3,46412; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{2}{3}\pi$$



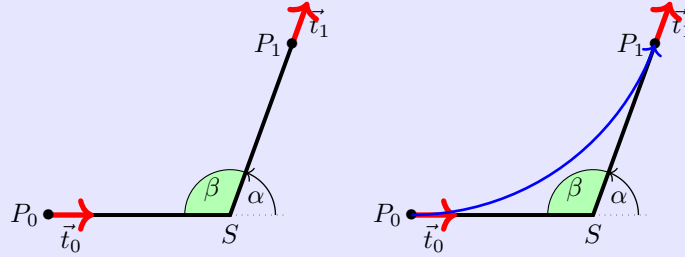
Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{7}{18}\pi\right) \\ 6.0 \cdot \sin\left(\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{7}{18}\pi\right) \\ \sin\left(\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = \frac{7}{18}\pi$.

For the blending arc follows:

$$M = \begin{pmatrix} 0 \\ 8,5689 \end{pmatrix}; \quad r = 8,5689; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{7}{18}\pi$$



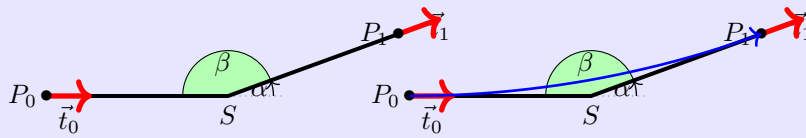
Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{1}{9}\pi\right) \\ 6.0 \cdot \sin\left(\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{1}{9}\pi\right) \\ \sin\left(\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = \frac{1}{9}\pi$.

For the blending arc follows:

$$M = \begin{pmatrix} 0 \\ 34,0277 \end{pmatrix}; \quad r = 34,0277; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{1}{9}\pi$$

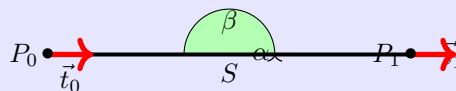


Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 12.0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = 0$.

There is no blending arc here.



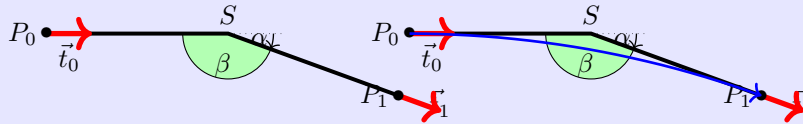
Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(-\frac{1}{9}\pi\right) \\ 6.0 \cdot \sin\left(-\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(-\frac{1}{9}\pi\right) \\ \sin\left(-\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = -\frac{1}{9}\pi$.

For the blending arc follows:

$$M = \begin{pmatrix} 0 \\ -34,0277 \end{pmatrix}; \quad r = 34,0277; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = \frac{1}{9}\pi$$



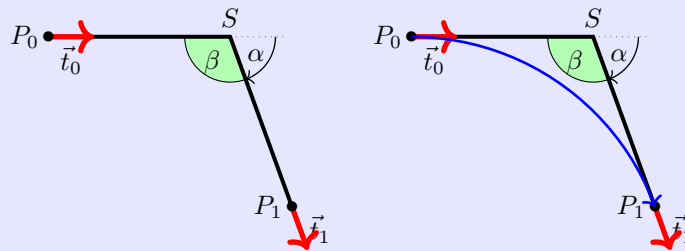
Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(-\frac{7}{18}\pi\right) \\ 6.0 \cdot \sin\left(-\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(-\frac{7}{18}\pi\right) \\ \sin\left(-\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = -\frac{7}{18}\pi$.

For the blending arc follows:

$$M = \begin{pmatrix} 0 \\ -8,5689 \end{pmatrix}; \quad r = 8,5689; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = -\frac{7}{18}\pi$$



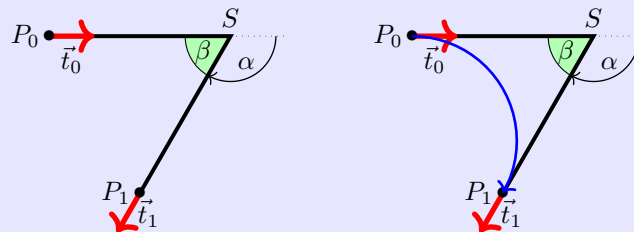
Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(-\frac{2}{3}\pi\right) \\ 6.0 \cdot \sin\left(-\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(-\frac{2}{3}\pi\right) \\ \sin\left(-\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = -\frac{2}{3}\pi$.

For the blending arc follows:

$$M = \begin{pmatrix} 0 \\ -3,4641 \end{pmatrix}; \quad r = 3,46412; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = -\frac{2}{3}\pi$$



40. Maple files

[Wat17c; Wat17d; Wat17b; Wat17e; Wat17a; Wat17f]

40.1. Geometries with Maple

The algorithms presented can be well represented using geometries. Two aspects can be investigated. On the one hand, the effort for an implementation, the computational accuracy and the stability of an algorithm are interesting; on the other hand, the methods are to be evaluated and compared with regard to their usefulness. For this purpose, modules for the formula manipulation system Maple [Wat17c] were developed. Maple offers the environment to implement algorithms easily and quickly. Furthermore, graphical representation is possible with simple means. However, a simple workbook of Maple is not designed for very large software projects. Here, one must resort to the possibility of creating and using libraries. On the one hand, Maple offers the possibility of creating one's own libraries, so-called modules. In this way, one remains within the environment and syntax of Maple. This path will be pursued further here. Another possibility is the use of libraries created by means of a higher programming language, e.g. C++, the so-called DLLs.

In the following, the creation and use of a test environment with the help of modules is described. First, the geometry elements that are stored in various modules and their use are described. The structure and use of a module in Maple is then explained so that own extensions and additions are possible.

40.2. Geometry elements used

This is a test environment for the algorithms presented, only geometries that have been described are also used.

- points (`MPoint`)
- lines (`MLine`)
- arcs (`MArc`)
- Bézier curves (`Bezier`)
- polygon courses (`MPolygon`)
- Geometry List (`MGeoList`)
- Hermite problems (`MHermiteProblem`)
- Symmetric Hermite Problems (`MHermiteProblemSym`)

For each geometry element a corresponding module has been created, the name of which is given in the list above. The list of which functions are available is presented in another section.

When implementing such a project, it quickly becomes clear that when using several geometry elements, a clear data structure and well-defined access to the data is essential. For example, when representing straight lines, the decision must be made

whether the representation should be point-directional or by means of two points. The choice is generally made depending on the application. Here, the path is taken that, due to a well-defined access to the data, the use of both representations is possible.

40.3. Building the data structure

The idea is to use a data structure that is as uniform and simple as possible. Maple does offer the possibility to define objects. However, it is difficult to use within a procedural environment. Therefore, all data is basically represented as lists. The first list element always contains an identifier of the element `??`. This is followed by the data, which in turn can be geometry elements. It should be noted that direct access to the data cannot be prevented; here the user is responsible.

Access to the data should be exclusively via procedures of a module. The following is an example of the data structure for points:

```
[MVPOINT, [x, y]]
```

The creation of a point then takes place via a procedure `New`:

```
NeuerPunkt := MPoint:-New(10,15);
```

When called up in the test file, the following is then output:

```
TestPO := ["Point", [10,15]]
```

These data structures are used for the calculation of geometries. They are used in functions or procedures. Unlike variables, the data structures and procedures are defined globally and not locally. Under the command `export` the procedures are declared at the beginning of the module and can thus also be used outside the module. If, for example, a data structure from `MPoint` is to be used in another module or outside the modules, it is called as follows:

```
P0 := MPoint:-New(x,y);
```

In addition to the modules for the geometries, there is a module (`MConstant`) for saving constants and names. There, for `MVPOINT`, for example. „Point “ or e.g. a constant for the comparison to zero for real numbers.

Finally there is the test file, which is not a module, in which the modules are loaded, called and tested in their function. More about this file later in „1.4 Maple test file “.

40.4. Structure of a module

The following section deals with the purpose of modules and their structure.

The project is about capturing the above geometries in a data structure and representing them in Maple. However, the calculations and formulas that have to be used are a hindrance to the final representation. Modules are very well suited for summarising and hiding the calculations that take place in functions. This way, the important functions can be accessed in Maple without seeing what is in them.

Example:

The function `Angle` from the module `MPoint`: Function to calculate an angle between location vector and x-axis:

```
Angle := proc(P)
  local alpha, x, y;
```



```

x := GetX(P);
y := GetY(P);
alpha := 0;
if abs(x) < 0.00001
then
    alpha := 3*Pi/2;
else
    alpha := Pi/2;
end if;
else
    alpha := arctan(y/x);
    if x < 0
    then
        alpha := alpha + Pi;
    else
        if y < 0
        then
            alpha := alpha + 2*Pi;
        end if;
    end if;
end if;
return factor(alpha);
end proc;

```

This function is long, but it only represents a small part of the programme for the representation in question. That is why it is in the module and can thus be called externally with a single command:

```
Winkel := MPoint:-Angle(P)
```

The following section describes the structure of a module. Since Maple offers a wide range of possibilities, a restriction is made. Only the structure of the modules used is described, here on the basis of the module `MPoint`. For more detailed possibilities, please refer to the Maple manual [Wat17c].

structure:

The module must first be started. This is done with the name (here always a capital M and the geometry) of the module and the following command:

```
MPoint := module()
```

Then, similar to procedures, the variables and functions must be defined. You can declare them either locally or globally. If the variables or procedures are only used and changed in the module, the declaration is made with the command `local` as follows:

```
local Variable names, with, comma, separated;
```

If the variables or procedures are also to be usable outside the module, this is defined with `export`:

```
export Function names, with, comma, separated;
```

This is followed by the specification of the options:

```
option package;
```

the description of the module:

description "Self-selected module description, e.g. module for points ";
and the initialisation of the module:

```
ModuleLoad := proc
  MVPOINT:=MConstant:-GetPoint();
  print("Module MPoint is loaded");
end proc;
```

From here on, programming is done as usual in Maple. The previously defined procedure and variable names are used and programming is done with commands that are also used outside of a module in Maple. It is important to know that with modules, each function can be called constantly, so the order of the functions is irrelevant.

To finally end the module, use the following command:

```
end module;
```

40.4.1. General structure of a module

In summary, the modules have the following structure:

```
Modulname := module()

  local Names, with, comma, separated;

  export Names, with, comma, separated;

  global Names, with, comma, separated;1

  option package;

  description „ Self-selected module description“;

  ModuleLoad := proc()2
    MVPOINT:=MConstant:-GetPoint();
    print("Modul MPoint is loaded");
  end proc;

  Procedure1 := proc (Passing parameters)
    inhalt;
  end proc;

  Procedure2 := proc (Passing parameters)
    content;
  end proc;

  ...

end module;
```

¹Possible, but not used in the modules

²Example `MPoint`

40.4.2. Saving a module

The management of modules is done automatically by Maple. However, since the modules are passed on and have to be edited individually, some settings have to be made. Therefore, the following prefix is used for each Maple file of the project:

```
1 restart;
2 with(LibraryTools);
3 lib := "C:/FH/Tools/Maple/MyLibs/Blending.mla";
4 march('open', lib);
5 ThisModule := 'MArc';
```

The first line initialises the system. The next 3 lines enable working with archives. In the second line, the tools are loaded so that the variable `lib` can be occupied. All modules are stored in an archive; in this example it is the file `Blending.mla` in the directory `C:/FH/Tools/Maple/MyLibs/`. The command `ThisModule := 'MArc'`; contains the name of the current module.

A module is then saved using the command `savelib('ModuleName')`. A file `ModuleName.mla` is then created. Depending on the configuration of Maple, the set path where the file is automatically created may not be writable. Then you can configure the system so that the mla file is stored in the current directory or in a directory of your choice.

If the above prefix is used for a Maple file, all that is required to save the module is `savelib(ThisModule, lib)`;

40.4.3. Verwendung eines Moduls

A module that has been saved can now be used in other Maple worksheets. To do this, the command

```
with(ModuleName)
```

is used. If you have used a special path, Maple cannot find the file `ModuleName.mla`. Then the path must be made known, e.g.:

```
savelibname := "c:/Maple/MyLibs/"; savelibname;
```

Now the procedures of the module can be accessed. An overview of all exported procedures is provided by the command

```
Describe(ModuleName)
```

is displayed. A list of the names including the names of the transfer parameters is displayed. If a procedure is equipped with the field `description`, this text is also displayed.

40.4.4. Creating a Maple module

Creating your own module is done quickly if you use the framework above. However, one should make some considerations beforehand and maintain a standard.

Before creating an own module, the data model and the corresponding procedures should be worked out. A programme flow chart is useful here. The central task of the module is usually quickly determined. In addition, however, the following rules should be observed.

Rule 1 Basically, both the module and each procedure receive a description. This is not limited to the optional argument `description`, but is placed in front of each procedure. The description basically contains the description of the task. The prerequisite is also mentioned or an error handling is described. Then follows the description of all input parameters, their function and their data structure. The return value is then described.

rule 2 Each module receives a procedure `Version()`, which returns the current version number.

rule 3 Data is not global. To access data, procedures `Set*` and `Get*` are provided. These procedures are also used within the procedures of a module.

rule 4 At least one test function is written for each procedure. The test function can be used to illustrate the use and any special features.

40.5. Functions of the modules

In the following, the individual modules are listed. The data structure of each module and all functions with associated tasks are listed.

40.5.1. Module `MPoint`

`MPoint` is a module for points and works with a data structure and with procedures/functions. The data structure `New` for a point is a list, which is structured as follows:

```
[MVPOINT, [x,y]]
```

Its first element is a name to identify the module. Your second element is again a list containing the elements of the geometry. In this case the name is "Point" and the elements are the x and y coordinates for a point. No distinction is made here between point and vector. A point can also be seen as a vector from the coordinate origin to the point.

Via the Get functions `GetX` and `GetY` one can capture the coordinates of the point and use them in other functions. The other functions can then be used to calculate with the points/vectors.

`MPoint`

E	<code>New</code>	Data structure for a point
E	<code>GetX</code>	Reading the x-coordinate
E	<code>GetY</code>	reading the y-coordinate
E	<code>Angle</code>	calculating the angle between the x-axis and the point
E	<code>Add</code>	Calculates a linear combination of two points/vectors
E	<code>Sub</code>	Calculates the difference of two points/vectors
E	<code>Cos</code>	Calculates the cosine between two vectors
E	<code>Sin</code>	Calculates the sine between two vectors
E	<code>Scale</code>	Scales a vector with a factor
E	<code>Perp</code>	Calculates a vector that is orthogonal to the given vector
L	<code>IllustrateXY</code>	Plot function to illustrate a blue point
E	<code>Illustrate</code>	Plot of a blue point
E	<code>Plot</code>	Plot of a green point
E	<code>Plot2D</code>	Plot of a point with own options
E	<code>Length</code>	Calculates the distance of the point to the coordinate origin
E	<code>Uniform</code>	Normalises the vector to length 1
E	<code>LinetoVector</code>	Calculates vector from a distance
E	<code>Distance</code>	Calculates the distance between two points

40.5.2. Module **MLine**

MLine is a module for lines and works with a data structure and with procedures/-functions. The data structure **New** for a line is a list which is structured as follows:

```
[MVLINE, [P0,P1]]
```

Its first element is a name to identify the module. Its second element is again a list containing the elements of the geometry. In this case the name is „Line“ and the elements are the start and end points for a line.

The data structure **NewPointVerctor** is also a data structure for a line, but here the line is defined by a start point and a direction vector.

The Get functions **StartPoint** and **EndPoint** can be used to capture the start and end points of the route and use them in other functions. The other functions can then be used to calculate with the routes.

MLine

E	New	Data structure for a line (two-point form)
E	NewPointVector	creation of a route (point-direction form)
E	StartPoint	reading the starting point
E	EndPoint	reading the end point
E	Position	Calculate a point that lies on the line
E	Plot2D	Plot a part of the route starting from the starting point
E	Plot2DTangent	Plot of a point with tangent
E	Plot2DTangentArrow	Plot of a point with tangent (as arrow)
E	LineLine	Calculation of the intersection of two straight lines.
E	AngleLine	Calculation of the angle between two straight lines

40.5.3. Module **MArc**

MArc is a module for circular arcs and works with a data structure and with procedures/functions. The data structure **New** for an arc is a list that is structured as follows:

```
[MVARC, [mx,my,r,phi,alpha]]
```

Its first element is a name to identify the module. Your second element is again a list containing the elements of the geometry. In this case the name is "Arc" and the elements are the x- and y-coordinate for the centre, the radius, the start angle and the angle change for an arc.

The Get functions **GetM**, **GetMX**, **GetMY**, **GetR**, **GetPhi**, and **GetAlpha** can be used to collect the data for an arc and use it in other functions. The other functions can then be used to calculate with the arcs.

MArc

E	New	Data structure for an arc
E	GetMX	Reading the x-coordinate of the centre point.
E	GetMY	read the y-coordinate of the centre point
E	GetR	read the radius
E	GetPhi	reading the start angle
E	GetAlpha	Reading the change of angle
E	GetM	reading the centre point
E	Position	Calculating a point on the arc
E	Plot2D	Plot a part of the arc starting from the start angle
E	Blend	calculating an arc from a symmetric Hermite problem

40.5.4. module **MBezier**

The **New** data structure for a polygon is a list constructed as follows:

```
[MVBEZIER, [PointList]]
```

Within the module **MBezier** exist the procedures listed in the following table.

MBezier

E	New	Manual input of control points for a Bézier curve
E	Version	Output of the versions
E	BlendCurvature	Determination of control points from symmetric Hermite problem
E	BlendCurvatureEpsilon	determination of control points from symmetric Hermite problem with given error
E	Position	position on the Bézier curve
E	GetTheta	Reading the angle
E	GetEpsilon	reading the tolerance
E	GetControlPoint	Read the control points
E	Plot2D	Read the Bézier curve
E	PlotControlPoints	representation of all control points

40.5.5. Module **MPolygon**

MPolygon is a module for polygons and works with a data structure and with procedures/functions. The data structure **New** for a polygon is a list that is structured as follows:

```
[MVPOLYGON, [PointList]]
```

Its first element is a name to identify the module. Your second element is again a list containing the elements of the geometry. In this case the name is "Polygon" and the elements are any number of points.

Using the Get functions **GetPoint** and **GetN** you can get the number of points and the points themselves and use them in other functions. The other functions can then be used to calculate with the points or the polygon course.

MPolygon

E	New	Data structure for a list of points
E	GetPoint	Reading the ith point from the point list
E	GetN	Determine the number of points in the point list
E	Length	Determination of the Euclidean length of the polygon
E	Position	Calculation of a point on the polygon course
E	Tangents	calculation of the tangent
L	Plot2DAll	Plot list of all points
E	Plot2D	Plot of all points as polygonal plots.
E	Plot2DTangent	representation of the polygon course with tangent
E	PlotPoints	representation of all points

40.5.6. module **MGeoList**

MeoList is a module for a geometry list and works with a data structure and with procedures/functions. The data structure **New** for a geometry list is a list that is structured as follows:

```
[MVGEOLIST, []]
```

Its first element is a name to identify the module. Its second element is again a list containing the elements of the geometry. In this case the name is „GeoList“ and the elements are any number of individual geometries.

Using the Get functions `GeoGeo` and `GetN`, the i-th element of the list and the number of elements in the list can be captured and used in other functions. The other functions can then be used to calculate with the geometries or the list. In the functions `Length`, `Plot2DAll`, `Plot2D` and `Position` the functions are called in themselves. This is possible because the functions work independently of each other.

MGeoList

E	New	Data structure for a geometry list
E	Append	Append a geometry element to the data structure
E	Prepend	Inserting a geometry element as the first element of the list
E	Replace	Replace a geometry element with another one.
E	GetN	Determine the number of geometry elements in the list
E	GeoGeo	Reading the i-th geometry element
E	Length	Calculate the Euclidean length of the geometry elements
E	Position	Calculates a point on the geometry
L	Plot2DAll	Plot function for plotting the geometry elements
E	Plot2D	Plot a part of the geometry list starting from the first element

40.5.7. Module **MHermiteProblem**

`MHermiteProblem` is a module for Hermite problems and works with a data structure and with procedures/functions. The data structure `New` for a route is a list, which is structured as follows:

```
[MVHERMITEPROBLEM, [P0,T0n,P1,T1n]]
```

Your first element is a name to identify the module. Its second element is again a list containing the elements of the geometry. In this case the name is „HermiteProb“ and the elements are two points with associated tangents (lines).

Using the Get functions `StartPoint`, `EndPoint`, `StartTangent` and `EndTangent`, the points and their tangents can be captured and used in other functions. The other functions can then be used to calculate with the data for the Hermite problem.

MHermiteProblem

E	New	Data structure for a Hermite problem
E	StartPoint	reading the start point
E	EndPoint	Reading the end point command
E	StartTangent	reading the start tangent
E	EndTangent	Reading the end tangent
E	Plot2D	Plotting the Hermite problem

40.5.8. Module **MHermiteProblemSym**

`MHermiteProblemSym` is a module for symmetric Hermite problems and works with a data structure and with procedures/functions. The data structure `New` for a symmetric Hermite problem is a list built as follows:

```
[MVHERMITEPROBLEMSYM, [P0,T0n,P1,T1n,S,L]]
```

Your first element is a name to identify the module. Its second element is again a list containing the elements of the geometry. In this case the name is „SymHermiteProb“

and the elements are two points with associated tangents, their intersection, and the distance of the points to the intersection.

Via the Get functions `StartPoint`, `EndPoint`, `StartTangent`, `EndTangent`, `CrossPoint` and `ParameterL` one can acquire the data and use it in other functions. The other functions can then be used to calculate with the data for the symmetric Hermite problem.

MHermiteProblemSym

E	New	Data structure for a symmetric Hermite problem
E	StartPoint	reading the start point
E	EndPoint	Reading the end point command
E	StartTangent	reading the start tangent
E	EndTangent	Reading the end tangent
E	ParameterL	reading of the distance
E	CrossPoint	reading of the intersection point
E	Plot2D	Plotting of the symmetrical Hermite problem
E	Create	creation of a Sym. Hermite problem by 3 points
E	BlendArc	rounding of the corner point by an arc

40.5.9. Module **MConstant**

`MConstant` is a module for storing fixed constants and names to identify data structures. In the locally declared functions, starting with CV, the constants for declaring the different geometries are defined. These are names for recognising the geometry elements in the test file. In the globally declared functions, starting with Get, the names for identifying the geometry elements are returned.

MConstant

L	NULLEPS	constant to compare to zero
L	CVPOINT	constant for geometry elements: Point
L	CVLINE	constant for geometry elements: Line
L	CVARC	constant for geometry elements: Arc
L	CVPOLYGON	constant for geometry elements: polygon
L	CVGEOLIST	constant for geometry elements: GeoList
L	CVHERMITEPROBLEM	constant for geometry elements: HermiteProb
L	CVHERMITEPROBLEMSYMMETRIC	Const. for geometry elements: SymHermiteProb
L	CVBIARC	Const. for geometry elements: Biarc
E	GetNullePs	return of the zero comparison command.
E	GetPoint	identifier for points
E	GetLine	Identifier for lines
E	GetArc	identifier for circular arcs
E	GetPolygon	identifier for polygons
E	GetGeoList	Identifier for geometry lists
E	GetHermiteProblemSymmetric	Identifier for symmetric Hermite problems
E	GetHermiteProblem	ID for Hermite problems
E	GetBiarc	identifier for Biarcs

40.5.10. Module **MGeneralMath**

`MGeneralMath` is a module for general mathematical functions and works with the data structures and procedures.

MeneralMath

E	MPoint	Data structure for a point
E	MPointX	Reading of the x-coordinate for a point
E	MPointY	reading the y-coordinate for one point
L	MPointIllustrateXY	plot structure for a blue point
E	MPointIllustrate	plot structure for a blue point
E	MPointPlot	Illustration of a green point
E	MLine	Data structure for a line
E	MLineStartPoint	Reading of the start point for a draw frame
E	MLineEndPoint	Reading the end point for a line
E	MPointOnLine	Calculation of a point on the line
E	MLinePlot2D	Plot the part of a line starting at the starting point
. E	MLineLine	calculating the intersection of two linesline

40.5.11. Module Biarc**Data Structure**

Requirements and specifications:

The Biarc is to be used to solve a Hermite problem. A Hermite problem is defined as follows:

Given two points P_0 and P_1 with associated normalised tangents $\vec{t}_0$ and $\vec{t}_1$. The biarc must connect the points such that the tangents of the start and end points of the biarc coincide with the tangents of the Hermites problem.

Data structure:

The data structure **New** for a biarc must contain two arcs.

So the data structure for one arc is needed: **MArc:-New**.

This in turn contains the coordinates for the centre, the radius, the start angle and the angle change.

40.5.12. Module MBiarc

MBiarc is a module for Biarcs and works with a data structure and with procedures/functions. The data structure **New** for a Biarc is a list which is structured as follows:

```
[MVBIARC, [Arc0, Arc1]]
```

Its first element is a name to identify the module. Your second element is again a list containing the elements of the geometry. In this case the name is „Biarc“ and the elements are two arcs.

Using the Get functions **GetArc0** and **GetArc1** one can capture the two arcs for the biarc. The functions listed below can be used to calculate the data for the biarc.

MBiarc

E	New	Data structure for a biarc
E	GetArc0	Reading of the first arc
E	GetArc1	Reading the second arc
E	Circle	calculating the circle K_J from the Hermite problem data.
E	Plot2DCircle	plot of the circle K_J
E	angle	Calculate the angle from the centre of the circle to points on the circle
E	Plot2D	Plot of the biarc
E	ConnectionPoint	Calculation of the connection point J (Equal Chord)
E	TangentTj	Calculation of the tangent to J
E	Tangent Biarc	rotation of Tj for the biarc.
E	BiarcCenter	Calculate the centres of the arcs of the biarc
E	Biarc	calculate the centre of the biarc.
E	Position	Determine a point on the biarc
E	Blend	Calculate the biarc only from the Hermite problem

40.6. Programme Flowchart**40.6.1. Overall Flow****40.6.2. Verrundung der Kurve**

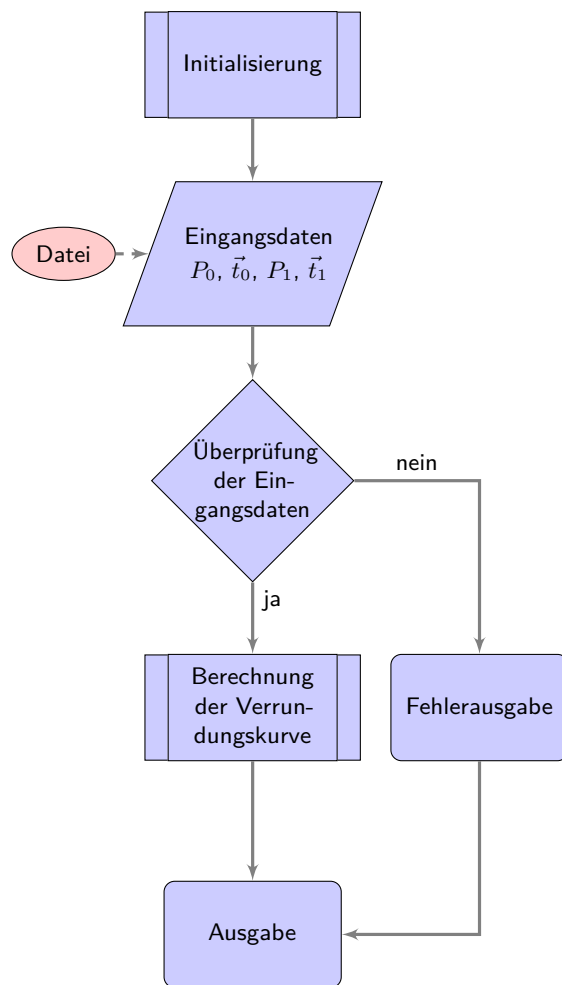


Figure 40.1.: Programme flow chart „Corner rounding“

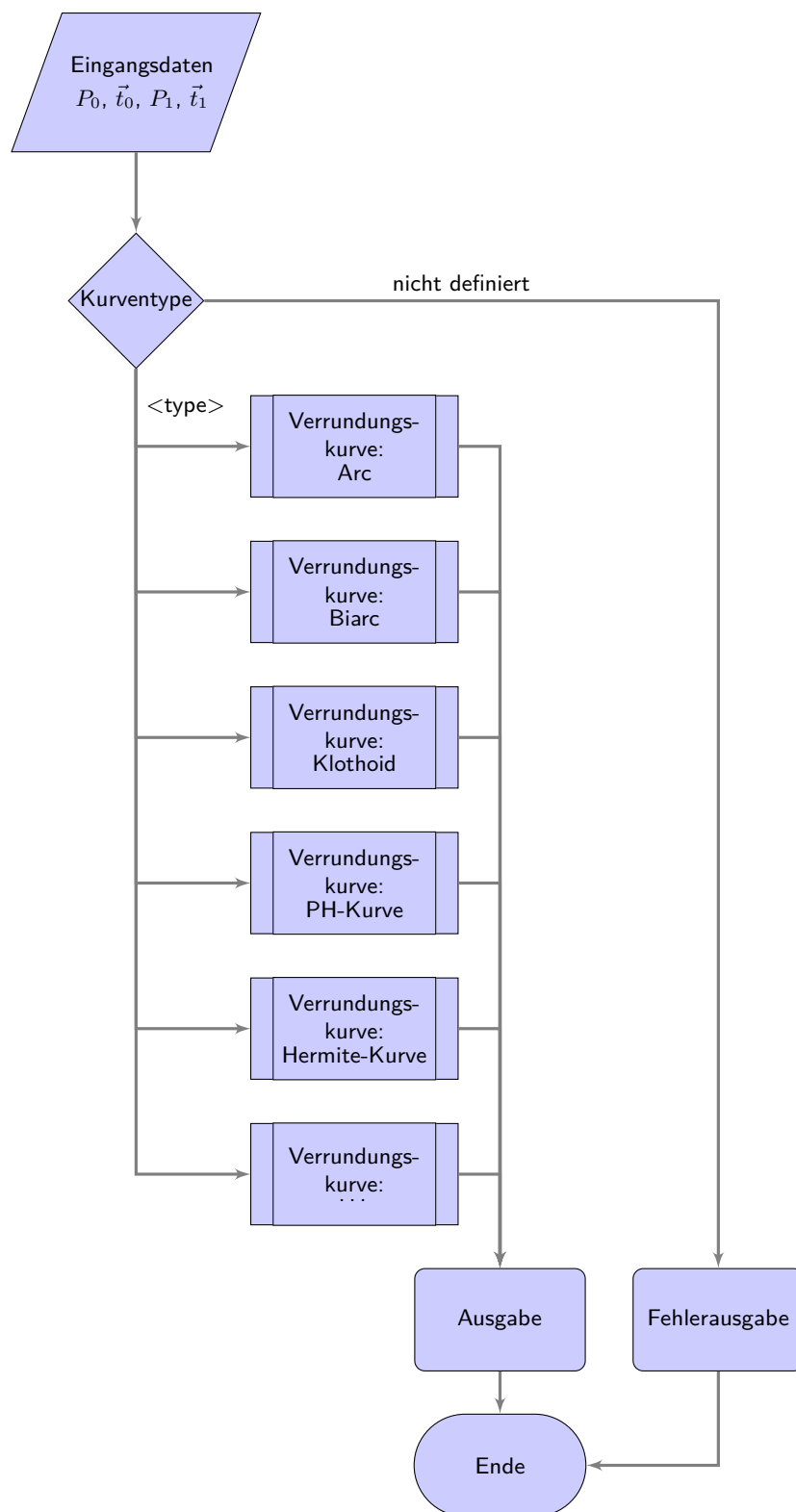


Figure 40.2.: Programme flowchart „Selection of the rounding strategies“

41. Representation of Python Programs

It is possible to integrate a part into the document, see 41.1. This is the most elegant method and is also preferable. Individual lines and line ranges can also be selected in the list of options. If a file is integrated, it is always as up-to-date as the document. It may also be useful to integrate program lines: `redLED = pyb.LED(1)`.

Attention: The integration of programs with the help of images is pointless.

Listing 41.1.: The program “Hello World” in Python for microcontroller boards is inserted from the file `Blink.py`.

```

1000 %%
1001 %% This is file 'rename-to-empty-base.tex',
1002 %% generated with the docstrip utility.
1003 %%
1004 %% The original source files were:
1005 %%
1006 %% fileerr.dtx (with options: 'return')
1007 %%
1008 %% This is a generated file.
1009 %%
1010 %% The source is maintained by the LaTeX Project team and bug
1011 %% reports for it can be opened at https://latex-project.org/bugs/
1012 %% (but please observe conditions on bug reports sent to that address!)
1013 %%
1014 %%
1015 %% Copyright (C) 1993–2024
1016 %% The LaTeX Project and any individual authors listed elsewhere
1017 %% in this file.
1018 %%
1019 %% This file was generated from file(s) of the Standard LaTeX ‘Tools
1020 %% Bundle’.
1021 %%
1022 %%
1023 %% It may be distributed and/or modified under the
1024 %% conditions of the LaTeX Project Public License, either version 1.3c
1025 %% of this license or (at your option) any later version.
1026 %% The latest version of this license is in
1027 %% https://www.latex-project.org/lppl.txt
1028 %% and version 1.3c or later is part of all distributions of LaTeX
1029 %% version 2005/12/01 or later.
1030 %%
1031 %% This file may only be distributed together with a copy of the LaTeX
1032 %% ‘Tools Bundle’. You may however distribute the LaTeX ‘Tools Bundle’
1033 %% without such generated files.
1034 %%
1035 %% The list of all files belonging to the LaTeX ‘Tools Bundle’ is
1036 %% given in the file ‘manifest.txt’.
1037 %%
1038 \message{File ignored}
1039 \endinput
1040 %%
1041 %% End of file 'rename-to-empty-base.tex'.

```

../Code/HelloWorld/Blink.py

42. Representation of Programs written for Arduino Boards

It is possible to integrate a part into the document, see 42.1. This is the most elegant method and is also preferable. Individual lines and line ranges can also be selected in the list of options. If a file is integrated, it is always as up-to-date as the document.

Attention: The integration of programs with the help of images is pointless.

Listing 42.1.: The program “Hello World” for an Arduino microcontroller board is inserted from the file `Blink.ino`.

```

1000 /**
1001  *
1002  *   @file Blink.ino
1003  *
1004  *   @brief Simple program for testing the configuration
1005  *
1006  *   Turns an LED on for one second, then off for one second, repeatedly.
1007  *
1008  *   Most Arduinos have an on-board LED you can control. On the UNO, MEGA
1009  *   and ZERO
1010  *   it is attached to digital pin 13, on MKR1000 on pin 6. LED_BUILTIN is
1011  *   set to
1012  *   the correct LED pin independent of which board is used.
1013  *   If you want to know what pin the on-board LED is connected to on your
1014  *   Arduino
1015  *   model, check the Technical Specs of your board at:
1016  *
1017  *   https://www.arduino.cc/en/Main/Products
1018  *
1019  *   modified 8 May 2014
1020  *   by Scott Fitzgerald
1021  *   modified 2 Sep 2016
1022  *   by Arturo Guadalupi
1023  *   modified 8 Sep 2016
1024  *   by Colby Newman
1025  *
1026  *   This example code is in the public domain.
1027  *
1028  *   https://www.arduino.cc/en/Tutorial/BuiltInExamples/Blink
1029  */
1030
1031
1032 /**
1033  *   @brief the setup function runs once when you press reset or power the
1034  *   board
1035  *
1036  *   standard function of Arduino sketches
1037  *
1038  *   Initialization of the pin LED_BUILTIN as output
1039  *
1040  *   @param —
1041  *
1042  *   @return void
1043  */
1044 void setup() {
1045     // initialize digital pin LED_BUILTIN as an output.
1046     pinMode(LED_BUILTIN, OUTPUT);
1047 }
1048
1049 /**
1050  *   @brief the loop function runs over and over again forever
1051  *
1052  *   standard function of Arduino sketches
1053  *
1054  *   swiching the led on / off for 1sec.
1055  *
1056  *   @param —
1057  *
1058  *   @return void
1059  */
1060 void loop() {
1061     digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the
1062     voltage level)
1063     delay(1000); // wait for a second
1064     digitalWrite(LED_BUILTIN, LOW); // turn the LED off by making the
1065     voltage LOW
1066     delay(1000); // wait for a second
1067 }

```

../Code/Arduino/Blink/Blink.ino

43. First Chapter

...

A Speicherprogrammierbare Steuerung (SPS) is ...

A Computerized Numerical Control (CNC) needs a SPS (PLC) to ...

44. CAGD

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

The book CAGD by Gerald Farin is a classic on splines. [Far02]

The standard 66025 for programming CNC machines is also a classic; however, it does not deal with splines. [DIN66025-2]

Mr. F. Farouki has dealt with both CNC machine programming and splines. His article¹ on a real-time interpolator also shows this. [FS17]

A new dimension of machine tools have emerged with the invention of 3D printers. [Rus+07]

Another aspect of automation technology is communication. Another milestone has been reached with 5G technology. another milestone has been reached.²

¹co-author is J. Srinathu

²Zaf+20.

A. drawings with tikz

The package `tikz` is a powerful tool for creating graphics. Many introductions exist. Here only the first steps are shown, so that you can easily create flowcharts.

Drawing a line and arrows

```
\begin{tikzpicture}
  \draw (0,0) — (1,0);

  \draw[>-] (2,0) — (3,0);

  \draw[<->] (5,0) — (6,0);
\end{tikzpicture}
```


Drawing a thick blue line

```
\begin{tikzpicture}
  \draw[line width=2pt, blue] (0,0) — (1,0);

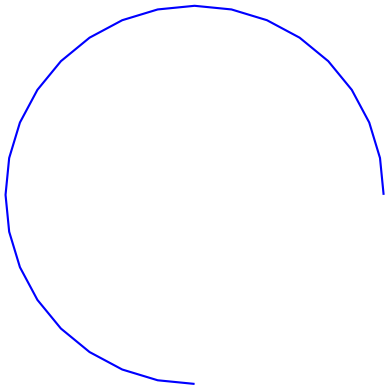
  \draw[line width=2pt, red, dotted] (2,0) — (3,0);

  \draw[line width=2pt, dashed, green] (4,0) — (5,0);

  \draw [thick,dash dot] (0,1) — (5,1);
  \draw [thick,dash pattern={on 7pt off 2pt on 1pt off 3pt}] (0,2) — (5,2);
\end{tikzpicture}
```

Drawing an arc

```
\begin{tikzpicture}
  \draw [blue,thick,domain=0:270] plot ({5+2.5*cos(\x)}, {1+2.5*sin(\x)});
\end{tikzpicture}
```



Draw a function

```
\begin{tikzpicture}[
  declare function={%
    F(\x)                =3-2*pow(2.7979,-0.8*\x);
  }
]

% Zeichnen der Funktion
\draw[red,line width=2pt,domain=0:4] plot({\x},{F(\x)});

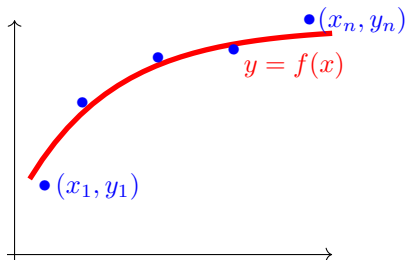
% Bezeichnung
\node[red] (O) at (3.5,2.5) {$y=f(x)$};

% Punkte
\node[blue] (P1n) at (0.9,0.9) {$(x_1,y_1)$};
\node[blue] (P1) at (0.2,0.9) {$\bullet$};

\node[blue] (P2) at (2.7,2.7) {$\bullet$};
\node[blue] (P3) at (1.7,2.6) {$\bullet$};
\node[blue] (P4) at (0.7,2) {$\bullet$};

\node[blue] (P5n) at (4.4,3.1) {$(x_n,y_n)$};
\node[blue] (P5) at (3.7,3.1) {$\bullet$};

% Koordinatensystem
\draw[black,->] (-0.2,-0.1) — (-0.2,3.1);
\draw[black,->] (-0.3,0) — (4,0);
\end{tikzpicture}
```



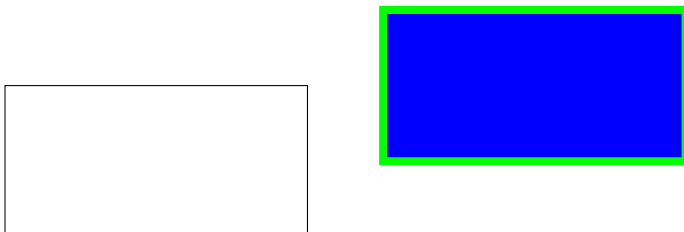
Drawing rectangles and moving objects

```

\begin{tikzpicture}
  \draw (0,0) — (4,0) — (4,2) — (0,2) — cycle;

  \begin{scope}[shift={(5,1)}]
    \draw[green,fill=blue,line width=3pt] (0,0) — (4,0) — (4,2) — (0,2) —
  \end{scope}
\end{tikzpicture}

```



Use of variables

```

\begin{tikzpicture}
\pgfmathsetmacro{\PHI}{-15}
% Now use \PHI anywhere you want -15 to appear,
% can also be used in calculations like 2*\PHI
\def\x{10};

\draw[red] (0,4) — (1-\PHI*0.5,4);
\draw[green] (0,2) — (1+1/\x,2);
\draw[blue] (0,0) — ({1+3*(\x/5+1)},0);
\end{tikzpicture}

\bigskip

%\usetikzlibrary{math} %needed tikz library

\begin{tikzpicture}
  \pgfmathsetmacro{\drehpunkt}{11.63815573}
  %Variables must be declared in a tikzmath environment but
  % can be used outside
  \tikzmath{\x1 = 1; \y1 =1;
    %computations are also possible
    \x2 = \x1 + 1; \y2 =\y1 +3; }
  \draw[->] (\x1, \y1)--(\x2, \y2);
\end{tikzpicture}

```



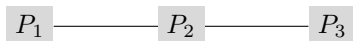


Use of points

```
%\usetikzlibrary{backgrounds} % is needed
```

```
\begin{tikzpicture}
  \node [fill=gray!30] (P1) at (0,0) { $P_1$ };
  \node [fill=gray!30] (P2) at (2,0) { $P_2$ };
  \node [fill=gray!30] (P3) at (4,0) { $P_3$ };

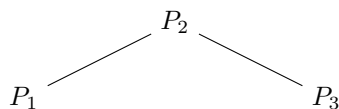
  \begin{scope}[on background layer]
    \draw (P1) — (P3);
  \end{scope}
\end{tikzpicture}
```



Use of nodes

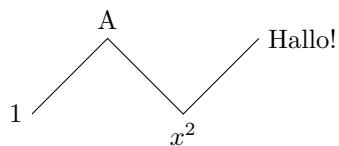
```
\begin{tikzpicture}
  \node (P1) at (0,0){ $P_1$ };
  \node (P2) at (2,1){ $P_2$ };
  \node (P3) at (4,0){ $P_3$ };

  \begin{scope}
    \draw (P1) — (P2) — (P3);
  \end{scope}
\end{tikzpicture}
```



Use of nodes

```
\begin{tikzpicture}
  \draw (0,0) node[left]{1}
    — ++(1,1) node[above]{A}
    — ++(1,-1) node[below]{ $x^2$ }
    — ++(1,1) node[right]{Hallo!};
\end{tikzpicture}
```



B. Criteria for a good L^AT_EX project

A good report is not only characterised by good content, but also fulfils formal aspects. The following list should help to comply with basic rules. Before submitting, all points should be checked and ticked off.

- ☐ Is a suitable directory structure used?
- ☐ Is an appropriate division into files used?
- ☐ Are meaningful directory and file names used?
 - ☐ Are directory and file names such as report or term paper avoided?
 - ☐ Are meaningful names used for images and not „image1“?
 - ☐ Are directory and file names not too long?
 - ☐ Are special characters and spaces avoided?
- ☐ Are commands like `\newline` and `\\` avoided?
- ☐ Are the L^AT_EX files clearly laid out?
 - ☐ Are indentations used in the L^AT_EX files?
 - ☐ Are free lines inserted?
 - ☐ Is the project mentioned in the header of the files?
 - ☐ Does the header of the files mention the main sources?
 - ☐ Is the author mentioned in the header of the files?
- ☐ Are all necessary files given?
- ☐ Are the temporary files deleted?
- ☐ Are graphics created by yourself?
- ☐ Are the sources of the images given?
- ☐ Are citations made with the command `\cite`?
- ☐ Is a bib file used?
- ☐ Are the entries of the bib-file clearly arranged?
- ☐ Are the entries of the bib-file edited to show them correctly?
- ☐ Are the correct types used for the bib entries?
- ☐ Are meaningful keys used in the bib files?

Unfortunately, the list cannot be complete, but it provides some clues.

Literaturverzeichnis

- [Aba+16] M. Abadi et al. “TensorFlow: A System for Large-Scale Machine Learning”. In: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. Savannah, GA: USENIX Association, Nov. 2016, pp. 265–283. ISBN: 978-1-931971-33-1. URL: <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi>.
- [Arda] *Arduino ABX00031 Nano 33 BLE Sense Module Benutzerhandbuch*. [pdf]. 2022. URL: <https://de.manuals.plus/arduino/abx00031-nano-33-ble-sense-module-manual> (visited on 06/26/2023).
- [Ardb] *Arduino Libraries – LCD_I2C*. 2023. URL: https://www.arduino.cc/reference/en/libraries/lcd_i2c (visited on 07/09/2023).
- [Ardc] *Arduino Libraries – Arduino_LPS22HB*. 2024. URL: https://www.arduino.cc/reference/en/libraries/arduino_lps22hb/ (visited on 06/14/2024).
- [Ardd] *Arduino Libraries – PDM*. 2023. URL: <https://docs.arduino.cc/learn/built-in-libraries/pdm> (visited on 06/14/2023).
- [Arde] *Arduino Reference – Interrupt*. 2019. URL: <https://www.arduino.cc/reference/de/language/functions/external-interrupts/attachinterrupt/> (visited on 06/26/2024).
- [Ardf] *Arduino Reference – loop()*. 2019. URL: <https://www.arduino.cc/reference/en/language/structure/sketch/loop/> (visited on 06/26/2023).
- [Ardg] *Arduino Reference – setup()*. 2019. URL: <https://www.arduino.cc/reference/en/language/structure/sketch/setup/> (visited on 06/26/2023).
- [Ardh] *Arduino Tiny Machine Learning Kit*. [pdf]. 2022. URL: <https://store.arduino.cc/products/arduino-tiny-machine-learning-kit> (visited on 06/23/2023).
- [Ard19] Arduino, ed. *Arduino Library – Kalman*. 2019. URL: <https://www.arduino.cc/reference/en/libraries/kalman/> (visited on 07/09/2023).
- [Ard21] Arduino, ed. *Arduino Nano 33 BLE Sense Rev2 with headers*. 2021. URL: <https://store.arduino.cc/products/nano-33-ble-sense-rev2-with-headers>.
- [Ard21] Arduino, ed. *Check out Arduino Docs!* 2021. URL: <https://www.arduino.cc/en/Guide>.
- [Ard23a] Arduino, ed. *Arduino Nano 33 BLE Sense Rev1 – Product Reference Manual*. [pdf]. 2023. URL: <https://docs.arduino.cc/resources/datasheets/ABX00031-datasheet.pdf>.
- [Ard23b] Arduino, ed. *Arduino Nano 33 BLE Sense Rev2 – Product Reference Manual*. [pdf]. 2023. URL: <https://docs.arduino.cc/resources/datasheets/ABX00069-datasheet.pdf>.

- [Ard24a] Arduino, ed. *Getting started with the Arduino Nano 33 BLE Sense*. 2024. URL: <https://wiki-content.arduino.cc/en/Guide/NANO33BLESense>.
- [Ard24b] Arduino, ed. *Language Reference*. 2024. URL: <https://docs.arduino.cc/language-reference/>.
- [Ard24] Arduino, ed. *Arduino APDS9960*. 2024. URL: https://docs.arduino.cc/libraries/arduino_apds9960/ (visited on 09/24/2024).
- [Ari21] D. Aristi. *LCD_I2C library on GitHub*. 2021. URL: https://github.com/blackhack/LCD_I2C (visited on 07/09/2023).
- [Ava15] Avago Technologies, ed. *APDS-9960 - Digital Proximity, Ambient Light, RGB and Gesture Sensor*. [pdf]. 2015. URL: <https://docs.broadcom.com/doc/AV02-4191EN>.
- [Az] *Terminal Adapter für Nano V3*. [pdf (de)] [pdf (en)]. AZ-Delivery, 2024. URL: <https://www.az-delivery.de/en/products/terminal-adapter-board-mit-schraubklemmen>.
- [BN11] H. Babovsky and W. Neundorf. *Numerische Approximation von Funktionen*. Tech. rep. TU Ilmenau, 2011. URL: <https://www.tu-ilmenau.de/fileadmin/media/num/neundorf/Dokumente/Preprints/NumApp1.pdf> (visited on 01/13/2017).
- [Bab+23] N. R. Babu et al. “Case Study on Ni-MH Battery”. In: *2023 2nd International Conference on Applied Artificial Intelligence and Computing (ICAAIC)* (2023), pp. 1559–1564.
- [Bas16] S. Basler. *Encoder und Motor-Feedback-Systeme*. Wiesbaden: Springer Fachmedien Wiesbaden, 2016. ISBN: 978-3-658-12843-2. DOI: 10.1007/978-3-658-12844-9. URL: <https://link.springer.com/book/10.1007/978-3-658-12844-9>.
- [Ber18] H. Bernstein. *Elektrotechnik/Elektronik für Maschinenbauer - Einfach und praxisgerecht*. 3rd ed. Berlin Heidelberg New York: Springer-Verlag, 2018, pp. 235–236. ISBN: 978-3-658-20838-7.
- [Box21] J. Boxall. *Arduino Workshop - A Hands-On Introduction with 65 Projects*. 2. No Starch Press, 2021. ISBN: 9781593274481.
- [Bro24] Broadcom. *APDS-9960*. 2024. URL: <https://www.broadcom.com/products/optical-sensors/integrated-ambient-light-and-proximity-sensors/apds-9960>.
- [Chi+06] H.-H. Chiang et al. “The Human-in-the-loop Design Approach to the Longitudinal Automation System for the Intelligent Vehicle, TAIWAN iTS-i”. In: *2006 IEEE International Conference on Systems, Man and Cybernetics*. [pdf]. IEEE. 2006, pp. 2839–2844. DOI: 10.1109/ICSMC.2006.384384. URL: <https://ieeexplore.ieee.org/abstract/document/4273859>.
- [DIN66025-2] DIN Deutsches Institut für Normung e.V. *DIN 66025-2:1988-09: Programmaufbau für numerisch gesteuerte Arbeitsmaschinen: Wegbedingungen und Zusatzfunktionen*. Norm. Berlin, Sept. 1988.
- [Din] *DIN EN ISO 13850: Sicherheit von Maschinen - Not-Halt-Funktion - Gestaltungsleitsätze (ISO 13850:2015)*. 2016. URL: <https://www.din.de/de/mitwirken/normenausschuesse/nam/veroeffentlichungen/wdc-beuth:din21:233572513>.
- [FD22] P. Filipi and Dozie. *A Difference between Arduino Nano 33 BLE Sense vs. Arduino Nano 33 BLE Sense Lite*. [pdf]. 2022. URL: <https://forum.arduino.cc/t/a-difference-between-a-n-33-ble-sense-vs-sense-lite/1030305>.

- [FH14] R. Faragher and R. Harle. “An Analysis of the Accuracy of Bluetooth Low Energy for Indoor Positioning Applications”. In: *Proceedings of the 27th International Technical Meeting of The Satellite Division of the Institute of Navigation (ION GNSS+ 2014)*. 2014, pp. 2018–2027.
- [FS17] R. Farouki and J. Srinathu. “A real-time CNC interpolator algorithm for trimming and filling planar offset curves”. In: *Computer-Aided Design* 86 (Jan. 2017). DOI: 10.1016/j.cad.2017.01.001.
- [Far02] G. Farin. *Curves and Surfaces for CAGD*. 5. [pdf]. San Diego, CA: Academic Press, 2002.
- [Fet21] R. J. Fetick. *Kalman*. 2021.
- [Fou24a] P. S. Foundation, ed. *threading: Thread-based parallelism*. 2024. URL: <https://docs.python.org/3/library/wave.html> (visited on 06/13/2024).
- [Fou24b] P. S. Foundation, ed. *wave: Read and write WAV files*. 2024. URL: <https://docs.python.org/3/library/wave.html> (visited on 06/13/2024).
- [Gan24] T. Gan. “Capacitive Flexible Pressure Sensors and Their Application”. In: *E3S Web of Conferences* 553 (July 2024). [pdf]. DOI: 10.1051/e3sconf/202455305038.
- [Gua21] A. Guadalupi. *Machine LEarning Carrier V1.0*. [pdf]. 2021.
- [HEG21] E. Hering, J. Endres, and J. Gutekunst. *Elektronik für Ingenieure und Naturwissenschaftler*. 8. [pdf]. Springer Vieweg, 2021. ISBN: 978-3-662-62697-9. DOI: 10.1007/978-3-662-62698-6.
- [HMS21] E. Hering, R. Martin, and M. Stohrer. *Physik für Ingenieure*. 13th ed. Springer Vieweg, Aug. 2021, p. 548. ISBN: 978-3-662-63177-5.
- [HS23] E. Hering and G. Schönfelder. *Sensoren in Wissenschaft und Technik*. 3. [pdf]. Springer Vieweg, 2023. ISBN: 978-3-658-39490-5. DOI: 10.1007/978-3-658-39491-2978-3-662-62697-9.
- [Hil22] G. Hiller. *Color measurement - the CIE color space*. [pdf]. Datacolor, 2022.
- [Hym24] S. Hymel. *APDS-9960 RGB and Gesture Sensor Hookup Guide*. [pdf]. 2024. URL: <https://learn.sparkfun.com/tutorials/apds-9960-rgb-and-gesture-sensor-hookup-guide/all>.
- [Jak+10] G. Jaklic et al. “On Interpolation by Planar Cubic G^2 Pythagorean-Hodograph Spline Curves”. In: *Mathematics of Computation* (2010). [pdf].
- [Jsta] *Corporate Profile*. 2021. URL: <https://www.jst-mfg.com/product/index.php?series=262>.
- [Jstb] *List of JST Connectors Series*. 2020. URL: <http://www.jst.com/home8.html>.
- [Jstc] *PH connector*. [pdf]. J.S.T. Deutschland GmbH, 2022. URL: <https://www.jst-mfg.com/product/index.php?series=199>.
- [Jstd] *VH connector*. [pdf]. J.S.T. Deutschland GmbH, 2021. URL: <https://www.jst-mfg.com/product/index.php?series=262>.
- [KKV14] K. Karvinen, T. Karvinen, and V. Valtokari. *Sensoren – messen und experimentieren mit Arduino und Raspberry Pi*. dpunkt, 2014. ISBN: 97833864901607.
- [Küh24] C. Kühnel. *Arduino - Das umfassende Handbuch*. 3rd ed. [pdf]. Rheinwerk Verlag GmbH, 2024. ISBN: 978-3-367-10279-2.

- [Kur21] A. Kurniawan. *IoT Projects with Arduino Nano BLE Sense: Step-By-Step Projects for Beginners*. [pdf]. Berkeley, CA: Apress, 2021. ISBN: 978-1-4842-6457-7. DOI: 10.1007/978-1-4842-6458-4. URL: <https://doi.org/10.1007/978-1-4842-6458-4>.
- [LAH22] C. Liu, M. A. Alif, and G. He. “Shoulder Motion Detection Algorithm Based on MPU6050 Sensor and XGBoost Model”. In: *2022 International Conference on Computing, Communication, Perception and Quantum Technology (CCPQT)*. [pdf]. IEEE. 2022, pp. 1–5. DOI: 10.1109/CCPQT54534.2022.9939285. URL: <https://ieeexplore.ieee.org/document/9939285>.
- [LBSK05] X. Lin, Y Bar-Shalom, and T Kirubarajan. “Multisensor-multitarget bias estimation for general asynchronous sensors”. In: *IEEE Transactions on Aerospace and Electronic Systems* 41.1 (2005). [pdf], pp. 24–45. DOI: 10.1109/TAES.2005.1419586.
- [LP18] R. Lukac and K. N. Plataniotis. *Color Image Processing: Methods and Applications*. Image Processing Series. CRC Press, 2018. ISBN: 9781420009781. URL: <https://books.google.de/books?id=oD8qBgAAQBAJ>.
- [Lib21] A. Libraries, ed. *Arduino_LSM6DSOX Library for Arduino*. 2021. URL: https://github.com/arduino-libraries/Arduino_LSM6DSOX (visited on 07/09/2023).
- [MAB22] J. Martínez, D. Asiain, and J. R. Beltrán. “Self-Calibration Technique with Lightweight Algorithm for Thermal Drift Compensation in MEMS Accelerometers”. In: *Micromachines* 13.4 (2022). [pdf], p. 584. DOI: 10.3390/mi13040584. URL: <https://www.mdpi.com/2072-666X/13/4/584>.
- [Mae24] S. Maetschke. *APDS-9960 Gesture and Color Sensor with Arduino*. [pdf]. 2024. URL: <https://www.makerguides.com/apds-9960-gesture-and-color-sensor-with-arduino/>.
- [Mal15] Mallon, Edward and Beddows, Patricia. *How to calibrate a compass (and accelerometer) with Arduino*. accessed date 19.05.2022. 2015. URL: <https://thecavepearlproject.org/2015/05/22/calibrating-any-compass-or-accelerometer-for-arduino/>.
- [McF+23] B. McFee et al. “librosa/librosa: 0.10.2.post1”. Version 0.10.2.post1. In: (2023). DOI: 10.5281/zenodo.11192913.
- [NKG13] A. Nouredin, T. B. Karamat, and J. Georgy. *Fundamentals of Inertial Navigation, Satellite-based Positioning and their Integration*. Springer Science and Business Media LLC, 2013. DOI: 10.1007/978-3-642-30466-8.
- [Pha06] H. Pham. *PyAudio*. 2006. URL: <https://people.csail.mit.edu/hubert/pyaudio/> (visited on 04/06/2024).
- [Poh17] R. O. Pohl, ed. *Pohls Einführung in die Physik: Band 1: Mechanik, Akustik und Wärmelehre*. 21st ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2017. ISBN: 9783662486634. URL: <http://nbn-resolving.org/urn:nbn:de:bsz:31-epflucht-1552851>.
- [RS17] J. Rehder and R. Siegwart. “Camera/IMU calibration revisited”. In: *IEEE Sensors Journal* 17.11 (2017). [pdf], pp. 3257–3268. DOI: 10.1109/JSEN.2017.2674307.

- [Raj19] A. Raj. *Arduino Nano 33 BLE Sense Review - What's New and How to Get Started?* 2019. URL: <https://circuitdigest.com/microcontroller-projects/arduino-nano-33-ble-sense-board-review-and-getting-started-guide>.
- [Rei] *Batterieclip für 9-Volt-Block*. 11445. Das Datenblatt liefert allgemeine Informationen zum Batterieclip. Die Höhe des Batterieclips ist nicht aufgeführt. Reichelt Elektronik GmbH. July 2011.
- [Rei24a] Reichelt Elektronik GmbH, ed. *Arduino - Grove Universal-Kabel, 4-Pin, 5cm, fixiert (5er-Pack)*. Online; accessed 17.04.2024. 2024. URL: <https://www.reichelt.de/arduino-grove-universal-kabel-4-pin-5cm-fixiert-5er-pack--grv-cable4pin5f-p191140.html>.
- [Rei24b] Reichelt Elektronik GmbH, ed. *Batterieclip für 9-Volt-Block, vertikal*. Online; accessed 17.04.2024. 2024. URL: <https://www.reichelt.de/batterieclip-fuer-9-volt-block-vertikal-clip-9v-p6665.html>.
- [Rei24c] Reichelt Elektronik GmbH, ed. *GRV 4PIN F2GRV Arduino - Grove 4-Pin Female Jumper zu Grove 4-Pin (5er-Pack)*. Online; accessed 06.06.2024. 2024. URL: <https://www.reichelt.de/arduino-grove-4-pin-female-jumper-zu-grove-4-pin-5er-pack--grv-4pin-f2grv-p191166.html>.
- [Rus+07] D. Russell et al. *Apparatus and Methods for 3D Printing*. United States Patent, Patent N0.: US 7,291,002 B2. 2007.
- [Ryb13] J. Rybach. *Physik für Bachelors: mit 92 durchgerechneten Beispielen, 176 Testfragen mit Antworten sowie 93 Übungsaufgaben mit kommentierten Musterlösungen*. Carl Hanser Verlag, 2013. ISBN: 978-3-446-43529-2.
- [ŠŠ17] T. Štefanička, R. Ďuračiová, and C. Seres. “Development of a Web-Based Indoor Navigation System Using an Accelerometer and Gyroscope: A Case Study at The Faculty of Natural Sciences of Comenius University”. In: *Slovak Journal of Civil Engineering* 25.2 (2017). [pdf], pp. 30–38. DOI: 10.1515/sjce-2017-0005.
- [SO20] L. Sach and M. O’Hanlon. *Create Graphical User Interfaces with Python*. [pdf]. Raspberry Pi Press, 2020. ISBN: 978-1-912047-91-8.
- [SS14] B. Sencer and E. Shamoto. “Curvature-continuous sharp corner smoothing scheme for Cartesian motion systems”. In: *2014 IEEE 13th International Workshop on Advanced Motion Control (AMC)*. [pdf] und [Version 2 - pdf]. Piscataway, NJ: IEEE, 2014, pp. 374–379. ISBN: 978-1-4799-2323-6. DOI: \url{10.1109/AMC.2014.6823311}.
- [STM21] STMicroelectronics International NV, ed. *MP34DT05-A – MEMS audio sensor omnidirectional digital microphone*. [pdf]. 2021. URL: <https://www.st.com/resource/en/datasheet/mp34dt05-a.pdf> (visited on 06/23/2024).
- [Sab11] A. M. Sabatini. “Estimating three-dimensional orientation of human body parts by inertial/magnetic sensing”. In: *Sensors* 11.2 (2011), pp. 1489–1525.
- [Sch05] P. Scholz. *Softwareentwicklung eingebetteter Systeme*. 1st ed. Springer, 2005.
- [Sch07] J. Schanda. *Colorimetry: Understanding the CIE System*. Wiley, 2007. ISBN: 9780470175620. URL: <https://books.google.de/books?id=uZadszSGe9MC>.

- [Sch22] T. Scherer. “Batteriemanagement”. In: *Elektor special: Stromversorgung und Batterien* (2022), pp. 6–8.
- [See12] Seeed Technology Co.,Ltd., ed. *Introduction to Grove*. [pdf]. Shenzhen Headquarters, 2012.
- [See13] Seeed Technology Co.,Ltd., ed. *Preface - Getting Started*. [pdf]. Shenzhen Headquarters, 2013.
- [See16] Seeed Technology Co.,Ltd., ed. *Grove System*. [pdf]. Shenzhen Headquarters, 2016.
- [She] *Voltage Sensor / 170640 - Data Sheet*. 170640. [pdf]. Shenzhen Global Tech Co. 2015.
- [Sima] *Joy-IT@MotoDriver2*. [pdf]. Simac Electronics GmbH. Apr. 2019.
- [Simb] *Joy-IT@Ultrasonic Distance Sensor*. SEN-US01. [pdf]. Simac Electronics GmbH. Aug. 2017.
- [Sim19] Simac Electronics GmbH. *COM-KY040RE-Datenblatt: Drehencoder mit Tasterfunktion*. [pdf]. 2019. URL: www.joy-it.net.
- [Smy21a] R. J. Smythe. *Advanced Arduino Techniques in Science - Refine Your Skills and Projects with PCs or Python-Tkinter*. [pdf]. Berkeley, CA: Apress, 2021. ISBN: 978-1-4842-6786-8. DOI: 10.1007/978-1-4842-6784-4. URL: <https://doi.org/10.1007/978-1-4842-6784-4>.
- [Smy21b] R. J. Smythe. *Arduino in Science - Collecting, Displaying, and Manipulation Sensor Data*. [pdf]. Berkeley, CA: Apress, 2021. ISBN: 978-1-4842-6777-6. DOI: 10.1007/978-1-4842-6777-6.
- [Stma] *LSM6DSOX Datasheet*. [pdf]. 2018. (Visited on 06/23/2023).
- [Stmb] *LSM9DS1 Data Sheets*. 2015. URL: <https://www.st.com/resource/en/datasheet/lsm9ds1.pdf> (visited on 06/11/2022).
- [Stmc] *MP34DT06J – MEMS audio sensor omnidirectional digital microphone*. [pdf]. 2021. URL: <https://www.st.com/resource/en/datasheet/mp34dt06j.pdf> (visited on 06/23/2023).
- [Sto24] P. Stoffregen. *Encoder Library*. Ed. by PJRC. [pdf]. 2024. URL: https://www.pjrc.com/teensy/td_libs_Encoder.html.
- [TM24] P. A. Tipler and G. Mosca. *Tipler Physik*. 9th ed. Springer Spektrum Berlin, May 2024, pp. 423–427. ISBN: 978-3-662-67936-4.
- [TPM14] D. Tedaldi, A. Pretto, and E. Menegatti. “A Robust and Easy to Implement Method for IMU Calibration without External Equipment”. In: *Proceedings of the IEEE International Conference on Robotics and Automation*. [pdf]. IEEE. 2014, pp. 3040–3045. DOI: 10.1109/ICRA.2014.6907275.
- [The22] The NumPy Community, ed. *NumPy User Guide*. [pdf]. 2022. URL: <https://numpy.org/doc/stable/user/whatisnumpy.html>.
- [Tri+22] H. K. Tripathy et al. “Smart COVID-shield: An IoT driven reliable and automated prototype model for COVID-19 symptoms tracking”. In: *Computing* (2022). [pdf], pp. 1–22. DOI: 10.1007/s00607-022-00975-7.
- [Vec] *Inertial Navigation Primer*. <https://www.vectornav.com/resources/inertial-navigation-primer/specifications-and-error-budgets/specs-imuspecs>. [pdf]. 2021.

- [Voš24] A. Voš. “Volumio mit Drehgebern erweitern”. In: *Make Magazin* (2024). [\[pdf\]](#).
- [Wah+11] W. Wahyudi et al. “Inertial Measurement Unit using multigain accelerometer sensor and gyroscope sensor”. In: *2011 International Conference on Electrical Engineering and Informatics*. [\[pdf\]](#). IEEE. 2011, pp. 1–6. DOI: 10.1109/ICEEI.2011.6021711.
- [Wan+22] Y. Wang et al. “Multi-position wearable human activity recognition dataset”. In: (2022). DOI: 10.21227/z8bc-pt92. URL: <https://dx.doi.org/10.21227/z8bc-pt92>.
- [Wat17a] Waterloo Maple Inc. 2017. *Description*. [letzter Zugriff 14.09.2017](#). 2017. URL: <https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/description>.
- [Wat17b] Waterloo Maple Inc. 2017. *Export*. [letzter Zugriff 14.09.2017](#). 2017. URL: <https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/export>.
- [Wat17c] Waterloo Maple Inc. 2017. *Module*. [letzter Zugriff 14.09.2017](#). 2017. URL: <https://de.maplesoft.com/support/help/maple/view.aspx?path=module>.
- [Wat17d] Waterloo Maple Inc. 2017. *ModuleLoad*. [letzter Zugriff 14.09.2017](#). 2017. URL: <https://de.maplesoft.com/support/help/Maple/view.aspx?path=ModuleLoad>.
- [Wat17e] Waterloo Maple Inc. 2017. *Option*. [letzter Zugriff 14.09.2017](#). 2017. URL: <https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/option>.
- [Wat17f] Waterloo Maple Inc. 2017. *Procedures*. [letzter Zugriff 14.09.2017](#). 2017. URL: <https://de.maplesoft.com/support/help/Maple/view.aspx?path=procedure>.
- [Wei20] S. Weinzierl. *Handbuch der Audiotechnik*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2020. ISBN: 978-3-662-60357-4. DOI: 10.1007/978-3-662-60357-4.
- [Wil03] S. Wilson. *WAVE PCM soundfile format*. 2003. URL: <https://ccrma.stanford.edu/courses/422-winter-2014/projects/WaveFormat/> (visited on 04/15/2023).
- [Wil22] W. M. Willems. *Lehrbuch der Bauphysik*. 9th ed. Springer Vieweg Wiesbaden, Nov. 2022, pp. 567–568. ISBN: 978-3-658-34093-3.
- [Zaf+20] A. Zafeiropoulos et al. “Benchmarking and Profiling 5G Verticals’ Applications: An Industrial IoT Use Case”. In: *IEEE Conference on Network Softwarization, NetSoft 2020*. 2020. DOI: 10.1109/NetSoft48620.2020.9165393.

Index

- 3D printer, 311
- Programmable Logic Controller, 309
- File
 - Arduino LSM9DS1.h, 192
 - Arduino_APDS9960, 83, 88
 - Arduino_LSM9DS1.h, 191
 - ArduinoBLE, 230
 - ArduinoBLE.h, 228
 - Blink.ino, 308
 - Blink.py, 306
 - BuiltinLED.cpp, 44
 - BuiltinLED.h, 44
 - Kalman.h, 191, 192
 - LED.cpp, 49
 - LED.h, 48
 - LSM6DSOXSensor.h, 181
 - LSM9DS1, 230
 - Mag_raw.txt, 34
 - Nano33BLE_Led_A0.ino, 224
 - output.wav, 124
 - PackageExample.py, 273
 - PDM.h, 144
 - pdm.h, 142, 143
 - PowerLED.cpp, 50
 - PowerLED.h, 50
 - RGBLED.cpp, 57
 - RGBLED.h, 56
 - SignsOfLife.cpp, 52
 - SignsOfLife.h, 52
 - SimpleAccelerometer, 165
 - Sketch -> Include Library -> Manage Library, 226
 - smoothedAudio.wav, 131
 - sound1.wav, 141
 - sound2.wav, 141
 - sound3.wav, 141
 - SSD1306Ascii.h, 167
 - TestBattery.ino, 210, 212
 - TestHCSR04 UltraschallsensorTest.ino, 247
 - TestHCSR04.ino, 249
 - Wire.h, 167, 191, 193
 - wire.h, 192
- Inertial Measurement Unit
 - see* IMU, 21, 30
- Acoustic Overload Point
 - see* AOP, 21, 35
- AOP, 21, 35
- CAGD, 311
 - Splines, 311
- CNN, 21, 309
- Computerized Numerical Control
 - see* CNN, 21, 309
- Example, 273
 - Description, 273
 - Installation, 273
 - Introduction, 273
- General Purpose Input Output
 - see* GPIO, 21, 246
- GPIO, 21, 246, 247
- I²C, 21, 246, 247
- IDE, 21, 212, 249
- IMU, 21, 30, 33
- Integrated Development Environment
 - see* IDE, 21
- Inter-Integrated Circuit
 - see* I²C, 21, 246
- Japan Solderless Terminal
 - see* JST, 21, 218
- JST, 21, 218
- LED, 21, 47, 48, 52, 55–59, 61, 62
 - Brightness, 61
 - Built-in LED, 43
 - Built-in RGB-LED, 55
 - Colors, 58
 - Power LED, 47
- Light Emitting Diode
 - see* LED, 21, 47
- mcd, 21, 56
- Millicandela
 - see* mcd, 21, 56
- PCB, 21, 213
- PDM, 21, 35, 144
- Pin
 - Pin 11, 65, 66

- Pin 13, 43–45
- Pin 22, 56
- Pin 22,23,24, 56
- Pin 23, 56
- Pin 24, 56
- Pin 25, 48, 50
- PLC, *see* Programmable Logic Controller
- Printed Circuit Board
 - see* PB, 21, 213
- Pulse Density Modulation
 - see* PDM, 21, 35
- Pulse with Modulation
 - see* PWM Signal, 21, 48
- Push Button
 - Built-in Push Button, 65
- PWM Signal, 21, 48, 55, 58

- SCL, 21, 246, 247
- SDA, 21, 246, 247
- Serial Clock
 - see* SCL, 21, 246
- Serial Data
 - see* SDA, 21, 246
- Serial Peripheral Interface
 - see* SPI, 21, 219
- Speicherprogrammierbare Steuerung
 - see* SPS, 21, 309
- SPI, 21, 219
- SPS, 21, 309

- UART, 21, 246, 247
- Universal Asynchronous Receiver Transmitter
 - see* UART, 21, 246

- VCC, 21, 210, 246, 247
- Voltage at the Common Collector
 - see* VCC, 21, 210